

Public Transport Facial Recognition System Project

Public Transport Facial Recognition Payment System	7
Analysis	7
Problem Recognition.....	7
Stakeholders	7
Why computational methods are suitable:	8
Computation Methods:.....	8
Abstraction.....	8
Thinking ahead	8
Decomposition	9
Thinking procedurally	9
Research.....	9
Standard rail ticket systems (UK)	9
Apple Face ID	10
Cruise ship disembarking recognition.....	10
School Reception sign in system	11
Approaches to my solution	12
Open cv	12
Google cloud vision API.....	12
Tkinter	12
PyQt.....	13
Interview	13
Frequent Rail User	13
Interview – Frequent rail user.....	14
John	14
Sandra	14
Analysis	15
Interview – Rail Ticket Officer	15
Mike	15
Mark	16
Analysis	16
Proposal	16
Features	16
Limitations.....	17

Hardware requirements.....	17
Software requirements	17
Success Criteria	18
Design.....	21
Structure of solution	21
Whole program structure diagram	21
Explanation of system diagram.....	21
Barrier application.....	21
Kiosk application	22
Account Creation	22
Account management.....	22
Explain/justify - why you have chosen to break the problem down into these subproblems.	Error! Bookmark not defined.
Justification of system diagram	23
Kiosk application flow diagram	23
Explanation of Kiosk Flow Diagram.....	23
Barrier application flow diagram	26
Explanation of barrier application flow diagram	26
Justification of flow diagrams	26
Proposed screen designs and Usability features	28
Database	28
Kiosk program	28
Barrier application interface	32
Usability Features	34
Inputs and Outputs	34
Key variables	34
Validation	35
Entry Fields.....	35
Dropdown menu boxes.....	36
Menu buttons	36
Camera feed.....	36
Testing method	36
Whitebox testing.....	36
Blackbox testing	37
Stakeholder testing	37
Alpha and beta testing	37

Testing layout.....	37
Development.....	38
Stage 1 : Facial recognition of user	38
Step 1: Read camera input.....	38
Step 2: Recognise face from the camera feed	39
Step 3: Find face encodings of the face	41
Stage 1 Review	45
What's been developed:	45
How it was tested:	45
How it meets my success criteria:.....	45
Where it doesn't meet my success criteria so far:	46
Any changes from the design in this section:	46
Where the project is so far:	46
Stage 2: Option Menu	46
Testing.....	49
Stage 2 Review	51
What's been developed:	51
How it was tested:	51
How it meets my success criteria:.....	51
Where it doesn't meet my success criteria so far:	51
Any changes from the design in this section:	51
Where the project is so far:	52
Stage 3: New user creation	52
Step 1: Confirmation window	52
Step 2: Database declaration	54
Step 3: New user picture taken.....	55
Step 4: New user details entry.....	65
Stage 3 Review	82
What's been developed:	82
How it was tested:	82
How it meets my success criteria:.....	82
Where it doesn't meet my success criteria so far:	83
Any changes from the design in this section:	83
Where the project is so far:	83
Stage 4: Login Window	84
Step 1: Log the user in.....	84

Step 2: User options menu	91
Step 3: Login fail window	105
Stage 4 Review	108
What's been developed:	108
How it was tested:	108
How it meets my success criteria:.....	108
Where it doesn't meet my success criteria so far:	109
Any changes from the design in this section:	109
Where the project is so far:	109
Stage 5: Admin menu.....	110
Step 1: Admin button.....	110
Step 2: Admin login.....	112
Step 3: Admin page menu.....	120
Step 4: Admin email option	124
Step 5: Admin name option	132
Step 6: Admin date option	139
Step 7: Admin Credit option	149
Stage 5 Review	156
What's been developed:	156
How it was tested:	156
How it meets my success criteria:.....	156
Where it doesn't meet my success criteria so far:	156
Any changes from the design in this section:	156
Where the project is so far:	156
Stage 6: Barrier program.....	157
Step 1: Barrier setup and facial detection	157
Step 2: Barrier face recognition	161
Step 3: Charging the user the correct amount	164
Stage 6 Review	172
What's been developed:	172
How it was tested:	172
How it meets my success criteria:.....	172
Where it doesn't meet my success criteria so far:	173
Any changes from the design in this section:	Error! Bookmark not defined.
Where the project is so far:	Error! Bookmark not defined.
Testing.....	173

Post-development testing.....	173
Kiosk	173
Barrier	180
Stakeholder testing.....	188
Stakeholder testing review	190
Evaluation	190
Success criteria comparison.....	190
Why some criteria weren't met.....	191
Why some criteria were partially met	192
Changes during Development.....	192
Improvements to the program	193
Future additional features	193
Usability features	194
Future maintenance.....	196
Limitations	196
Finished program	198
"Stage1.py"	198
"face_encode.py"	201
"Thekiosk.py"	202
"new_user.py"	203
"log_in.py"	210
"update_details.py"	213
"admin.py"	217
"Barrier.py"	225

Public Transport Facial Recognition Payment System

Analysis

Problem Recognition

Public transport is easily defrauded by the public and initiating ticket checks can be expensive and time consuming, often causing traffic. Traditional ways to check tickets is by paying employees to manually check every customer's ticket once they have boarded, meaning that they must pay daily wages to every transport officer per vehicle. Using AI recognition for ticket collection has a wide range of benefits: less money spent on workers' wages, no chance of passengers losing tickets, no ability to board without pay.

There would be physical limitations to the implementation of AI, since transport stations would need to install stations able to deny or allow access to passengers with a built-in camera to recognise the face of the customer. Payment/account initialisation stations would also be required to be implemented. The purpose of these would be to scan the face and allow the barrier to open after the account has been charged or decline access if an account does not have enough money present. Also, there must be enough storage space for the database to store landmark points or 3D depth maps of each user's face for later use yet quick retrieval, which would cost quite a lot.

Another limitation would be the risk of hacking and having to defend against that by in the future implementing code which can reduce the risk of hackers. If hackers were able to access my program, customers data and information would be at risk. Facial scans are sensitive pieces of information that can be used to create deepfakes and identity fraud. Encryption would be needed to be implemented to make the data less accessible to potential hackers.

The problem requires computational methods since it utilises artificial intelligence, cameras, and interaction with a computer. The program will use artificial intelligence to detect faces from a camera, match them to a face on a database and then charge or not charge a corresponding account, depending on the amount of credit it has.

Stakeholders

The stakeholders from this software range from the owners of rail/transport systems to the consumers who use these forms of transportation.

The biggest focus of a business to its owners and investors is profitability. Owners and investors of rail networks would benefit from this software because implementation could decrease costs, increase profitability, and decrease train queues. Salaries of train conductors and security personnel could be reduced as the need to have people checking tickets on each train would be unnecessary. By having an automatic ticket system, there is no need to scan anything or spend long amounts of time trying to find change for a specific ticket amount. Instead, money would be stored as credit on a database, decreasing how long it takes to purchase a ticket. Both would help in increasing profits, as it increases the rate at which revenue comes in and the amount of outgoing costs, increasing profits.

For rail users, they would benefit from having an easier way to purchase transport tickets and faster queuing times for trains/metro. A credit system will be much easier than individual tickets since you can make a lump payment instead of having to purchase individual tickets. Facial recognition only takes a couple of seconds to walk through instead of minutes that purchasing, researching, and then authenticating a ticket can take. The average rail/metro users are students and commuters going to work who all value their time and punctuality which the software could assist in. Having prepaid credit

will also ease commutes since there is no need to make payment beforehand. The average rail/metro user would benefit from such software as it would speed up their day and make commuting faster and easier.

Facial recognition ticket bases do not only have to be applied to metro or rail, but they could also have a wide range of uses, such as in the workplace. Workers could “clock in” through a facial recogniser which logs them on their database and logs when they leave the workplace, giving accurate times for work hours and time spent overtime, also including lunch breaks. This could have a dual effect also as security for the workplace, not permitting entry to anyone who is not authorized. The database could also calculate total hours worked, overtime hours and minutes late. Better monitoring of staff tardiness by the managers would benefit companies' efficiency and provide computerised assessment of attendance. In this model, credit would not be necessary.

Why computational methods are suitable:

The project is best solved by computational methods since computers can swiftly recognise and authenticate users on large databases, unlike humans. Digital accounting using databases is also much simpler and faster rather than manual cash/card transactions which computers and computer databases can do very swiftly. Computers also do not require salaries, lunch breaks or holidays, being much less maintenance for company owners.

The problem the computational methods are solving is speed and efficiency, which computers are better at due to computers being able to execute simple instructions quicker than humanly possible.

Computers allow for automation of a process, repeating the same set of instructions without decreasing speed or efficiency. Computer's function is to receive inputs, process it and then return outputs to the user, a lot quicker than a human can. This would be suitable for my project since my project will include inputs, that must be processed and then outputs to the user, which a computer will be able to do with a lot more ease.

The problem can be solved by a computer since it is able to take the image of the user and process and store data about the user and output the data when it's been processed. The computer can do this at speeds faster than humans and cheaper than humans and would also use validation to erase human errors that can occur. Computational methods should be used for this reason, creating a more efficient and safe system for users.

Computation Methods:

Abstraction

When detecting a face, the computer must analyse a lot of data, a lot of it being useless colour data that does not assist in detection, just slows down the process by increasing the amount of data that is required to process. By grey scaling the image, there is less data for the computer to process at a time and allows smoother tracking of facial landmarks from the live feed.

Thinking ahead

The main user input for the main program would be the user's camera. The use of the camera allows for recognition of facial landmarks to detect a user's face and corresponding account data. I will have other inputs from computer such as typing into entry fields and clicking buttons and dropdown boxes selections.

The data that is input by the user's needs to also be processed and inputted into the database for further usage.

During account creation, the user needs to input details about their account including date of birth, email, name, gender, and a full scan of their face. These details will be stored on the database.

Decomposition

The problem can be broken down into 3 main components: the logging in/entry/credit system, the database, and the account creation.

The database needs to be made first due to each both other components require use of the database to run. The database will require information about ticket prices and timings and then a table for user information.

The logging in system can be further broken down into these steps:

- Scan face
- Identify landmark points
- Match to a user on the database (if not redirect to account creation)
- Charge credit if required
- Allow entry to the user

Thinking procedurally

The problem can use functions and procedures to create reusability in the program and section off and isolate pieces of code from the rest of the code inside of it. Putting code into functions and procedures allows for the code to be further broken down and decomposed, increasing the readability of the code and making development and bug fixing more simple.

An example of a function I would have in my code would be a function to find the face map of the user, which would take an image as a parameter and then returning the face map of the user from the function. The reason I would have this as a function and not just inside of the code when I need to use it is because I most likely will need to use this function multiple times, reducing the time spent writing the code since I can call the function.

Research

Standard rail ticket systems (UK)



The typical ticket system used for rail systems in the UK is where you have barriers and a scanner which either accepts an existing ticket, scans a digital ticket, or accepts a contactless payment. For this you need three different input devices in one machine: card reader, ticket input and digital ticket reader. A ticket printing/purchasing device would also be necessary for users who would like to purchase a physical ticket before the barrier.

Applications to my software:

For my software to work it would also need similar hardware as currently used in majority of rail ticket systems in the form of barricades with a form of authentication, but I would not need 3 input systems. The use of a card reader is something I could apply to my solution as it can allow for easy top up of an account without much difficulty to the user.

Apple Face ID

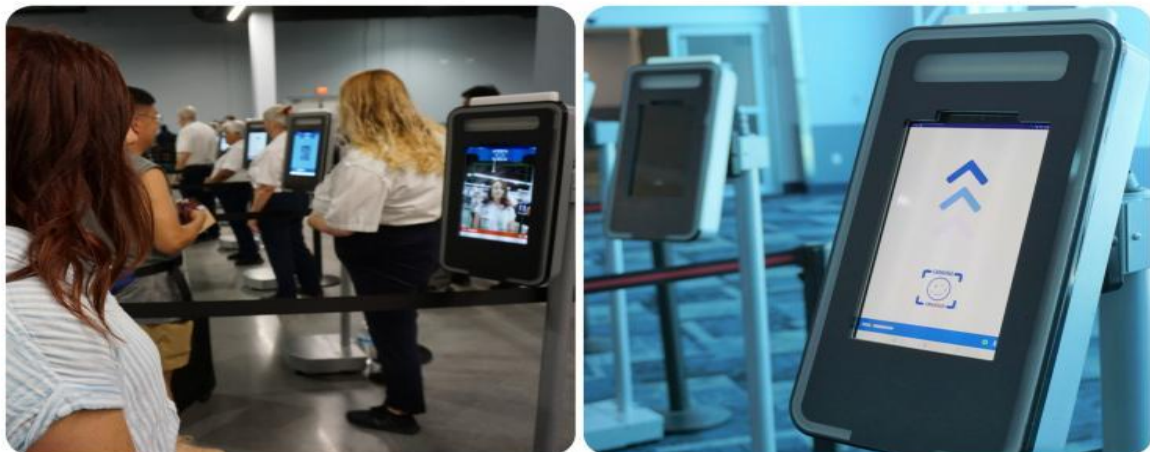


Facial recognition for a login system is already used in majority of smartphones (not only apple). It works by creating a 3D depth map of your face beforehand from multiple angles and saves the scan that is made for further use to unlock your phone or authorise payments on your smartphone. Face ID's depth map is specifically developed so it cannot be unlocked with photos to discern whether it is the user unlocking the phone or someone trying to maliciously unlock it. Many advanced Face ID smartphones have a way to detect if it is the user's intention to authorise using face id or someone is trying to unlock their phone by flashing it up at the user. This is usually by a setting, for example for the user to always look at the camera to unlock the device/ authorise the payment.

Applications to my Software:

The use of a depth map to stop fraud would be especially useful as it stops people from just holding up a piece of paper of somebody who has credit on their account and paying for their ticket that way. The way the facial scan is made would be applicable to my software since it is easy to understand and captures each angle to create said depth map. A user's intention of authorisation would be unnecessary since it would be hard logistically to defraud someone out of a ticket in the same way the phone can be pointed at a user's due to phones being mobile and barrier machines being immobile.

Cruise ship disembarking recognition



When passengers first arrive on the ship, their face is scanned using a handheld scanner and then it is added onto their cruise card using NFC or a barcode. Anytime passengers disembark the cruise, their cruise cards are scanned, and their status is changed to state that they are currently not on the ship. When passengers return to the ship, their cruise cards are scanned yet again to change their status back to being on the ship. This also authorises the passenger and can inform cruise staff of any warrants out against the passenger.

Application of this to my software:

The use of a handheld scanner would not be more efficient or useful due to needing a staff member present to initiate the scan and then apply it to the database. It would be much faster to have the computer do all the processing. Physical cards are also a negative due to them being losable, whilst you are not able to lose your face which is completely unique.

School Reception sign in system



In my school's reception, how all staff and visitors sign into the school is using an NFC reader which is attached to an iPad. When you scan your pass onto the NFC reader, it recognises the user and automatically signs them into the system if they are not currently signed in, displaying a welcome message with their image and name. If the user is already signed in, then the screen will display a goodbye message also displaying their image and name. Users who do not have a pass can also use the screen to manually find their name and then sign in that way or visitors who do not have an account can create a temporary pass into the system.

Applications to my software:

The use of an NFC reader would not be beneficial to me since users can easily lose their passes and it is also too similar to the current transportation system, which most of them use NFC readers and cards to store credit of users. I would also have hardware limitations of requiring an NFC reader and card which I can put data onto to test it and then issuing multiple of these to test on stakeholders. The sign in/out screen where it sees the users face would be beneficial to my software, displaying the users name and image letting the user know that they have certainly signed in/out since they can see their details being displayed.

Facial recognition

From my research I can conclude that facial recognition will be most applicable to my project. Other forms of logging in (other than using a username and password) prove to be a lot harder to develop and I would have to source out other forms of hardware such as a fingerprint scanner or NFC reader, but I already have a camera readily available to use for facial recognition and facial recognition does not pose any major problems, such as someone stealing your face from you, such as with an NFC reader. This also means that family members cannot share accounts, ensuring authenticity.

Approaches to my solution

Open cv

Open cv is the facial recognition and camera feed capture system that I will use for my project. Open cv has been used in majority of facial recognition python projects due to its easy-to-understand commands included inside of the module and its easy integration with other python modules such as tkinter. It contains many pre-existing functions inside of it which make coding much easier.

Open cv is a fairly old system and there are more efficient forms of face camera capture and facial recognition out there but open cv is the most heavily used and tested one, reducing errors and increasing optimisation. I do not need the best library out there, but I do need one that is heavily optimised for use since the computer I will be developing on is not particularly fast.

Open cv is an open-source library which enables its users to create and adapt new libraries based on the open cv library, setting it up for more customisation and usability within the code.

From open cv I will mainly utilize its ability to take camera images and use it within python code.

Google cloud vision API

Google cloud vision is another alternative to open cv where it allows facial recognition and displaying camera feeds. Google cloud vision includes pre trained models for recognition of images and other objects.

Google cloud vision API is a paid library so I would have to pay a certain amount of money for every single face detected. This is not suitable at all to my project since I am making something that is free for me to create since I am not getting paid for it. Since it is paid, it has a smaller number of users who use it, therefore it does not have as much documentation that is unofficial, but it does have more official documentation.

Google cloud vision API also does not include any form of image vision into the python code so even if I used this module, I would have to include something like open CV to use the computer camera input inside of my python code.

Tkinter

Tkinter is a GUI module for python which is included inside of the official documentation part of python documentation, included inside of python before installing it. Tkinter has a large userbase, therefore it has a lot of tutorials and documentation ready for it since it is the most widely used GUI for tkinter.

I personally have experience with tkinter as well, which will make developing a lot easier since I won't have to learn how to use a lot of commands since I already am well versed in tkinter's documentation.

Tkinter also has integration made with open cv camera usage, meaning that I can take open cv variables and display them within tkinter windows.

Tkinter also has a newer developed version by users of it called custom tkinter, which uses the backbone of tkinter for its code with updated visuals, making the program look more modern and more appealing to users.

I will be using custom tkinter for most of my code since I already know how to use it and it is simple to use and has a lot of helpful documentation surrounding it if I encounter any errors. The GUI itself is easy to use and isn't slow.

PyQt

An alternative to Tkinter would've been a module called PyQt, which works and looks similarly to tkinter but it has a lot smaller userbase. Due to this, there exists a lot less documentation and tutorials than tkinter which would make programming for someone completely new to it a lot harder than tkinter. PyQt is also more complex than tkinter, which means that coding it in general is a lot harder and may take me longer to use.

PyQt has integration also with OpenCV, but it is not as widely talked about across the internet so error fixing as I make the program would most likely be harder.

I have never used PyQt before but I have used tkinter so it would be a lot easier for me personally to create a program using tkinter.

Interview

Frequent Rail User

I have chosen a frequent rail user for two of my stakeholders since these will be the people impacted most by the software since they are the users of it. Frequent rail users would use this software every time they frequent the rail system, speeding up their daily routine with a hands-free alternative to typical ticket systems. This solution is appropriate for busy commuters who may have their hands full

These are the questions I will propose to them.

1. How frequently do you use facial recognition technology?
2. What is your experience with facial recognition?
3. Would you consider using your face as a ticket? (Face is scanned instead of a QR code or ticket)
4. If not, why?
5. Do you have any fears regarding facial recognition?
6. Would you be fine with a 3D depth map of your face to be stored?
7. How could current rail tickets be improved upon?
8. How long on average does it take you to buy and use a regular rail ticket?
9. Any other comments?

Questions one and two establish the user's current knowledge and experience with facial recognition, if they're familiar with it and how often they use it will affect how open they would be to using it in the context of a ticket, which is question three.

Question three determines if they would be open to the idea of facial recognition, heavily influenced by their prior experiences, either with the rail system of facial recognition or new technology in general.

Question four would provide reasoning behind a negative perception, whether it is due to facial recognition or rail experiences. Question five would help to understand the thoughts of the user if there are any worries relating to facial recognition, so they could possibly be disproven in the future.

Question six establishes if the average user would be afraid of a rail company storing data about a user's face, since some people do not want certain information, such as your facial blueprints, to be stored on large databases.

Question seven and eight establish if there is anything wrong or anything that could be improved upon in the current railway systems, such as queue times or accessibility.

Interview – Frequent rail user

John

Q1) How frequently do you use facial recognition technology?

"I often use facial recognition to open my phone. I use apple Face ID every day."

Q2) What is your experience with facial recognition?

"Overall positive experience, I don't have any issues with Face ID."

Q3) Would you consider using your face as a ticket? (Face is scanned instead of a QR code or ticket)

"Yes, if it would make my commute faster definitely. I am easily prone to lose my ticket or spend time trying to search for it."

Q4) If not, why?

N/A

Q5) Do you have any fears regarding facial recognition?

"I'm not so sure about my face being stored."

Q6) Would you be fine with a 3D depth map of your face to be stored?

"As long as it is held securely, I'm not sure about the government storing a depth map of my face."

Q7) How could current rail tickets be improved upon?

"Nothing really to improve upon aside from the barriers. They can be easily jumped."

Q8) How long on average does it take you to buy and use a regular rail ticket?

"Around a couple minutes on average, I usually purchase them online which is fast but sometimes there can be queues for the barriers"

Q9) Any other comments?

"No."

Sandra

Q1) How frequently do you use facial recognition technology?

"Everyday."

Q2) What is your experience with facial recognition?

"My phone."

Q3/4) Would you consider using your face as a ticket? (Face is scanned instead of a QR code or ticket) If not, why?

"No, because you can easily scan a QR code so I would rather not have the rail lines have a scan of my face."

Q5) Do you have any fears regarding facial recognition?

"Yes, could be used to create things such as deepfakes which could be used against us."

Q6) Would you be fine with a 3D depth map of your face to be stored?

"I would rather it not be saved in like a data base somewhere but if it was the only way to use it, then yes."

Q7) How could current rail tickets be improved upon?

"I'm not sure."

Q8) How long on average does it take you to buy and use a regular rail ticket?

"Only a few minutes since it is all online."

Q9) Any other comments?

"No."

Analysis

The rail users have indicated both that they are already familiar with facial recognition and use it daily. Since this is true, it would be less necessary to have an introductory tutorial (or one in less depth or length) since majority of younger users will also be familiar with facial recognition from their smart phones.

Only one of the interviewees said they would prefer facial recognition but both expressed concern about how the data is held and stored. The risk of facial depth map data being stolen is low but can be minimised by implementing encryption and sufficient security so the data cannot be reached in the first place. If facial data were stolen, people's identities could be stolen or turned into deepfakes for malicious intentions.

Interview – Rail Ticket Officer

The reason I chose rail ticket officers

Mike

Q1) What is the biggest issue with security in the rail system?

"People just don't have any respect for us officers. Were just trying to do our jobs."

Q2) Would updating rail barrier security assist in your job?

"Yea probably."

Q3) Do people frequently hop over barriers/ avoid paying for tickets?

"Yea around one person a week."

Q4) How do you think this can be more strictly enforced?

"People just need to do the right thing I suppose. There's nothing more we can do except find the people without tickets."

Q5) Do you believe your job would be at risk if tickets were removed?

"Yea then I wouldn't have nothing to do and be made redundant."

Q6) What are your views on facial recognition technology?

"I don't want all these robots scanning my face and storing it everywhere, I just don't feel safe knowing someone has my face somewhere out there."

Mark

Q1) What is the biggest issue with security in the rail system?

"People can easily hop over the barriers when staff are not around during less busy times, and sometimes even the barriers are just left open"

Q2) Would updating rail barrier security assist in your job?

"Yea, we'd need to remove people a lot less and it would help us to get our actual job done."

Q3) Do people frequently hop over barriers/ avoid paying for tickets?

"Quite frequently yea, I often find younger ones doing it but sometimes it begs the question of is it worth really chasing them after with a fine if they can't afford the ticket in the first place."

Q4) How do you think this can be more strictly enforced?

"Better barrier security I suppose, it's too busy and overcrowded for the staff to see the people trying to get on illegally so it often goes on unnoticed."

Q5) Do you believe your job would be at risk if tickets were removed?

"I don't think so since you still need someone on the train in case anything kicks off, and they couldn't just fire the lot of us."

Q6) What are your views on facial recognition technology?

"Quite neutral, it's easier to log into my phone I suppose but I don't like knowing that my face is out there somewhere in a database in the hands of some company."

Analysis

Both officers agreed that people not paying for tickets is a problem within the rail system and both agree that there are issues with the rail barrier system which could be improved upon. If those issues were taken care of, they both suggested that it would increase the efficiency of their jobs.

Only one out of two of the employees interviewed believe said that by removing physical tickets that their job would be at risk.

One of the interviewees stated that they are not comfortable with facial recognition technology. Both stated that they are concerned about the storage of their facial map data.

Proposal

Features

My solution will feature two applications, one which allows account creation, credit top up and view account information and another which detects faces. I have chosen to split it into two different applications because this encapsulates the programs more, protecting the code from each other so they do not interfere with each other if anything goes wrong. It also is more practical to have two separate programs, since, if the account creation and management modules were implemented at the barriers, it would cause a lot of queues and cause people to pass through at a slower rate than before the implementation of this software.

The first program will open to a screen giving the user the option to log in or create an account. After log in is selected, the face of the user is scanned and if it matches to the database, the user has a variety of options to choose from: update details, view history, add credit or exit. Each option brings up a new interface. Update details fetches the account information from database and allows the user to choose which they would like to change and once they are ready, they can click the return button. View history fetches and displays all the previous transactions from the credit on a user's account. Add credit allows the user to enter a sum and payment details and adds the amount to the users account. Exit brings the user back to the first screen.

The user cannot interact with the second application. This is to decrease human traffic at the barriers and stop queues from forming from people spending time interacting with an interface. The user walks into the frame of the camera, the camera detects and displays the users face and if it is detected and has enough credit, the user is allowed to pass through. If the user is not detected, it will display a message to the user directing the user into the frame. If the user is detected but not matched on the database, it will display a message to the user to create an account.

Limitations

A limitation to my software would be the actions of the users. Whilst majority of users would use the program and hardware correctly, there may be multiple users who disagree with the concept and harm the hardware. If the cameras were not intact or too shattered to recognise faces, it would limit the use of that specific barrier. The users also being ignorant of instructions on the displays would limit the use of the program to themselves and possible other users who are waiting in line to use that machine. If a user who hasn't got an account is denied access at the barricade and told to make an account and they don't read the monitor, they would be limiting the use of the application for themselves and others who might be waiting to use it.

The hardware itself is a limitation to me currently, as I am unable to afford or gain access to an electronic barricade, I am unable to program the barricade to open as there is not a barricade to open. This will limit the completion of my program as I will not be able to completely solve the program to open the doors. I also lack the specific devices which would be used to create accounts, but they would mostly exist as large touchscreen computers, which can be emulated in computers/tablets.

The implementation of touchscreen is also a limitation, due to me not having access to a touchscreen device which is powerful enough to run the code I am creating, I am unable to test the touchscreen feature of my program which makes knowing if the code works very difficult.

Hardware requirements

Web camera – The software will require a webcam to smoothly record and process live video, for this a minimum of 30 FPS would be needed from the camera or more. A resolution of 1280 x 720 (720p/HD Ready) would also be necessary to clearly detect every face without any lagging or interference. Camera quality is important since a low-quality camera (less than 720p) will not be able to smoothly track the face and pinpoint exactly where landmarks are, and the tracking may jitter and loose accuracy due to a lower quality.

Processor – A processor that is capable of running the program would be essential. The processor will need to be able to track and recognise faces and then fetch the facial information from a database and perform calculations. This is necessary since you cannot run a program without a processor.

Software requirements

Python supported operating system – An operating system which is either Window 7 or later, macOS 10.9 or later, or any Linux distribution.

Python interpreter – An interpreter present on the pc which will be able to read python.

Python open CV library – This library is necessary to run the program as it utilises machine learning to create facial recognition.

Success Criteria

Criteria	Measure	Justification
Display taken image on new user window	Screenshot of the picture from camera being displayed on the main window	This is so that the user can see their own image so they can verify it to be adequate and accurate to their face and to ensure it is their face.
Recognise a face from camera feed	Screenshot of facial box being displayed	This is important so that the camera can pick up human faces exactly and map out the dimensions of the image.
Be able to store details of a user on a database.	Screenshot evidence of user details on a database	This is crucial to the application because it cannot work without storing details of new users onto the database.
Be able to retrieve details of a user on a database	Screenshot evidence of details being fetched from a database and received by the user	Without this, users are not able to log into their accounts if their details are not able to be retrieved, making the program useless.
Store facial encodings of users	Screenshots of user faces being stored	Storage of new user images is crucial to the program since logging in is dependent on being able to retrieve these images.
Ability to compare a face to an already stored face	A recognised face having a correct match with one on the database and outputting the value true. Screenshot of value	This allows for the program to identify which user is trying to log in so that the updates to the account are accurate.
A clear user display which conveys correct information based on the inputted data	Screenshot of the correct display after data being inputted.	This allows for the user to receive outputted data which is necessary to them.
Values on the database being altered depending on the user's actions	Screenshot of values prior and values after the action has taken place	This is core to the functionality of the program as without database alterations, credit will not be deducted, or necessary details will remain unchanged.

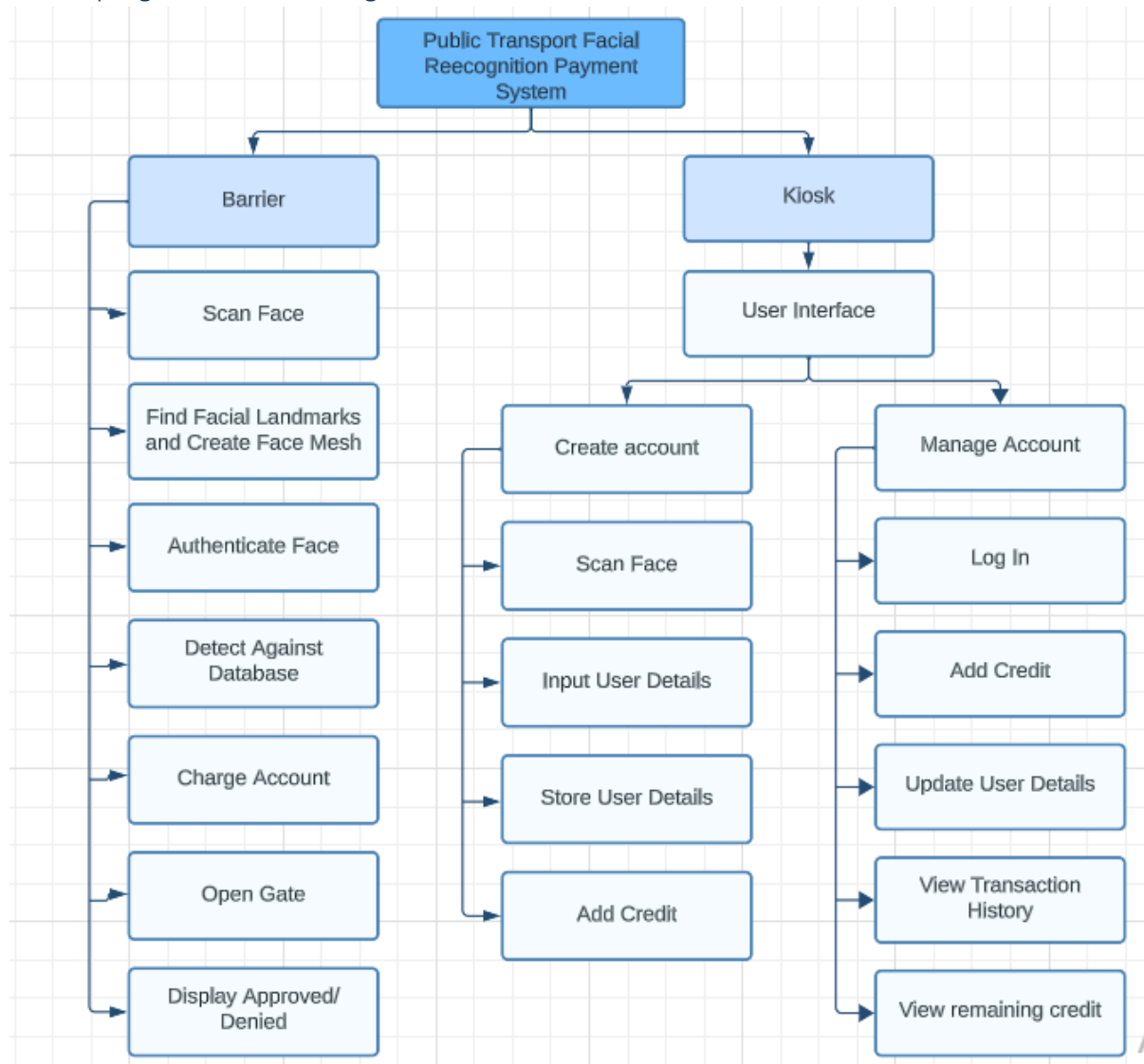
Be able to create a new entry on the database including all relevant details that would be required to identifying a new user	Screen recording of new entries appearing on the database after user's interactions.	This allows for new users to enter onto the system, without users the rest of the program cannot run so this is also crucial.
User interface which is easily navigate-able	Screenshot of interface, A new user using the interface with ease and not being confused. Buttons being clearly labelled	An easily navigable interface means that users cannot get lost within the program and clearly understand what they must do so they don't encounter errors.
Clearly displayed instructions for the user-on-user creation window.	Screenshot of instructions	The instructions guide the user on how to use the application, so they do not encounter errors or enter the wrong details.
Choice of credit for user after creating a window	Screenshot of window containing credit options	Adding credit to their account allows for the user to utilise the functionality of the barrier application, without credit they will encounter errors.
Correctly display user history in user history window	Screenshot of user history being display on the user history window	This criterion is important for the users to see their history incase they have been charged the incorrect amount or charged for a ticket that they didn't use.
Correctly displayed information when an account is charged for money	Screenshot of displayed information	This is to let the user know if their account has been successfully charged or not so they can proceed through the barrier.
Charge the correct ticket price depending on user age and save the amount charged	Screenshot of account credit before and after	This is to ensure users do not get charged incorrect amounts from their credit and to save the amount they have been charged to the database.
Displayed if barrier is open or closed	Screenshot of both open and closed	This is so that users know that it is ok to enter or not to enter through the barrier.
Barrier program refreshes after a few seconds to allow for the next user to be detected	Video of an account being charged and then it refreshes, and the next account being charged	This is so that every time that a new user enters through the barrier that the program doesn't have to restart so it can be constantly running.

Inform user when lacking credit for a ticket	Screenshot of lacking credit window	This is to communicate to the user that they must add more credit via the kiosk application.
Functional buttons throughout the program	Video of button being clicked and action happening	The buttons must be functional otherwise the program will not be able to run.
Working entry boxes for user to enter details	Video of entry boxes entering in data and the data being saved	Entry boxes must be working so that the information inputted into them can be retrieved for use in the database.
Functional dropdown boxes that allow a user to only select a fixed number of options to choose from	Screenshot of dropdown boxes and their options	Dropdown boxes must be working for when only a select number of entries are valid for a certain type of data to reduce user error and issues the program may encounter.
Validation of email addresses entered in the new user entry stage	Screenshot of invalid data not being accepted	This is not necessary to my current code as it will not affect the functionality of it but in real world application this would be necessary since it would be required to communicate with the customers from the owners of the transportation system.

Design

Structure of solution

Whole program structure diagram



Explanation of system diagram

I have split the applications into 2 subsections: the barrier application and the kiosk application.

Barrier application

The Subtasks of the Barrier Application compose of:

- Facial scan
 - This task will use a camera input to detect a human's face
- Find facial landmarks and create face mesh
 - This task will detect and determine the landmarks on the users face and use them to create a 3D facial mesh
- Authenticate Face

- Using machine learning, this task will determine if the face that is being scanned is an authentic human face or an image
- Detect against Database
 - This task will use the data gathered from facial landmarks and facial mesh and compare it to the database to find a match in accounts
- Charge Account
 - After finding the account that the facial mesh corresponds to, it will be charged the necessary amount for a ticket.
- Open Gate
 - After the account is charged the barrier gate will open to allow entry for the passenger
- Display Approval or Denial
 - If the account is found and charged, a screen will output approval, if an account is not found it will display denial

Kiosk application

The kiosk application is further split into two separate categories under user interface: account management and account creation.

Account Creation

The account creation subtasks compose of the following procedures:

- Facial scan
 - This task will use a camera input to detect a human's face and locate its landmarks.
- Input of user details
 - This task will allow the user to add personal details to their account including name, date of birth and an email address.
- Storage of user details on the database
 - This task will create a new field on the database and store the details provided on it.
- Option to add credit
 - Once the account has been made, the option to add credit onto the account will be provided.

Account management

The account management subtasks compose of the following procedures:

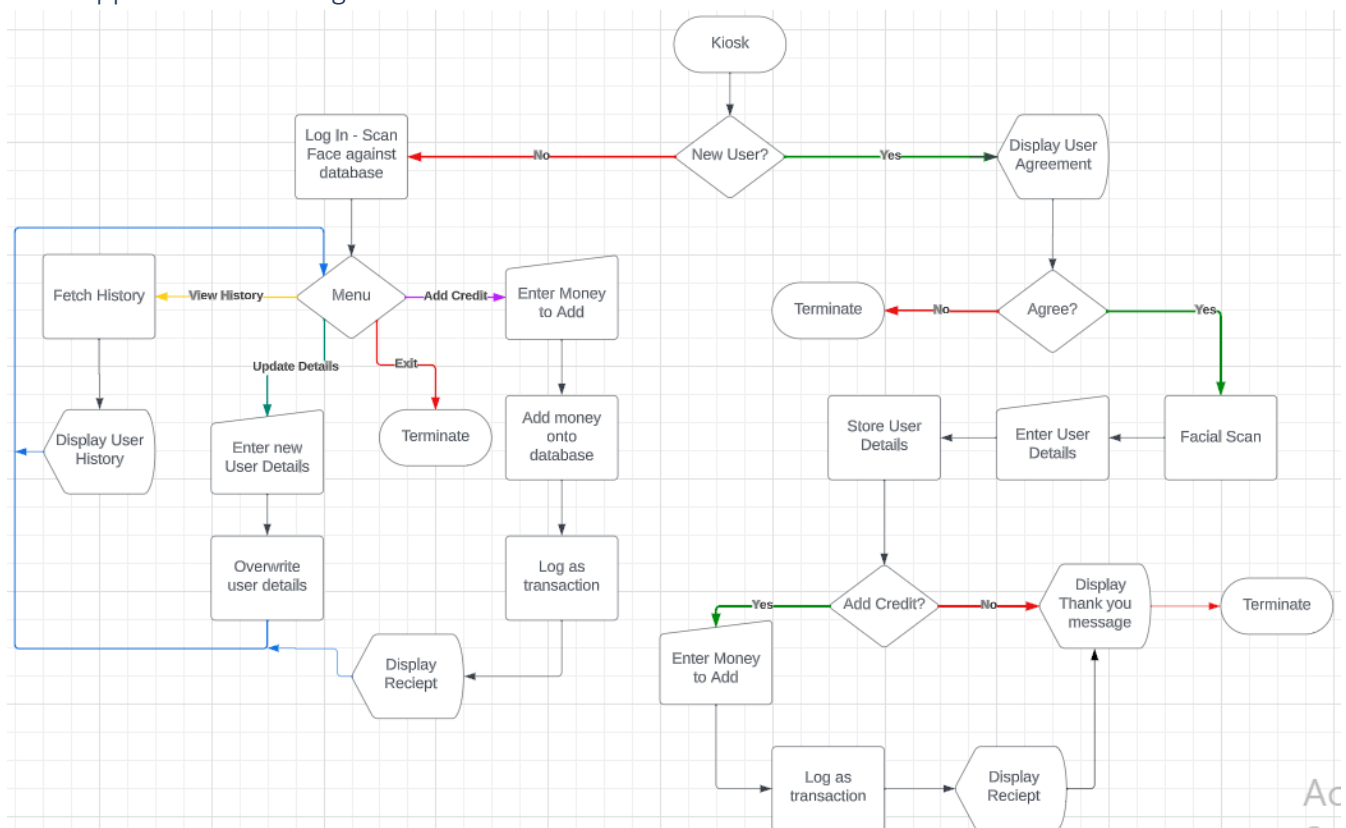
- User log in
 - This task will scan the users face, detect it against the database and then allow access to the account.
- Option to add credit
 - This task will allow the user to input an amount of money that will be charged from them and added to their account on the database for ticket transactions.
- Option to update user details
 - This task allows for the user to change any details of their account on the database.
- Option to view transaction history
 - This task will allow the user to view all the prior transactions including date and amount charged.
- Display of remaining credit on the user's account
 - This task will display the remaining amount of credit to the user.

Justification of system diagram

The use of a system diagram allows me to decompose the application into subtasks and those subtasks further into subtasks. Decomposition allows for individual tasks to be recognised so they can be more easily dealt with individually but also to group those tasks into separate sections to see which tasks must work together to create the larger solution.

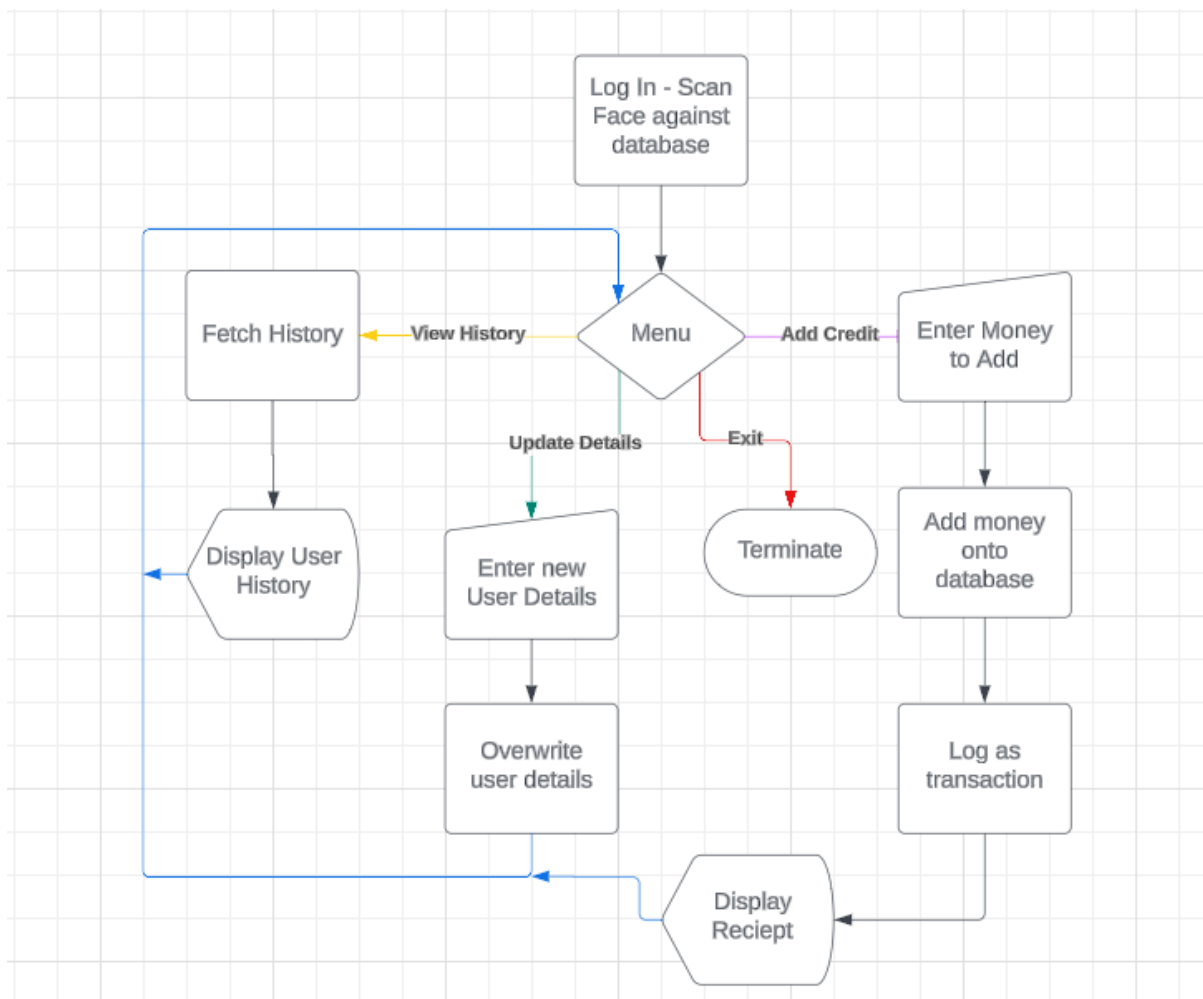
Each subgroup that I have chosen for the program is one distinct part that needs to be coded as a part of the functionality of the program, majority of these steps are dependent on each other being coded and the program will not be fully complete and functional without majority of these subgroups inside of the diagram.

Kiosk application flow diagram

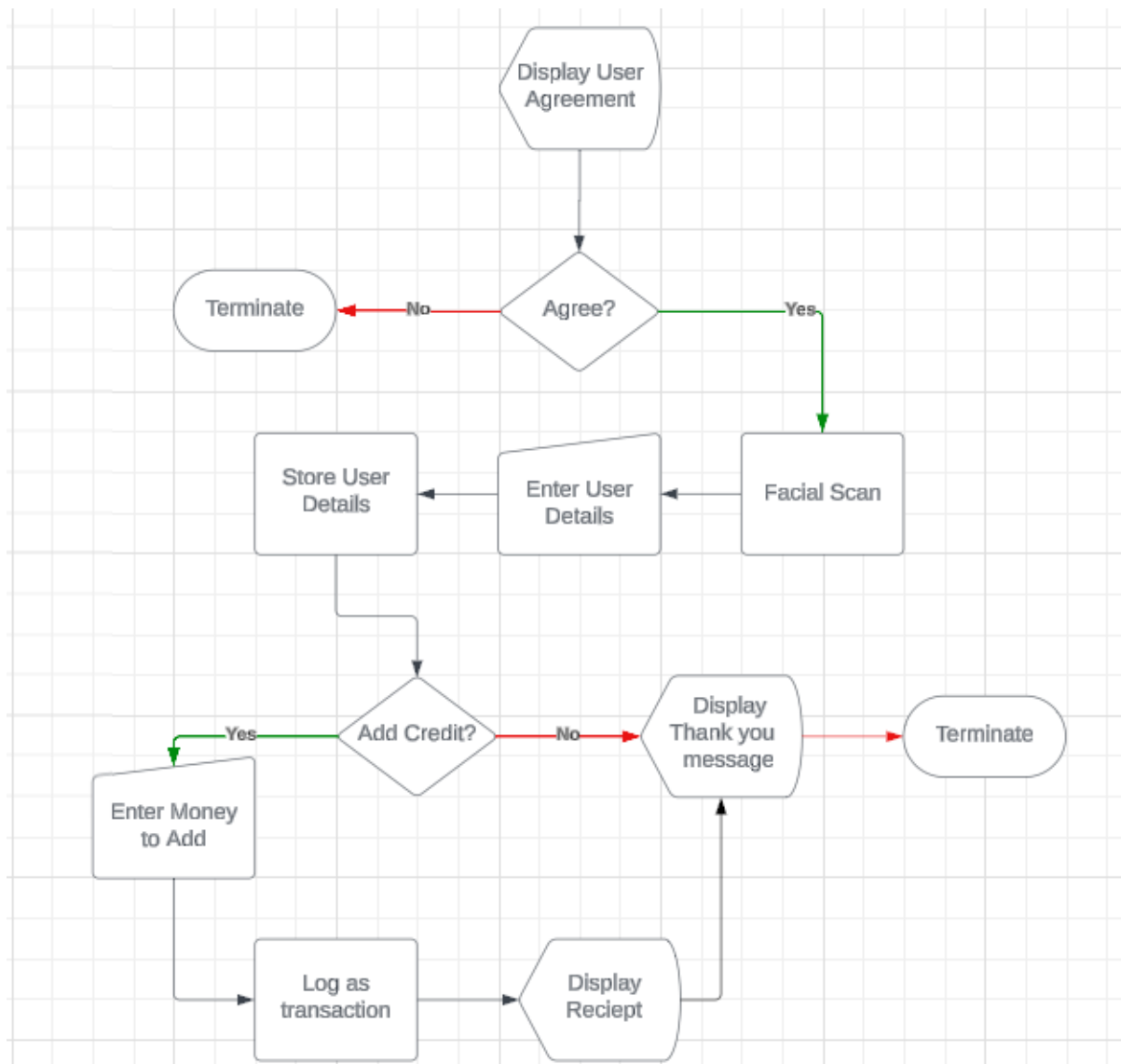


Explanation of Kiosk Flow Diagram

The application will begin with a user selection to decide if the account is new or not. This will allow the application to know if the user requires a new account to be made or to allow for the user to log in.

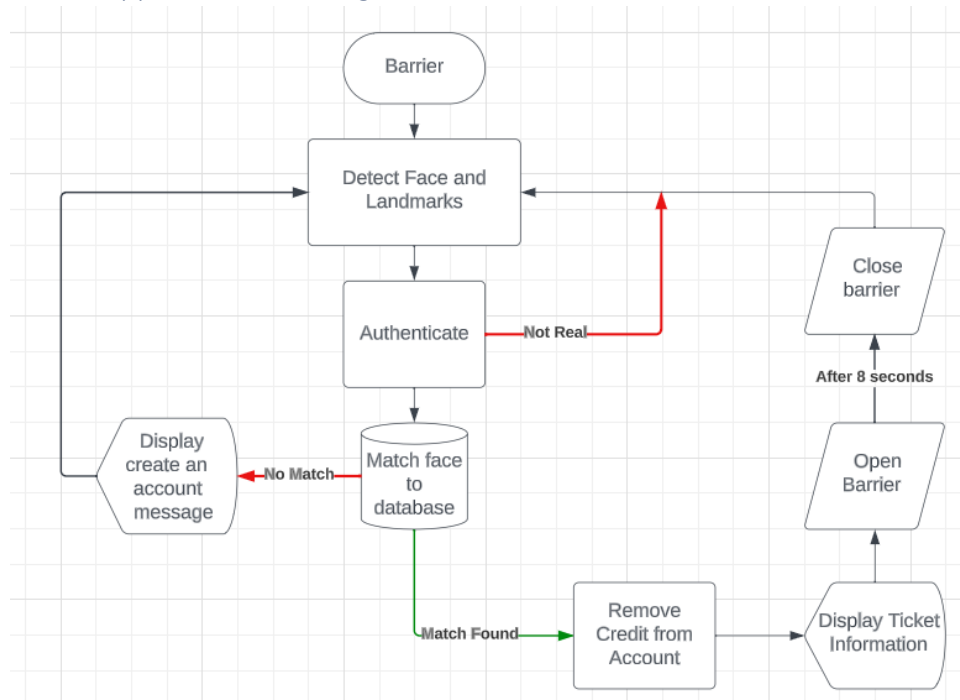


If the user has an existing profile, the system will process their logging in by scanning their face and matching it to the database. Once the face is matched the user is sent to a menu where they have a selection of 4 options (view history, update details, add credit and exit). When view history is selected, the application will fetch the user history and display it to the user. Once the user is finished, it will bring the user back to the menu where the same options are presented again. When update details is chosen, the user will enter the new details that they would like to be changed, then they are overwritten to the database and the user is returned to the menu. When add credit is chosen, the user will enter amount to pay, this value is changed on the database and recorded as a transaction on the user's account history and then the receipt is displayed to the user. When exit is chosen, it will log the user out of the application and go back to the main screen, ready for the next user.



If the user has not got a profile, it will at first display a user agreement that they must agree to, agreeing to the storage of their facial depth map and landmarks and other private details such as name, username, address and email address. If they disagree the application will return to the main menu screen. If they agree, the application will initiate a facial depth and landmark scan to create a face mesh for the user and store that as their password. Once the facial scan is finished, the user must enter person details such as name, address, date of birth and email address and then store those on the database. Once the account has been created, the user will have the option to add some credit onto their account. If they do not wish to add credit currently, the application will display a thankyou message for a brief period of time and then return to the start menu for the next user. When the user chooses to add credit, the user will enter amount to pay, this value is changed on the database and recorded as a transaction on the user's account history and then the receipt is displayed to the user and brought back to the menu screen.

Barrier application flow diagram



Explanation of barrier application flow diagram

The application begins by automatically attempting to detect a face, it will not be able to progress until a face has been found. Once the face is found, the application will authenticate the face to be a real person's instead of an image of a face. If the authenticator deems it to be not a human face, the program will go back to the beginning and try to find another face. If it is a real face, the program will then attempt to match the face to the existing database of accounts. If there is no match, the application will display an account creation message, telling the user to create an account at a kiosk and then return to searching for a new face. If a match is found on the database, then the application will charge the user's account then display the ticket information and open the barrier. After 8 seconds, the barrier will close the program will start again.

Justification of flow diagrams

The use of a flow diagram allows me to break down the application into its individual sequences, selections and iterations in a visualised form as a followed plan. Flow diagrams allow for applications to be fully broken down into their individual components which allows me to identify every input and output from every section and it also allows me to analyse the logic inside of the diagram for any faults and errors that may be in my code by preplanning it.

Each section of the flow diagram is crucial since the sections that connect from it cannot often happen without the previous step happening in the program. Many aspects of the code are dependent on each other, either to run or to explain things to the user, meaning without exactly these subgroups, the program would not be complete.

Algorithms

The following is a pseudocode algorithm to compare two images to be true

```
image1 = get(image1.png)
Image2 = get(image2.png)

Same = image2-image1
If same == true:
    Print("The images are the same)
Else:
    Print("The images are not the same)
```

First I get both of the images and store them inside variables of image 1 and image 2, then I find a variable called same by finding the difference between the two images and if the images are close to each other it will print that the images are the same and if they are not, it will print that they are different.

The following is a pseudocode algorithm for the barrier application

```
Video = getCameraFeed()

While getCameraFeed == true:
    if facePresent == true:
        credit = getUserCredit()
        credit = credit - 5
        setUserCredit(credit)
        print("Barrier open")
    else:
        print("Barrier closed")
```

First it will store the camera feed inside a variable called video. Then it will loop through a while loop that will continue until the camera feed turns off. Inside of the loop will be an if statement, determining if the face is present. If the face is present, it will deduct 5 credits from the user's current credit and then set that to be the user's new credit and print the barrier is open. If there is no face present, it will print the barrier is closed.

The following is a pseudocode algorithm for a birthday validation.

```
If day == 31 or 30 or 29:
    if day == 31 and month == 2 or 4 or 6 or 9 or 11:
        print("Error")
    elif day == 30 or 29 and month == 2:
        print("Error")
else:
    print("Correct")
```

If the date of the birthday is equal to 31, 30 or 29, then the program will go into another if statement. If the day is 31 and the month is February, April, June, September or November, it will print that the birthday is an error. Else if the date is equal to 30 or 29 and the month is February (not account for leap years), then the program will print error. If it does not meet these conditions, it will print correct.

Proposed screen designs and Usability features

Database

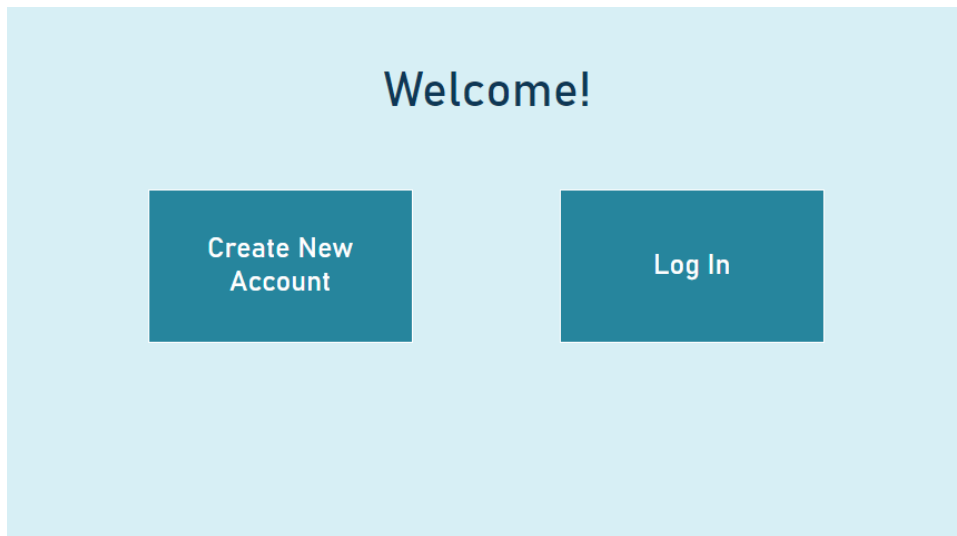
User count	Name	Email	Year born	Month born	Date born	Credit
1	User 1	Email 1	Year 1	Month 1	Date 1	Credit 1
2	User 2	Email 2	Year 2	Month 2	Date 2	Credit 2
3	User 3	Email 3	Year 3	Month 3	Date 3	Credit 3

I will use an excel spreadsheet as my database for this project since I already have knowledge of how to import data from spreadsheets into python. This is quicker than learning how to use SQL inside of python since I already possess the skills required. For implementation into the real world, SQL would be needed to be substituted in for excel but for a small-scale project excel will be fine.

The first entry in the first row will be the User Count square, which will hold the current number of users in the database currently. This makes it a lot easier when adding a new user to the database since I can return that square and then add 1 to it to increase the count each time.

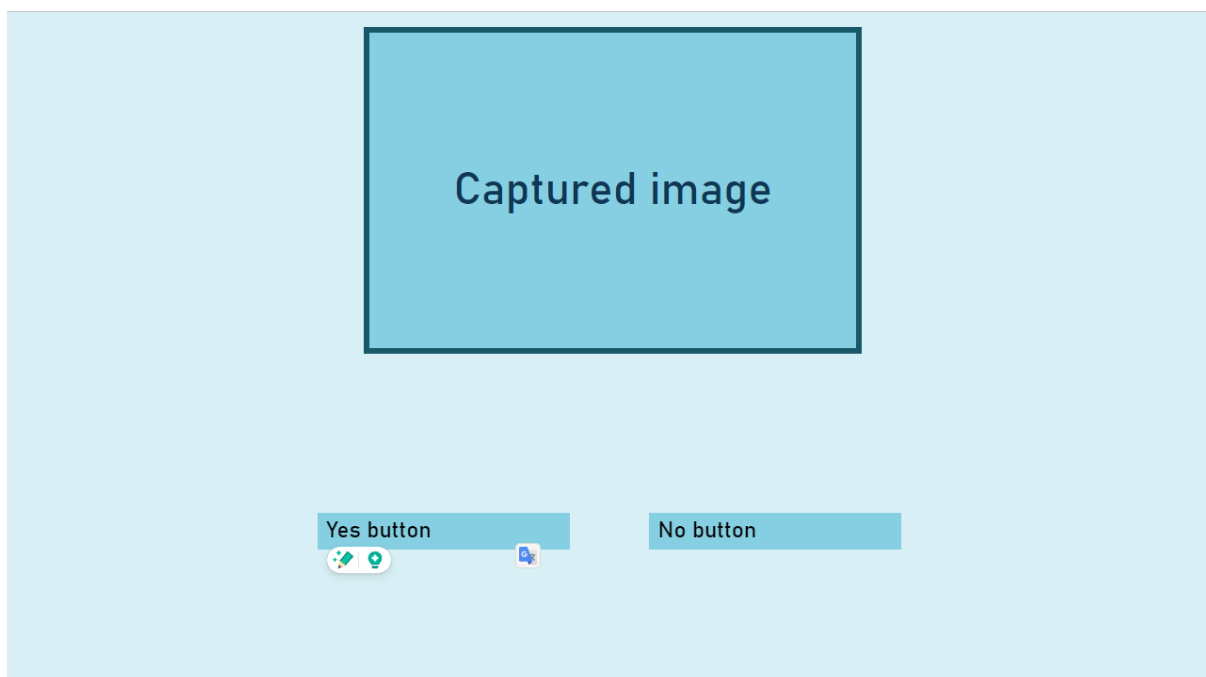
The date of birth is split into 3 individual columns instead of all being a part of one column for easier retrieval and comparison of data fields. It also allows for easier formatting when asking for the date of birth from users.

Kiosk program



The first screen will allow the user to make a choice between two interactable buttons: a log in button and a create account button. Creating an account will be for new users who do not already have an account created and it will allow them to scan their face onto their database with their personal details and facial scan included. The log in button will be for returning users who have a preexisting account where they would like to either view history, update details, or add credit.

Success criteria meets the criteria "Functional buttons throughout the program" and "User interface which is easily navigate-able".



This screen will be used after a user's face is scanned to be inputted into the database when creating a new account. This will be the screen that will be showed after the user confirms that they are ready for the camera to capture their image, allowing the user to confirm or deny the captured image.

Captured image window

This window will display all the video that is inputted from the window. Once the user confirms that they are ready to have their picture be taken, the camera will take a picture of a user and display that image to the user back on the next page which will be this page.

This meets the success criteria “Display taken image on new user window”, “Recognise a face from camera feed”, “Store facial encodings of users”

Confirmation Buttons

This screen will have two confirmation buttons. One of the buttons will be to accept the captured image when the user agrees that the image is suitable for them, and the other image will be to deny the captured image. If the user denies the captured image, the image will be retaken, and it will redisplay this screen. If the user chooses to accept the image that has been taken the user will be taken to the next screen to setup their details.

The buttons meet the success criteria of “Functional buttons throughout the program”

```
graph TD; IB[Instruction box]; BYD[Birth year dropdown]; BMD[Birth month Dropdown]; BD[Birthdate dropdown]; NE[Name Entry]; EE[Email Entry]; CB[Complete button];
```

This screen will be the new user detail entry screen where the user will be able to enter details about themselves to be added onto the database to confirm their identity. This screen will be shown after the user face scan screen once a new user has decided on an image that is appropriate for their user entry.

The instruction box will display instructions to complete the user entry to the user, that they need to a valid name, email and birthday into each box which is accurate. This will meet the success criteria of “Clearly displayed instructions for the user-on-user creation window” and an “Easily navigable user interface”.

The dropdown boxes will be 3 boxes which when hovered or clicked over they will show all the options for a user to select from and the user can only select from those set options. For a year drop down, this would be all years from the current year up until around 100 years ago since there are not many people alive over the age of 100. This is to stop users from falsifying data that is on the database. For the month dropdown, it will have all the 12 months of the year and for the date dropdown it will be however many days are available in the current month that the user chose.

The dropdown boxes meet the success criteria of “Functional dropdown boxes that allow a user to only select a fixed number of options to choose from”.

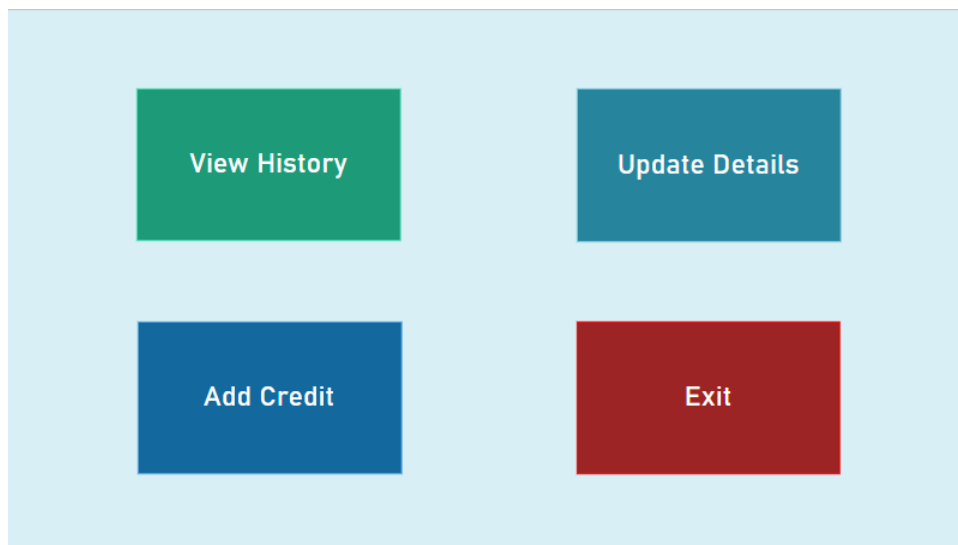
The name and email entry fields will allow for users to add their names and email addresses by typing it manually into the entry box. The entry box should be validated to make sure that all emails that are entered into the database are valid such that in a real-world application, if the customer would be needed to be contacted, the admins of the application would be able to easily find a way to communicate with the user.

The email entry box meets the success criteria of “Validation of email addresses entered in the new user entry stage” and box entry boxes meet the success criteria of “Working entry boxes for user to enter details”.

The complete button will be there for the user to confirm that the details that they have entered are correct and those are the details that they wish to enter the database. Once the button is pressed, all the users' details that they have entered in this page will be saved into the database under the user number of the user for later retrieval.

This meets the success criteria of “Functional buttons throughout the program” and “Be able to store details of a user on a database.”

If the details that the user has entered are not valid or any of the fields are empty, the user will be presented with an error window.

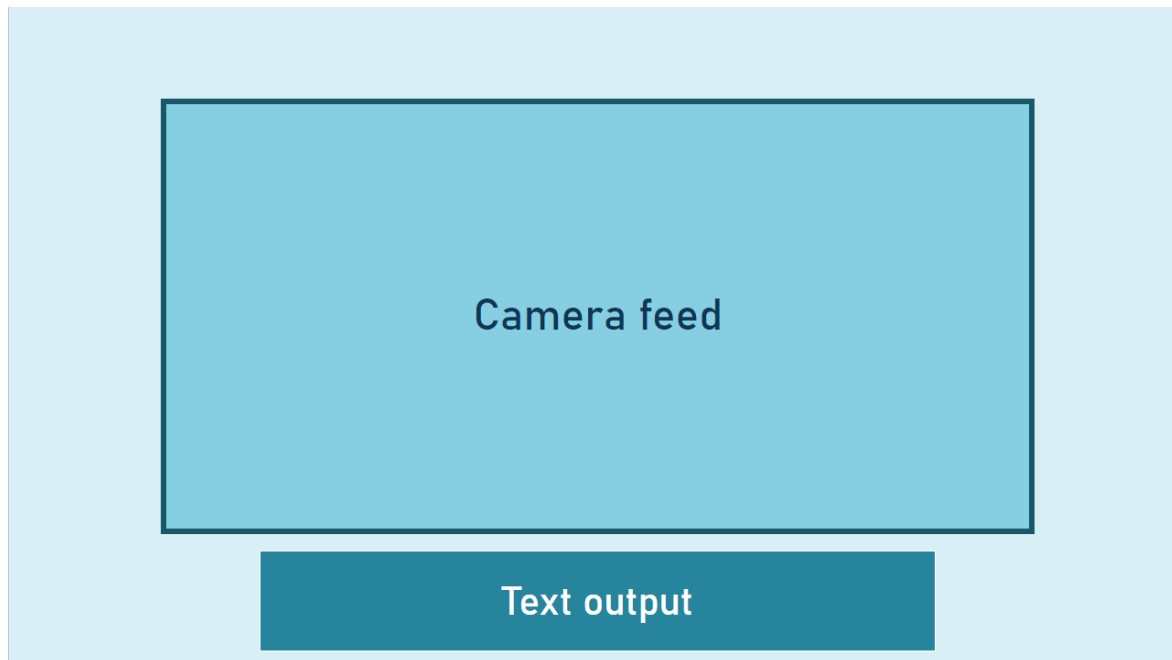


This menu will have four different buttons for once a user logs into their account so they can choose what they wish to do once they have logged in. One of the options is an exit button so they can log out of their account and once they return from each of the actions they will be taken back to this exact menu.

The “Exit” button has a red colour, so users associate the button with leaving the application and exiting out of it, so it is more easily recognisable. Each individual action is a different colour so that the application is clearly split into 4 different categories and the users can more easily navigate and see the difference in the buttons.

This meets the success criteria: meets the criteria “Functional buttons throughout the program” and “User interface which is easily navigate-able”.

Barrier application interface

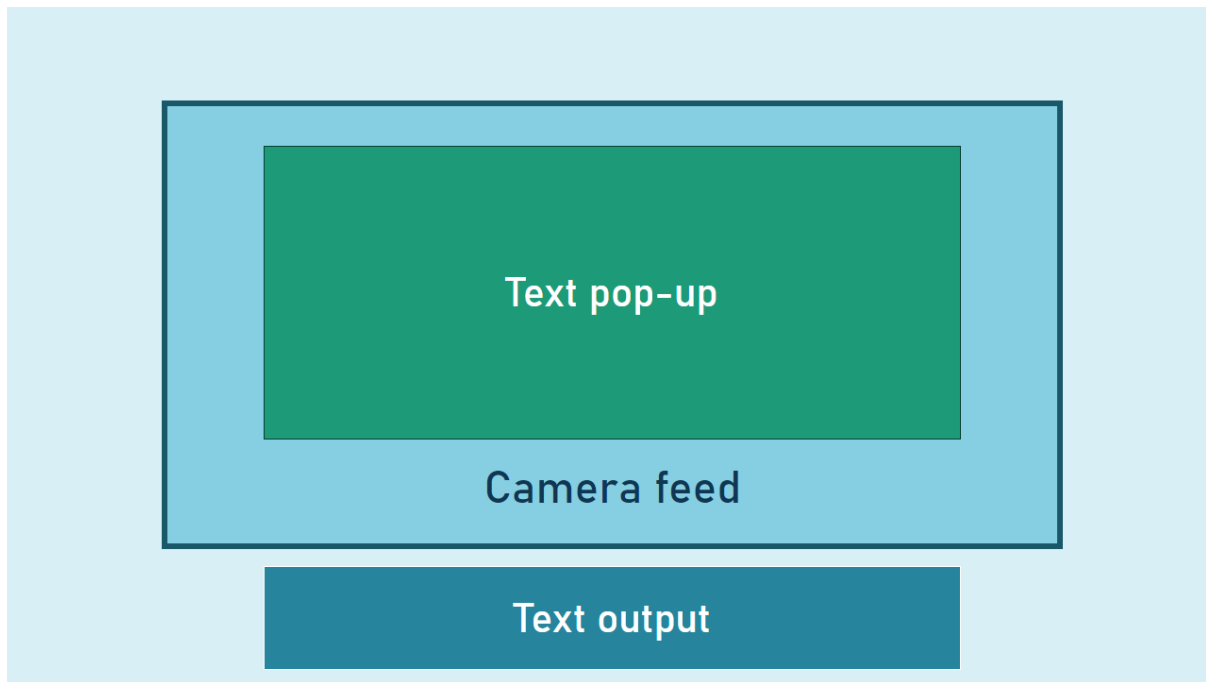


This will look very similar to the facial scan interface from the kiosk application, but it will be the only window for the program since it will not be interactable. This is to reduce waiting times in queues and remove confusion from users who are not very “tech-savvy”.

The main part of the application will be a view window for the user of the camera feed where they are able to see the camera input. This will track their face and tell the user if the face is present and visible. The user view window is smaller than the view window created in the kiosk application so it will give more room for the text box with instructions. This is crucial since it will bring more attention to the instruction box so users who are stuck and unsure what to do will more easily be able to realise their mistake.

The text output box will inform the user if the barrier is open or closed and if the face is detected or not.

This meets the success criteria: “Recognise a face from camera feed”, “Ability to compare a face to an already stored face”, “Displayed if barrier is open or closed”,



This pop up will open once the user has entered into the frame of the barrier application and their account has enough credit to be charged. The popup will cover the camera feed for a set amount of time before disappearing to hide the identity of the person that has just passed and so that other users know to not approach it yet. Once the time has passed the popup will disappear again.

The text pop up will output the text of "your account has been charged....xyz" with the ticket type being a variable which changes depending on the age of the user and the ticket that corresponds with that age.

If the users face is not found in the database, the text pop-up will tell the user that they are not found in the database and that they need to create an account.

Success criteria: "User interface which is easily navigate-able", "Ability to compare a face to an already stored face", "Recognise a face from camera feed", "Correctly displayed information when an account is charged for money", "Charge the correct ticket price depending on user age and save the amount charged"

Usability Features

I have increased usability greatly in my barrier program, by not including any input from the user other than the camera feed. There are no buttons or entry fields that the user can press inside of the program, making the application incredibly simple and straightforward to use, with instructions in each step detailing what the user specifically must do in that section.

Inside of my kiosk application, there are quite a few buttons and entry fields that the user must utilize throughout the program but reduced only to the most crucial inputs. For example, I could design that after each individual entry that the user must make that they must confirm each entry but instead all entries are confirmed together from a single button. I have reduced the amount of user keyboard input and reduced the options for users by using dropdown boxes and buttons wherever I can to decrease user error and increase the usability of the code. Every window will include instructions telling the user specifically what they must do at each stage.

Inputs and Outputs

Input	Process	Output
Camera feed	To display to the user and to retrieve facial details of the user to be stored onto the database or to use to recognise a user.	Tkinter window displaying it back to the user.
Menu button clicks	Upon clicking the button, the process that is related to the button will be run.	Actions that are involved in the process that the button holds will be processed and outputted to the user, such as opening new windows
Dropdown entry selection	The data will be retrieved from the entry, ready to be stored.	The data that is retrieved will be output into the database
Entry fields	The data will be retrieved from entry and validated.	The validated and retrieved data will be output into the database.

Key variables

This is a list of the key variables that will be included in my program.

Variable Name	Data Type	Variable Usage
Grayscale_Image	2D Array	This variable will be used to convert the image from a coloured image into a grayscale image, so the image is easier for the application to process. The variable stores the now grayscale version of the previous image
Faces_Number	Integer	This variable will store the number of faces that are currently in the frame of the camera input. If the variable is less than 1 then it means that there is no person present in the frame currently so actions cannot be performed, but if its greater than 1, that means that there are too many people in the frame so any actions cannot be performed with my application.

User_Count	Integer	This variable will store the number of current users that are on the database so it will know what the new users ID number will be and so it knows how many people to loop for when checking faces.
Face_Encodings	2D Array	This variable will be used to store the found face encodings of the image that is currently being scanned or detected by the program to compare it against the face encodings of other images.
Screen_width	Integer	This variable will store the size of the current computers screen width so it can set the tkinter window to be the exact size of the width of the user's screen.
Screen_height	Integer	This variable will store the size of the current computers screen height so it can set the tkinter window to be the exact size of the height of the user's screen.
Coordinate	String	This variable will store the coordinate location in the database that any piece of data needs to be saved or retrieved from.
Capture	2D Array	This variable will store the current video feed coming from the computer's camera so it can be used for retrieval of frames and to output back to the user.

Validation

Users will be able to enter data into the program, but users can often enter the wrong data (by accident or purpose) and to ensure that all data that is incorrect is invalid it must be validated. If data is not validated the program can encounter errors when attempting to compare different data types of certain variables.

Entry Fields

The user will have to enter data into entry fields when updating or adding a new email address or name.

For name entries, the program must ensure that the data is all characters and does not contain numbers since names cannot contain numbers. Names must also be in title case. If a name entry contains numbers, the program should not accept that as a possible entry, but if the entry is not in title case, the program should just automatically set it to title case itself

For email entries, the program must ensure that the data is in the format of an email, with a email prefix/ username and an email domain following it. Case for emails does not matter since it works in any case and numbers are allowed, but other special characters are not. For emails that are entered into my program, the program must ensure that they are in the appropriate email format and do not contain special characters. If the email is not in an email format it should be denied by the application.

Dropdown menu boxes

The user will be able to choose data from dropdown menu boxes to choose their birthdate. This will consist of 3 different boxes, for the year, month and day that the user is born in. Since the options in the menu boxes are already preset, the input itself doesn't need to be validated but the combination of options that the user chooses must be valid.

Since every month doesn't contain the same amount of days, the application must reject all invalid dates (e.g. the 30th of February) that the user provides since they do not exist.

Menu buttons

The nature of buttons does not allow for incorrect data to be inputted from them since they only have two states, clicked or not clicked. When the button is clicked it does not process or output any data and once it is clicked it only processes the procedure that it is programmed to process. The only way this could incorrectly return a value is if I have incorrectly coded it.

Camera feed

The camera feed can only be incorrect if there is not a face currently present in the program or there are too many faces being detected by the program. This can be validated by ensuring that the program only works if exactly one face is being detected at the present moment and the face is being correctly detected.

Testing method

Whitebox testing

To reduce the number of errors and to increase the ease of solving errors, after each individual section of the code has been written, I will test it to ensure its functionality. This is called Whitebox testing, where you test each piece of code individually. I will utilize whitebox testing during the development process and once I have fully finished the development, I will utilize Blackbox testing.

Since testing during development will be to ensure functionality, I will start a test by simply running the program (or part of program) that I am attempting to test, and I will record the result of the testing by screenshotting or recording the outputs. By doing this for each newly developed part of my code, I am ensuring that any errors I encounter are caused by the newly written code and not previously written code, making troubleshooting a lot easier. I will be the only tester during the development phase since it will consist mainly of small-scale tests that does not require another individual to test the code and I assume I will encounter multiple errors prior to completion, where testing would be harder with another individual if they had to wait until every problem was resolved. When the code runs as expected, I will record the expected result and continue on another aspect of the code. If the code is incorrectly working, I will record the incorrect output or error and the code that caused the issue for reference in the future if I encounter the same or similar problem again and once the issue is resolved I will record the correct code.

By utilizing whitebox testing, I am decreasing the number of errors during my blackbox testing phase and errors that may occur during other parts of my development of the program. This will speed up the development process since fixing errors will be a lot easier when it is broken into individual sections immediately after writing the code for it. This will also speed up the development process since I will have to spend less time searching for the cause of potential errors, since I will know exactly which section it is caused by during whitebox testing,

Blackbox testing

Once I have completed my whitebox testing and ensure the entire program has functionality, I will begin the process of blackbox testing, where I test the entirety of the program as a final test. This will be to ensure that there are no issues combining the different parts of my code together to form the completed solution, making sure to eliminate any errors that occur. When completing the final test, I will utilize destructive testing to attempt to break my code and find any weak points in my validation of user inputted data. This is to ensure that even with incorrect user inputs, the code does not break whatsoever. I will record the entire testing process in screenshots and screen capture recordings, including and changes I make to fix the code.

Stakeholder testing

Once I have developed the program to be error free after finishing the blackbox and whitebox testing, I will be asking for input from the stakeholders for aspects of the program which they think are suitable and which they think are not and need improvements. I will focus my questions around usability features instead of functionality, since I would already be testing and confirming the functionality of the program in my whitebox and blackbox testing. I will ask the users for input on the usability of the program and any adjustments that they would like to be made to make the application more user-friendly. My stakeholder testing will not be the most important aspect of my program since my program is more centred around its functionality and ability to perform the task it needs to, rather than looking good and being easy to use for the user, but it is still important.

Alpha and beta testing

During my whitebox and blackbox testing, I will utilize alpha testing, where I will be the sole user testing the functionality of the code. This is to make testing quicker, simpler and easier, since it is much faster for me to just click play and utilize the code, versus a different user having to have the specific functionality explained to them at every step that needs to be tested. I do not need multiple people to test the functionality of the code since it will be primarily the same for all users (unless they do not have an existing account) and I can manipulate the test data to serve my exact part that I am testing (such as changing my date of birth on the database to test for varying ticket prices or removing my face from the database to test how it reacts to an unknown face).

During my stakeholder testing, I will utilize beta testing, where I use a small selection of users to test out my product for their input. My stakeholder testing will be mainly centred around the usability since each individual tester has varying opinions on different usability features so they will all be able to give valued feedback. I cannot do the stakeholder testing alone since I am biased to the program, having written it, which is why it is important to conduct the stakeholder testing in the form of beta testing.

Testing layout

This is an example of how I will set out my testing data for both the whitebox testing and the blackbox testing.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Retest Number	Solution	Fig

Development

Stage 1: Facial recognition of user

To be able to create aspects of my login and user creation window I will need to be able to detect the face of a user and find the facial landmarks of the user.

Step 1: Read camera input

To be able to return details from a user's face, first we need to be able to process the users face.

```
# import the opencv library
import cv2

# define a video capture object
vid = cv2.VideoCapture(0)

while (True):

    # Capture the video frame
    # by frame
    ret, frame = vid.read()

    # Display the resulting frame
    cv2.imshow( winname: 'frame', frame)

    # the 'q' button is set as the
    # quitting button you may use any
    # desired button of your choice
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

I started this by importing open cv, and then defining vid as a variable to store each frame that is captured from my video feed. I set a while loop to keep looping through until the user interrupts it by pressing q on the keyboard and then to read the camera by each frame and display each frame.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Camera feed	Camera feed window to open	Camera feed window opens	Pass		1.0
2.0	'Q' Exit key	When q is pressed on the keyboard the program closes	The program closes	Pass		2.0

Fig 1.0

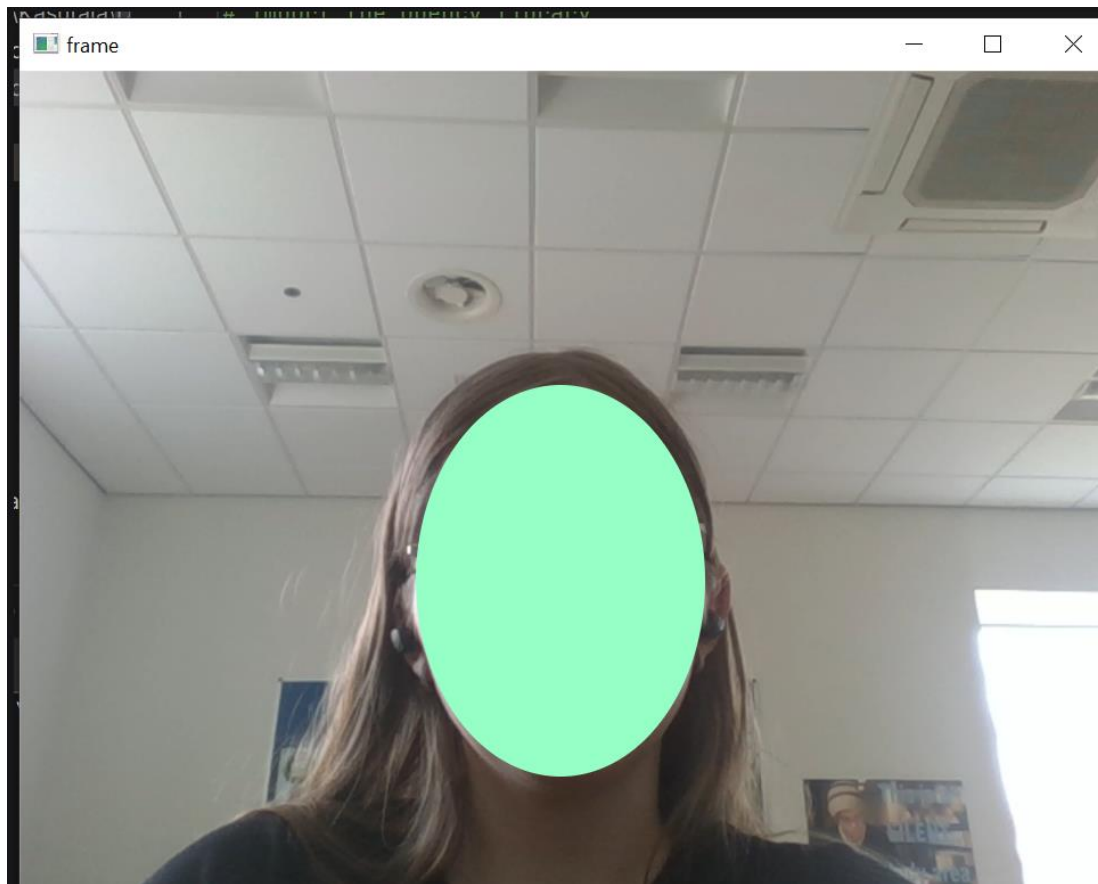


Figure 2

```
C:\Users\Kasutaja\Documents\THEproje  
  
Process finished with exit code 0
```

Step 2: Recognise face from the camera feed

To make sure I am not detecting anything else, I will set my program to recognise where each face is so it can save exactly that section.

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
```

First I set a variable to store the haarcascade which I will be using to detect the faces in my program.

```
# Convert the frame to grayscale
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Detect faces in the frame
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5
                                       , minSize=(30, 30))

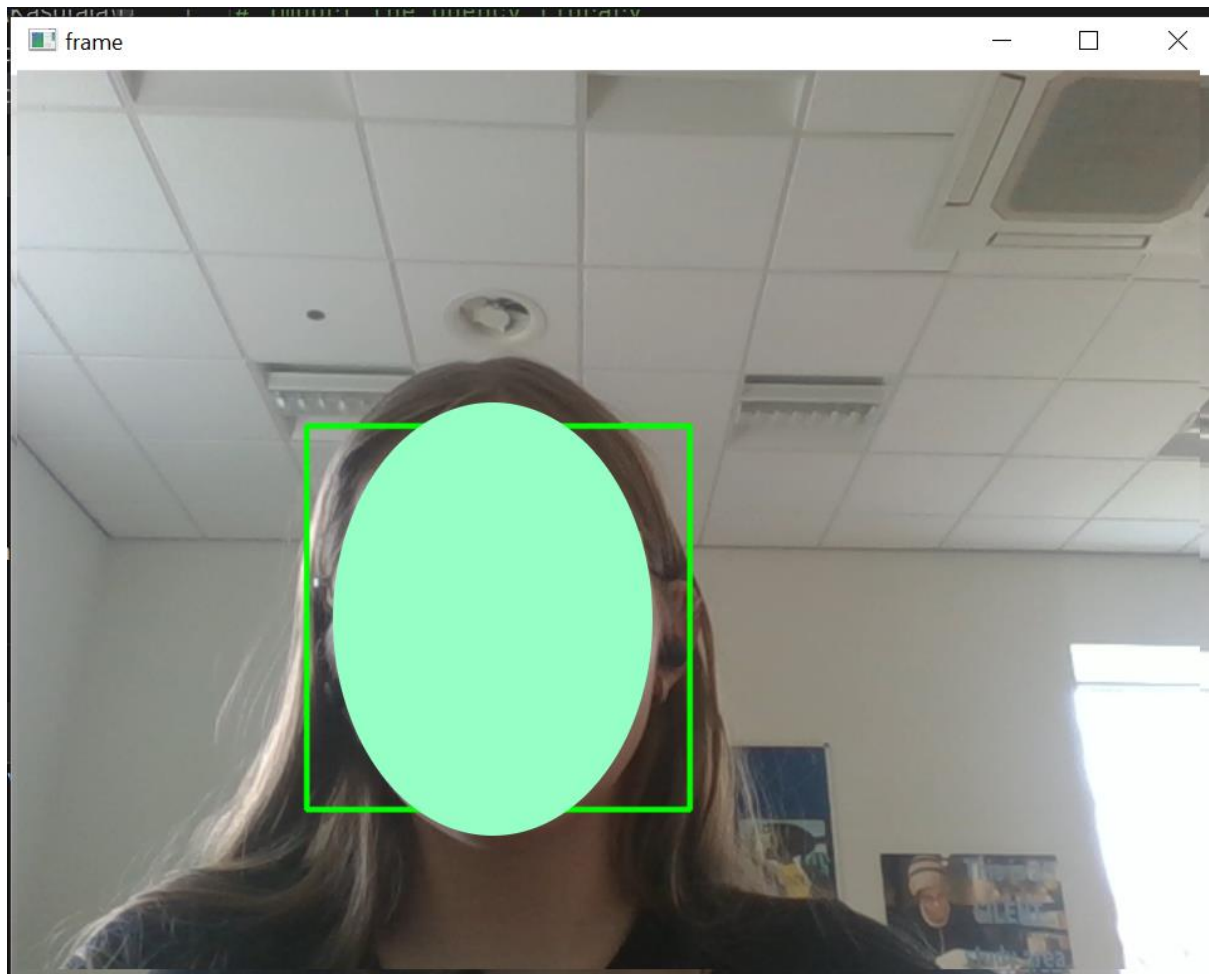
# Draw a rectangle around the detected faces
for (x, y, w, h) in faces:
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

# Display the resulting frame
cv2.imshow('frame', frame)
```

then set the camera feed to be grey so that it can be more easily read and detected by the algorithm since there is less data to process. I then created a variable called faces which detects each face in the frame. Once the faces are detected it will draw a square around the face, showing that the face is being detected in real time. I put this all before the cv2.imshow line because that is the line that displays the current frame, if it was before then the green box would not show up of the screen that is displayed to the user.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Green box to appear in camera feed	Green box around face	Green box around the face	Pass		1.0
2.0	Green box to follow user	Green box around face as the user moves	Green box following user	Pass		Capture1.mp4

Fig 1.0



Step 3: Find face encodings of the face

Once the program has found the user, I will get the program to return the face encodings of the user and then compare them to an existing image of mine to see if it is able to correctly identify me as being the same person.

```
def find_face_encodings(image_path):
    # reading image
    image = cv2.imread(image_path)
    # get face encodings from the image
    face_enc = face_recognition.face_encodings(image)
    return face_enc[0]
```

The first step was to create a function which finds the face encodings of any image that calls the function. First the function uses open cv to read the image that is given by the parameters and then uses the face recognition module to find the face encodings of the selected image. After that it returns the first face encodings of the 2D array since the comparison cannot work with 2D arrays.

```
if len(faces) > 0:
    face_frame = frame[faces[0][1]:faces[0][1] + faces[0][3], faces[0][0]:faces[0][0] + faces[0][2]]
    cv2.imwrite(filename="temp.jpg", face_frame)
```

Next I created an if statement to process if there is a face present in the camera feed at the present moment. If there is a face present, the program will find the bounding region of the face and save it as a temporary image named "temp.jpg". The use of the bounding region saves the computer processing since it abstracts the image of unnecessary details such as the background and allows the face encodings to be processed quicker since there is a smaller region.

```
# getting face encodings for an existing image
image_1 = find_face_encodings("1.jpg")
# getting face encodings for temporary frame
image_2 = find_face_encodings("temp.jpg")

# checking both images are same
is_same = face_recognition.compare_faces(known_face_encodings: [image_1], image_2)[0]
print(f"Is Same: {is_same}")
```

After that, using the function I previously wrote, I will find the face encodings of both the images and compare them for accuracy and then it will output whether the faces are the same or not "false" if they are not and "true" if they are the same.

The image I used to compare the frames to is the following.



Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Face comparison of two of the same faces	True to be outputted	True was outputted	Pass		1.0,1.1
2.0	Comparison of two different faces	False to be outputted	False was outputted	Pass		2.0

Fig 1.0 - Camera feed

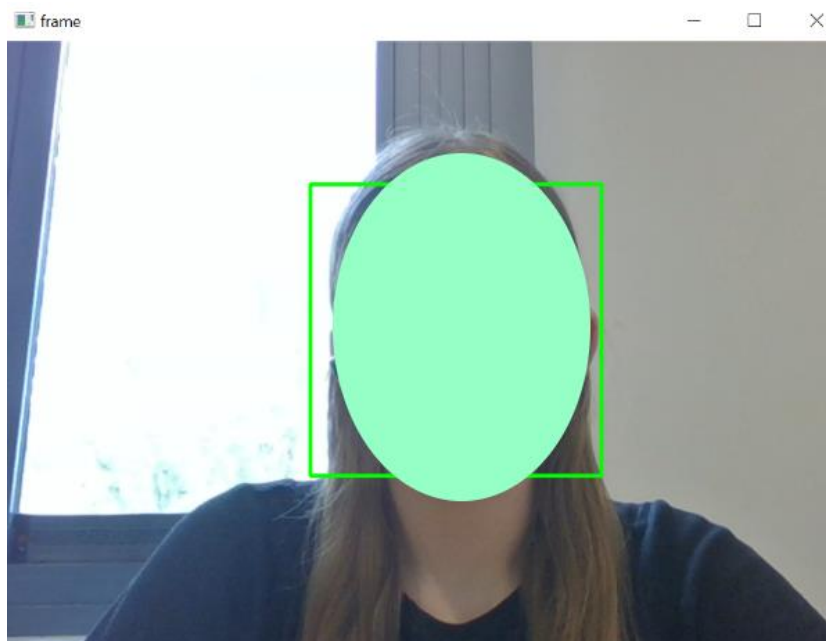
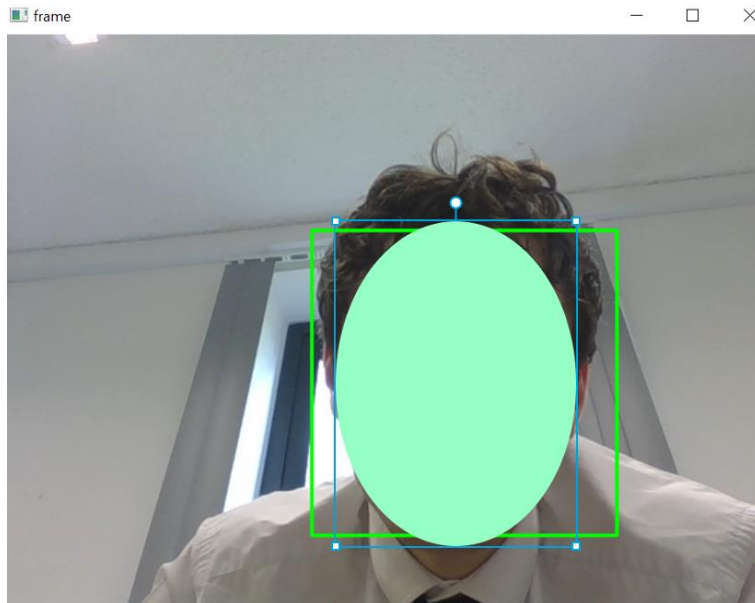


Fig 1.1 - Program output

```
C:\Users\Kasutaja  
Is Same: True  
Is Same: True
```

Fig 2.0



Stage 1 Review

What's been developed:

In this stage, I have successfully created the code to recognise a face from a live camera feed and be able to find a value for face encodings of a user to compare an image for a camera feed to an existing image. I created this so that when I code the other parts of my program that require facial recognition, I can easily reuse the code from this without having to recode it every time, reducing time spent coding and possibly problem solving.

How it was tested:

I first tested the functionality of using open cv to open and read the camera frame and close it when necessary and then tested being able to recognise a human face inside of the camera feed. After I was confident in my ability to use open cv, I integrated the face recognition module to compare an image taken from live feed to an existing image to confirm that the images are the same person. I used the wrong image as well to generate a false result when the images are not the same.

How it meets my success criteria:

This stage has met the success criteria "Recognise a face from camera feed". I have proved this by using screenshots to demonstrate how the program is viewing and isolating my face by placing a square around my face.

I have also met the criteria of "Ability to compare a face to an already stored face". This was proved by using an image that I have already stored and finding the face encodings of that and comparing it to a picture taken from live open cv camera feed and returning a value for the similarity of the images.

Where it doesn't meet my success criteria so far:

- "Display taken image on new user window"
- "Store facial encodings of users"

Any changes from the design in this section:

So far, I have not encountered any issues writing my code, so I have not found the need to make any changes to the design of the code.

Where the project is so far:

So far, the project has part of the backbone of the code created which is part of the main functionality of the code, being able to correctly recognise and compare faces of humans. From here I will need to add the database and then the design aspects of my code.

Stage 2: Option Menu

The program begins with presenting an option to the user to choose whether they are a new or returning user who wants to log in. I will develop this screen first before because both the new user and returning user side of the program require the menu screen to access that side of the program.

I will first create a basic window to be the backbone of the option menu.

```
import customtkinter as ctk

# selecting gui color and theme
ctk.set_appearance_mode("light")
ctk.set_default_color_theme("dark-blue")

font20 = ("Comic Sans MS", 20)
font25 = ("Comic Sans MS", 25)
font30 = ("Comic Sans MS", 30)

app = ctk.CTk() # initialise the app
```

I started by importing tkinter and setting customisation features such as setting the appearance mode to light and the colour theme to dark blue. I also declared three font types for later use in the program. I named them according to the size of the font, so it is easier to remember.

```

app = ctk.CTk() # initialise the app

# screen size geometry
screen_width = app.winfo_screenwidth()
wcentre = (screen_width // 2) - (screen_width // 2)

screen_height = app.winfo_screenheight()
hcentre = (screen_height // 2) - (screen_height // 2)

geometry = f"{screen_width}x{screen_height}+{hcentre}+{wcentre}"

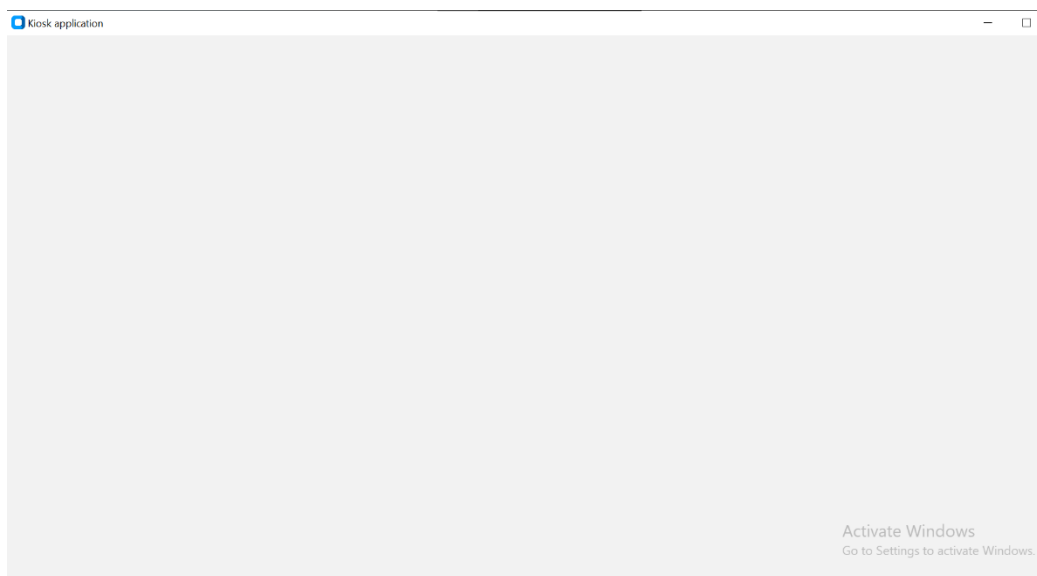
app.geometry(geometry) # sets the size of the screen
app.title("Kiosk application") # application title bar

app.mainloop()

```

Then I initialise the application in the variable “app”. I created another variable called “geometry” which finds the height and width of the screen and then the centre of the screen and then puts it in a format accessible by tkinter for the size of the window and set the window size to be that. I also set the title of the application and then completed the app loop.

I first ran the code to see if it would open and this was opened.



The screen opened in the correct size of my screen and was placed in the centre of the screen.

I then proceeded to add the necessary buttons and text that were required in my program.

```

def login():
    print("Login Pressed")
1 usage
def newuser():
    print("New user button pressed")

```

First, I created two procedures which will be called when their respective buttons will be clicked. When clicked they will output that the button which it is has been pressed (e.g. login button should print login pressed)

```
label = ctk.CTkLabel(app, text="Please choose an option", font=font20)

label.pack(pady=20)

frame = ctk.CTkFrame(master=app)
frame.pack(pady=20, padx=40, fill='both', expand=True)

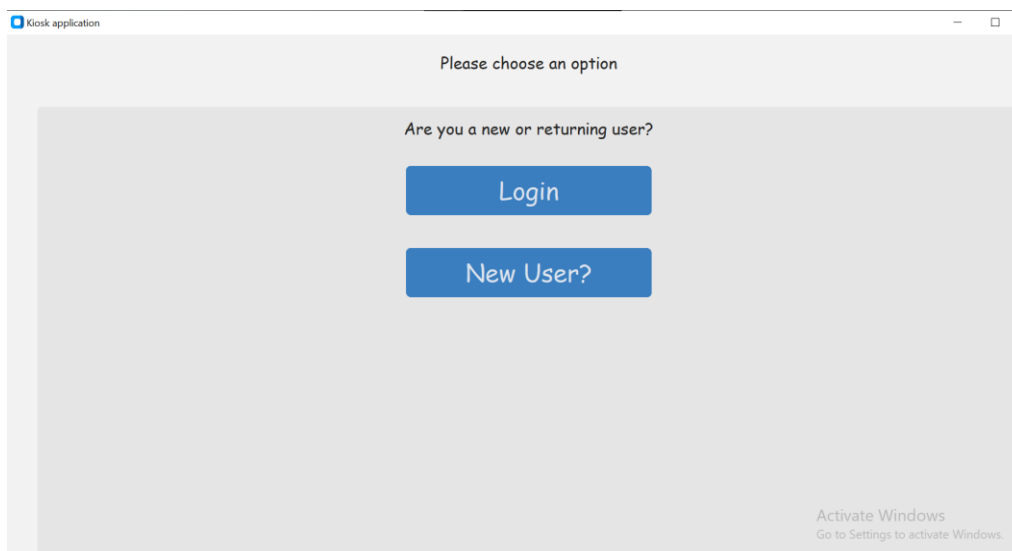
label = ctk.CTkLabel(master=frame, text='Are you a new or returning user?', font=font20)
label.pack(pady=12, padx=10)

loginButton = ctk.CTkButton(master=frame, text='Login', command=login(), font=font30, height=60, width=300)
loginButton.pack(pady=20, padx=5)

newUserButton = ctk.CTkButton(master=frame, text='New User?', command=newuser(), font=font30, height=60,
                               width=300)
newUserButton.pack(pady=20, padx=15)
```

I created a label for the top of the screen to instruct the user to choose an option from the options presented to them. I then declared the label into the application using the pack method. I created a frame for the buttons and the second labels and put the procedures that I created into both of the buttons, making sure everything is declared into the label by the pack method.

I ran the program and then was outputted this screen.



Testing

I will now test the buttons for functionality.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Login Button press	"Login Pressed" to be printed	"Login pressed" and "New user button pressed" printed	Fail	Both functions are called even before the button is clicked	1
1.1	Login Button press	"Login Pressed" to be printed	"Login pressed" printed	Pass	Procedures were called with brackets in the command for the button	1.1
2.0	New user Button press	"New user button pressed" to be printed	"New user button pressed" printed	Pass	N/A	2

Fig 1.0

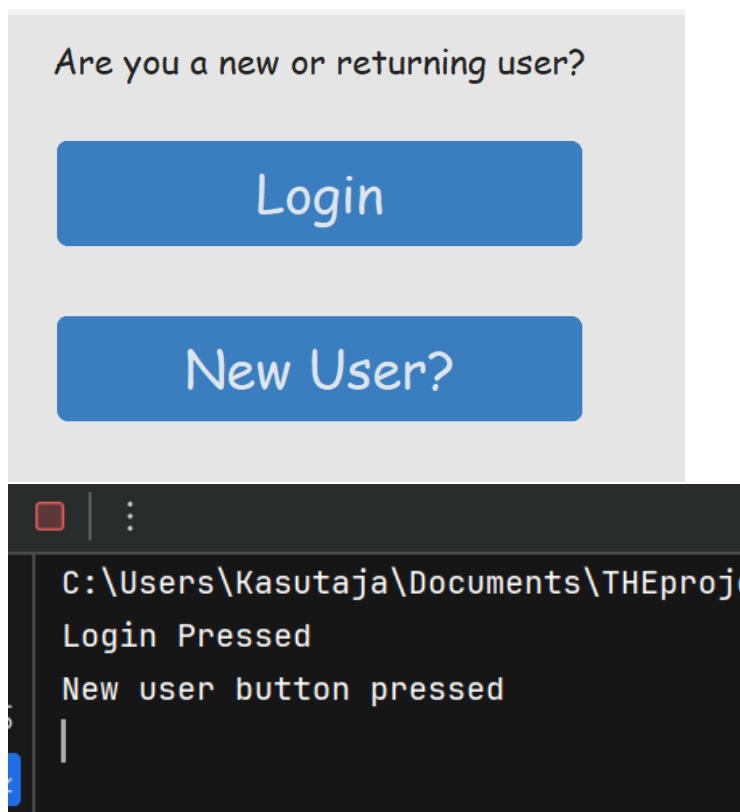


Figure 1.1

Please choose an option

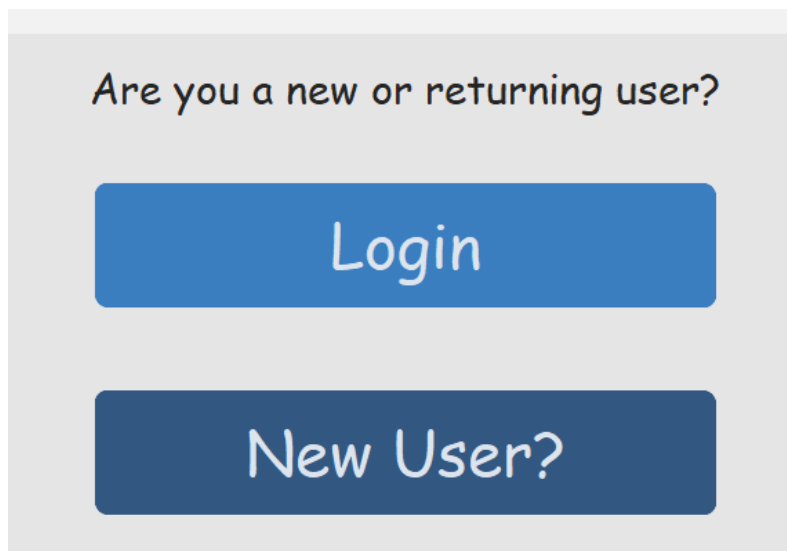
Are you a new or returning user?

Login

New User?

```
C:\Users\Kasuta  
Login Pressed  
  
in', command=login, font=font  
  
ew User?', command=newuser, f
```

Fig 2.0



```
C:\Users\Kasutaja\Document
New user button pressed
```

Stage 2 Review

What's been developed:

In this section I created the basic menu that opens when a user enters the kiosk program. This will provide the user with two options in the form of buttons and guide them to which button is the correct button for the user to press.

How it was tested:

I tested the code by creating temporary functionality for both buttons created to test if they are working by printing certain values as the command. I took screenshots of what was outputted and the specific button that was pressed to cause that output.

How it meets my success criteria:

This meets the success criteria of "User interface which is easily navigate-able". The user interface is easily navigate-able since it has clearly labelled buttons for what part of the program they will run, and it has labels telling the user what to do.

Where it doesn't meet my success criteria so far:

- "Functional buttons throughout the program"
- "Be able to create a new entry on the database including all relevant details that would be required to identifying a new user"

Any changes from the design in this section:

Instead of placing the buttons side by side like I planned in my draft of the design in the design section, I decided to place the buttons vertically on top of each other since it was easier to code with the pack method, if I wanted to place the buttons next to each other horizontally, then I would need to use the grid method which requires a lot more time spent coding the grid.

Where the project is so far:

So far, I have the start of my design created for the application which is the first menu that is involved in my program. This menu will branch off to form other parts of the program which will be designed and backed up by the processing that is made inside of the first stage.

Stage 3: New user creation

The next section I will work on is making the new user button have correct functionality that when it is pressed the user will be able to create a new account to go onto the database, since you cannot log in without already having a user account present.

Step 1: Confirmation window

Before I take the picture of the user, I would like to inform them that their picture will be taken so that they are sure that they need to create a new account.

```
def newuser():
    newuser_confirmation = ctk.CTk()
    newuser_confirmation.title("New User Confirmation")
    newuser_confirmation.geometry(geometry)

    label = ctk.CTkLabel(newuser_confirmation, text="Are you sure you wish to continue?\nFor account creation")
    label.pack(pady=200)

    confirm_button = ctk.CTkButton(newuser_confirmation, text="Confirm", height=60, width=250, font=font25)
    confirm_button.pack(pady=20)

    cancel_button = ctk.CTkButton(newuser_confirmation, text="Cancel", height=60, width=250, font=font25)
    cancel_button.pack(pady=20)

    newuser_confirmation.mainloop()
```

I initialised the new windows and then created a label with text informing the user that an image of them will be taken and stored, giving them the option to either cancel or confirm. I put all this code into the already existing newuser function which acts as the command for the new user button in the user menu so that once that button is clicked, this menu will open.

I created two functions for the confirm and cancel buttons

```
def newuser():
    def cancel():
        newuser_confirmation.destroy()
        print("Cancel")

    def confirm():
        app.destroy()
        newuser_confirmation.destroy()
        print("Confirm")
```


When cancel is called upon, the current program will destroy, taking the user back to the menu page. If the confirm function is called, the previous menu page will be destroyed as well as the current page. I added prints to both functions, so it is easier to test.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/ Fail	Issue/ Resolution	Fig
1.0	Confirmation window	Confirmation window opens upon pressing the new user button in the menu	Confirmation window opens	Pass		1.0
2.0	Confirm button	Confirm to be printed and all windows close	All windows close and confirm is printed	Pass		
3.0	Cancel button	Cancel to be printed and the current window to close	Menu is the only window left open and cancel was printed	Pass		

Fig 1.0

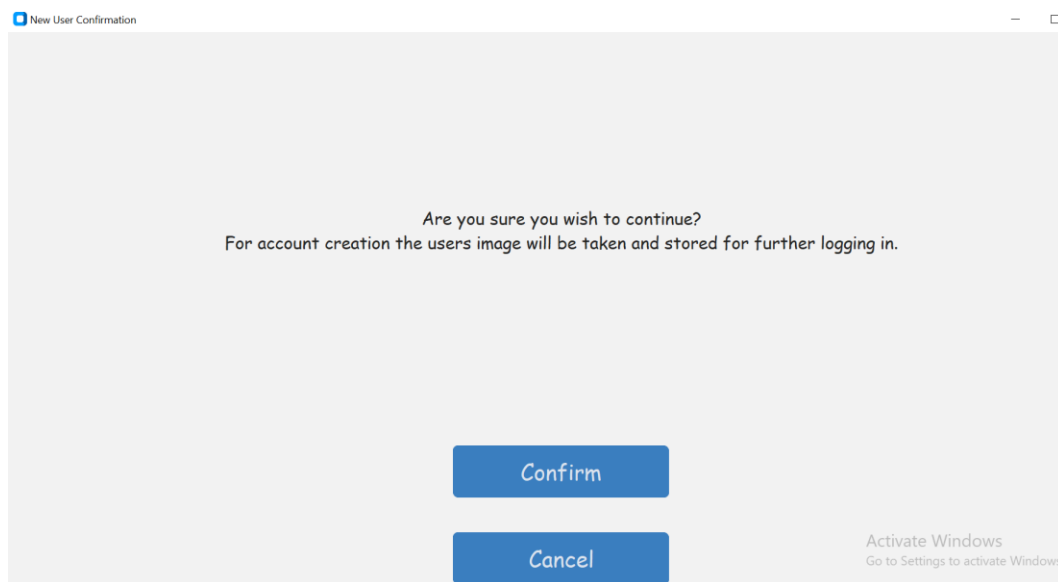
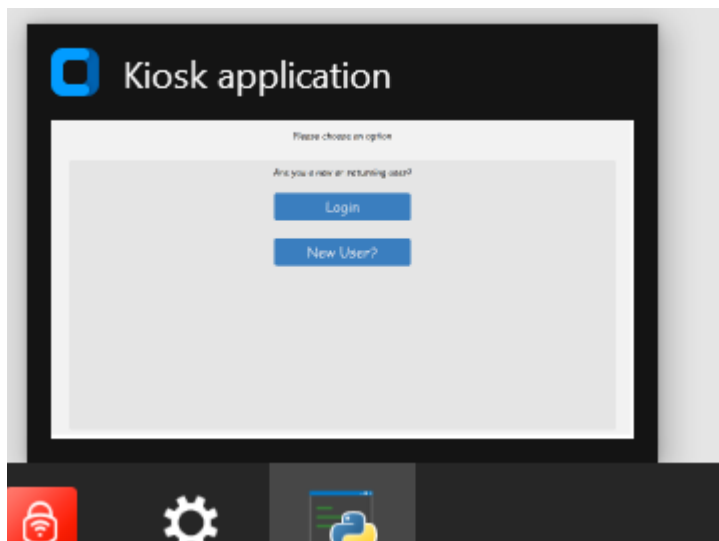


Fig 2.0

```
C:\Users\Kasutaja\Documents\THEproject
Confirm

Process finished with exit code 0
```

Fig 3.0



Only the menu is left open

```
Cancel
cancel printed
```

Step 2: Database declaration

To be used in the next section, I must create my database ready for the new users details to be stored on it. I will be using an excel spreadsheet for my database since it will be a lot easier to put into python than SQL.

	A	B	C	D	E	F	G
1	2	testmail	testname	2000	1	1	50
2	number	email	name	year	month	date	credit

I set the A1 square to be the total user count, so I made the rest of the row to be test data if I need to test the data later. I have set the current user count to 2 so once a new user is created, they do not overwrite the already present data. I left the titles of each of the rows into the database encase I need to reference it later, so it is a lot easier and quicker for me to understand. I will save this file as userbase.xlsx

Step 3: New user picture taken

After the user confirms that they are sure that they want to proceed, I will first have a window telling the user to get ready for the image to be taken. Initially I was going to have it be a live feed of the video and then the program takes a picture but the program is too fast at finding the face as I found out in stage 1 when the program would output true even before I could see the camera feed, it implies that I do not need to create a live feed window, instead I can just show the picture taken to the user and they confirm if the picture is fine.

I will put the code for this section inside of the confirm function.

```
wb = load_workbook(filename='userbase.xlsx') # load workbook
userBase = wb['Sheet1'] # load the correct workbook
userCount = userBase['A1'] # get the value for the amount of current users
userCount = userCount.value
```

Before creating the code for the new window, I will first load up the current workbook and the right sheet of the workbook. I will also retrieve the user count to be used to increment by one.

```
def confirm():
    global userCount
    userCount += 1
    userBase['A1'] = userCount # update user count

    strUserCount = str(userCount)
    filename = strUserCount + '.jpg'
```

At the start of the confirm function, I used the global user Count variable which I created earlier by fetching it from the database and I then incremented it by one for the new user count since there is now a new user. I set the filename of image to be taken to be the user Number of the user who's image it is for easier retrieval.

```

newuser_window = ctk.CTk()
newuser_window.title("New User Window")
newuser_window.geometry(geometry)

label = ctk.CTkLabel(newuser_window, text="Please stand still and look at the camera.",
                     font=font20)
label.pack(pady=200)

capture_button = ctk.CTkButton(newuser_window, text="Capture Image", command=call_take_picture, height=60,
                              width=250, font=font25)
capture_button.pack(pady=20)

newuser_window.mainloop()

```

I then initiate the newuser window and create a button to take a picture of the current user. The screen will also display instructions to the user to stand still and look straight ahead into the camera. The button calls the command call take picture.

```

def call_take_picture():
    new_user.take_picture(filename, newuser_window)

```

The call take picture calls the function takes picture but it's necessary since the button cannot have parameters as a part of its command, so I have to place it into a separate function already calling it. I placed the function take picture into a separate module since it keeps the code neater. If it was inside of the same module then I would have to place the function ahead of the current function in the code and that would make the code harder for me to track and follow and make developing later down the line harder.

```

while v1 == False:
    current_time = time.time()
    total_time = start_time - current_time
    if total_time > 10:
        msg = CtkMessageBox(title="Detection Error!", message="Unable to detect face, adjust your enviro
                             icon="warning", option_1="Exit", option_2="Retry")

        if msg.get() == "Exit":
            newuser_window.destroy()
            break

```

The first part of the function will be the cap variable, which I have already tested in phase 1 of the program to store the current camera feed. I then load the face cascade algorithm which I will be using to detect the faces. I then use the time function to set the start time of the loop. Once the loop starts looping, I will find a variable called total_time which will be the current time of calculation minus the start time, and if that time is greater than 10 the program should display an error box to the user asking them if they want to retry or exit.

Retrying will reset the timer but exiting will break the loop and destroy the current window that the user is on.

```

ret, frame = cap.read()
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

# Draw a rectangle around the detected faces
for (x, y, w, h) in faces:
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

# Check if a face is detected
if len(faces) > 0:
    # Save the frame as an image file without the green box
    face_frame = frame[faces[0][1]:faces[0][1] + faces[0][3], faces[0][0]:faces[0][0] + faces[0][2]]
    cv2.imwrite(filename, face_frame)

```

After that I reused the code that I wrote in stage 1 to read the camera and in grayscale and detect the number of faces in the frame with the face's variable.

```

if len(faces) > 0:
    # Save the frame as an image file without the green box
    face_frame = frame[faces[0][1]:faces[0][1] + faces[0][3], faces[0][0]:faces[0][0] + faces[0][2]]
    cv2.imwrite(filename, face_frame)

    # display the image to the user and ask if it's okay
    img = Image.open(filename)
    img = img.resize((int(img.width * 3), int(img.height * 3))) # Increase the size of the image by
    tk_img = ImageTk.PhotoImage(img)

    img_window = ctk.CTkToplevel()
    img_window.title("New User Image")
    img_window.geometry(geometry)
    img_window.transient(newuser_window) # make img_window a transient window of newuser_window

    img_label = ctk.CTkLabel(img_window, image=tk_img)
    img_label.pack()
    check_result()

    img_window.destroy()

```

This time if the face is being detected, I will also save the frame of the face and then increase the size of the image so it's easier for the user to see when shown on the window. I then create a top-level transient window of the current window to ask the user if the current image is acceptable to the user. I ask this by the check_result function.

```

def check_result():

    msg = CtkMessageBox(title="", message="Is this image fine?", font=font20,
                        icon="question", option_1="Yes", option_2="No")
    response = msg.get()

    if response == "Yes":
        global v1
        v1 = True

```

The check result function will open a tkinter message box asking the user if the displayed image is fine to them. If they respond yes, then it will break the loop by setting the variable v1 to be true. If the

answer is no, the code remains unchanged, and the program keeps looping through since v1 will still be false.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Capture image confirm window opens once previous button is clicked	The new user window to open once the button is clicked	The new user window opens	Pass		1
2.0	When capture image button is pressed, the next window opens displaying the face	The next window opening displaying the image taken	An image is displayed but it is not the user	Fail	Program is detecting something that isn't a face	2.0
2.1	When capture image button is pressed, the next window opens displaying the face	The next window opening displaying the image taken	Image window is correctly displayed and the tkinter message box opens	Pass	2.0 test resolution	2.1
3.0	Program closes when "Yes" is pressed in the tkinter message box	Program closes	Program closes	Pass		3.0
4.0	Program retakes a picture when "No" is pressed in the tkinter message box	New picture is taken	New picture is taken	Pass		4.0
5.0	Program saves the image as the user count	New image saved	New image is saved	Pass		5.0

Figure 1

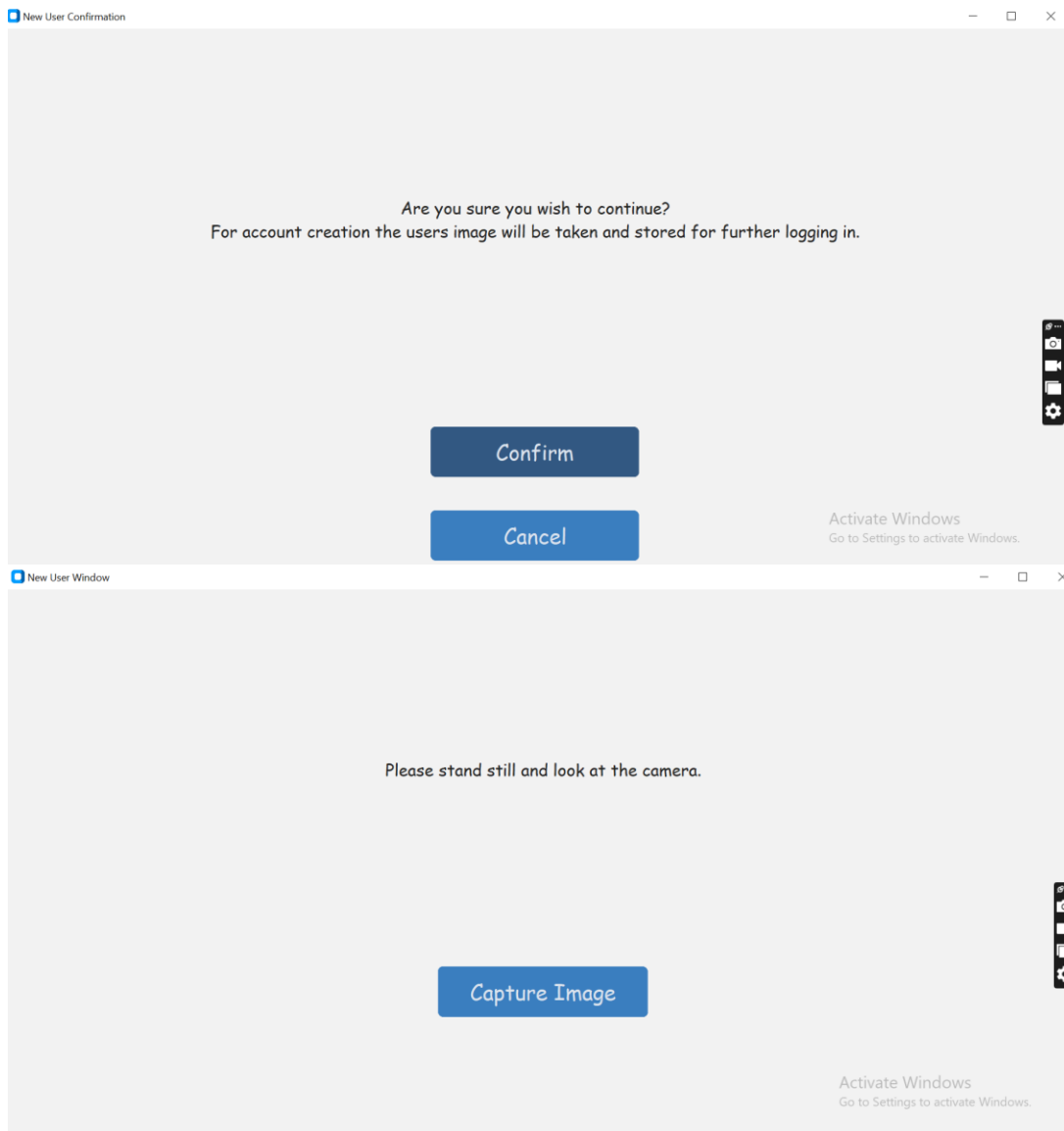
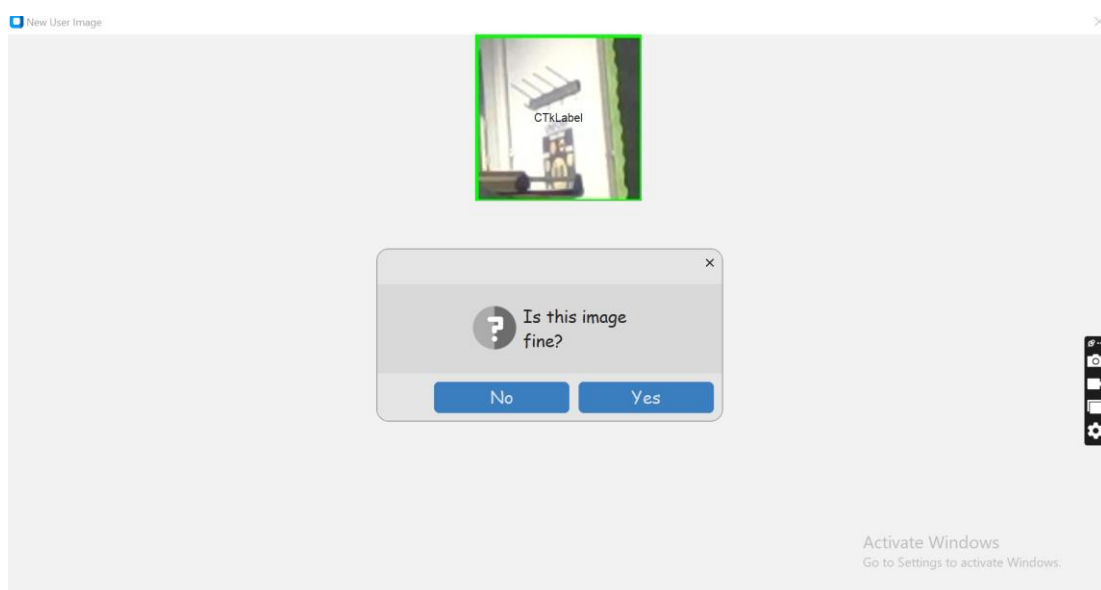


Figure 2.0



Test 2.0 resolution

To resolve it, I will need to change the confirmation to check if face encodings can be found for the image, which confirms it to be suitable image to use.

```
def check_result():
    image = cv2.imread(filename)
    # load the image taken

    face_enc = face_recognition.face_encodings(image) # find face encodings of the image taken

    if face_enc:
        msg = CTKMessageBox(title="", message="Is this image fine?", font=font20,
                             icon="question", option_1="Yes", option_2="No")
        response = msg.get()

        if response == "Yes":
            global v1
            v1 = True
```

First I will call the image that was made during the rest of the program and then I will find the face encodings of that image. If the face encodings exist, an if statement will be ran which involves the confirmation box, but if there are no face encodings present then the if statement will be skipped and a new image will be created.

Figure 2.1

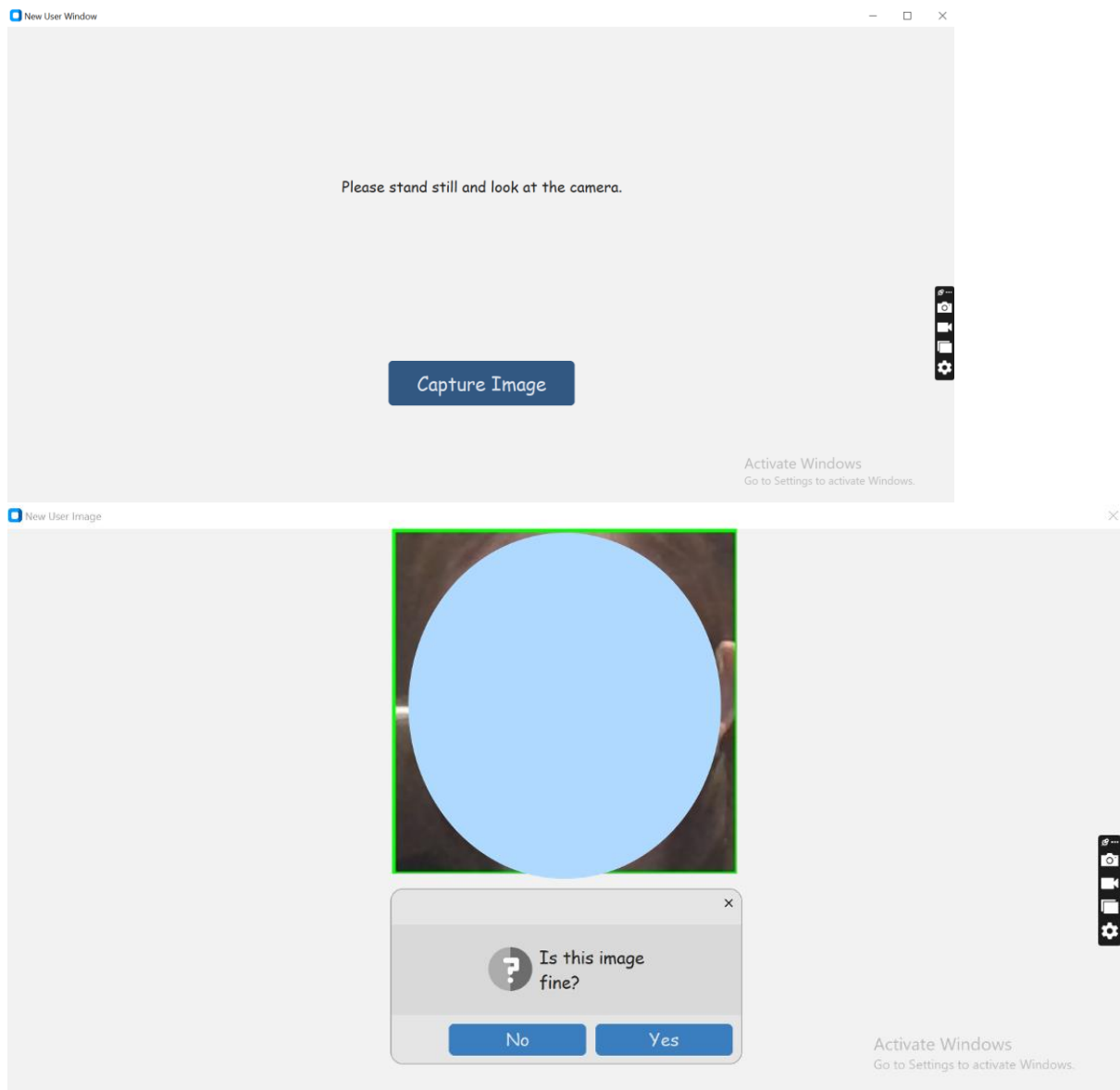
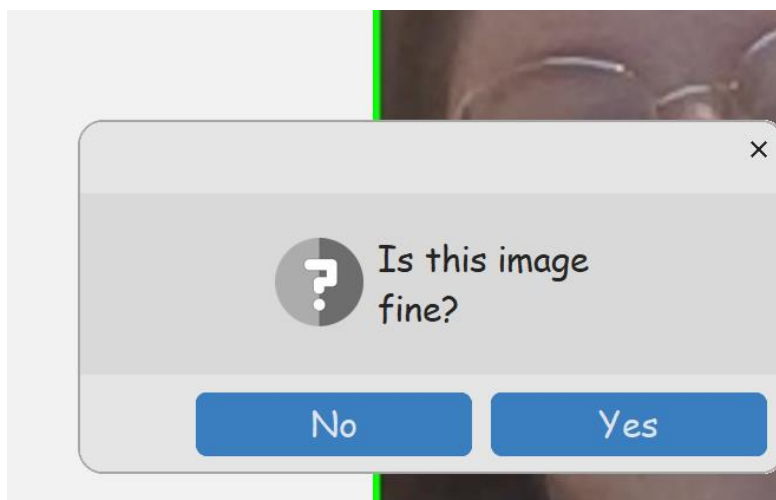


Figure 3.0



```
Process finished with exit code 0
```

Figure 4



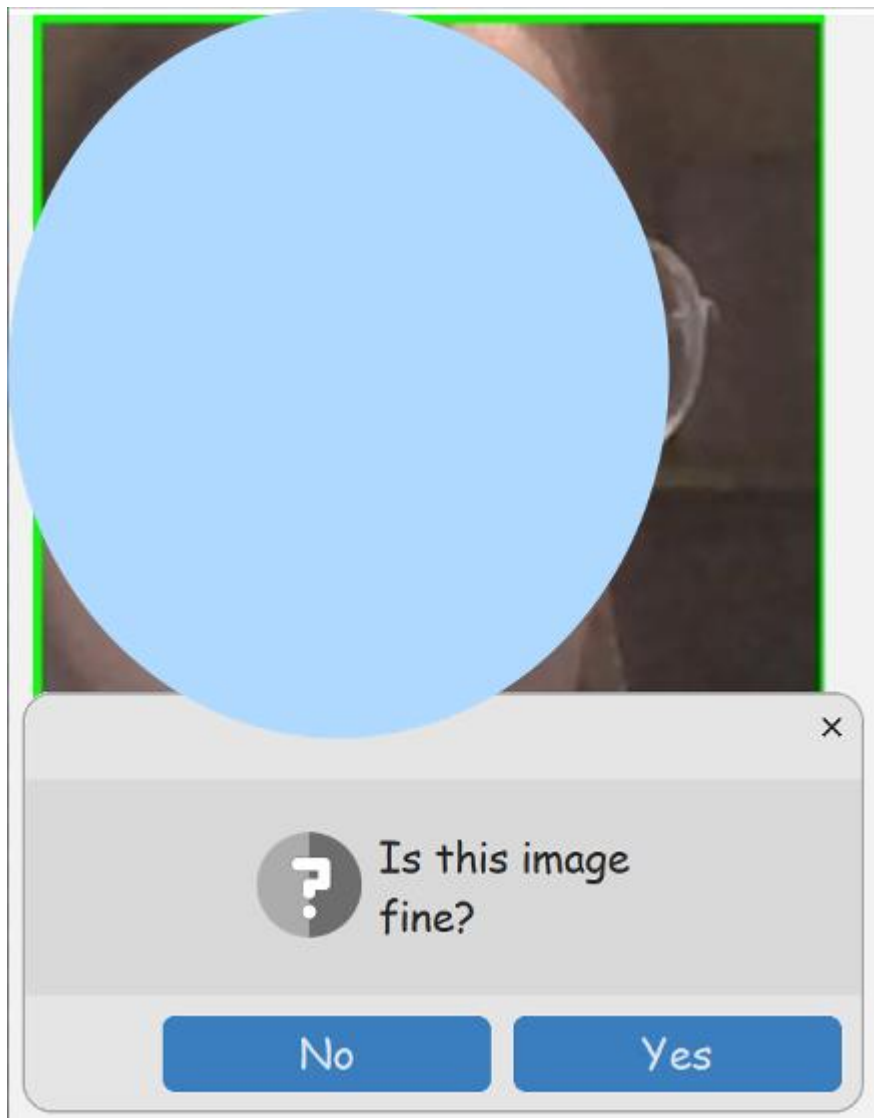
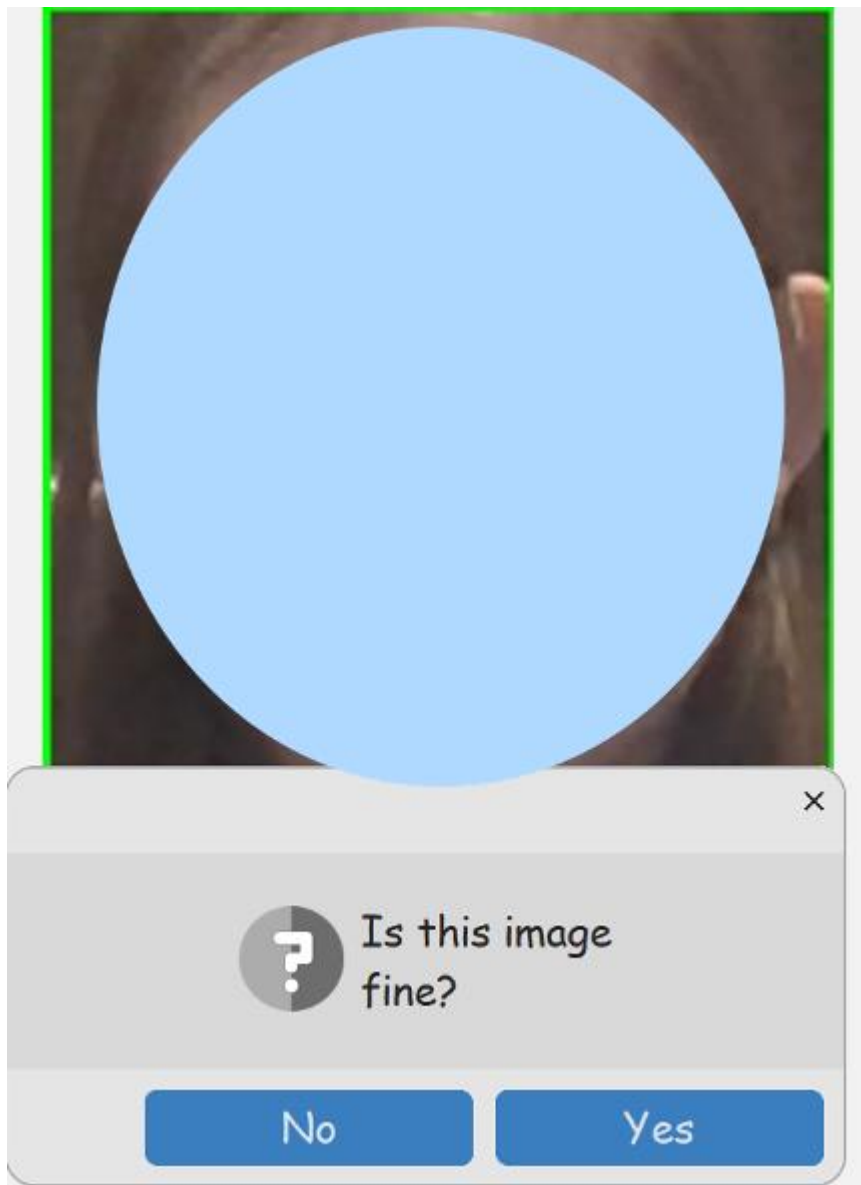
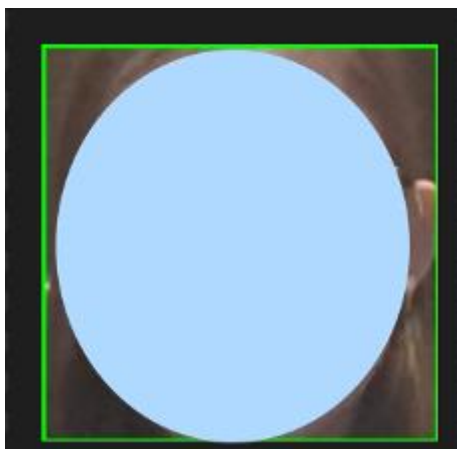


Figure 5



"3.jpg"



Step 4: New user details entry

After the user has created their picture, they will need to input details about themselves for use in contact or age verification.

```
img_window.destroy()

newuser_window.destroy()

newdetails_window = ctk.CTk()
newdetails_window.title("New User Window")
newdetails_window.geometry(geometry)

label = ctk.CTkLabel(newdetails_window, text="Please enter your name, email address and date of birth (
label.pack(pady=20)
```

In the same take picture function, after the existing code I started the new code. First, I will destroy the newuser window and then create a new window called newdetails window. Inside the window I will create a label which informs the user to enter their details such as name, email address and date of birth.

```
nameEntry = ctk.CTkEntry(master=newdetails_window, placeh
nameEntry.pack(pady=30)

emailEntry = ctk.CTkEntry(master=newdetails_window, place
emailEntry.pack(pady=30)
```

After that I created two entry fields, one for the user's name entry and the other for email entry and used the pack method on both of them.

```
dates = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
         15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31]

months = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

years = [2024, 2023, 2022, 2021, 2020, 2019, 2018, 2017, 2016, 2015, 2014, 2013, 2012, 2011, 2010, 2009,
         2008, 2007, 2006, 2005, 2004, 2003, 2002, 2001, 2000, 1999, 1998, 1997, 1996, 1995, 1994, 1993,
         1992, 1991, 1990, 1989, 1988, 1987, 1986, 1985, 1984, 1983, 1982, 1981, 1980, 1979, 1978, 1977,
         1976, 1975, 1974, 1973, 1972, 1971, 1970, 1969, 1968, 1967, 1966, 1965, 1964, 1963, 1962, 1961,
         1960, 1959, 1958, 1957, 1956, 1955, 1954, 1953, 1952, 1951, 1950, 1949, 1948, 1947, 1946, 1945,
         1944, 1943, 1942, 1941, 1940, 1939, 1938, 1937, 1936, 1935, 1934, 1933, 1932, 1931, 1930]
```

I then created 3 arrays to contain the data for days of the month, months of the year and years from now up until 1930 so the oldest person that can use my program would be currently 106 years old.

```

yearbox = ctk.CTkComboBox(newdetails_window, width=250, values="Year")
yearbox.pack(pady=30)
CTkScrollableDropdown(yearbox, values=years, justify="left", button_color="transparent")

monthbox = ctk.CTkComboBox(newdetails_window, width=250, values="Month")
monthbox.pack(pady=30)
CTkScrollableDropdown(monthbox, values=months, justify="left", button_color="transparent")

datebox = ctk.CTkComboBox(newdetails_window, width=250, values="Day")
datebox.pack(pady=30)
CTkScrollableDropdown(datebox, values=dates, justify="left", button_color="transparent")

```

After that, I created 3 scrollable dropdown menus for each of the arrays, allowing the user to choose from every open in the 1D array when clicking on it.

```

confirmbutton = ctk.CTkButton(newdetails_window, text="Confirm", command=confirm,
                             height=60, width=250, font=("Comic Sans MS", 25))

confirmbutton.pack(pady=20)
newdetails_window.mainloop()

```

Then I created a confirm button at the bottom of the page with the command confirm.

```

def confirm():
    email = emailEntry.get()
    global userCount
    if yearbox.get() != "Y" and monthbox.get() != "M" and datebox.get() != "D":
        if monthbox.get() == "2" and (
            datebox.get() == "29" or datebox.get() == "30" or datebox.get() == "31"):
            CtkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
        elif (monthbox.get() == "4" or monthbox.get() == "6" or monthbox.get() == "9" or monthbox.get() == "11") and datebox.get() == "31":
            CtkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
        elif not re.match(pattern=r"^[^@]+@[^@]+\.[^@]+", email):
            CtkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
        else:
            setData(letter="F", userCount, datebox.get())
            setData(letter="E", userCount, monthbox.get())
            setData(letter="D", userCount, yearbox.get())

            setData(letter="C", userCount, nameEntry.get())
            setData(letter="B", userCount, emailEntry.get())
            setData(letter="A", userCount, userCount)

    newdetails_window.destroy()

```

The command confirm will retrieve data from each of the entry boxes and then perform validation checks on the data inputted. First it will check if the month is February and the date is either 29, 30 or 31 then it will open a custom tkinter message box telling the user that it is an invalid entry. If the month is either march, may or September and the date is 31 it will also display invalid entries to the user.

It also validates the email to contain at least one @ symbol and one full stop and if it does not have both of those conditions, then it will show the same error message box.

If the user passes these validation checks, the program will use the set data function to save the data to the database using the user count as their number and the corresponding letter and value for the parameters. Once everything has been saved the program will close the window and run the credit function.

```
def setData(letter, userNumber, variable):
    coord = letter + str(userNumber)
    userBase[coord] = variable
```

The set data function works by taking the parameters of the letter in the database that the entry needs to be saved to, the users number and the variable that needs to be saved and it will create a coordinate by adding the letter to the user number and setting the variable to the coordinate on the database.

```
def credit(geometry, userCount):
    credit_window = ctk.CTk()
    credit_window.title("Credit Window")
    credit_window.geometry(geometry)

    creditTxt = ctk.CTkLabel(credit_window, text="Choose how much credit you would like to add",
                             font=font30)
    creditTxt.pack(pady=30)

    def nocredit():
```

The credit function takes the geometry that was found earlier in the function and the user number of the user. It then creates a new window called the credit window and then asks the user to choose how much credit they would like to add.

```
noButton = ctk.CTkButton(credit_window, text="No Credit", command=nocredit, height=60, width=250, font=font30)
noButton.pack(pady=30)

fiveButton = ctk.CTkButton(credit_window, text="£5", command=fivecredit, height=60, width=250, font=font30)
fiveButton.pack(pady=30)

tenButton = ctk.CTkButton(credit_window, text="£10", command=tencredit, height=60, width=250, font=font30)
tenButton.pack(pady=30)

twentyButton = ctk.CTkButton(credit_window, text="£20", command=twentycredit, height=60, width=250, font=font30)
twentyButton.pack(pady=30)

credit_window.mainloop()
```

After that I created 4 buttons for the user to choose from, a no credit, 5-, 10- and 20-pound button for the user to choose how much credit they want to choose. I used buttons instead of an amount because then it directly creates a minimum and maximum amount for the user to add and then validation doesn't have to be performed on the entry field.

```

def nocredit():
    setData( letter: "G", userCount, variable: 0)
    credit_window.destroy()

def fivecredit():
    setData( letter: "G", userCount, variable: 5)
    credit_window.destroy()

def tencredit():
    setData( letter: "G", userCount, variable: 10)
    credit_window.destroy()

def twentycredit():
    setData( letter: "G", userCount, variable: 20)
    credit_window.destroy()

```

The commands for the credit buttons also use the set data function and after the credit has been set it exits out of the credit_window.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Entry fields allow for data to be input	Entry field text enters it when typed	Both entry fields allow for data to be input	Pass		1.0
2.0	Year box shows appropriate data to choose from and allows the user to choose one	Range of options available in the dropdown menu and choose able when clicked	Options are available and it is chosen when clicked	Pass		2.0
3.0	Month box shows appropriate data to choose from and allows the user to choose one	Range of options available in the dropdown menu and choose able when clicked	Options are available and it is chosen when clicked	Pass		3.0

4.0	Date box shows appropriate data to choose from and allows the user to choose one	Range of options available in the dropdown menu and choose able when clicked	Options are available and it is chosen when clicked	Pass		4.0
5.0	Confirm button opens the next menu when clicked upon and closes the current one	Menu closes upon mouse click and credit window opens	Nothing happens	Fail	5.0 Issue	5.0
5.1	Confirm button opens the next menu when clicked upon and closes the current one	Menu closes upon mouse click and credit window opens	The menu closes and the credit window opens	Pass	5.0 resolution	5.1
6.0	Any credit button closes the credit window	Credit menu closes and the window closes	Menu closes	Pass		6.0
7.0	The details that are inputted from both windows are saved into the database	All correct data is inputted into the entry field	Nothing happens	Fail	Didn't save the spreadsheet after making changes	7.0
7.1	The details that are inputted from both windows are saved into the database	All correct data is inputted into the entry field	Only the user count is changed	Fail	Only the usercount is changed, retry	7.1
7.2	The details that are inputted from both windows are saved into the database	All correct data is inputted into the entry field	Only the user count is changed	Fail	I forgot to remove the save workbook from the main menu.	7.2
7.3	The details that are inputted from both windows are saved into the database	All correct data is inputted into the entry field	Details are saved	Pass		7.3
7.4	The details are inputted into the correct row	Data is inputted in the usercount + 1 row (after count is reset)	Not incremented by one	Fail	Must increment it when calling the usercount	7.4

7.5	The details are inputted into the correct row	Data is inputted in the usercount + 1 row (after count is reset)	Data is stored in the correct line and the count is incremented	Pass	Resolution 7.4	7.5
-----	---	--	---	------	----------------	-----

Fig 1.0

The figure consists of two screenshots of a registration form. The top screenshot shows the form with placeholder text: "Enter your Name" and "Enter your Email Address". The bottom screenshot shows the form with the test data "test name" and "testmail@mail.com" entered into the respective fields.

Fig 2.0

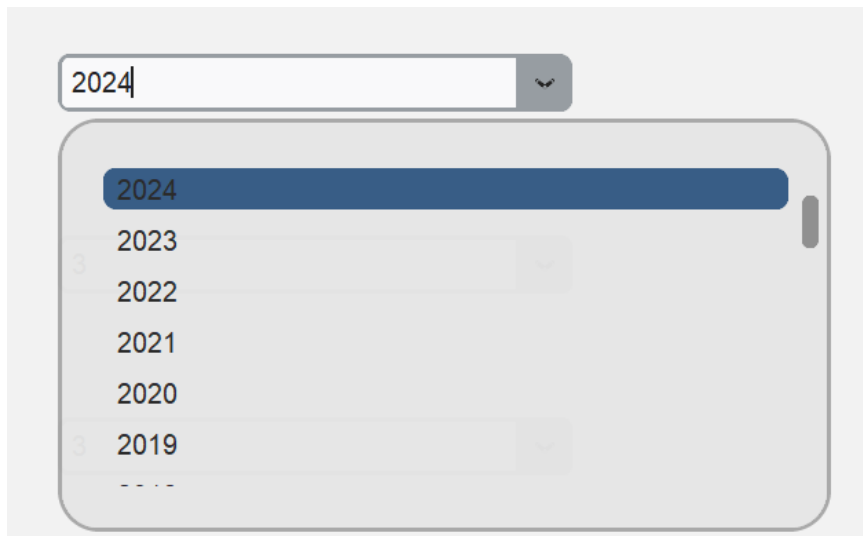


Fig 3.0

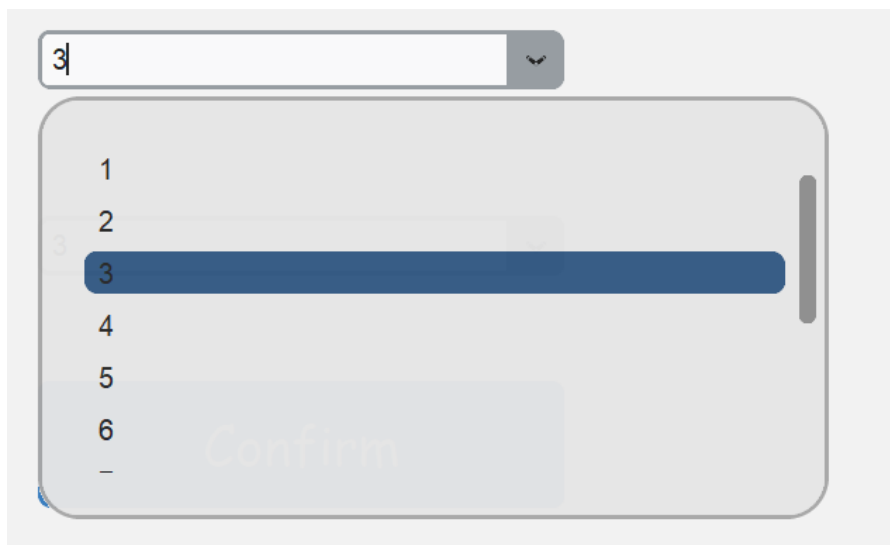


Fig 4.0

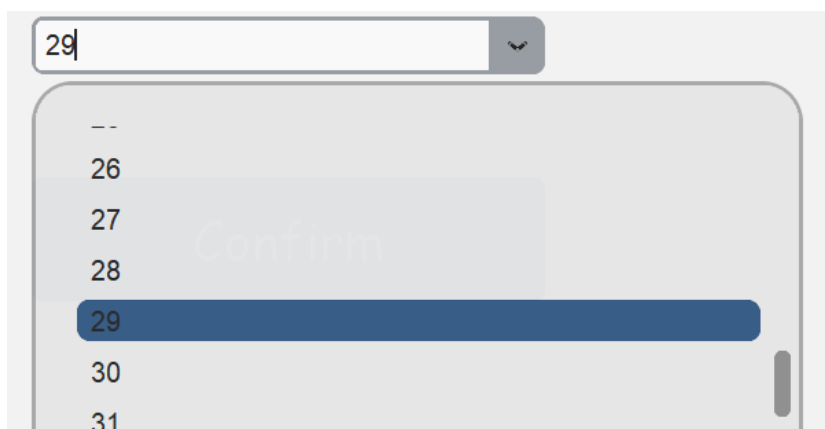


Fig 5.0

Please enter your name, email address and date of birth (Year, Month, Date)

test name

testmail@mail.com

2024

3

29

Confirm

Act
Go *

Figure 5.1

Please enter your name, email address and date of birth (Year, Month, Date)

test name

testmail@mail.com

2024

3

29

Confirm

Act
Go *

Choose how much credit you would like to add

No Credit

£5

£10

£20

Ac

Figure 6.0

Choose how much credit you would like to add

No Credit

£5

£10

£20

Acti

```
Process finished with exit code 0
```

Figure 7.0

name, email address and date of birth (Year)

	2	net@gmail.net	1951	4	2	500	
	username	email	name	year	month	day	credit

Figure 7.1

	A	B	C	D	E	F	G	H
1	3	net@gmail.net	1951	4	2	500		
2	username	email	name	year	month	day	credit	
3								

name, email address and date of birth (Year)

Confirm

Choose how much credit you would like to add

No Credit

£5

£10

£20

	4	net@gma	net	1951	4	2	500
	usernumb	email	name	year	month	day	credit

7.3

	7	test	test@mai	2022	3	4	10

7.4

	A	B	C	D	E	F	G
1	2	net@gmail	net	1951	4	2	500
2	2	test@mail	test	2022	5	4	20
3							

	A	B	C	D	E	F	G	H
1		2	net@gma	net	1951	4	2	500
2	usernumb	email	name	year	month	day	credit	
3								

	A	B	C	D	E	F	G	H
1		2	net@gma	net	1951	4	2	500
2		2	test@mai	test	2022	5	4	20
3								

Choose how much credit you would like to

No Credit

£5

£10

£20

7.5

	A	B	C	D	E	F	G
1	3	net@gmail.net		1951	10	10	500
2	2	test@mail.test		2022	5	4	20
3	3	test2@mail.test2		2021	2	2	20
4							

Issue 5.0

```
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3056.0_x64__qbz5n2kfra8p0\lib
    return self.func(*args)
  File "C:\Users\Kasutaja\Documents\THEproject\venv\lib\site-packages\customtkinter\windows\widgets\ctk_but
    self._command()
  File "C:\Users\Kasutaja\Documents\THEproject\new_user.py", line 189, in confirm
    setData("F", userCount, datebox.get())
NameError: name 'userCount' is not defined
```

The program cannot use the variable userCount if it does not exist inside of the code already since I have not declared it in this module, but I have in the main module.

Resolution 5.0

```
wb = load_workbook(filename='userbase.xlsx') # load workbook
userBase = wb['Sheet1'] # load the correct workbook
userCount = userBase['A1'] # get the value for the amount of current users
userCount = userCount.value
```

I declared the value of user Count at the start of the program before it is used so that value will be accessible.

Resolution 7.0

```
app.mainloop()
wb.save("userbase.xlsx")
```

At the bottom of the kiosk program, I put the line workbook.save so at the end of the entire program it saves anything changed to the database.

Resolution 7.2

I moved the save feature into the credit window function, so it saves it all directly after the credit window values are retrieved

```
twentyButton = ctk.CTkButton(credit
twentyButton.pack(pady=30)

credit_window.mainloop()
wb.save("userbase.xlsx")
```

Resolution 7.4

```
userCount = userBase['A1'] # get
userCount = userCount.value
userCount += 1
userBase['A1'] = userCount
```

I re-incremented the usercount by 1 and then stored the value in the new_user module.

Stage 3 Review

What's been developed:

I have developed the new user creation part of the code. This code is ran when the user clicks the new user button in the initial menu. When they do press the button, the application will start recording the user's information and then save it onto the database for further recalling, such as in the login stage or in the barrier application stage. In this stage I created and utilised the database in a format suitable for retrieving data later. I also stored user's pictures taken in a format that makes them easily retrievable for use later.

How it was tested:

I tested each individual section after I had coded it, describing the code that is being tested and its supposed functionality before and then testing the functionality by providing adequate screenshots of the target information that is being tested.

How it meets my success criteria:

This program meets the success criteria of "Display taken image on new user window". This was proved in step 3, where the image that has been taken by the program gets displayed back to the user after it has been taken, to be confirmed by the user.

It also meets the success criteria of "Be able to store details of a user on a database." This was evidenced in the step 4, where I showed adequate evidence proving that the userbase details are changed after a confirm button is clicked by the user and their correct details have been provided.

The criteria "Store facial encodings of users" has been also met in this section. In step 3 of this stage, I clearly demonstrated that the user's image was being stored as an image file, adequately named and able to retrieve it later on in the application.

The criteria of "Be able to create a new entry on the database including all relevant details that would be required to identifying a new user" has been met in the step 4 of this stage. This was proved as screenshots as after the new user side of the application has been run the data is added to a new row in the database.

The criteria "Clearly displayed instructions for the user-on-user creation window." has also been met, as in every stage of the user creation menu, there is text informing the user what they are required to input into the entries. If a user enters the wrong entries, then error boxes open informing them that what they are trying to input into the database is incorrect.

The criteria "Choice of credit for user after creating a window" is also met since after the user creates their account, they are given the option of how much credit they would like to add, including 0 credit, which is evidenced in screenshots showing screenshots of the user being able to choose from 4 credit options.

The criteria "Functional buttons throughout the program" is also met in this part of my program since every button in the code I have created so far has functional button which all finish an action upon clicking the button and they all have the function that they are intended to have. I have evidenced this by demonstrating me clicking the selected button and showing screenshots of what happens when that button is ran.

The criteria "Working entry boxes for user to enter details" is also met in this part of my program since the code contains multiple entry boxes that allow the user to add details they wish into the boxes and

then those values are saved into the database. I have evidenced this by showing the entry inside of the entry box and then showing the changed value on the database in the form of screenshots.

The criteria “Functional dropdown boxes that allow a user to only select a fixed number of options to choose from.” has been met when I allow for the user to enter in their birthday details when adding values to their database. The dropdown boxes were proven to be functional when I provided screenshot evidence of the boxes dropping down and the user being able to choose a value only from the set values that I have already created.

The criteria “Validation of email addresses entered in the new user entry stage” has been met as I allow the user to enter their email address from an entry box inside the code. Once the user clicks to confirm the email, it is checked for validation and if the validation criteria is not met the error box pops up. I have evidenced this through my screenshots of me entering an invalid email into the entry box and showing the error box that opens up.

Where it doesn't meet my success criteria so far:

- “Store facial encodings of users”
- “Be able to retrieve details of a user on a database”

Any changes from the design in this section:

I have changed the layout of the screen in my new user details entry section. Initially I designed the entry to be two columns of different entries but in my design, I created it to be only 1 column containing all the entries and the buttons. This was because it is a lot simpler to use the pack command instead of the grid command, which requires a lot more time spent on the coding of the grid and each individual coordinates of the room.

Where the project is so far:

So far, the kiosk project is halfway done, having half of the functionality and the design of the project complete since the new user side of it has been created. Next, I will still need to create the log in function side of the computer.

Stage 4: Login Window

Now that users are able to make an account within the program, they will also need to be able to login to change any details or add more credit.

Step 1: Log the user in

First, I will need to detect who the user is before it the log in features can be run.

```
wb = load_workbook(filename='userbase.xlsx')
userBase = wb['Sheet1'] # load the correct w
userCount = userBase['A1'] # get the value f
userCount = userCount.value
1 usage
def login_check():
```

In a separate module named log_in, I created a module called login_check. Before the module I loaded up the workbook and retrieved the usercount value from it.

```
total_time = 0
start_time = time.time()
while total_time < 25:
    cap = cv2.VideoCapture(0)
    success, frame = cap.read()
    cv2.imwrite( filename="temp.jpg", frame)
    is_same = False

    while not is_same:
        counter = userCount + 1
        for i in range(1, counter):
            current_time = time.time()
            total_time = start_time - current_time
            print(i)
            # getting face encodings for first image
            image_1 = face_encode.find_face_encodings(f"{i}.jpg")
            # getting face encodings for second image
            image_2 = face_encode.find_face_encodings("temp.jpg")

            # checking both images are same
            is_same = face_recognition.compare_faces( known_face_encodings: [image_1], image_2)[0]
            same = is_same
            print(f"Is Same: {is_same}")
```

At the start of the code, I declared the total time variable and then found the start time of the while loop. I set the loop to loop as long as the value for total time is less than 25 and at the end of the loop

to find the current time and subtract the start time from the current time to find the total time. Then I reused the stage 1 code and created a video capture using cv2 stored in the cap variable. I saved the frame into an image file called "temp.jpg" and declared the variable is_same to be false. Whilst the declared variable is false, another while loop will start looping through. Inside the while loop I created a linear search for the matching image by looping from 1 to the user count + 1 images. Inside of the loop I found the total time and then used the face encoding finder I made in stage 1 of the program to find the face encodings of the image and then compare the two and print the value.

```

same = is_same
print(f"Is Same: {is_same}")
if is_same:
    userNumber = i
    is_same = True
    total_time = 30
    break
else:
    print("The images are not same")

```

If the two images are the same, the current I value of the loop will become the user Number of the selected user and is same and the total time will be set to over the limit so the loops break. If they are not the same, it will print that the images are not the same.

```

if total_time > 15 and is_same == False:
    errorboxx = CtkMessageBox(title="Image Error", message="Would you like to try again?", option_2="No")
    if errorboxx.get() == "Yes":
        start_time = time.time()
        is_same = True

```

Outside of that loop I created another if statement to check if the time is still valid. If it is not valid it will open a message box asking the user if they would like to try again and if they say yes, then the start time is reset.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Login detection on a known face	1 to be outputted and is same to be true	1 was outputted and is same is true was outputted	Pass		1.0
2.0	Login detection on an unknown face	It to loop through the images and output not a match				2.0

3.0	Test if after 15 seconds the error box opens to show the message	After 15 seconds, error box opens	Nothing happened only looped through the images constantly	Fail	It is not reaching the time command since it is outside of the loop	3.0
3.1	Test if after 15 seconds the error box opens to show the message	After 15 seconds, error box opens	Message box opens	Pass	Resolution 3.0	3.1
4.0	No button on the message box	The program should stop looping	The program stops looping	Pass		4.0
5.0	Yes, button the message box	The program should continue looping until it hits the maximum count again	Program continues looping	Pass		5.0

Figure 1.0

```

C:\Users\Kasutaja\Documents\THEproje
1
Is Same: True

Process finished with exit code 0

```

Figure 2.0

```
C:\Users\Kasutaja\Documents
1
Is Same: False
The images are not same
2
Is Same: False
The images are not same
3
Is Same: False
The images are not same
```

Figure 3.0

```
2
Is Same: False
The images are not same
3
Is Same: False
The images are not same
1
Is Same: False
The images are not same
2
Is Same: False
The images are not same
3
```

Figure 3.1

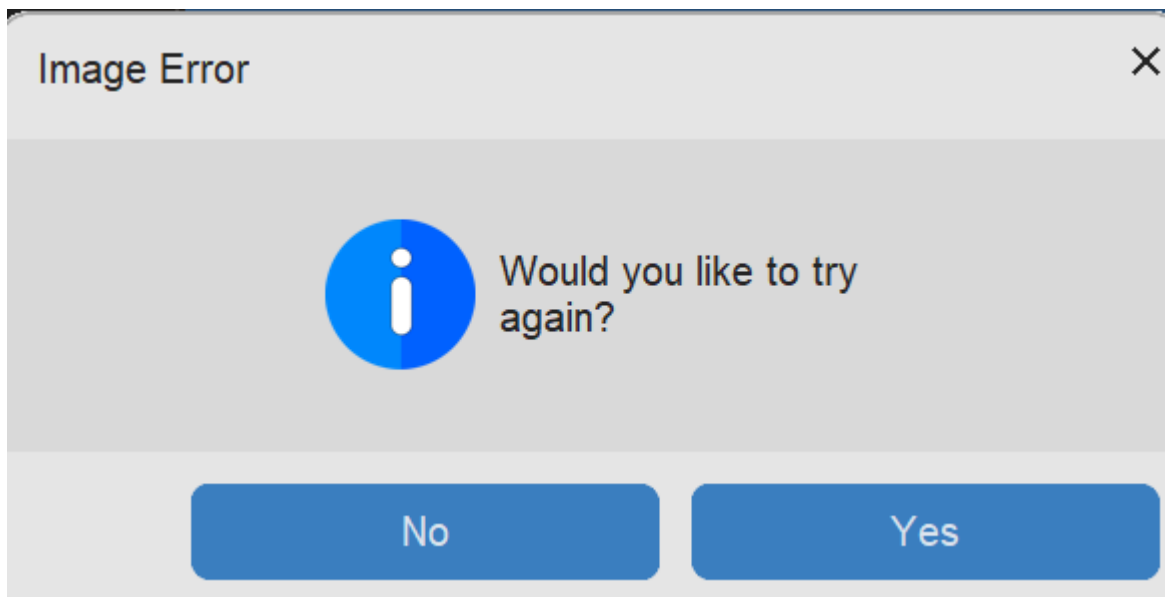
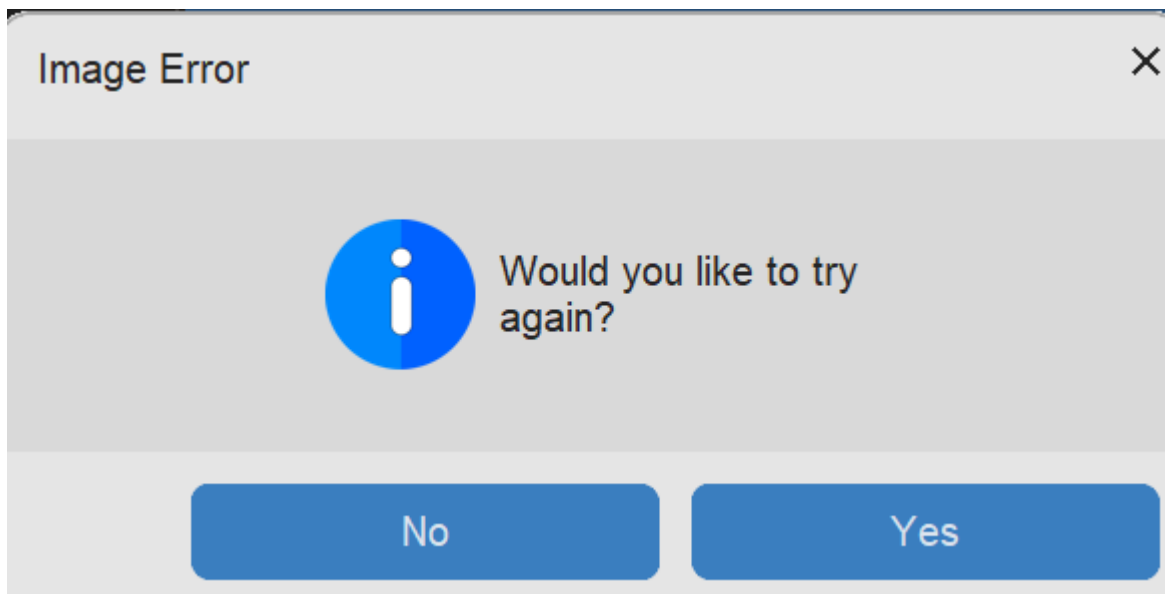


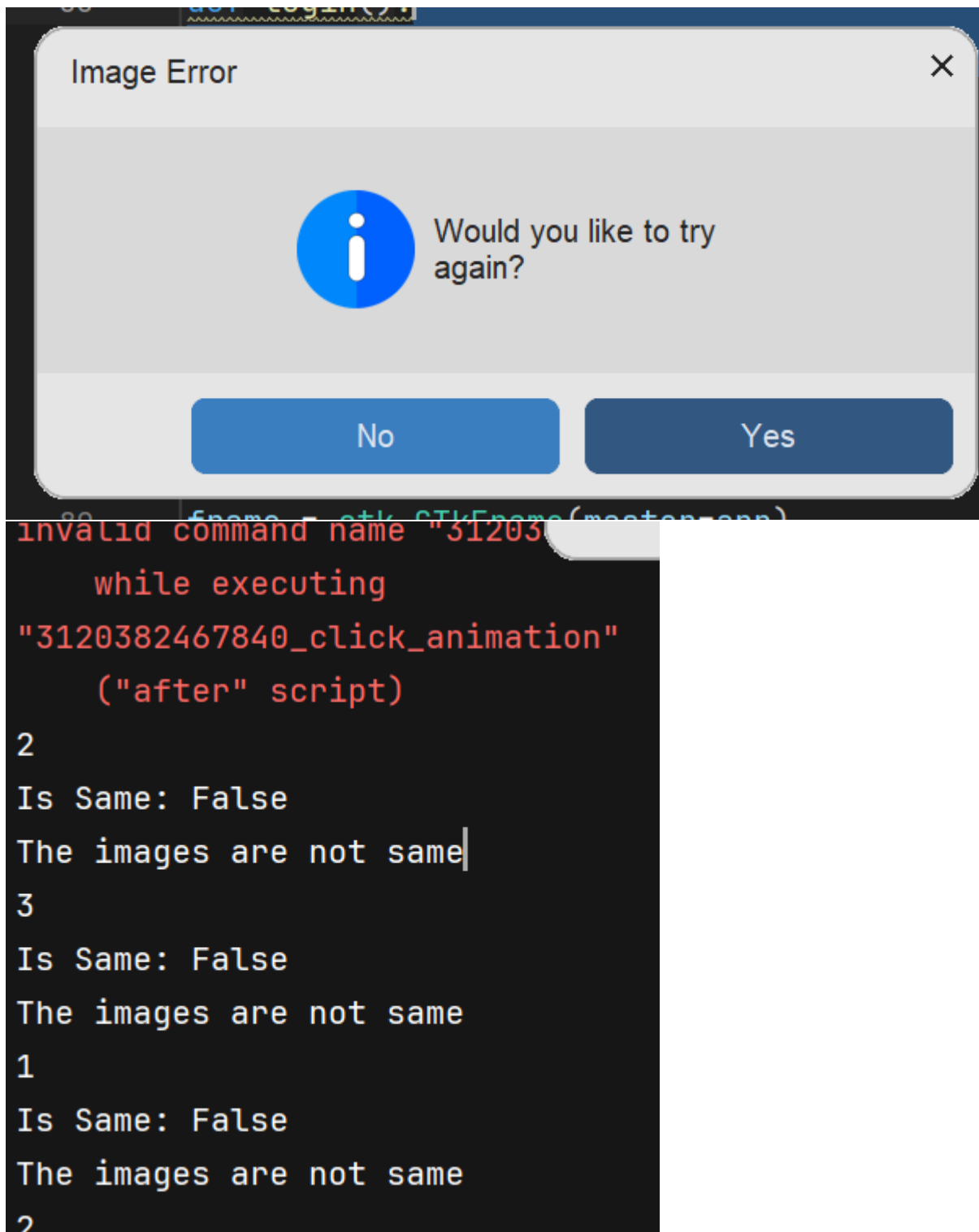
Figure 4.0



```
1
Is Same: False
The images are not same
2
Is Same: False
The images are not same
```

No more loops made

Figure 5.0



Resolution 3.0

Instead of a time command I will create a counter inside of the loop.

```

is_same = False
breaking = False
count = 0

while not is_same and not breaking:
    bound = userCount + 1
    for i in range(1, bound):
        count+=1
        print(i)

```

```

if count > userCount*3:
    errorboxx = CtkMessageBox(title="Image Error", message="Would you like to try again?", opt
                        option_2="No")
    if errorboxx.get() == "No":
        breaking = True
        break
    else:
        count = 0

```

Outside of the while loop, I set the count to be 0 and a new variable called breaking to also be false and created breaking to be a parameter of the while loop. If the count is greater than 3 lots of the user count, the same error box will show asking the user if they would like to try again. If the user replies no, then breaking will become true and break the while loop and I will use the break command to break the for loop but if the user says yes then I reset the count to 0.

Step 2: User options menu

After the user has successfully loaded into the program, they will need to be presented with an option screen with all the possible options for them to choose from.

```
        if errorboxx.get() == "":
            breaking = True
            break
        else:
            count = 0
    return is_same
```

First, I changed the login check function to return the value of is same at the end of its processing.

```
def login():
    app.destroy() # close the previous window
    same = log_in.login_check()
    if same == True:
        log_in.user_editmenu(geometry)
```

I then set the login check function to return the value of is same into the variable same inside of the kiosk program. If the value of same is True, which means that a face has been detected, then the program will run a new function which will be the user edit menu where the user can change details about themselves and it will call the parameter of geometry so I do not have to reconfigure it in the other window.

```
login_window = ctk.CTk()
login_window.title("Login Window")
login_window.geometry(geometry)

Name = userBase["C" + str(userNumber)].value

label = ctk.CTkLabel(login_window, text=f"Welcome, {Name}.\nPlease choose an option",
                     font=("Comic Sans MS", 20))
label.pack(pady=50)
```

Inside of the user edit menu function, first I will create a window, utilising the geometry variable and then I will find the name of the user who is logging in and display a welcome message using their name, with instructions to choose an option from the following options presented to the user.

```
updateDetailsButton = ctk.CTkButton(login_window, text="Update Details", command=updateDetails,
                                   height=60, width=250, font=("Comic Sans MS", 25))
updateDetailsButton.pack(pady=30)
```

The first button of the program will be the update details button which calls the command update details for which the function is above the current one in the program.

```
def updateDetails():
    msg = CtkMessageBox(title="Update details", message="What detail would you like to update?", icon="question",
                        option_1="Email", option_2="Name", option_3="Exit")
    response = msg.get()

    if response == "Email":
        update_details.updateEmail(userNumber)

    elif response == "Name":
        update_details.updateName(userNumber) #
```

The function for the update details command will create a tkinter message box which will allow the user to choose if they want to either update their name or their email or just exit incase they clicked the wrong button.

If the user clicks the email button it will call the update email function from the update details module.

```
email_window = ctk.CTk()
email_window.title("New Email")
email_window.geometry("300x100")

coord = "B" + str(userNumber)
current = userBase[coord].value

currentTxt = ctk.CTkLabel(email_window, text=f"Your current email is {current}", font=font20)
currentTxt.pack()
emailEntry = ctk.CTkEntry(master=email_window, placeholder_text="Enter your New Email Address",
                          width=200, height=25, border_width=2, corner_radius=10)
emailEntry.pack()
confirmButton = ctk.CTkButton(email_window, text="Confirm", command=newEmail,
                              height=60, width=250, font=font25)
confirmButton.pack()

email_window.mainloop()
```

The function for update email will create a new tkinter window which is smaller than the screen size and it will find the current email that is set by the user and then display that to the user and then create an entry field asking the user to input their new email address with the confirm button.


```
def newEmail():
    global newEmailVar
    if not re.match(pattern: r"^@+@[^@]+.[^@]+", emailEntry.get()):
        CTKMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
    else:
        coord = "B" + str(userNumber)

        newEmailVar = emailEntry.get()
        userBase[coord] = newEmailVar
        wb.save("userbase.xlsx")

        email_window.destroy()
```

The function that is called when the confirm button is the new Email function. First it will perform validation on the email entry field, I reused the code that was in the new user section used to validate the email entry field there. If the email is not valid then it will open a custom tkinter message box telling the user there is an error. If the email is valid then the program will set the new entry to be the email of the user in the database and then saves the database.

```
elif response == "Name":
    update_details.updateName(userNumber) #
```

Back inside of the update details function in the log in module, if the user chooses name, then it will call the function update Name from the update details module.

```

name_window = ctk.CTk()
name_window.title("New Name")
name_window.geometry("300x100")

coord = "C" + str(userNumber)
current = userBase[coord].value

currentTxt = ctk.CTkLabel(name_window, text=f"Your current name is {current}", font=font20)

currentTxt.pack()
nameEntry = ctk.CTkEntry(master=name_window, placeholder_text="Enter your New Name",
                        width=200, height=25, border_width=2, corner_radius=10)
nameEntry.pack()
confirmButton = ctk.CTkButton(name_window, text="Confirm", command=newName,
                             height=60, width=250, font=font25)
confirmButton.pack()

name_window.mainloop()

```

The

update name function works exactly like the update email function just replacing email with name. It first finds the current name of the user and displays that to the user and then tells the user to enter their new name. Once they enter their name and press confirm it will run the new name function.

```

def newName():
    global newNameVar
    coord = "C" + str(userNumber)

    newNameVar = nameEntry.get()
    newNameVar = newNameVar.title()
    userBase[coord] = newNameVar
    wb.save("userbase.xlsx")

    name_window.destroy()

```

The new name function is different from the new email function in that it doesn't need to validate the name. It sets the entry field data to be the data under the name section of the current user's user number in the database and saves the database and then destroys that window.

```
addCreditButton = ctk.CTkButton(login_window, text="Add Credit", command=addCredit, height=60, width=250,
                                font=font25)
addCreditButton.pack(pady=30)
```

Back in the user menu function inside of the login module, the next button in the window that will be displayed will be the add credit button. This button calls the add credit function.

```
1 usage
def addCredit():
    coord = "G" + str(userNumber)
    # get yes/no answers
    credit = userBase[coord].value
    msg = CtkMessageBox(title=f"You have £{credit}.", message="How much credit would you like to add?",
                        option_1="Exit", option_2="£10", option_3="£20")
    response = msg.get()

    if response == "£10":
        userBase[coord] = userBase[coord].value + 10
    elif response == "£20":
        userBase[coord] = userBase[coord].value + 20
    wb.save('userbase.xlsx')
```

The add credit function will first find the appropriate coordinate of the credit field for the user and then displays that back to them in a tkinter message box informing them how much credit they have and then asks them how much they would like to add. It gives the user an option of £10 or £20.

If the user chooses £10 then the function sets the current value of the coordinate in the userbase to be the same value with 10 added to it. If the user chooses 20, it does the same but adds 20 more. After the if statements the program saves any changes that have been made in the database. If the user chooses exit, then it does not do anything but closes the message box.

```
exitButton = ctk.CTkButton(login_window, text="Exit", command=exit,
                           height=60, width=250, font=font25)
exitButton.pack(pady=30)
login_window.mainloop()
```

The last button in the user menu is the exit button which calls the exit command when clicked.

```
def exit():
    login_window.destroy()
```

The exit command destroys the current login window.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	User menu opens when the program finds a correct match of a user to the database of users and correctly retrieves the name of the user	The user edit menu window opens and displays the correct name	User edit menu opens and the correct name is displayed	Pass		1.0
2.0	Update details button opens an option for the user to choose whether they want to update name or email	The message box opens giving the user three options.	The message box opens	Pass		2.0
3.0	The name edit window opens when name is pressed in the message box, and it displays the correct name	The window opens and the correct name is displayed to the user.	The window opened and the correct name was displayed.	Pass		3.0
4.0	The email edit window opens when email is pressed in the message box, and it displays the correct email.	The email window opens, and correct email is displayed	The email window opened, and the correct email is displayed	Fail	Half of the email is cut off so I will resize the elements	4.0
4.1	The email edit window opens when email is pressed in the message box, and it displays the correct email.	The email window opens, and correct email is displayed and clearly can be seen	The email window opened, and the correct email is displayed, and it is easy to read	Pass	4.0 Resolution	4.1
5.0	Add credit button opens a message box that allows the user to choose how much credit they want to add and displays current credit	Message box opens and the correct credit is displayed	The message box opens	Pass		5.0

6.0	Exit button closes the window	Window closes	Nothing happens	Fail	"Login window is not defined"	6.0
6.1	Exit button closes the window	Window closes	The window closes	Pass	Resolution 6.1	6.1
7.0	New name is saved to the database	Database name entry field is changed	The name is changed	Pass		7.0
8.0	New email is saved to the database	Database email entry field is changed	The email is changed	Pass		8.0
9.0	New credit is saved to the database	Database credit entry field is changed	The credit is changed	Pass		9.0

Figure 1.0

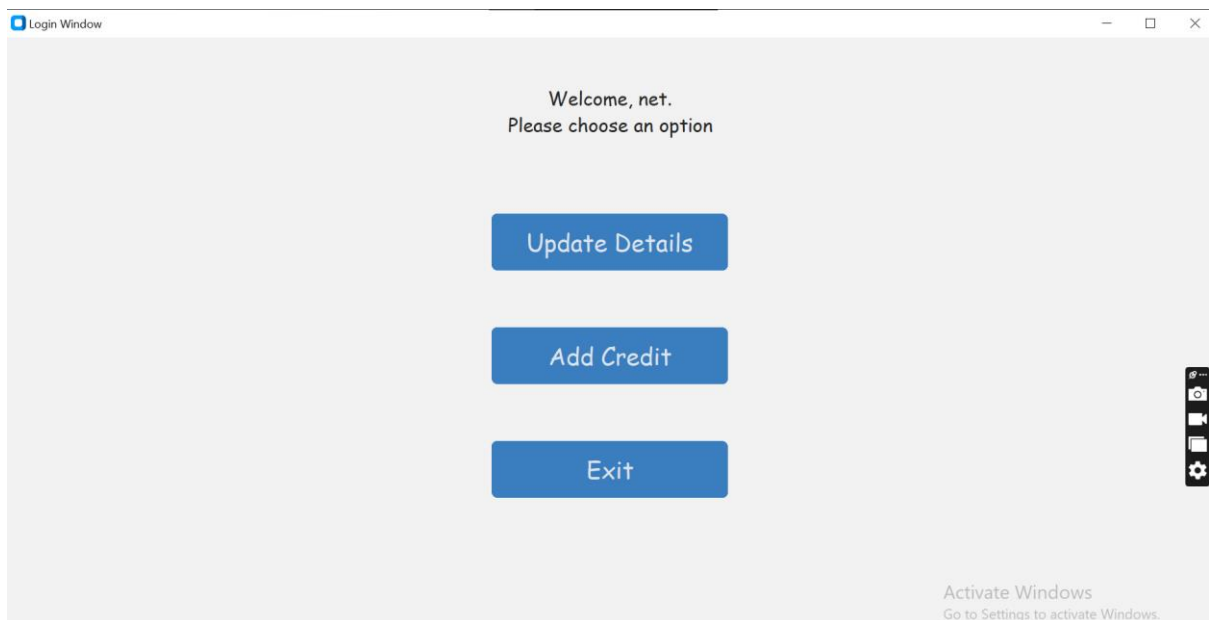


Figure 2.0

Welcome, net.
Please choose an option

Update Details

Update details



What detail would you
like to update?

Exit

Name

Email

fig 3.0

Update details



What detail would you
like to update?

Exit

Name

Email

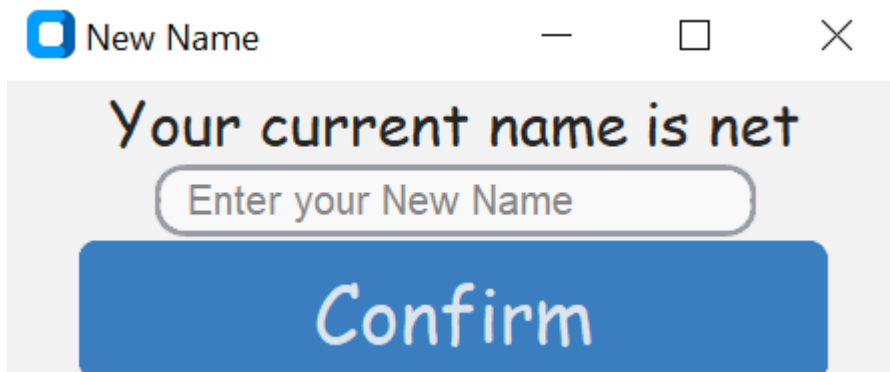


Fig 4.0

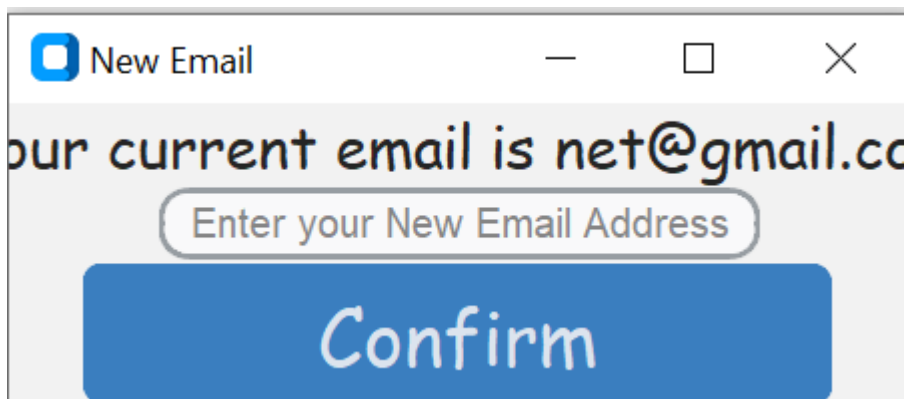


Fig 4.1

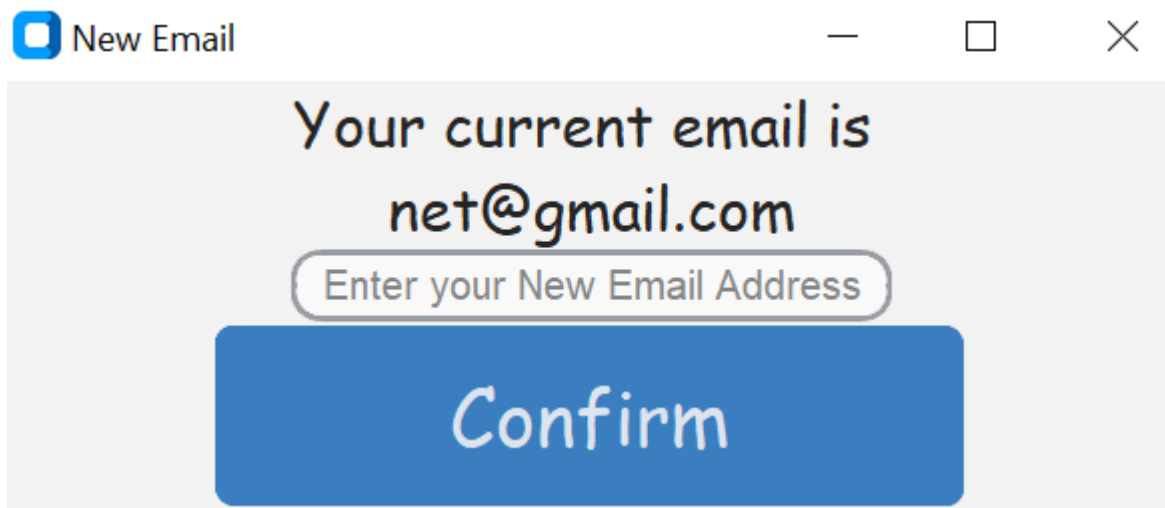


Fig 5.0

Welcome, net.
Please choose an option

Update Details

Add Credit

Exit

You have £500.



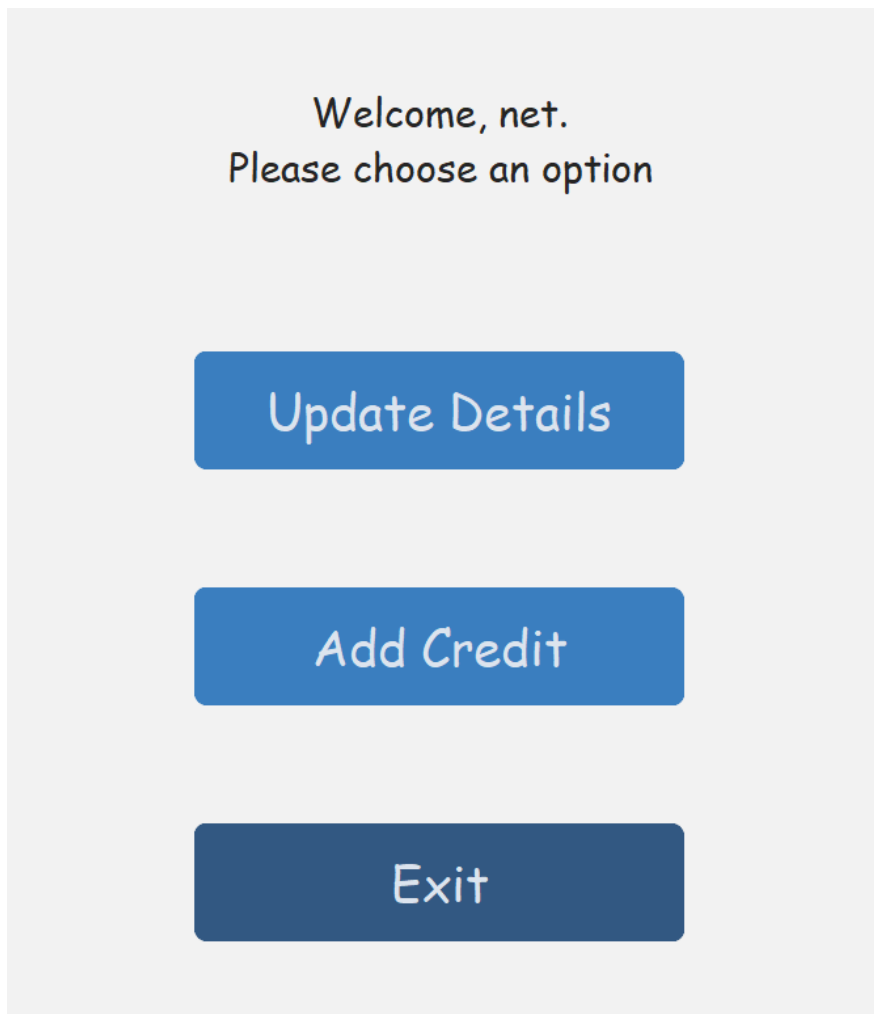
How much credit
would you like to add?

£20

£10

Exit

Figure 6.0



```
self._command()  
File "C:\Users\Kasutaja\Documents\THEproject\log_in.py", line 91, in exit  
    login_window.destroy()  
NameError: name 'login_window' is not defined
```

Figure 6.1

Welcome, net.
Please choose an option

Update Details

Add Credit

Exit

```
Process finished with exit code 0
```

Fig 7.0

	A	B	C	D	E	F	G	H
1	3	net@gma	net	1951	10	10	500	

New Name

Your current name is net

Anete

Confirm

	A	B	C	D	E	F	G
1	3	net@gma	Anete	1951	10	10	500
2	2	email	name	year	month	date	credit
3	3	test2@ma	test2	2021	2	2	20

Figure 8.0

	A	B	C	D	E	F	G
1	3	net@gma	Anete	1951	10	10	500
2	2	email	name	year	month	date	credit
3	3	test2@ma	test2	2021	2	2	20

New Email

Your current email is
net@gmail.com

nettest@gmail.com

Confirm

	A	B	C	D	E	F	G
1	3	nettest@gnet		1951	10	10	500
2	2	email	name	year	month	date	credit
3	3	test2@ma	test2	2021	2	2	20

Figure 9.0

	A	B	C	D	E	F	G
1	3	nettest@gnet		1951	10	10	500
2	2	email	name	year	month	date	credit
3	3	test2@ma	test2	2021	2	2	20

You have £500. ×



How much credit would you like to add?

£20
£10
Exit

	A	B	C	D	E	F	G
1	3	net@gma	net	1951	10	10	520
2	2	email	name	year	month	date	credit
3	3	test2@ma	test2	2021	2	2	20

Resolution 4.0

```
email_window = ctk.CTk()
email_window.title("New Email")
email_window.geometry("400x150")

coord = "B" + str(userNumber)
current = userBase[coord].value

currentTxt = ctk.CTkLabel(email_window, text=f"Your current email is \n{current}", font=font20)
```

I changed the geometry of the window to be slightly taller and larger and then set the email to be displayed on a new line in the label.

Resolution 6.0

```

1 usage
def user_editmenu(geometry):
    def exit():
        login_window.destroy()

    login_window = ctk.CTk()
    login_window.title("Login Window")
    login_window.geometry(geometry)

    Name = userBase["C" + str(userNumber)].value

```

I moved the function of exit into the user edit menu in which the login window is already existing so it should be defined, and an error should not occur.

Step 3: Login fail window

If the users face is not found, then the code will need to display a message to the user informing them that they have not been found

```

1 usage
def login():
    app.destroy() # close the previous window
    same = log_in.login_check()
    if same == True:
        log_in.user_editmenu(geometry)
    else:
        log_in.loginFail(geometry)
1 usage

```

First, if a match has not been found for the user's image, then it will need to run the login Fail function.

```

def loginFail(geometry):
    def exit():
        loginerror_window.destroy()
    loginerror_window = ctk.CTk()
    loginerror_window.title("Login Error Window")
    loginerror_window.geometry(geometry)

    label = ctk.CTkLabel(loginerror_window, text="There was an error with your log in.\nIf",
                        font=font20)
    label.pack(pady=50)

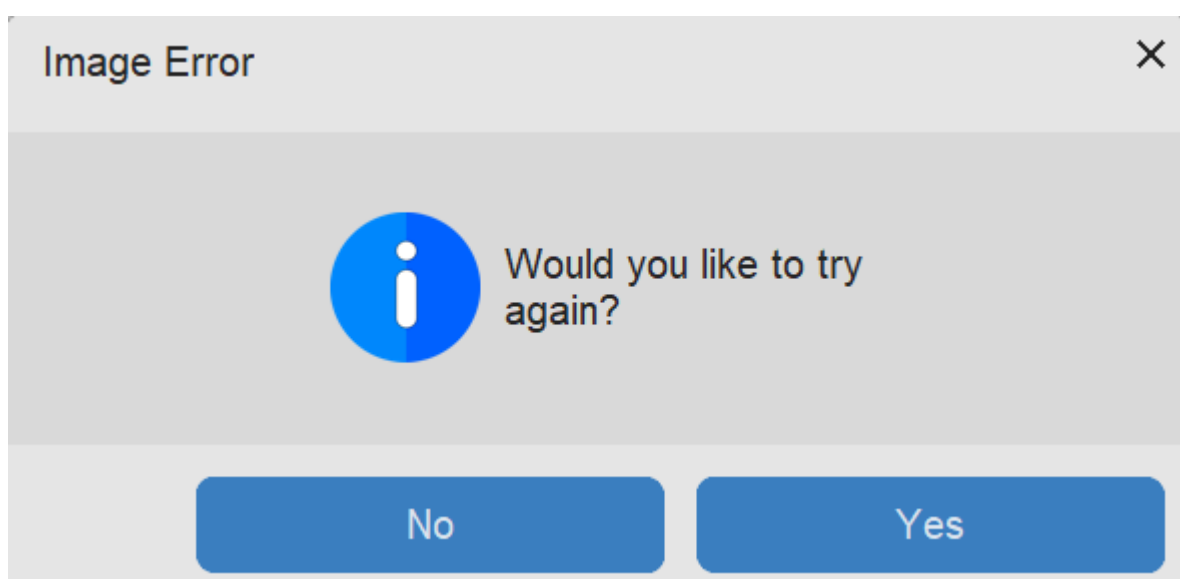
    exitButton = ctk.CTkButton(loginerror_window, text="Exit", command=exit,
                              height=60, width=250, font=font25)
    exitButton.pack(pady=30)
    loginerror_window.mainloop()

```

The login fail function will create a new tkinter window, displaying a message to the user that there was an error. At the bottom of the window there will be a button allowing the user to exit once they have read the full instructions. The button will call the exit function as its command and the exit function will close the login error window.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	If user image is not found, then the login error window opens	Login error window opens	Login error window opens	Pass		1.0
2.0	The exit button closes the window when it is clicked	Login error window closes	Login error window closes	Pass		2.0

Fig 1.0



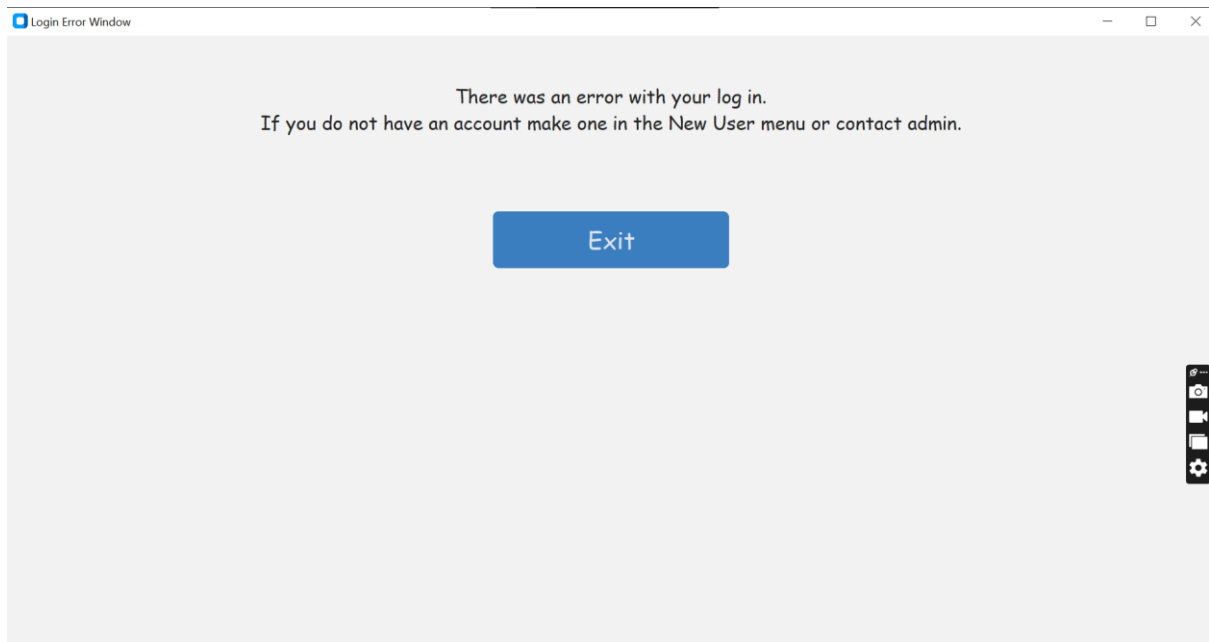
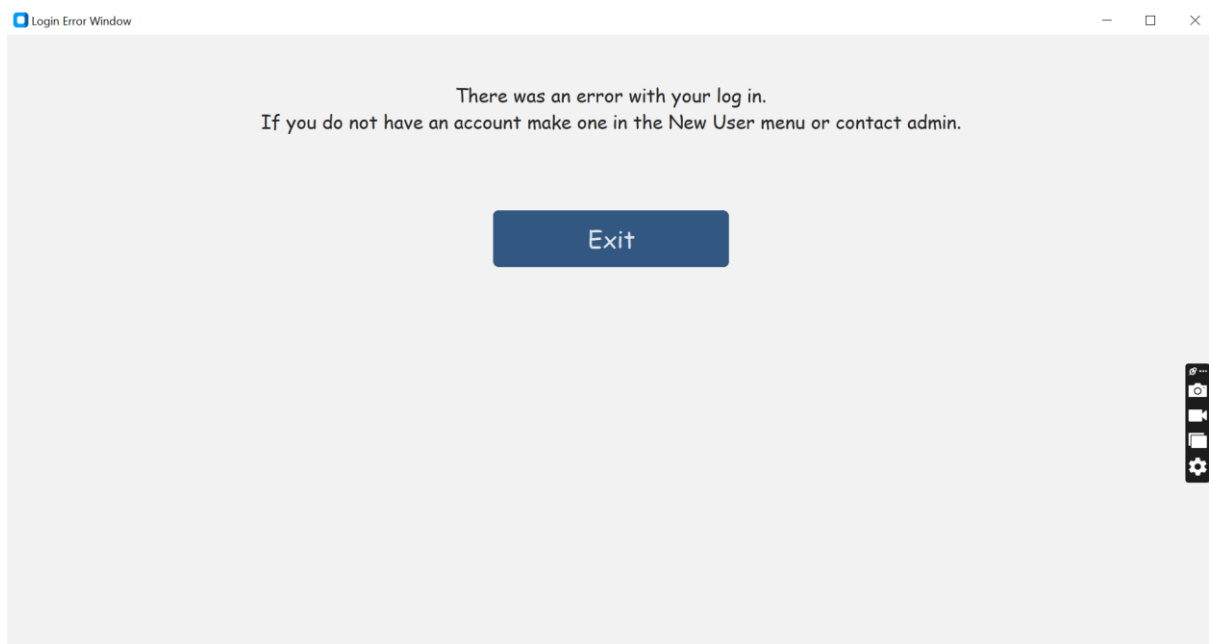


Fig 2.0



("after" script)

Process finished with exit code 0

Stage 4 Review

What's been developed:

In this stage, I created the log inside of the program, which allows users with existing accounts to log into their account and change any details that need to be changed. It logs the user in by scanning the users face and comparing the face to every face that is already on the database until it finds a match and then retrieves the details of the match and changes any of those if necessary.

How it was tested:

I tested each individual aspect of the code after I finished coding it by firstly describing what the code does and then checking the functionality of the code by running it and recording the outputs of the code and if that output is the desired outcome and what the desired outcome is.

How it meets my success criteria:

My program has met the success criteria of "Be able to retrieve details of a user on a database" when the program retrieves the details of users when logging in, displaying their name and displaying the current values of each field of that user before they change it.

It also meets the criteria "Ability to compare a face to an already stored face." when it loops through the images of the existing faces on the database and compares them to the image of the current user who the program is trying to detect.

It also meets the criteria of "Values on the database being altered depending on the user's actions." when the user uses the buttons available to the user to change their own details. I evidenced this in my user options menu step of this stage when I added screenshots showing the database before I made changes using the menu and after I made changes.

The criteria of "User interface which is easily navigate-able" is met in my solution since all buttons are clearly named for their functionality and all windows have instructions informing the user of what they should be doing. This is evidenced indirectly from the screenshots of the window that I have created, displaying the menus of the program.

The criteria "Values on the database being altered depending on the user's actions" has been met in this part of my code inside of the user login window, where users are able to change their details on the database. I have evidenced the change of details by displaying the details before the changes and the database after to show alterations made in screenshot evidence.

The criteria "Functional buttons throughout the program" has been met in my program since every button I have created in this section of my program has full correct functionality. The functionality assigned to the button is all working correctly upon clicking and I have demonstrated this by showing evidence in the form of screenshots as I click the button and then showing the output of clicking that button after it.

The criteria "Working entry boxes for user to enter details" has been met in my program inside of my log in menu where the user can change their name or email address and use entry boxes to enter the new value that they would like to change the current one to. I have evidenced that the entry boxes are working by showing screenshots of the data being entered into the box and then the same data being saved into the database.

Where it doesn't meet my success criteria so far:

- “Store facial encodings of users”
- “Correctly display user history in user history window”

Any changes from the design in this section:

I have slightly changed the design from the menu in my user interface again, by having all of the boxes all be in one vertical column instead of a grid layout since it is easier to use the pack method in tkinter than using the grid method since the grid method is a lot longer to use which would mean it would take longer to code for only a small difference in design.

I have also not included a user history section in the log in page since it would require too much work and too much storage for each user to have their own history section since it does not provide a lot for the user. When making the program larger and more complex for real life implementation I would probably integrate a history feature but for such small-scale usage and due to it being an OCR project I do not see this as a necessary part of my solution.

Where the project is so far:

So far this part of the project is close to completion, I will just need to add the administrator menu and then the Kiosk application will be finished. I still have all of the barrier application to be created.

Stage 5: Admin menu

Encase there is an error for a user putting in the wrong details, there will need to be an admin menu where administrators only can change a select users' details.

Step 1: Admin button

First, I will need to add a smaller button into the start menu which will be for admin only.

```
newUserButton = ctk.CTkButton(master=frame, text='New User?', command=newuser, f
                        width=300)
newUserButton.pack(pady=20, padx=15)

1 usage
def admin():
    print("Admin pressed")

adminButton = ctk.CTkButton(master=frame, text="Admin Only", command=admin)
adminButton.pack(pady=160)

app.mainloop()
```

After the new user button in the user menu, I created a new admin button which calls the command admin when clicked. The function admin for now prints "admin pressed" which will be called when the admin button is pressed.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Button shows up on the window	Button is on the menu window below the user button	Admin button is made below new user button	Pass		1.0
2.0	When button is pressed, "admin pressed" is printed	"Admin pressed" gets printed	"Admin pressed" is printed	Pass		2.0

Figure 1.0

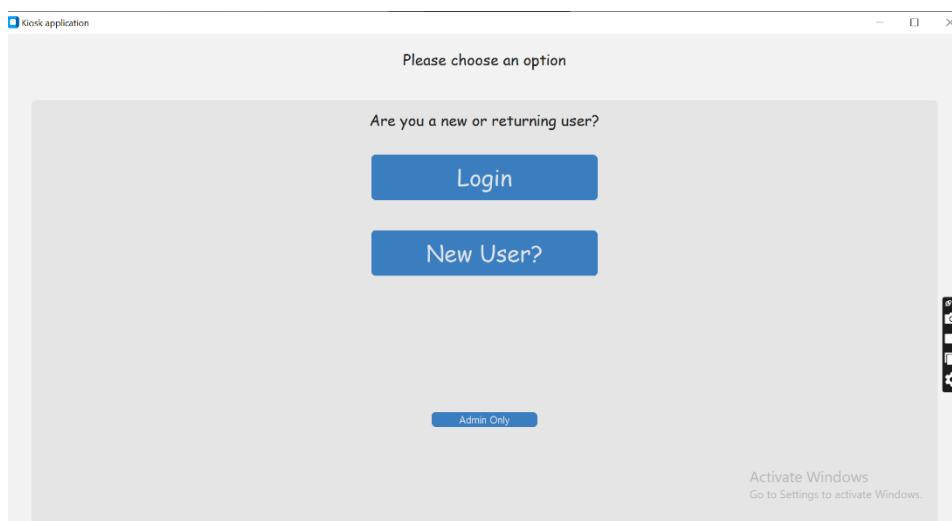
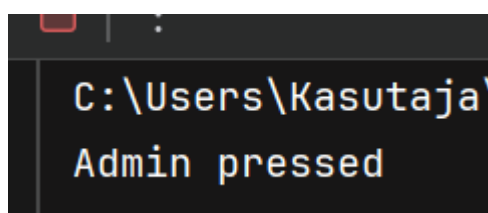
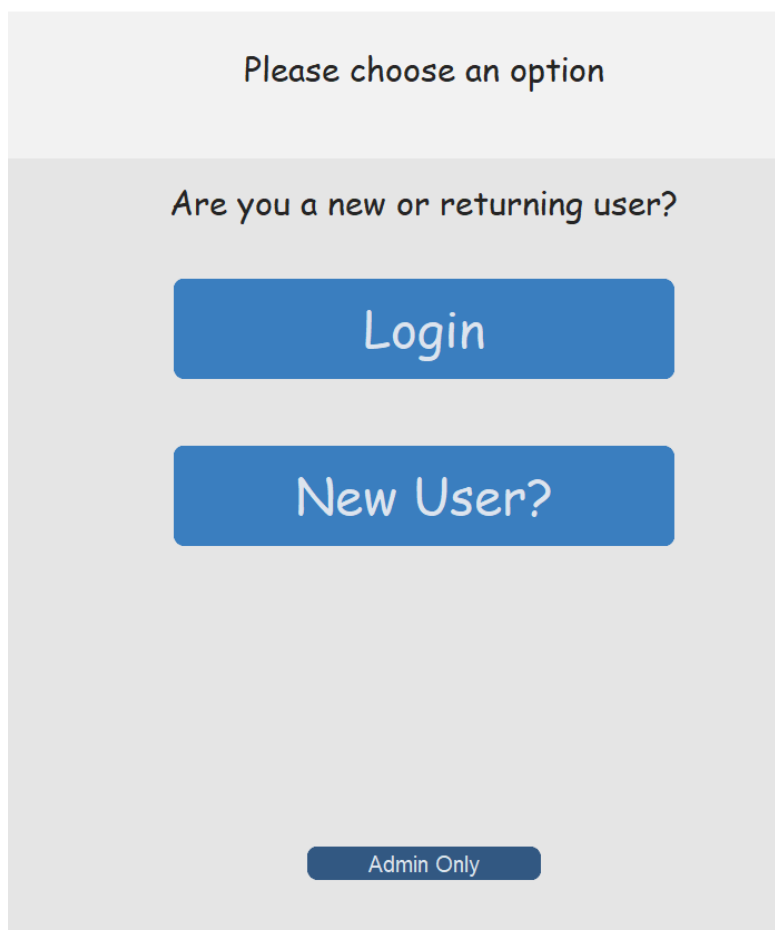


Fig 2.0



Step 2: Admin login

The first part of the admin function will be the admin login page where a password and username will need to be presented so that normal people cannot adjust other people's details without the correct allowance.

```
def admin():
    adminlog_window = ctk.CTk()
    adminlog_window.geometry("250x300")
    adminlog_window.title("Login Page")

    frame = ctk.CTkFrame(master=adminlog_window)
    frame.pack(pady=20, padx=40, fill='both', expand=True)

    label = ctk.CTkLabel(master=frame, text='Login Page')
    label.pack(pady=12, padx=10)

    user_entry = ctk.CTkEntry(master=frame, placeholder_text="Username")
    user_entry.pack(pady=12, padx=10)

    user_pass = ctk.CTkEntry(master=frame, placeholder_text="Password", show="*")
    user_pass.pack(pady=12, padx=10)
```

In a new module called admin, I created the admin function which creates a small window with the label Login page and two entry boxes with one having the text username and the other having the text password. When anything is written in the password entry field, all entries are shown with asterix instead of the actual letters inside to hide the password from any onlookers.

```
button = ctk.CTkButton(master=frame, text='Login', command=adminlogin)
button.pack(pady=12, padx=10)

def adminexit():
    adminlog_window.destroy()

exit_button = ctk.CTkButton(master=frame, text="Exit", command=adminexit)
exit_button.pack()
adminlog_window.mainloop()
```

Below the entry fields will be two buttons, one with the text login and the text exit. The exit button will call upon the command admin exit which will destroy the current admin window. The other will call the admin login function.

```
def adminlogin():
    username = user_entry.get()
    password = user_pass.get()

    if username == "admin" and password == "Password":
        CtkMessageBox(title="Login", message="Login Successful!",
                        icon="info", option_1="Okay")

    adminlog_window.destroy()
```

The admin login function will first get the values for the entry fields and store them in their respective variables. Then an if statement checking that the username is equal to admin and the password is equal to Password. If the if statement is true, then it will open a tkinter message box that will tell the user that the login is successful and when they press okay the box will close.

```
else:
    msg = CtkMessageBox(title="Detection Error!", message="Incorrect Password or Username",
                        icon="cancel", option_1="Exit", option_2="Retry")
    if msg.get() == "Exit":
        adminlog_window.destroy()
```

If the values are not correct, the if statement will enter into the else statement. The else statement will open a message box that will tell the user that the password or username is incorrect and give two options to retry or exit. If the user chooses exit, then it will destroy the window but if the user chooses to retry it will only close the message box.

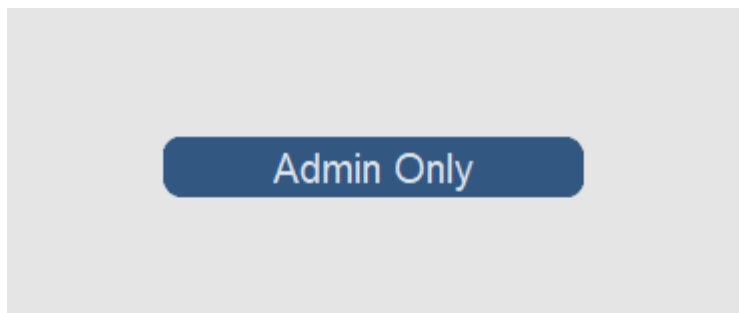
```
adminButton = ctk.CtkButton(master=frame, text="Admin Only", command=admin.admin)
adminButton.pack(pady=160)
```

I changed the button to called the command admin.admin and got rid of the admin command in the main kiosk module.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Admin button opens up the admin menu	Admin menu opens	Admin menu opened	Pass		1.0
2.0	When correct details entered and the login button is pressed, it opens a custom tkinter message box and closes the menu	Custom tkinter box opens and menu closes	Tkinter message box opens and the menu closes	Pass		2.0
3.0	When the incorrect details are entered and	Error message box opens	Error message box opened	Pass		3.0

	the login button is pressed, the tkinter box will say there is an error					
4.0	When "exit" is pressed on the message box, the admin window is closed	Admin window closes	Admin window closed	Pass		4.0
5.0	When the exit button is pressed in the admin menu, the admin window closes	Admin window closes	Admin window closes	Pass		5.0

Fig 1.0



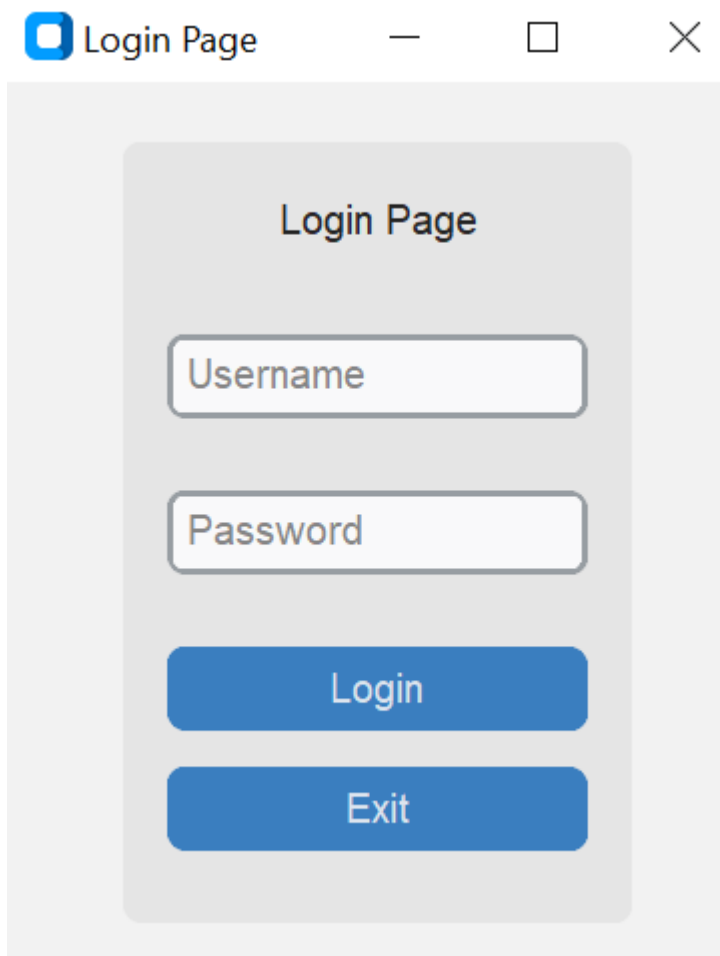


Fig 2.0

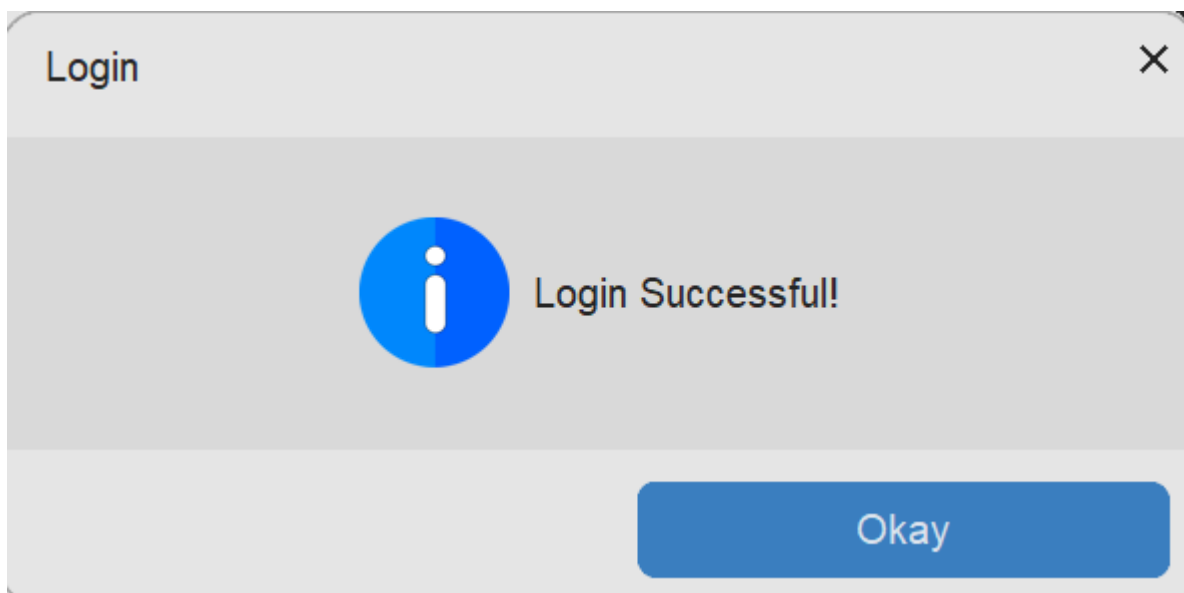
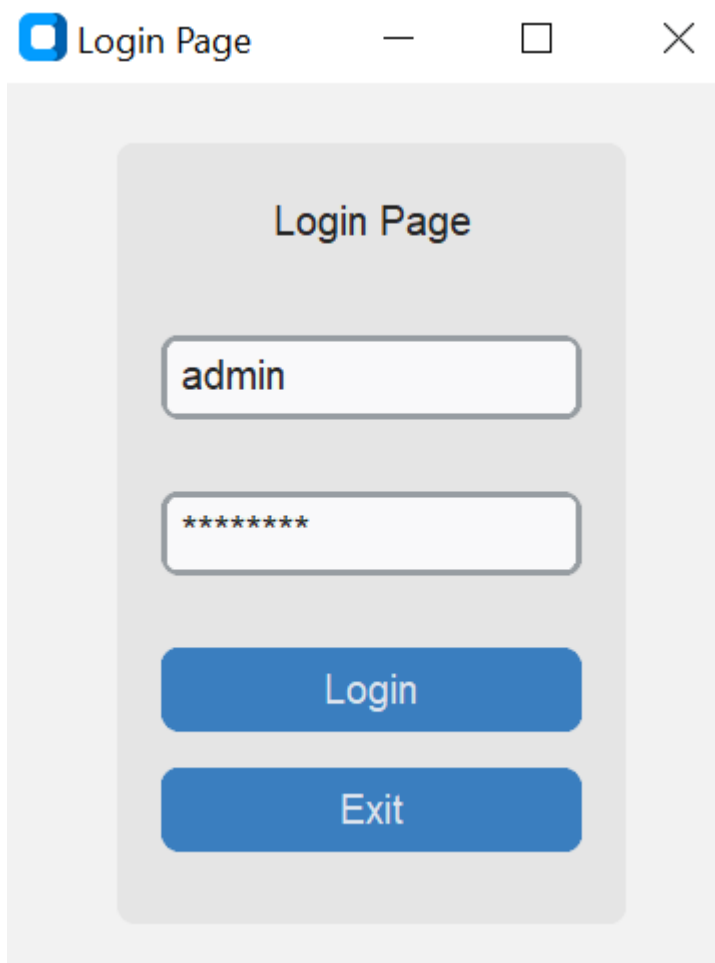


Fig 3.0

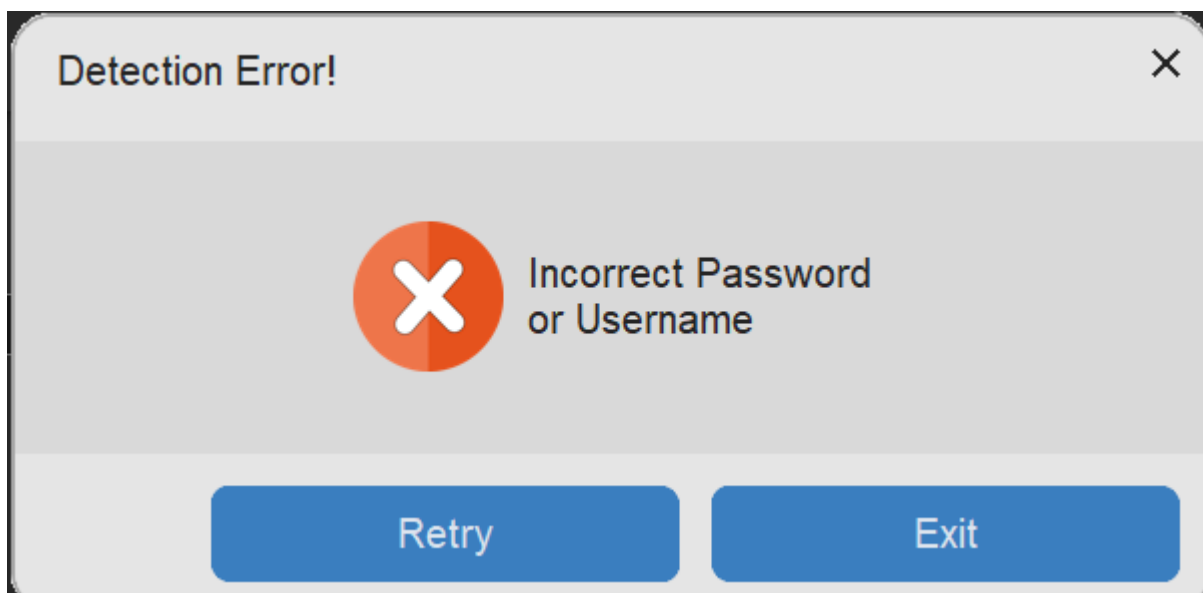
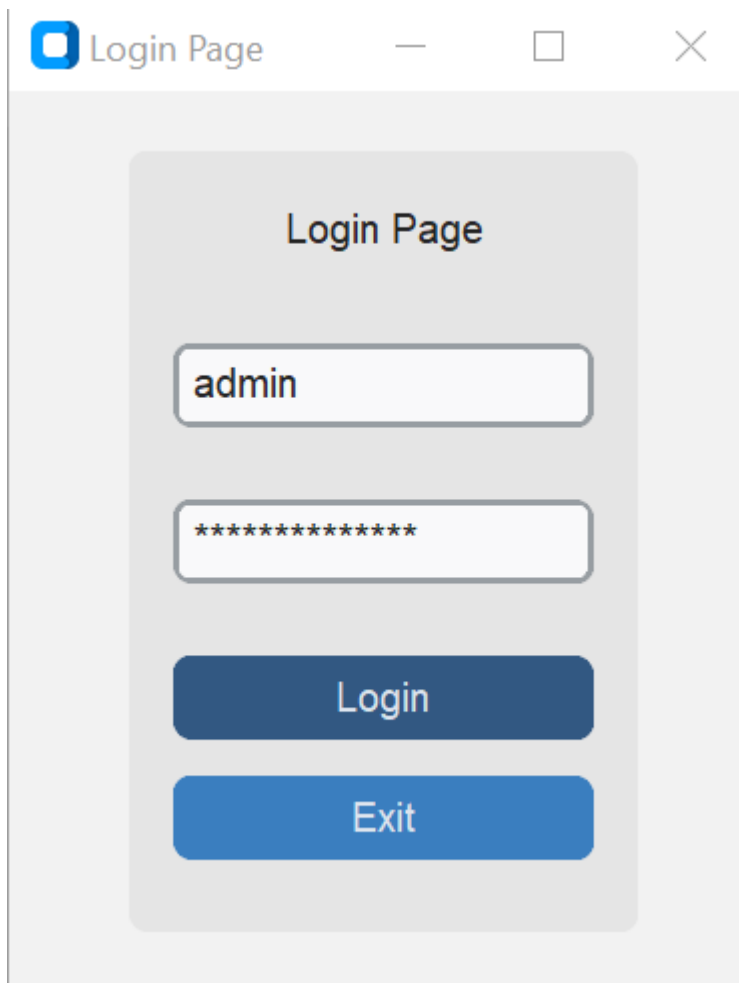


Fig 4.0

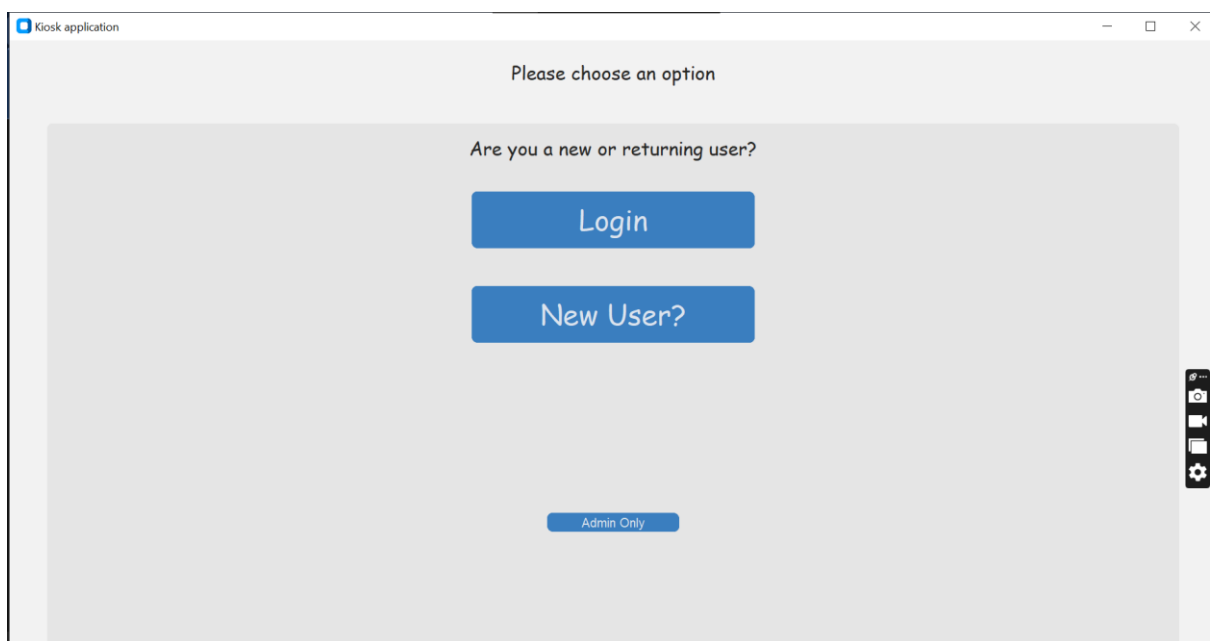
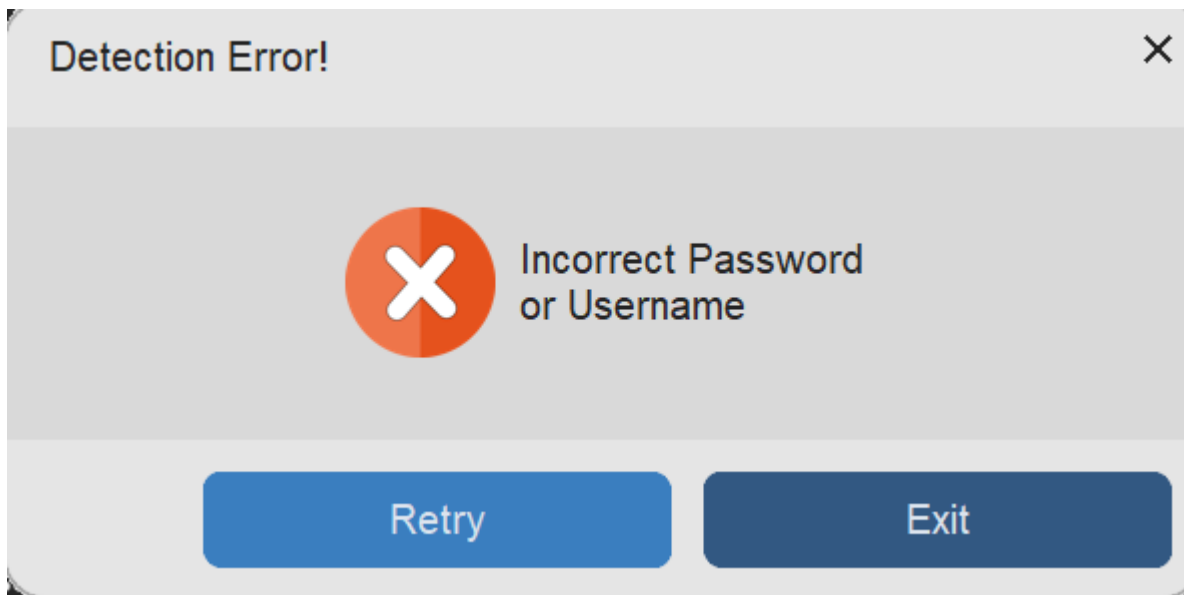
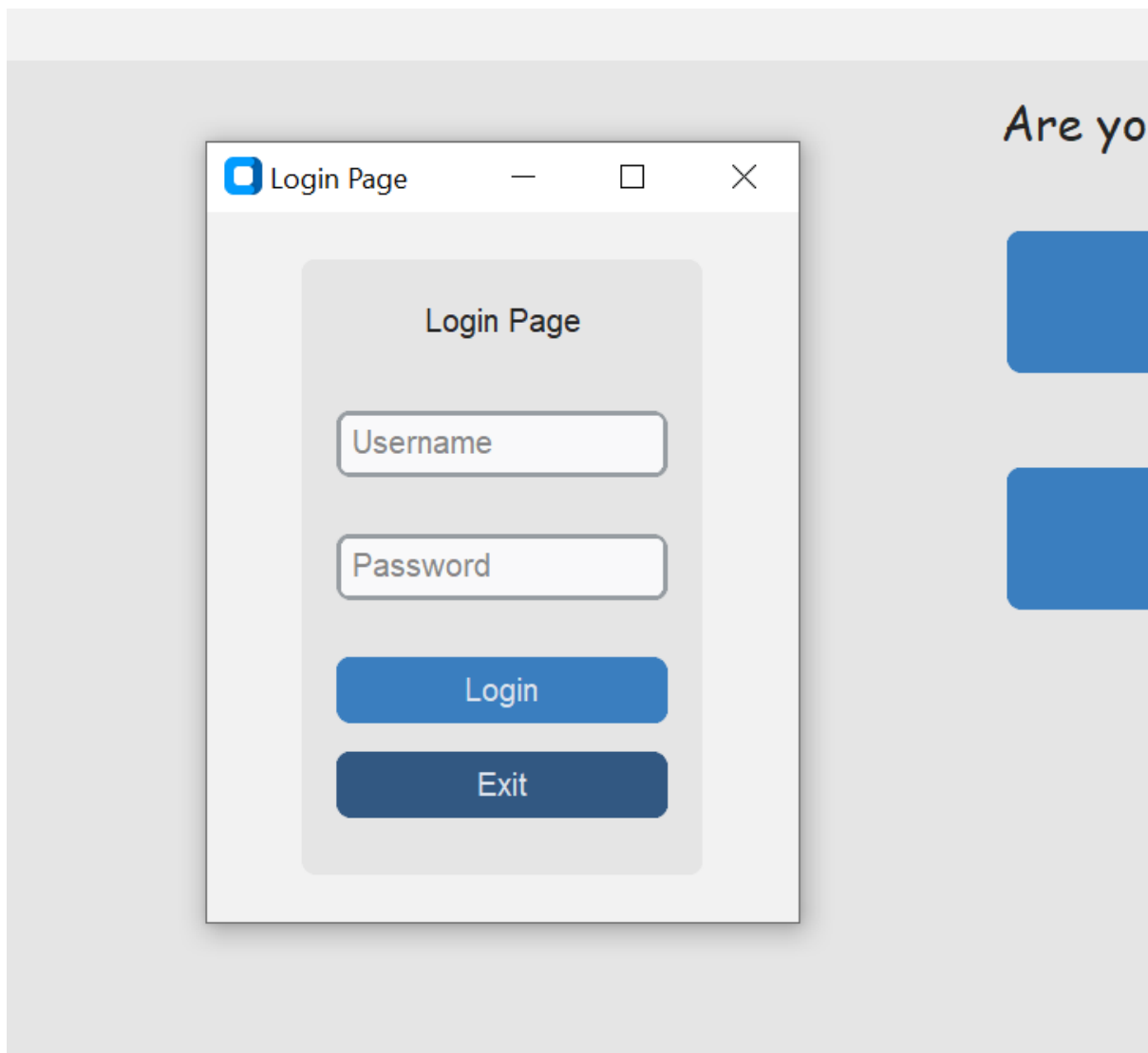
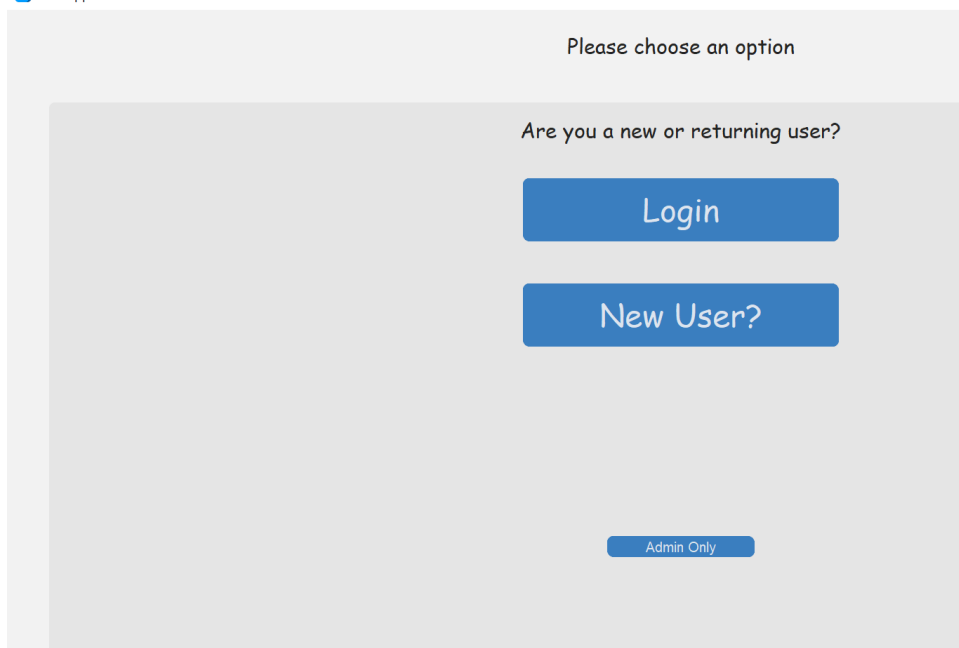


Fig 5.0



Kiosk application



Step 3: Admin page menu

Once the administrator has logged in, they will need to access a variety of options to choose what they would like to change from the user.

```
if username == "admin" and password == "Password":
    CtkMessageBox(title="Login", message="Login Successful!",
                  icon="info", option_1="Okay")

    adminlog_window.destroy()

    admin_page = ctk.CTk()
    admin_page.geometry("500x500")
    admin_page.title("Admin window")

    frame = ctk.CTkFrame(master=admin_page)
    frame.pack(pady=20, padx=40, fill='both', expand=True)
```

Under the login successful message box, I created a new admin window with a square geometry and created a frame for that window.

```
adminEmailButton = ctk.CTkButton(master=frame, text='Email', command=adminEmail)
adminEmailButton.pack(pady=12, padx=10)

adminNameButton = ctk.CTkButton(master=frame, text='Name', command=adminName)
adminNameButton.pack(pady=12, padx=10)

adminDateButton = ctk.CTkButton(master=frame, text='Date', command=adminDate)
adminDateButton.pack(pady=12, padx=10)

adminCreditButton = ctk.CTkButton(master=frame, text='Credit', command=adminCredit)
adminCreditButton.pack(pady=12, padx=10)

adminExitButton = ctk.CTkButton(master=frame, text='Exit', command=adminPageExit)
adminExitButton.pack(pady=12, padx=10)

admin_page.mainloop()
```

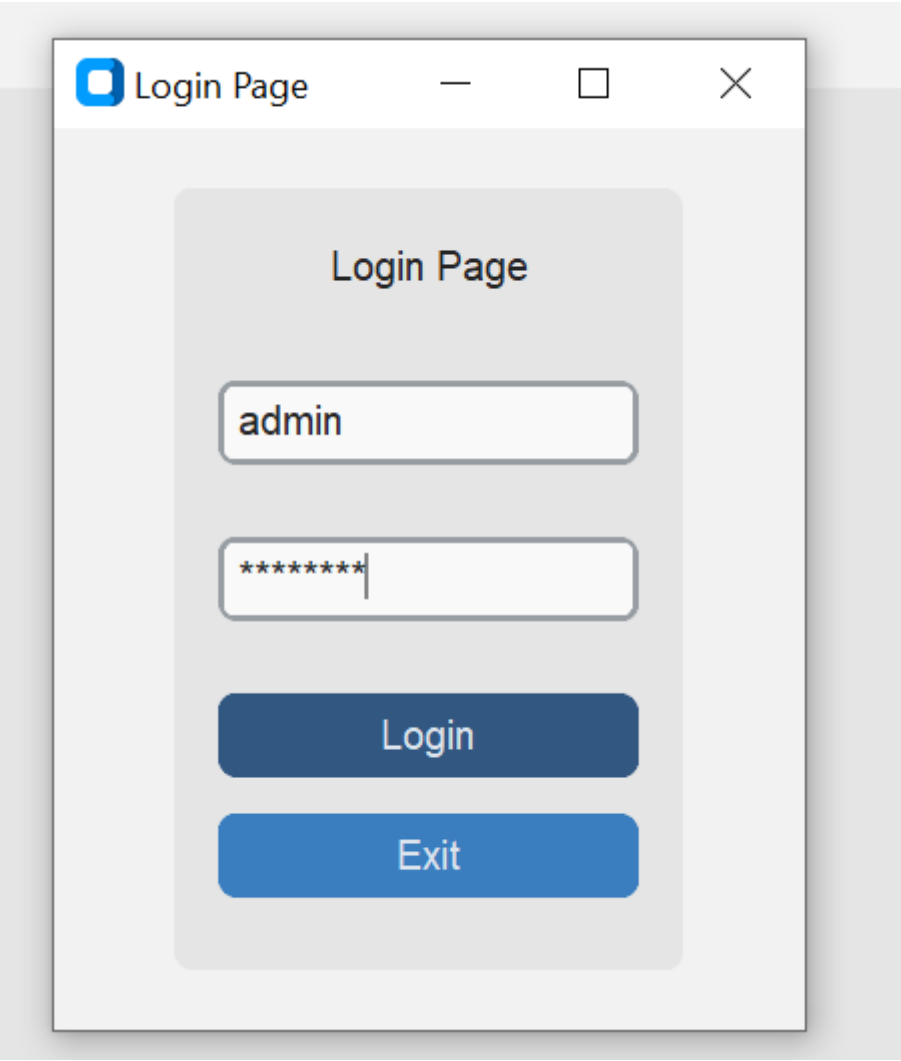
Inside of the frame I created 5 buttons, an email button, a name button, a date button, a credit button and an exit button. Each of these buttons will have functions which will be the commands for the respective buttons. First, I will make an exit button so that the user can close that window.

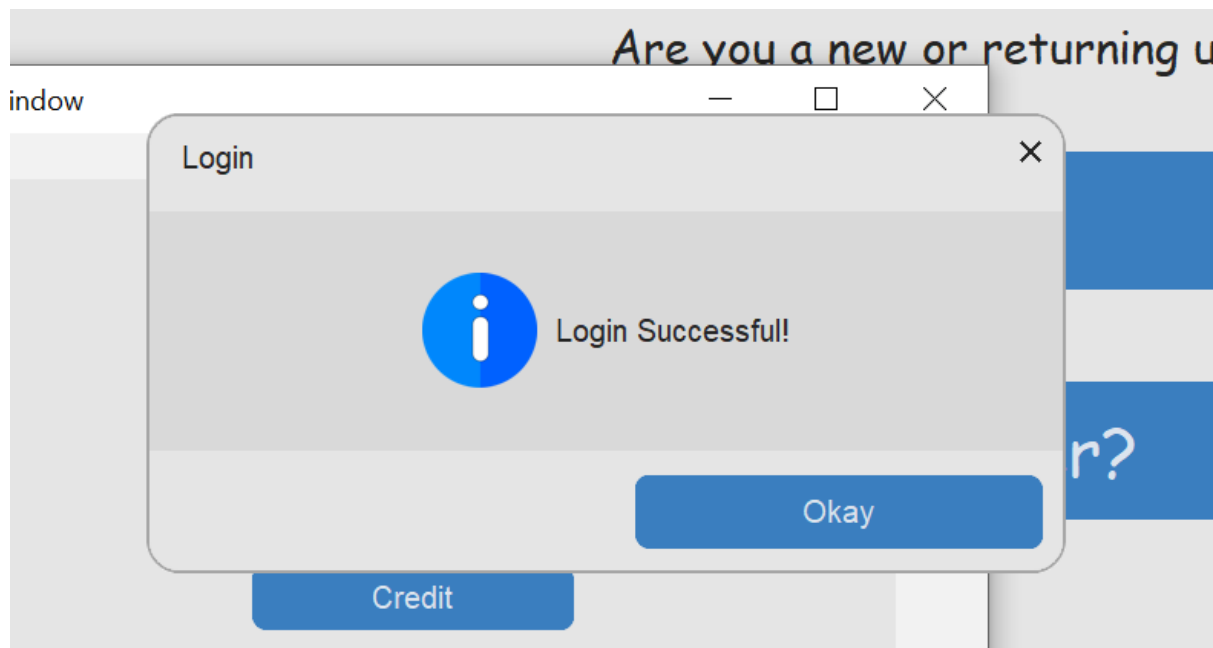
```
def adminPageExit():  
    admin_page.destroy()  
    admin()
```

For the admin page exit function, first I will destroy the admin page window. After the admin page window is destroyed, I will recall the admin function so that the login window opens again encase the certain administrator wants to log in again or a different administrator wants to log in.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Admin page opens when login is correct	Admin page opens	Admin page opened	Pass		1.0
2.0	Admin page exit button	Admin page closes	Admin page closed	Pass		2.0

Fig 1.0





Kiosk application

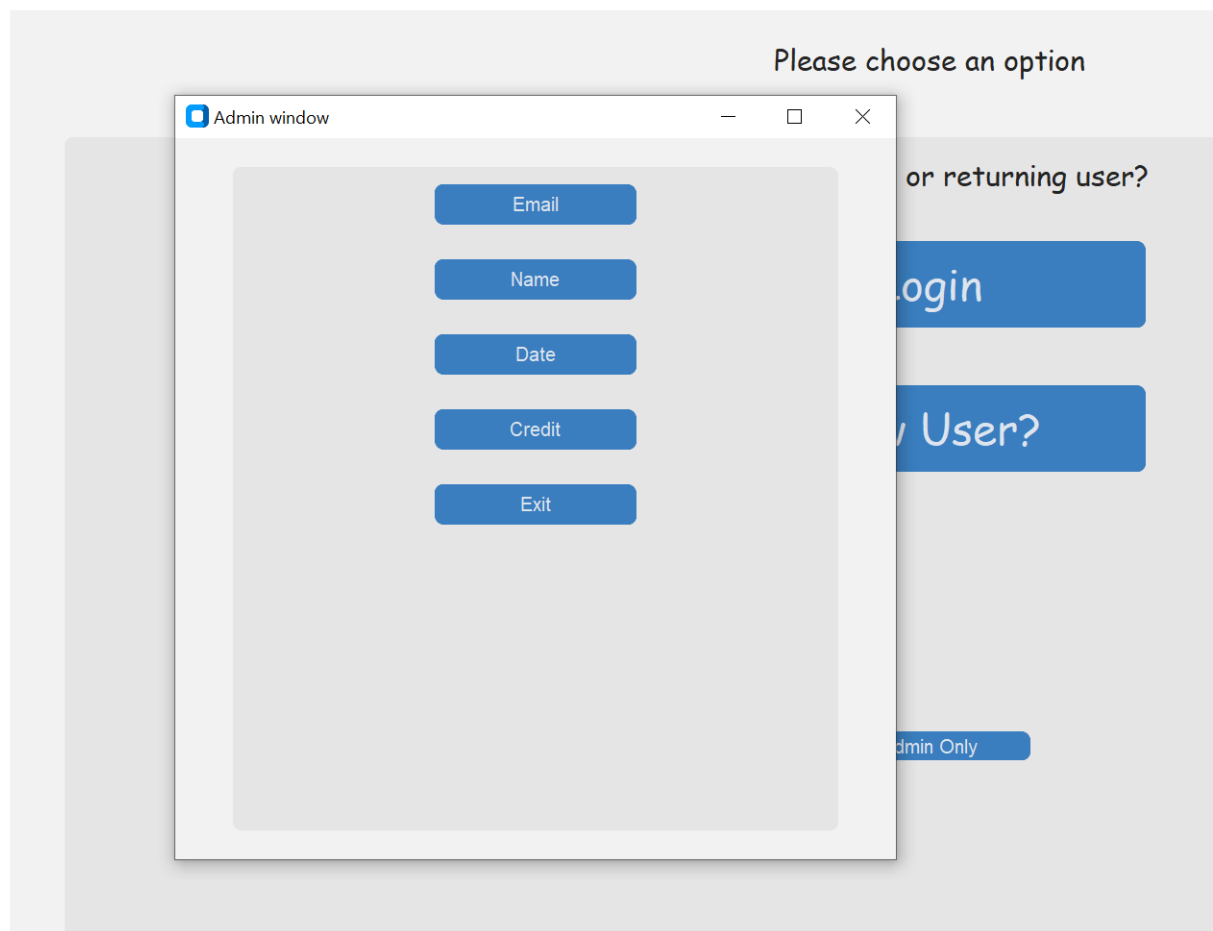
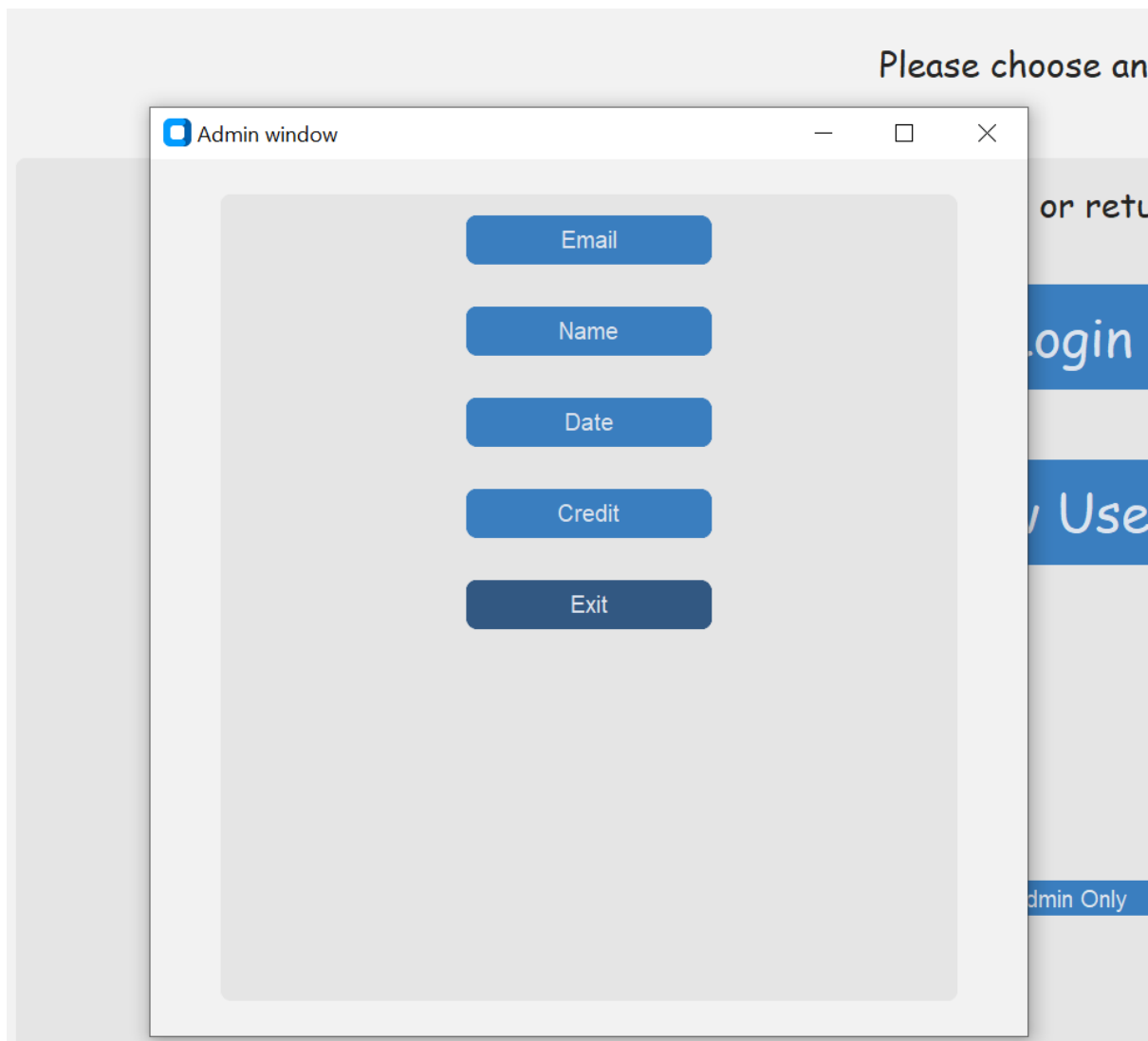
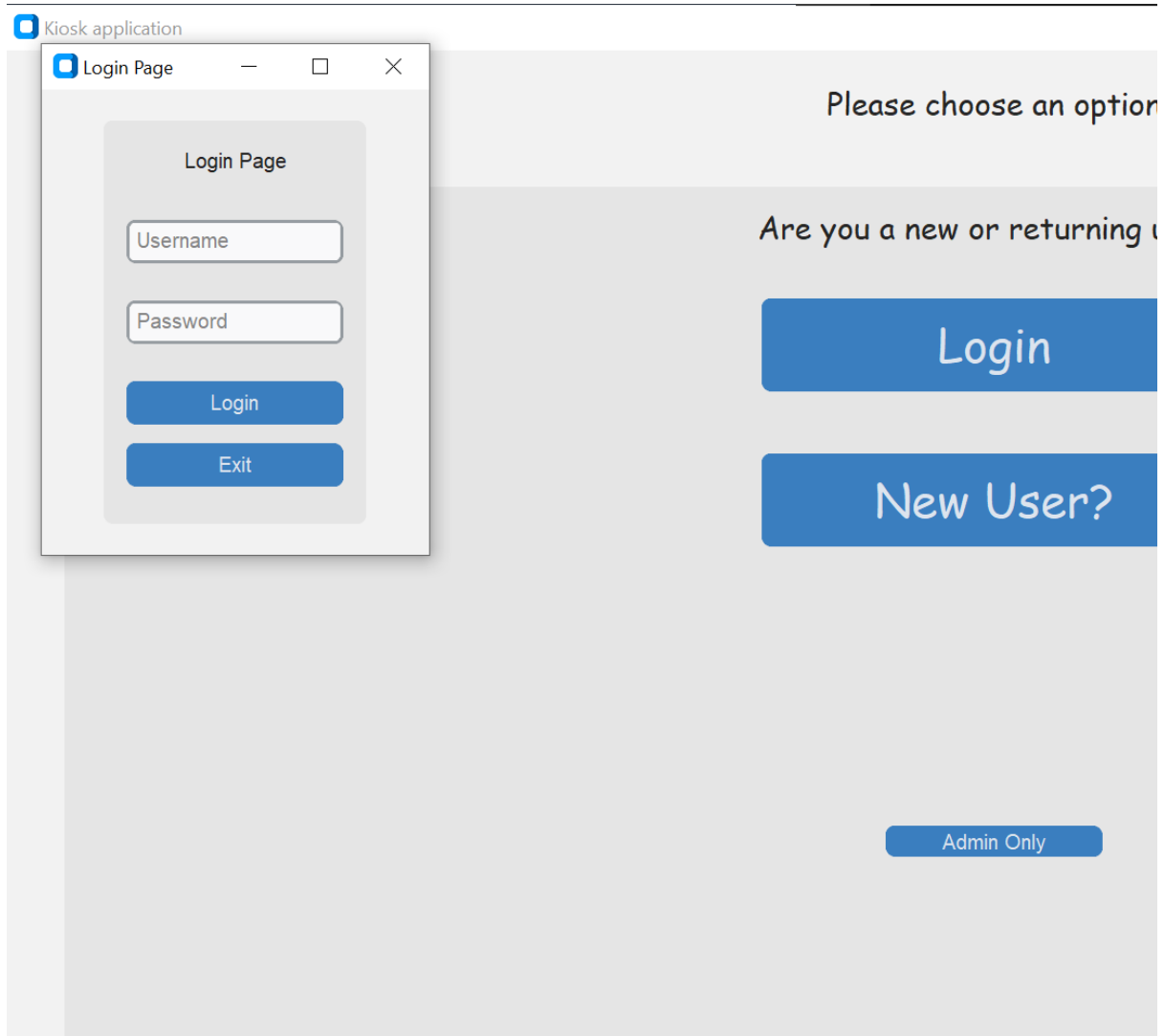


Fig 2.0





Step 4: Admin email option

Next, I will create the functions for the email button.

```
def adminEmail():  
    adminEmail_page = ctk.CTk()  
    adminEmail_page.geometry("500x500")  
    adminEmail_page.title("Admin Email Page")  
  
    frame = ctk.CTkFrame(master=adminEmail_page)  
    frame.pack(pady=20, padx=40, fill='both', expand=True)  
  
    userCount_entry = ctk.CTkEntry(master=frame, placeholder_text="User ID number")  
    userCount_entry.pack(pady=12, padx=10)  
  
    email_entry = ctk.CTkEntry(master=frame, placeholder_text="New Email Address")  
    email_entry.pack(pady=12, padx=10)
```

First I created the admin email function, starting with creating a window for the admin email page. I created a frame for inside of that window and then created two entry fields. The first entry field is for

the administrator to input the target users ID number. The second entry field is for the new email address that the current one needs to be changed to.

```
doneButton = ctk.CTkButton(master=frame, text="Done", command=done)
doneButton.pack(pady=12, padx=10)

exitButton = ctk.CTkButton(master=frame, text="Exit", command=emailExit)
exitButton.pack(pady=12, padx=10)

adminEmail_page.mainloop()
```

Below that, I created two buttons, a done button and an exit button.

```
def done():
    if not re.match(pattern: r"^[^@]+@[^@]+\.[^@]+", email_entry.get()):
        CTkMessagebox(title="Error", message="Invalid Entry!", icon="cancel")
    else:
        userID = userCount_entry.get()
        coord = "B" + str(userID)

        userBase[coord] = email_entry.get()

        CTkMessagebox(title="Email", message=f"User {userID} email successfully changed",
                        icon="info", option_1="Okay")
        wb.save('userbase.xlsx')
```

Upon clicking the "Done" button, the done function will run. First the function will validate the email entry to contain at least one @ symbol and one full stop and the full stop cannot be after the @ symbol. If the entry does not meet the validation requirements, then the user will be presented with an error message box. If the user entry meets the validation requirements, then it will save the entry field into the correct box for the user and display a message box telling the user that the email was successfully changed. After the if statement it will save the workbook.

```
def emailExit():
    adminEmail_page.destroy()
```

Upon clicking the "Exit button" the email exit function will be called. When it is called, the email admin page will be destroyed, and the user will be back to seeing the admin menu.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Email button	Upon clicking, it will open the email change menu	The email menu opens	Pass		1.0
2.0	Exit button in email menu	Upon clicking, it will close the email change menu and take the user back to the admin menu	Email menu closes and the admin menu is displayed	Pass		2.0
3.0	Done button in email menu	Upon clicking, it will correctly save the changes made to the database	The database is correctly altered	Pass		3.0
4.0	Email Validation	When an incorrect email is entered it will display validation error to the user.	The error message box opens	Pass		4.0

Fig 1.0

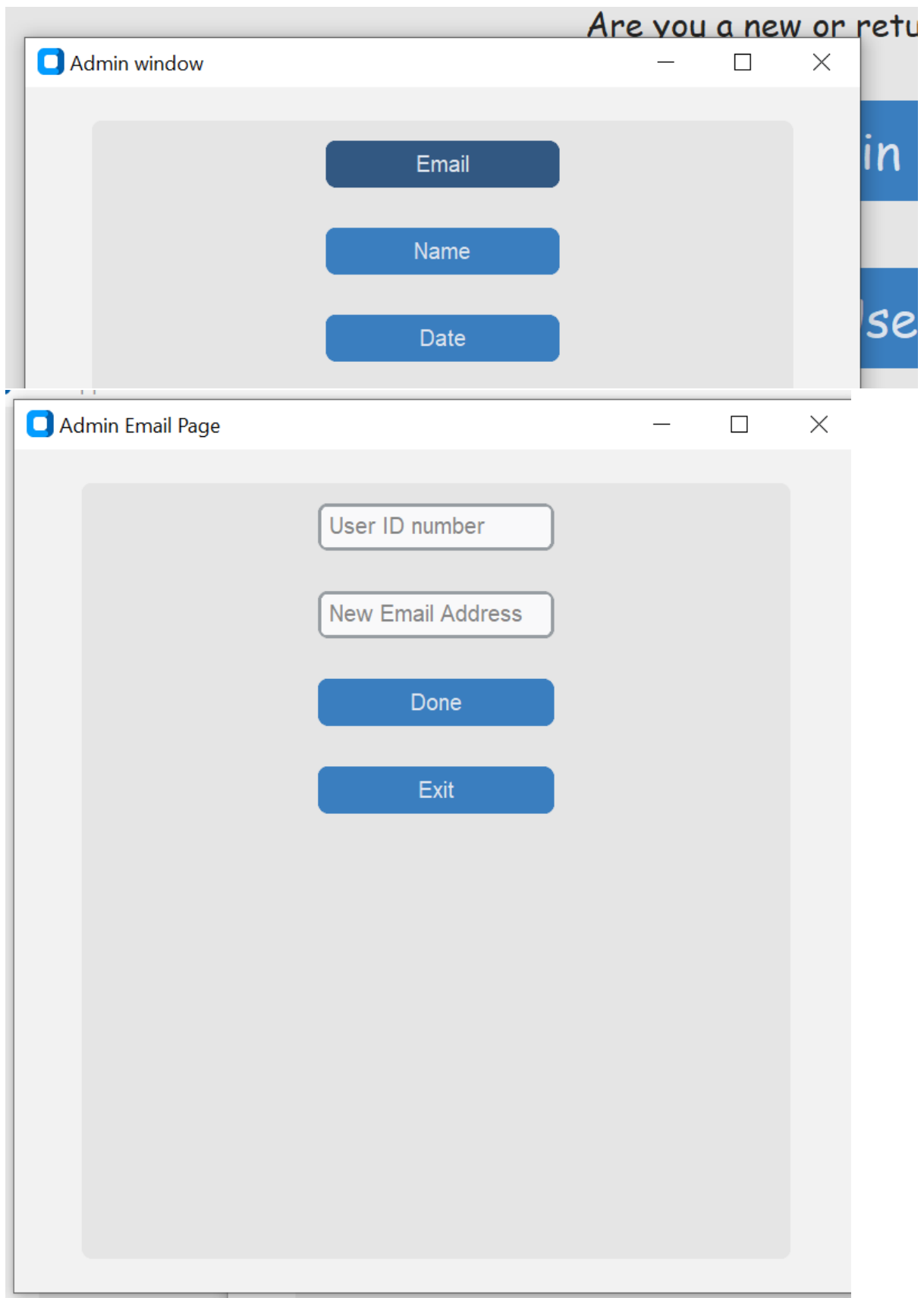
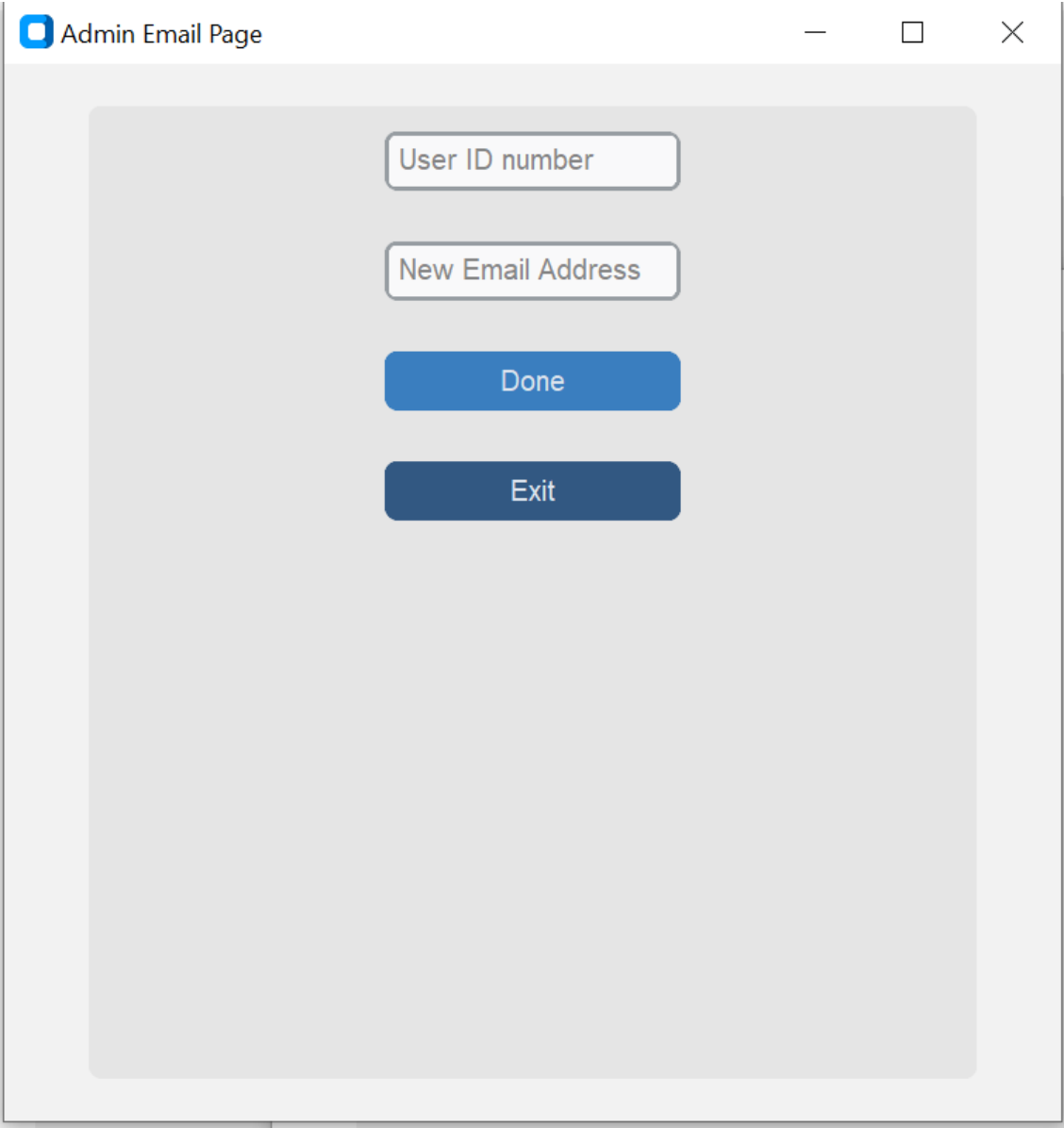


Fig 2.0



The image shows a window titled "Admin Email Page" with standard window controls (minimize, maximize, close). The main content area is a light gray rectangle containing four vertically stacked elements: a text input field labeled "User ID number", another text input field labeled "New Email Address", a blue button labeled "Done", and a dark blue button labeled "Exit".

Admin Email Page

User ID number

New Email Address

Done

Exit

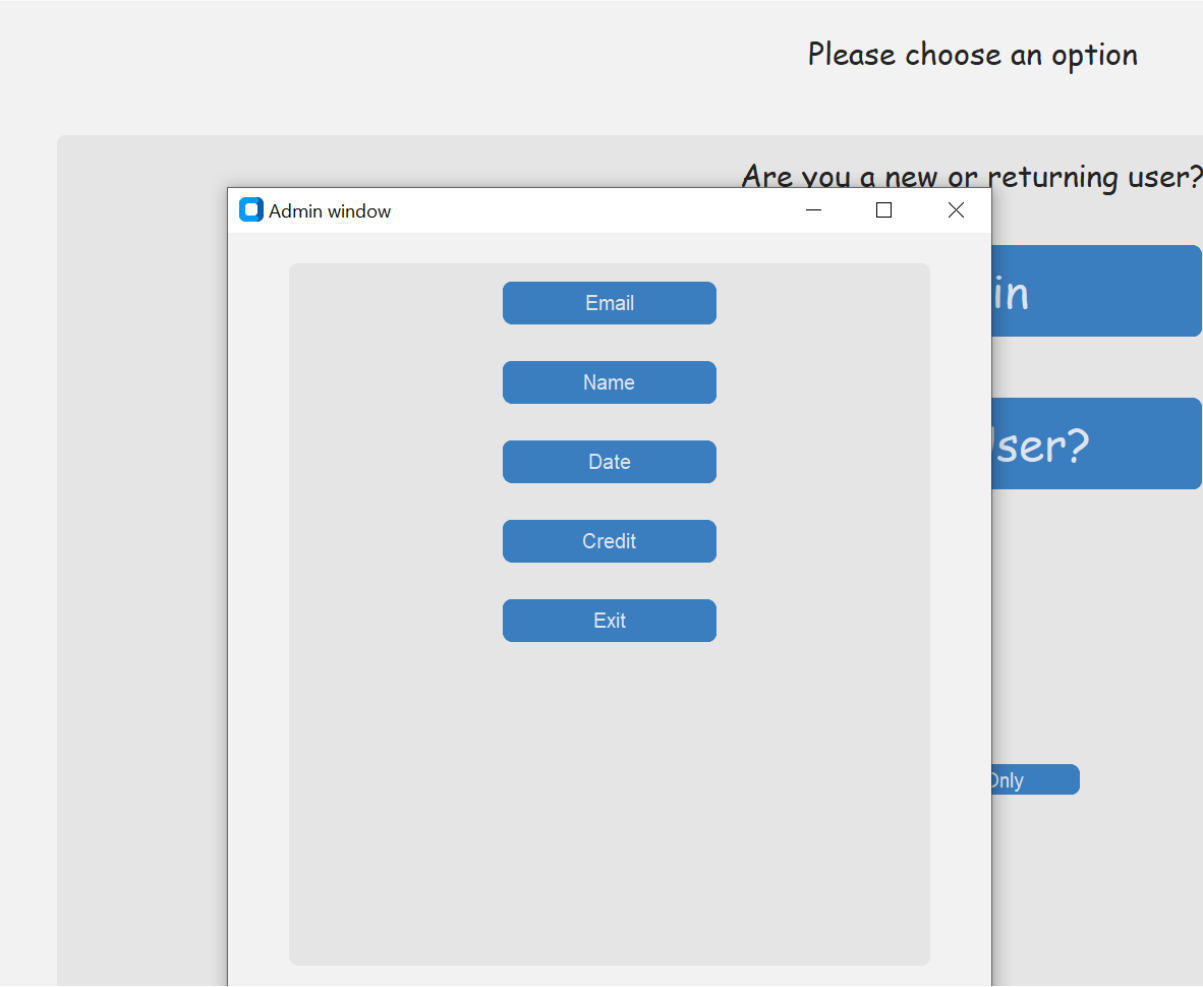


Fig 3.0

	A	B	C	D	E	F	G	H
1		3	net@gma	net	1951	10	10	520
2		2	email	name	year	month	date	credit
3		3	test2@ma	test2	2021	2	2	20
4								
5								

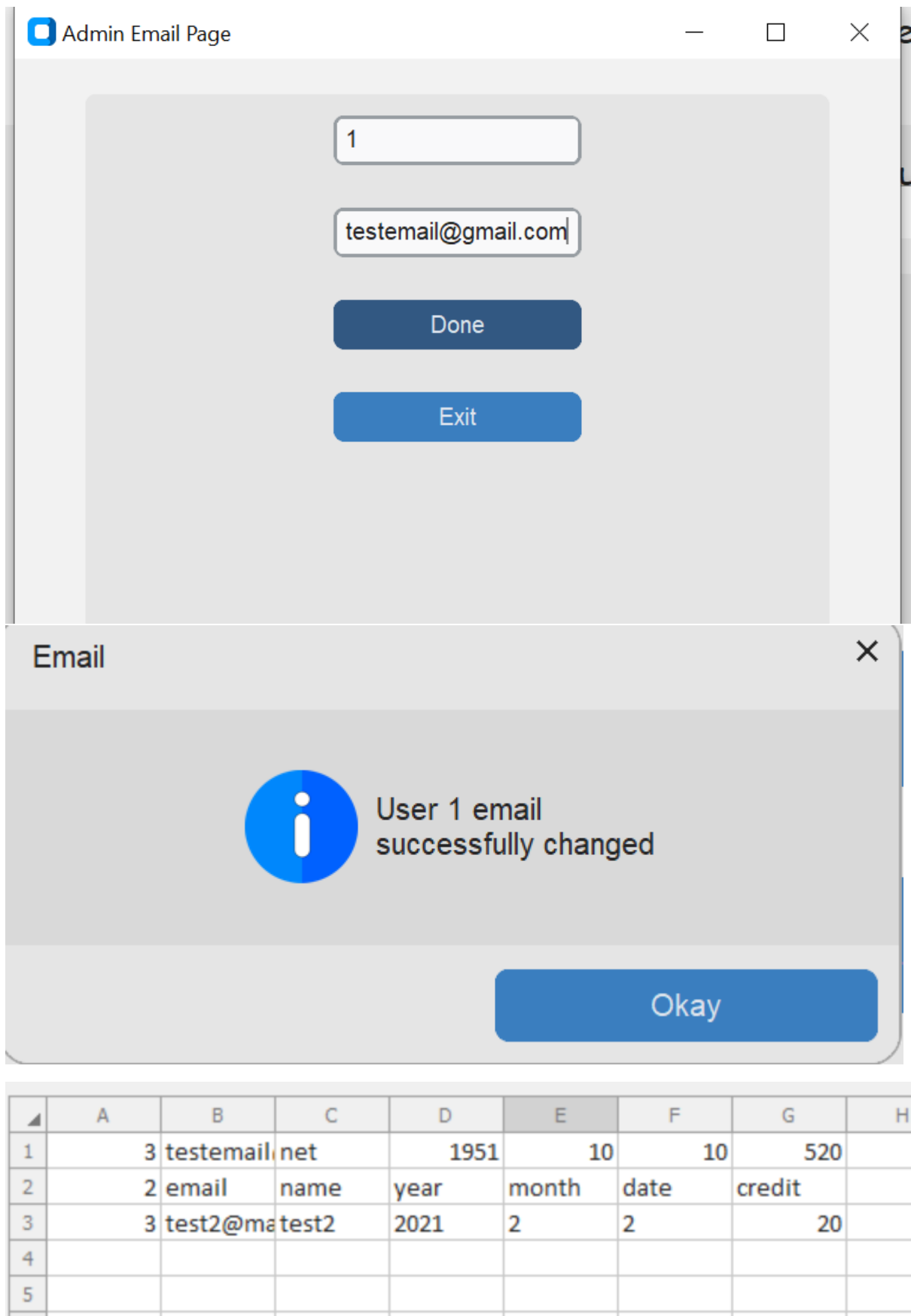


Fig 4.0

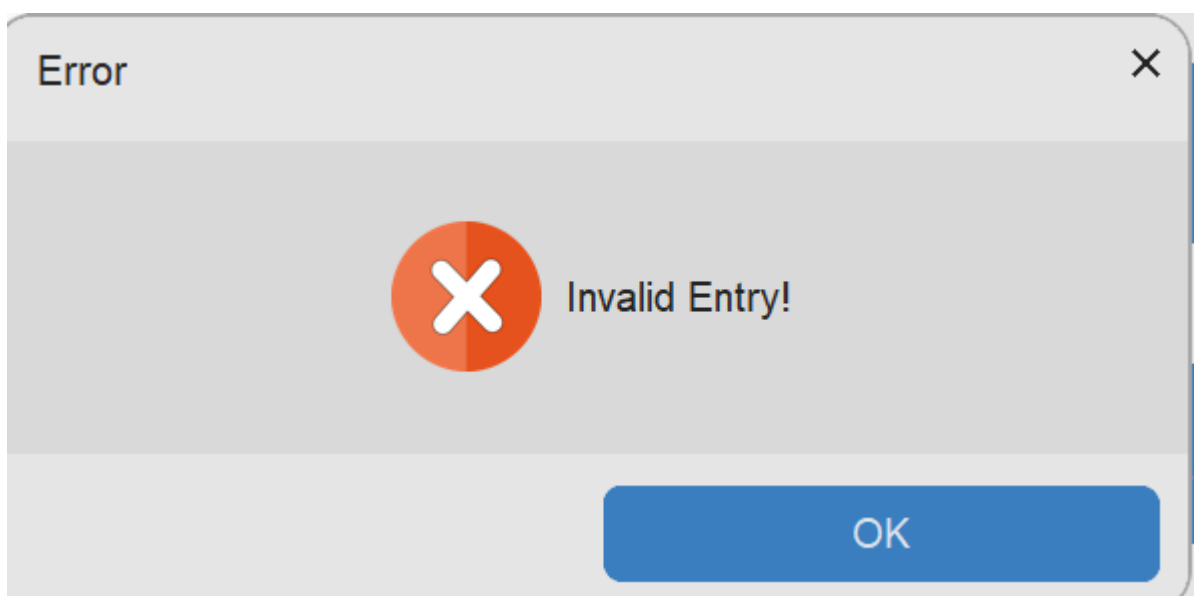
Admin Email Page

1

email.com

Done

Exit



Step 5: Admin name option

Next, I will create the function for the name button.

```
def adminName():  
    adminName_page = ctk.CTk()  
    adminName_page.geometry("500x500")  
    adminName_page.title("Admin Name Page")  
  
    frame = ctk.CTkFrame(master=adminName_page)  
    frame.pack(pady=20, padx=40, fill='both', expand=True)  
  
    userCount_entry = ctk.CTkEntry(master=frame, placeholder_text="User ID number")  
    userCount_entry.pack(pady=12, padx=10)  
  
    name_entry = ctk.CTkEntry(master=frame, placeholder_text="New Name")  
    name_entry.pack(pady=12, padx=10)
```

First I will initialise the name window and create a frame to store the contents inside. Next, I created two entry fields, one again for the user ID and the next for the name variable that the current name will be changed to.

```
doneButton = ctk.CTkButton(master=frame, text="Done", command=done)  
doneButton.pack(pady=12, padx=10)  
  
exitButton = ctk.CTkButton(master=frame, text="Exit", command=emailExit)  
exitButton.pack(pady=12, padx=10)  
  
adminName_page.mainloop()
```

I recreated a done button and an exit button again.

```
def done():  
    userID = userCount_entry.get()  
    coord = "C" + str(userID)  
  
    userBase[coord] = name_entry.get()  
  
    CtkMessageBox(title="Name", message=f"User {userID} name successfully changed",  
                  icon="info", option_1="Okay")  
    wb.save("userbase.xlsx")
```

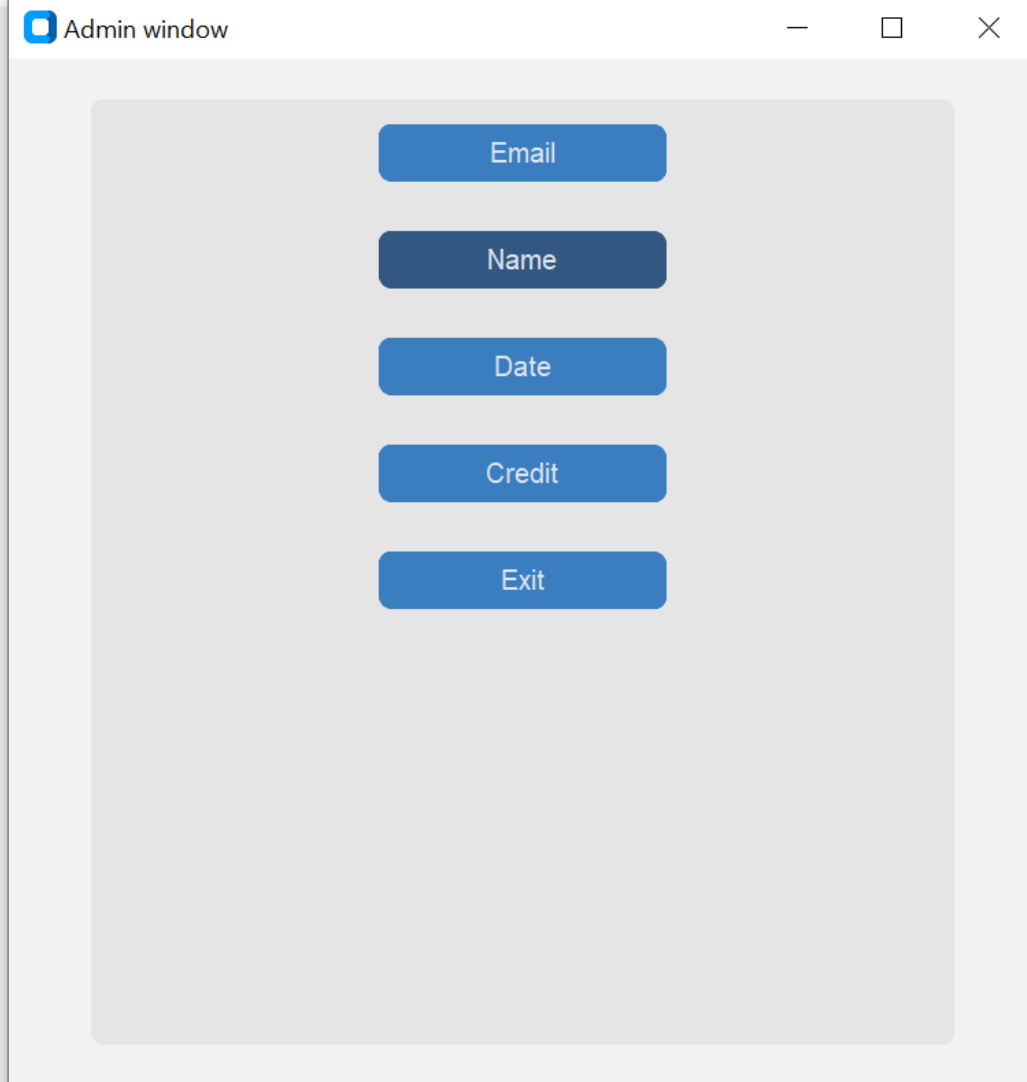
The function for the done button will change the email entry into the right place on the database and then display a message box informing the user that the name has been changed.


```
def emailExit():
    adminName_page.destroy()
```

The email exit function will destroy the name page.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Name button	Upon clicking, it will open the name change menu	The name menu opens	Pass		1.0
2.0	Exit button in name menu	Upon clicking, it will close the name change menu and take the user back to the admin menu	Name menu closes and the admin menu is displayed	Pass		2.0
3.0	Done button in name menu	Upon clicking, it will correctly save the changes made to the database	The database is correctly altered	Pass		3.0

Fig 1.0



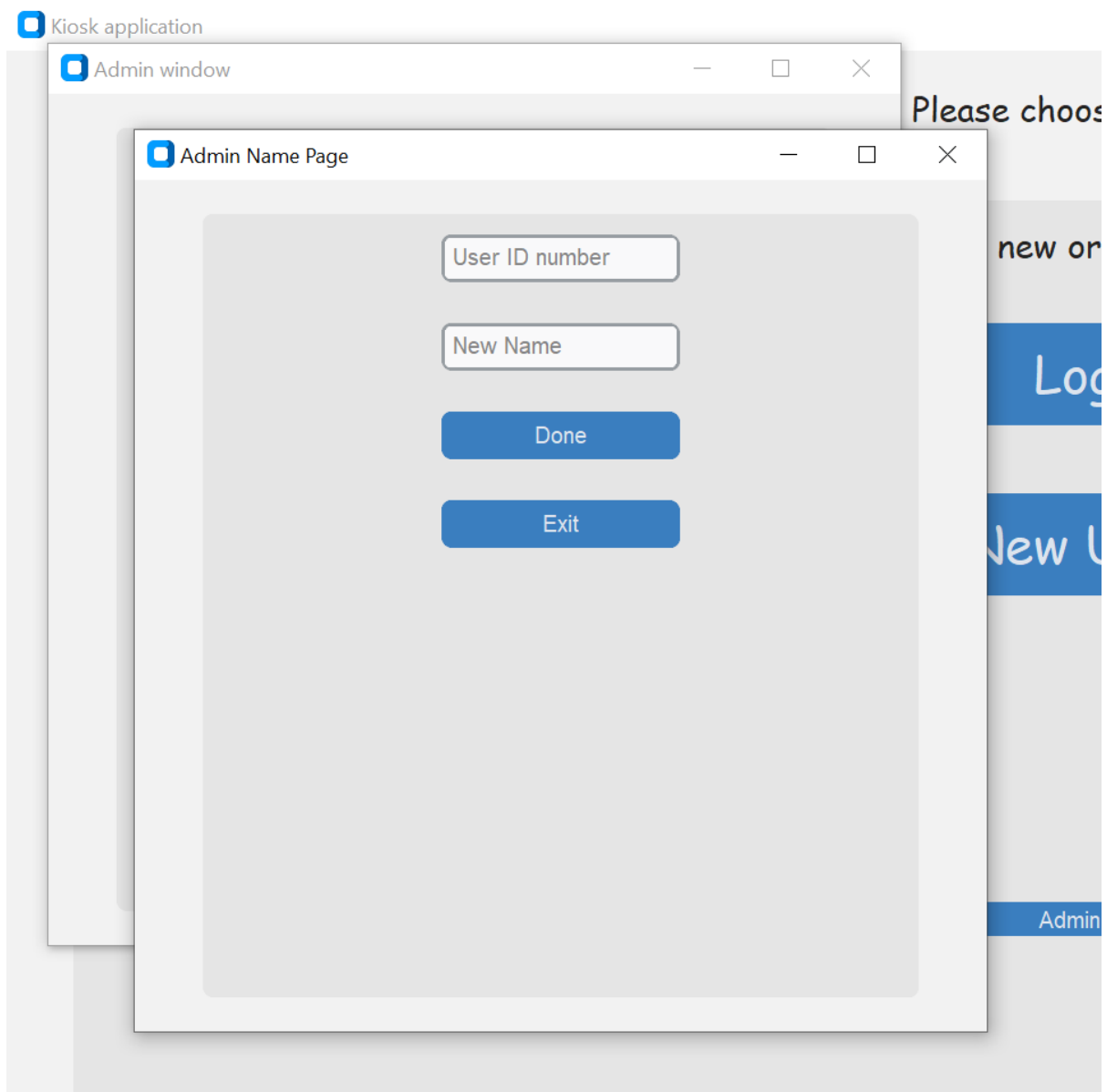
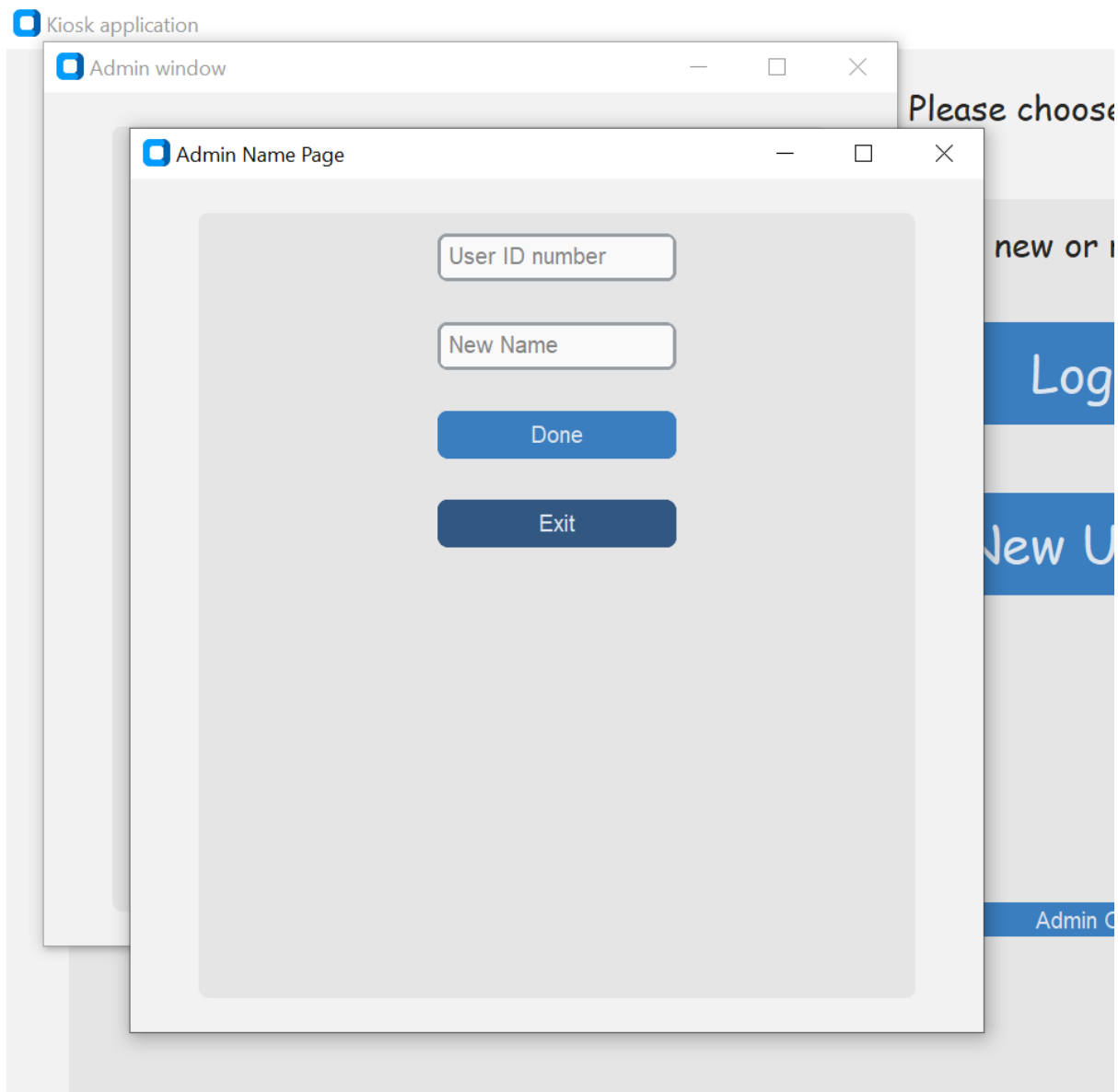


Fig 2.0



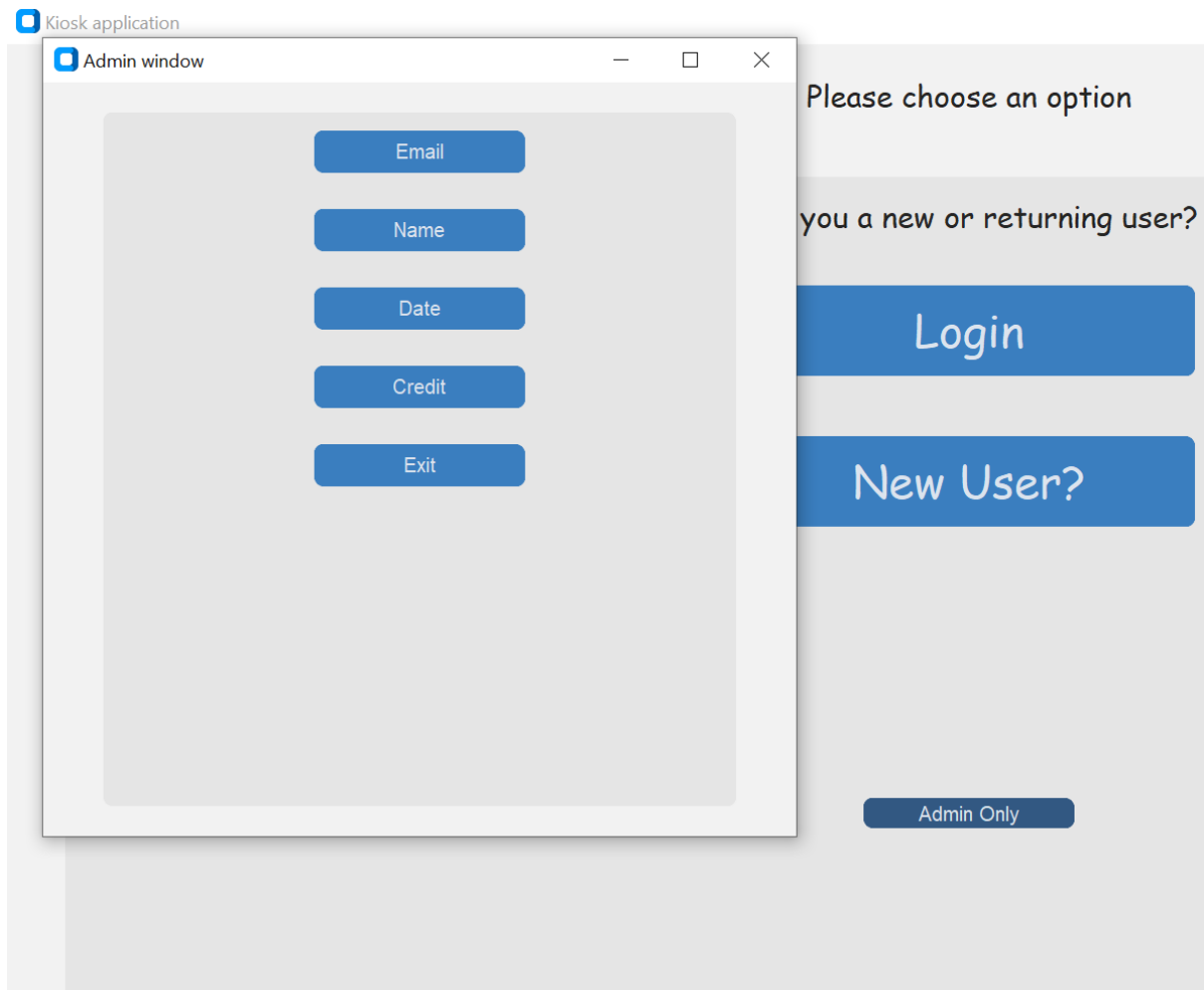
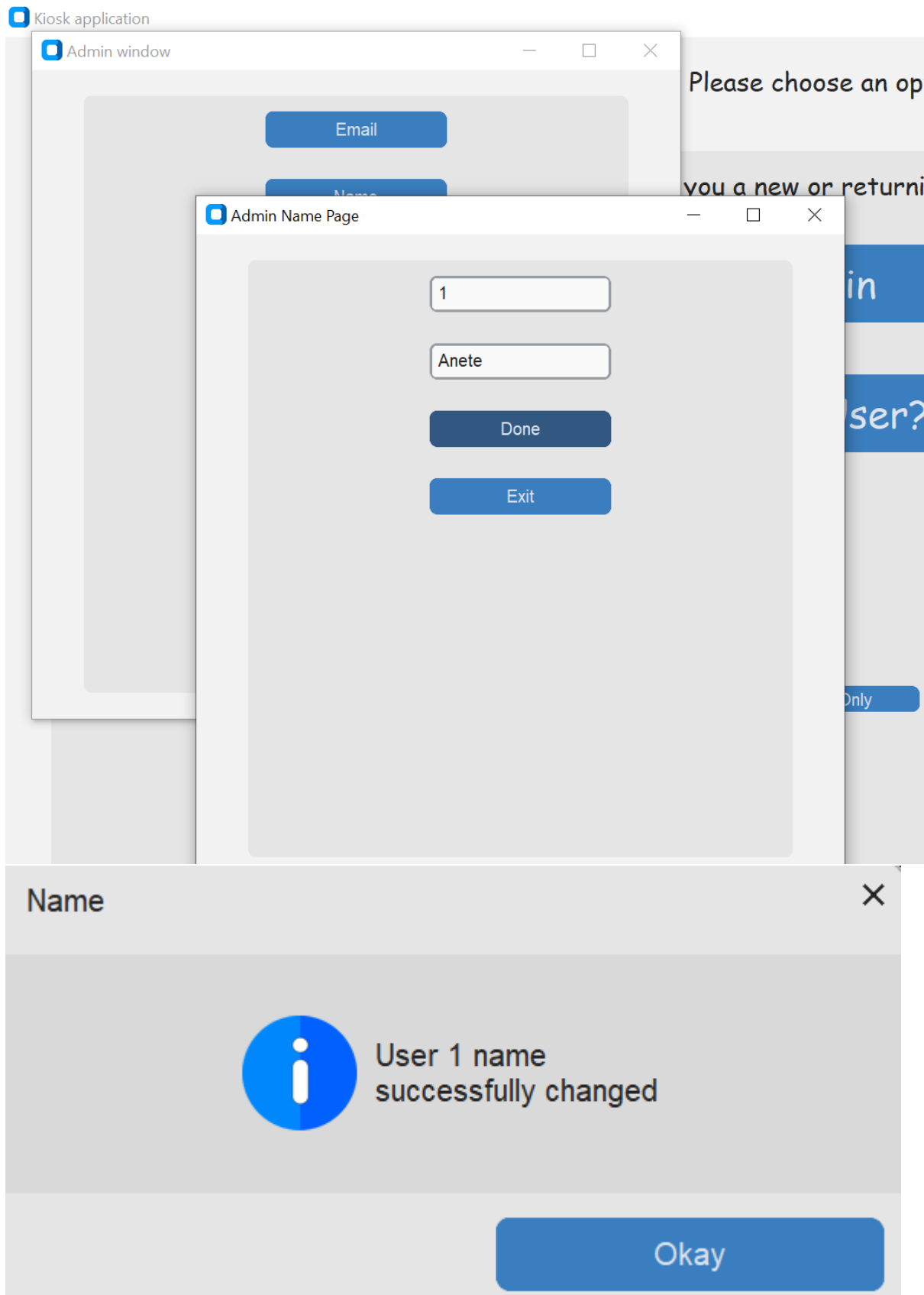


Fig 3.0

	A	B	C	D	E	F	G	H
1	3	testemail	net	1951	10	10	520	
2	2	email	name	year	month	date	credit	
3	3	test2@ma	test2	2021	2	2	20	
4								



	A	B	C	D	E	F	G	H
1	3	testemail	Anete	1951	10	10	520	
2	2	email	name	year	month	date	credit	
3	3	test2@ma	test2	2021	2	2	20	
4								

Step 6: Admin date option

I will next create the date function.

```
def adminDate():
    adminDate_page = ctk.CTk()
    adminDate_page.geometry("500x500")
    adminDate_page.title("Admin Date Page")

    frame = ctk.CTkFrame(master=adminDate_page)
    frame.pack(pady=20, padx=40, fill='both', expand=True)

    userCount_entry = ctk.CTkEntry(master=frame, placeholder_text="User ID number")
    userCount_entry.pack(pady=12, padx=10)
```

First I created a new tkinter window and a frame for inside of the window. The first thing inside of the frame was the ID entry field which is where the administration will put the target users ID.

```
yearbox = ctk.CTkComboBox(frame, values="Year")
yearbox.pack(pady=30)
CTkScrollableDropdown(yearbox, values=years, justify="left", button_color="transparent")

monthbox = ctk.CTkComboBox(frame, values="Month")
monthbox.pack(pady=30)
CTkScrollableDropdown(monthbox, values=months, justify="left", button_color="transparent")

datebox = ctk.CTkComboBox(frame, values="Day")
datebox.pack(pady=30)
CTkScrollableDropdown(datebox, values=dates, justify="left", button_color="transparent")
```

Next I created three scrollable dropdown boxes for the values of year, month and date and I reused the values used in the user creation menu for the lists of year month and date.

```

dates = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
         15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31]

months = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

years = [2024, 2023, 2022, 2021, 2020, 2019, 2018, 2017, 2016, 2015, 2014, 2013, 2012, 2011, 2010, 2009,
         2008, 2007, 2006, 2005, 2004, 2003, 2002, 2001, 2000, 1999, 1998, 1997, 1996, 1995, 1994, 1993,
         1992, 1991, 1990, 1989, 1988, 1987, 1986, 1985, 1984, 1983, 1982, 1981, 1980, 1979, 1978, 1977,
         1976, 1975, 1974, 1973, 1972, 1971, 1970, 1969, 1968, 1967, 1966, 1965, 1964, 1963, 1962, 1961,
         1960, 1959, 1958, 1957, 1956, 1955, 1954, 1953, 1952, 1951, 1950, 1949, 1948, 1947, 1946, 1945,
         1944, 1943, 1942, 1941, 1940, 1939, 1938, 1937, 1936, 1935, 1934, 1933, 1932, 1931, 1930]

```

```

doneButton = ctk.CTkButton(master=frame, text="Done", command=confirm)

doneButton.pack(pady=20)

def dateExit():
    adminDate_page.destroy()

exitButton = ctk.CTkButton(master=frame, text="Exit", command=dateExit)
exitButton.pack()

adminDate_page.mainloop()

```

Next I created a done button and an exit button. The exit button has the command date exit which will destroy the current exit page.

```

def confirm():
    userID = userCount_entry.get()
    if yearbox.get() != "Y" and monthbox.get() != "M" and datebox.get() != "D":
        if monthbox.get() == "2" and (
            datebox.get() == "29" or datebox.get() == "30" or datebox.get() == "31"):
            CtkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
        elif (
            monthbox.get() == "4" or monthbox.get() == "6" or monthbox.get() == "9" or monthbox.get() == "11") and date
            CtkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
        else:
            userBase['F' + str(userID)] = datebox.get()
            userBase['E' + str(userID)] = monthbox.get()
            userBase['D' + str(userID)] = yearbox.get()

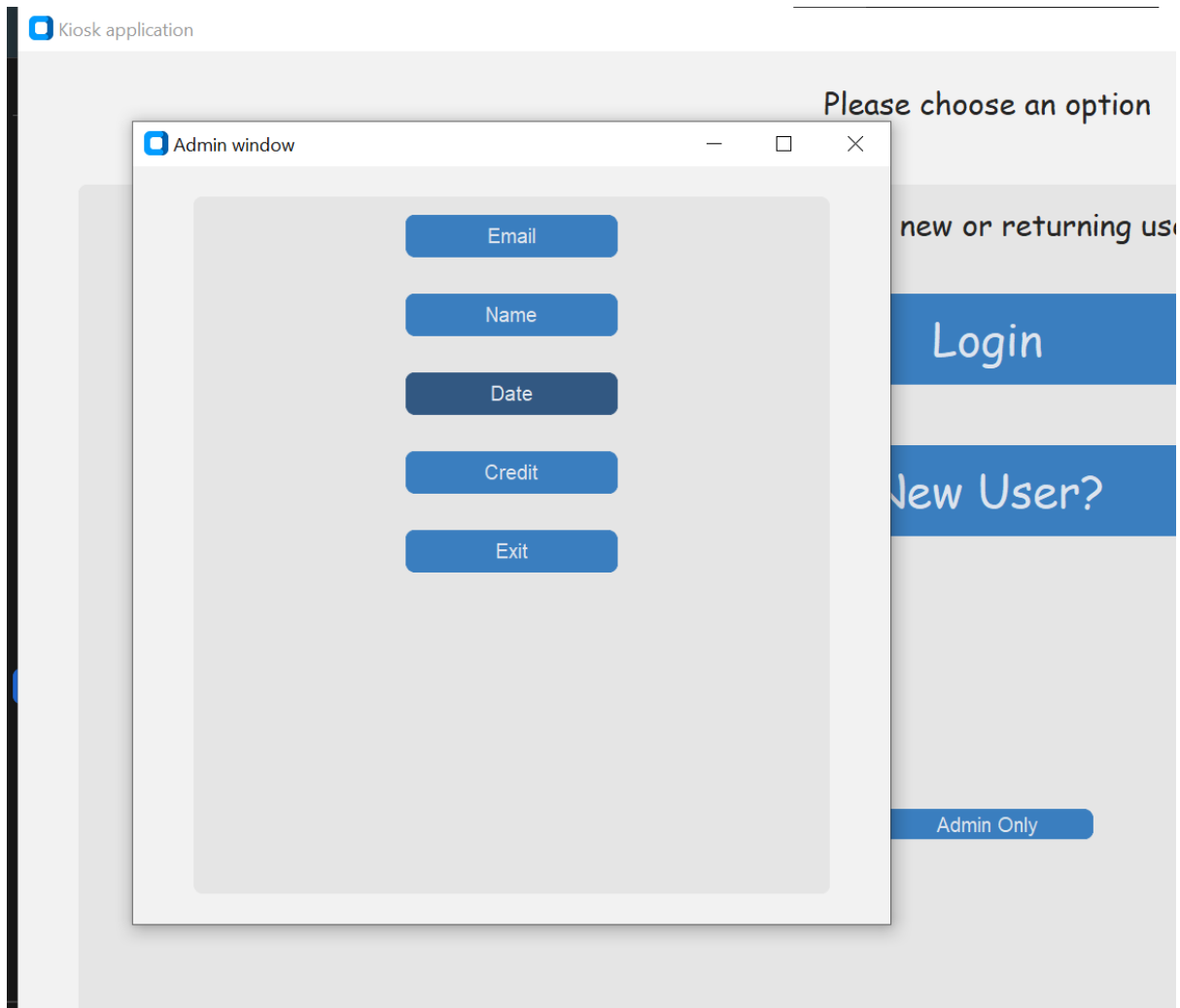
            CtkMessageBox(title="Date", message=f"User {userID} birthday successfully changed",
                          icon="info", option_1="Okay")

```

The function for the confirm button will first perform validation checks, making sure that the date and month combination is a valid combination. I reused the code from the new user creation to save time and testing at this stage. If the entries are valid, then they will get saved to the database and a message box will be displayed informing the user that the birthday has been successfully changed. If the validation finds the entry to be incorrect, a message box will open informing the user that the entry is invalid.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Date button	Date button opens the admin date menu	Date button opened the admin date menu	Pass		1.0
2.0	Year, month and date dropdown entry boxes	When clicking the dropdown boxes, they open dropdown menus showing all the possible options	All dropdown boxes showed their respective possible options	Pass		2.0
3.0	Done button	Done button saves the entry field into the database	Done button changes the fields in the database to the correct fields	Pass		3.0
4.0	Exit button	Exit button closes the date menu	Exit button closed the date menu	Pass		4.0
5.0	Date validation	When incorrect dates are entered, it displays an error message	Error message box opened upon entering invalid details	Pass		5.0

Fig 1.0



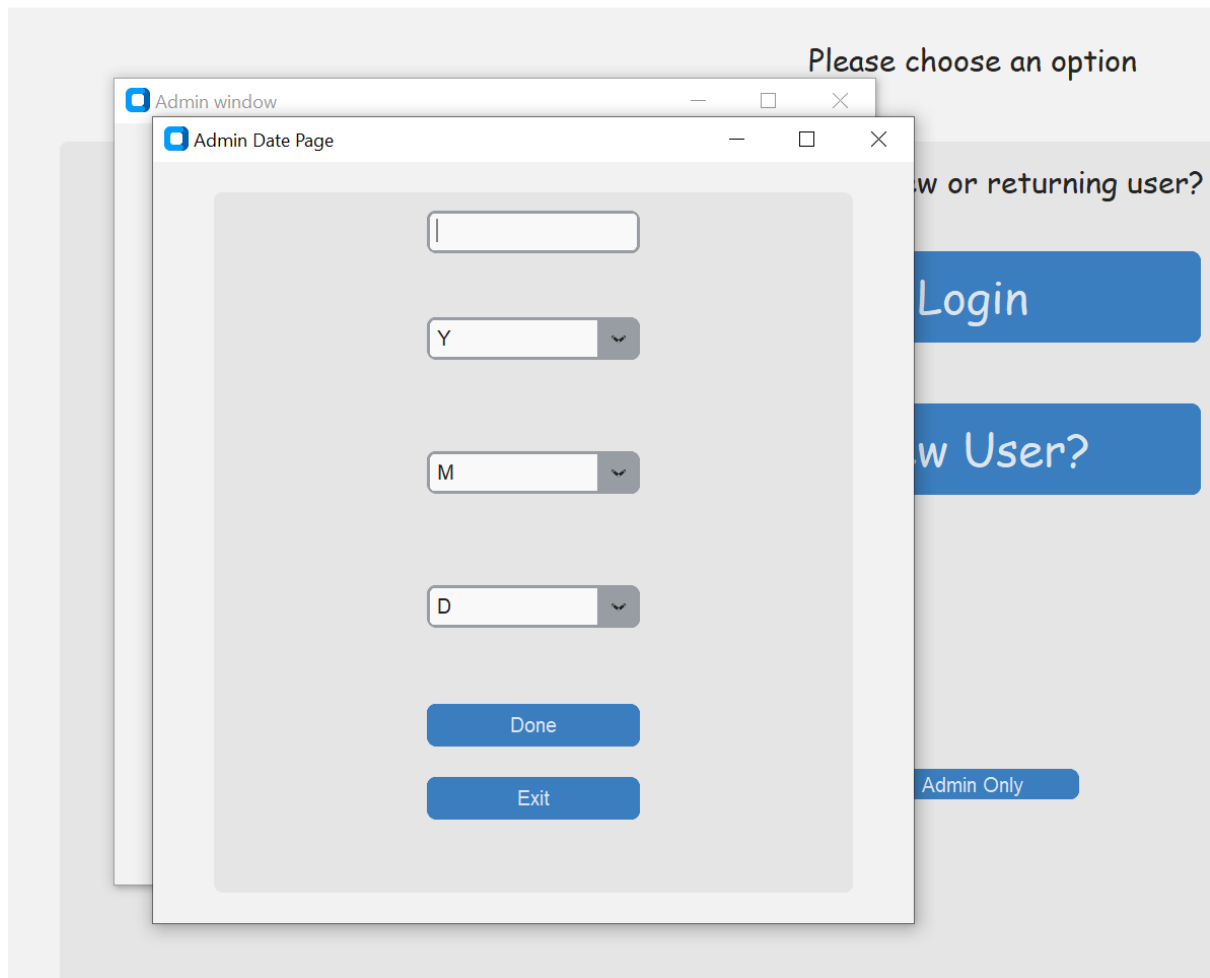
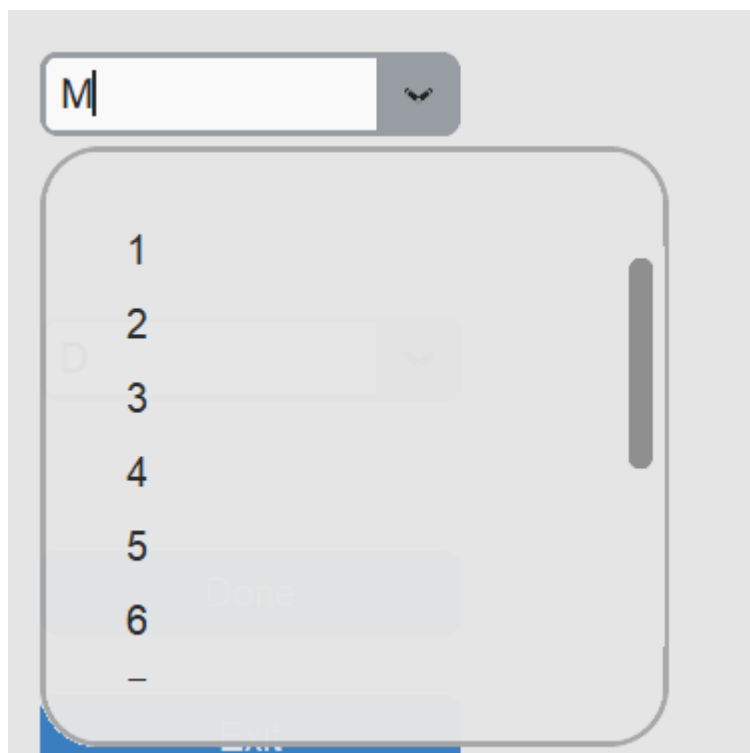


Fig 2.0



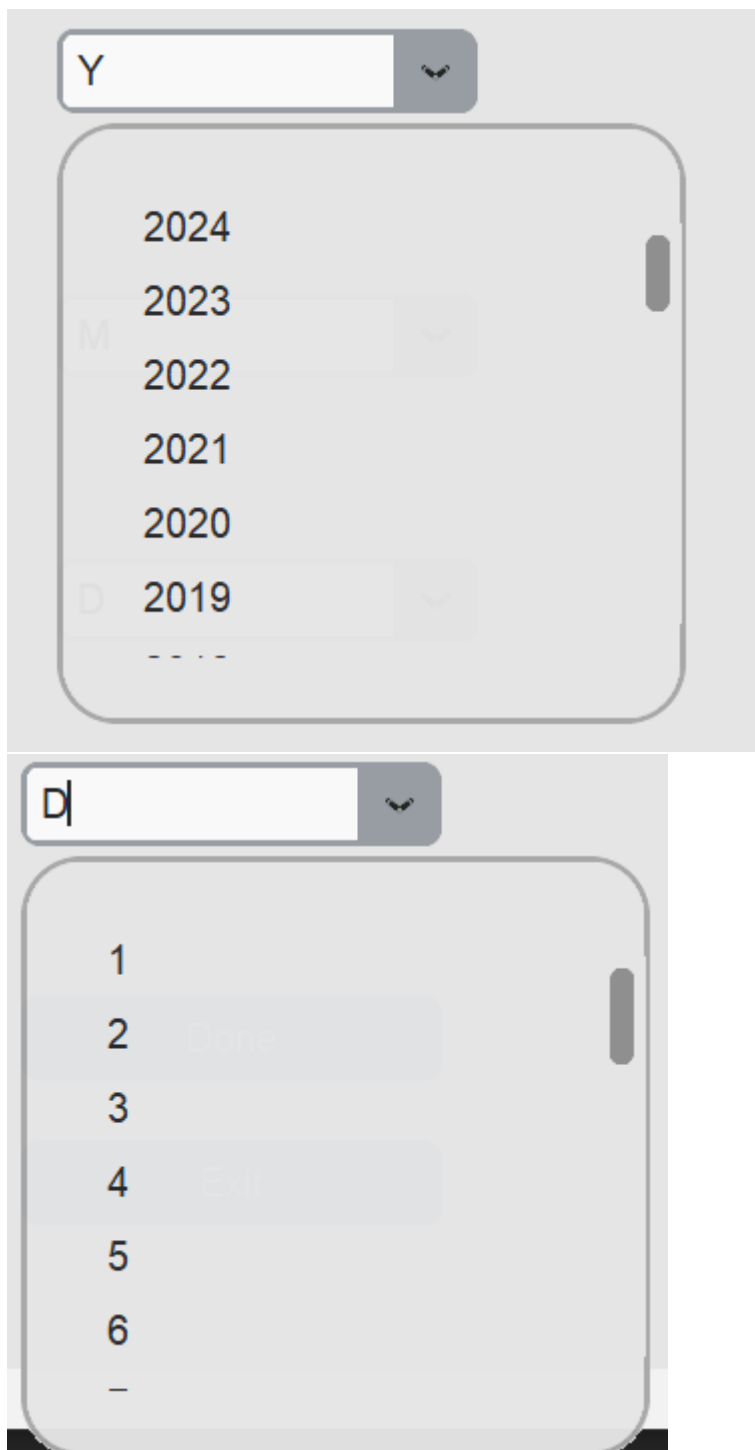


Fig 3.0

	A	B	C	D	E	F	G	
1	3	testemail	Anete	1951	10	10	520	
2	2	email	name	year	month	date	credit	
3	3	test2@ma	test2	2021	2	2	20	
4								

Done

Exit

Date



User 1 birthday
successfully changed

Okay

	A	B	C	D	E	F	G
1	3	testemail	Anete	2020	2	5	520
2	2	email	name	year	month	date	credit
3	3	test2@ma	test2	2021	2	2	20
4							

Fig 4.0

Admin Date Page

— □ ×

User ID number

Y

M

D

Done

Exit

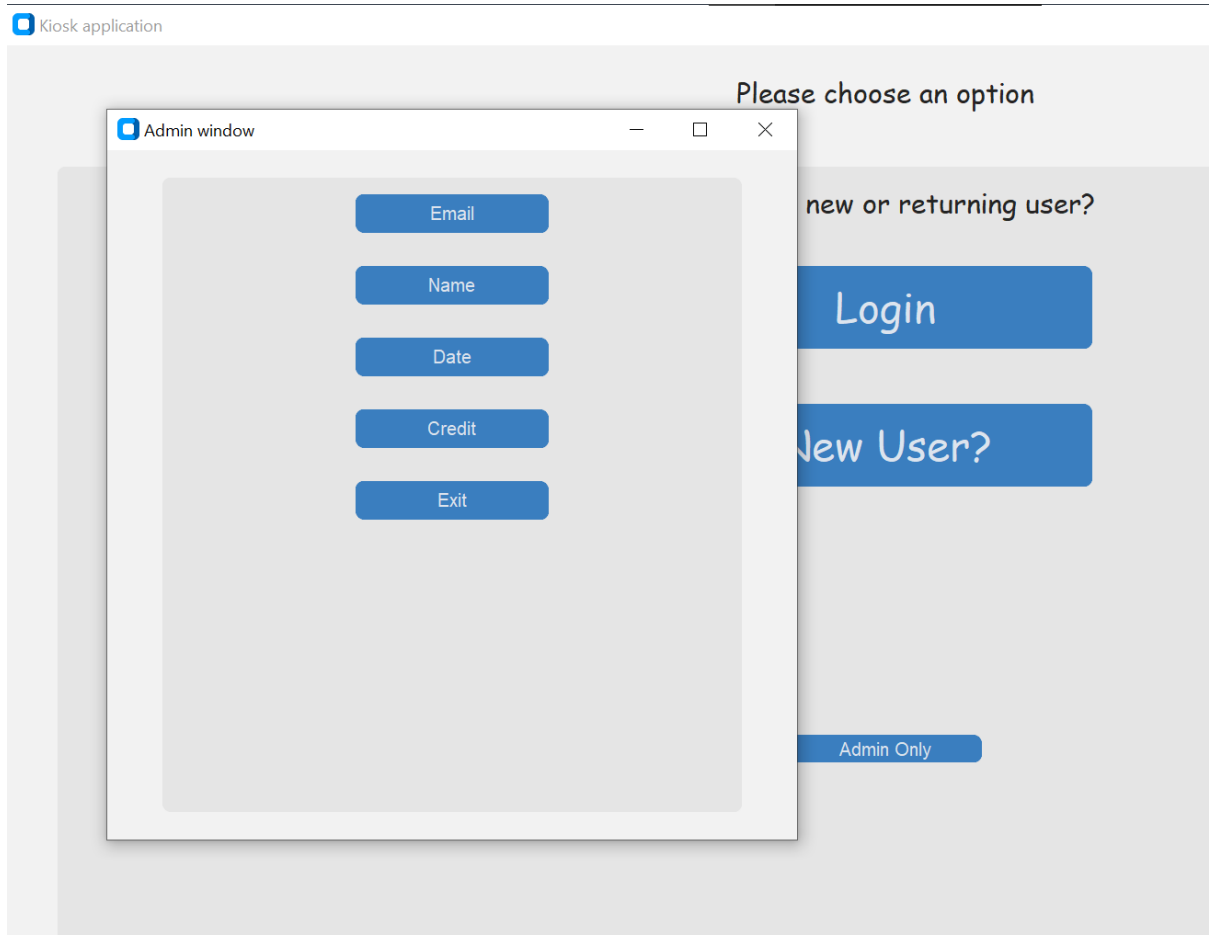


Fig 5.0

Done

Exit

Error



Invalid Entry!

OK

Step 7: Admin Credit option

Next, I will create the credit function.

```
def adminCredit():
    adminCredit_page = ctk.CTk()
    adminCredit_page.geometry("500x500")
    adminCredit_page.title("Admin Credit Page")

    frame = ctk.CTkFrame(master=adminCredit_page)
    frame.pack(pady=20, padx=40, fill='both', expand=True)

    userCount_entry = ctk.CTkEntry(master=frame, placeholder_text="User ID number")
    userCount_entry.pack(pady=12, padx=10)

    credit_entry = ctk.CTkEntry(master=frame, placeholder_text="New Credit Value")
    credit_entry.pack(pady=12, padx=10)
```

First I created the admin credit page and a frame for the features inside of the frame to be placed into. The first two features were a user count entry and a credit entry, for the users ID and the credit that it will be changed to.

```
def done():
    userID = userCount_entry.get()
    coord = "G" + str(userID)

    userBase[coord] = credit_entry.get()

    CtkMessageBox(title="Email", message=f"User {userID} credit successfully changed",
                  icon="info", option_1="Okay")

    wb.save("userbase.xlsx")

def creditExit():
    adminCredit_page.destroy()

doneButton = ctk.CTkButton(master=frame, text="Done", command=done)
doneButton.pack(pady=12, padx=10)

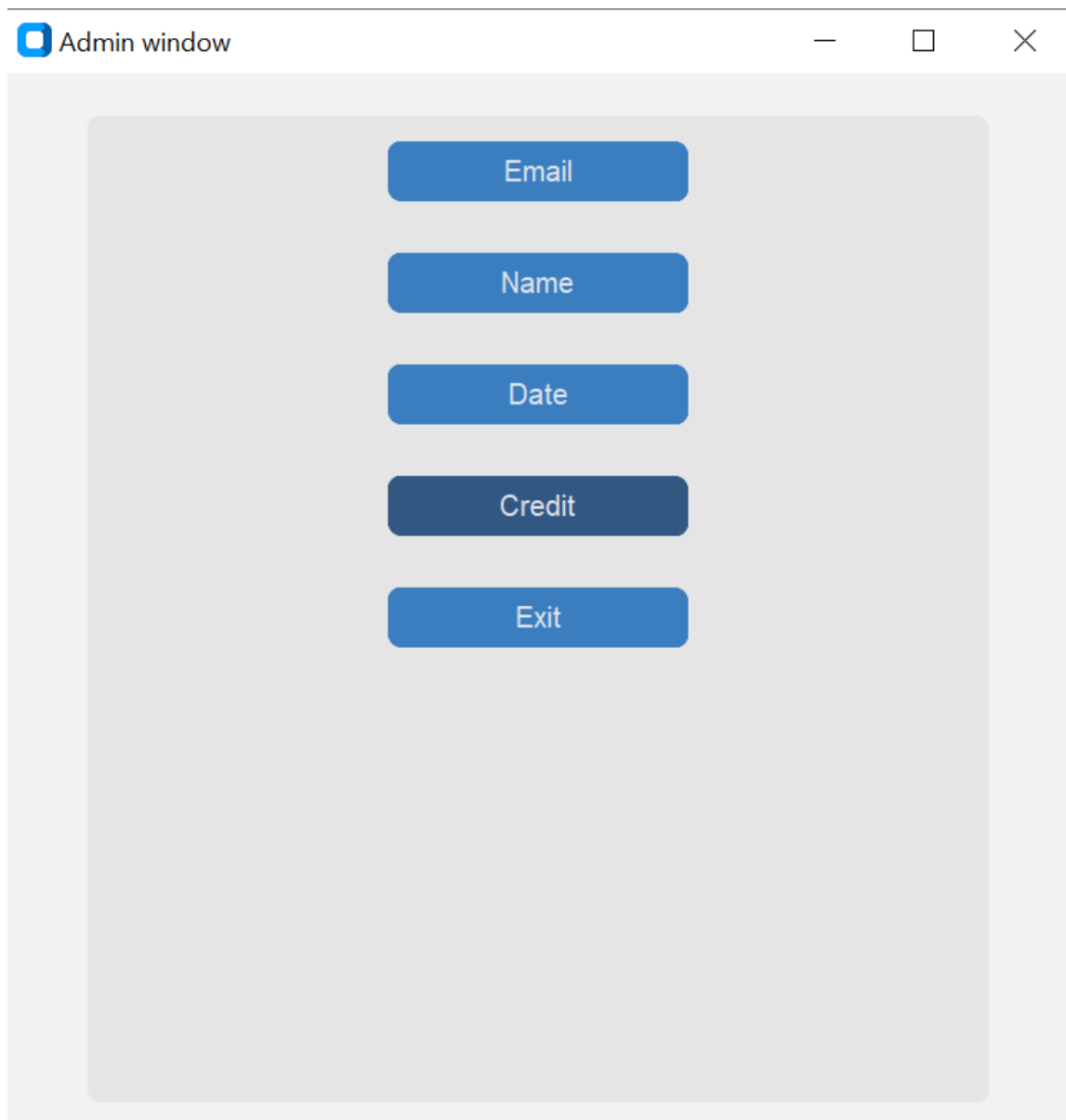
exitButton = ctk.CTkButton(master=frame, text="Exit", command=creditExit)
exitButton.pack(pady=12, padx=10)

adminCredit_page.mainloop()
```

I will also create two buttons a done button and an exit button. The function done will be ran when the button is clicked, and it will change the targeted place in the database to the new value and then display that credit has been changed to the user and then save the database. The credit exit function will be run when the exit button is clicked, causing the admin credit page to be destroyed.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Credit button in admin menu	Upon clicking, the credit menu opens	Credit menu opened	Pass		1.0
2.0	Done button	Upon clicking, the new credit variable gets saved to the correct place in the database.	The new credit is saved into the database	Pass		2.0
3.0	Exit button	Upon clicking the credit menu will close	Credit menu closed	Pass		3.0

Fig 1.0



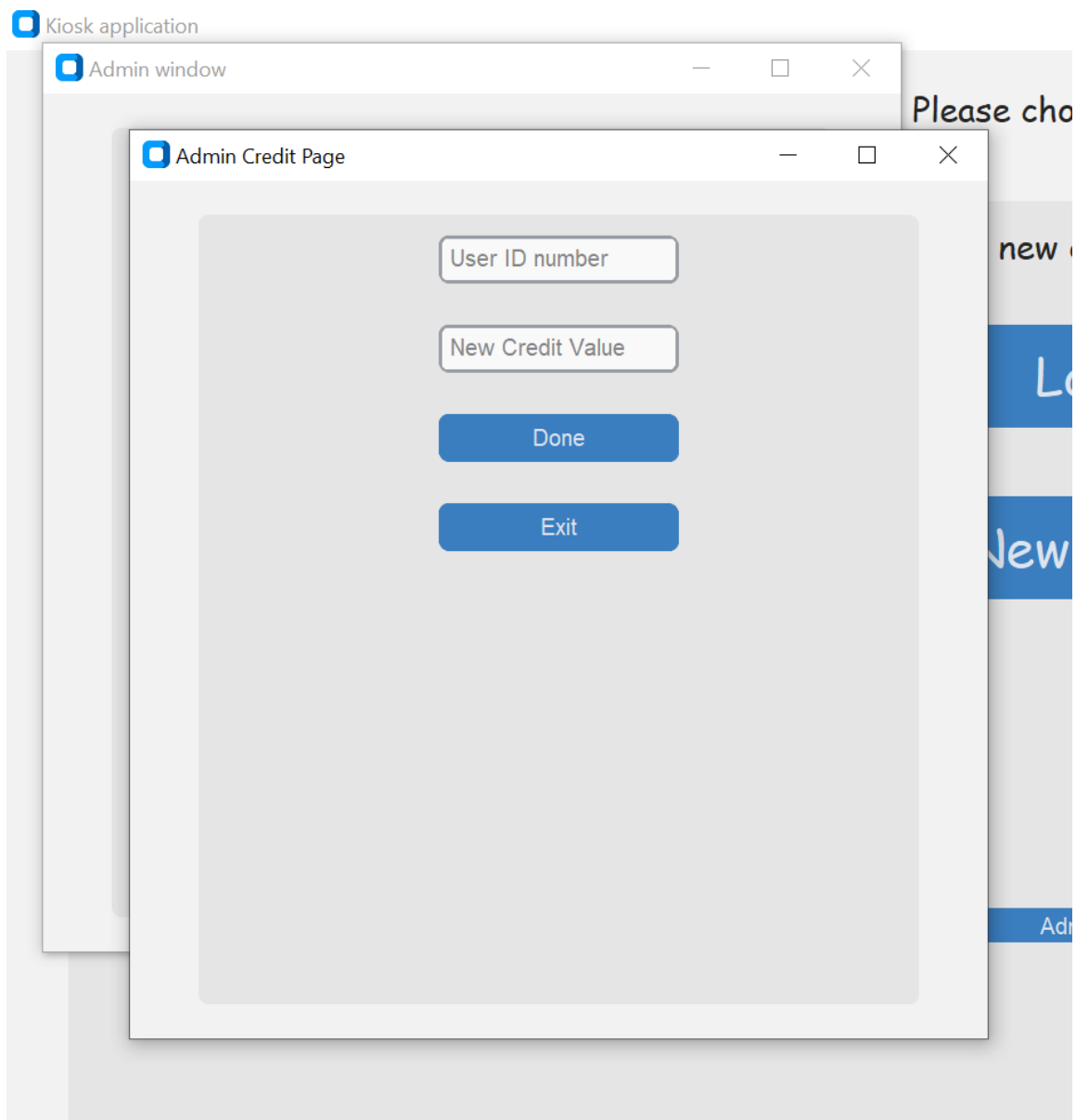
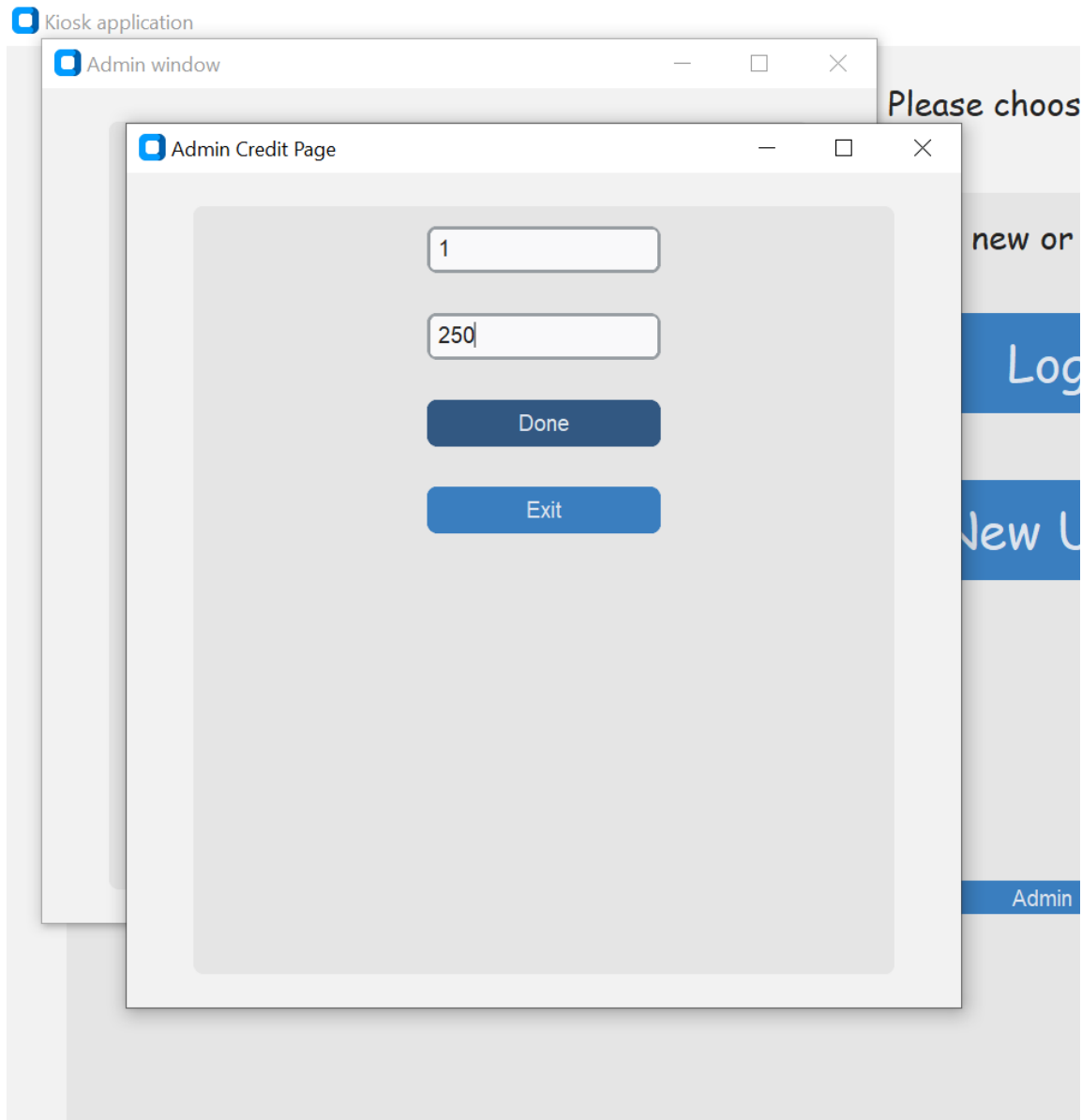
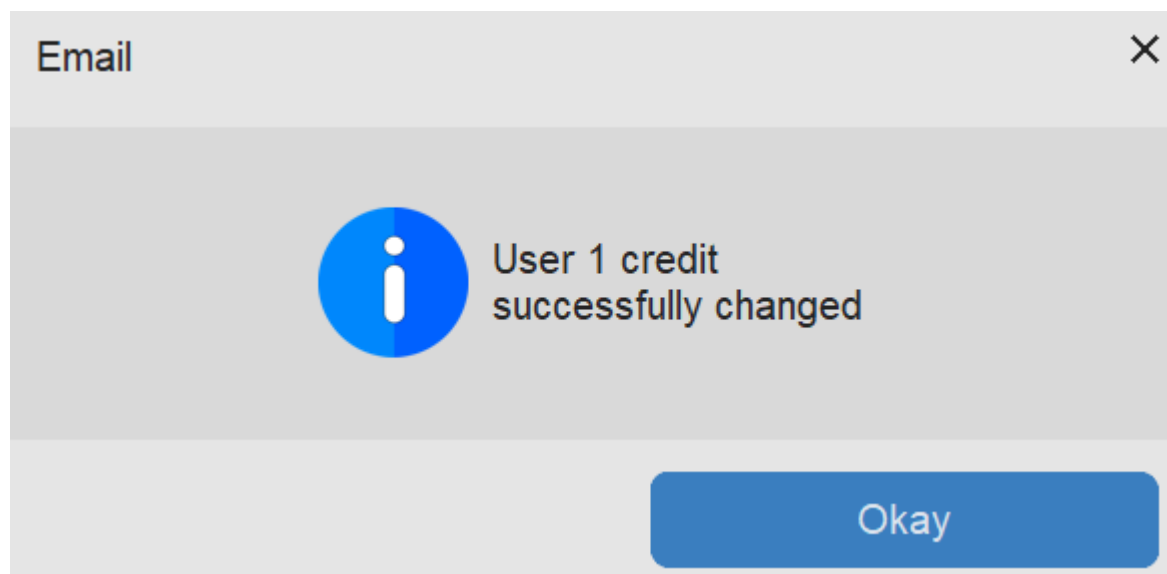


Fig 2.0

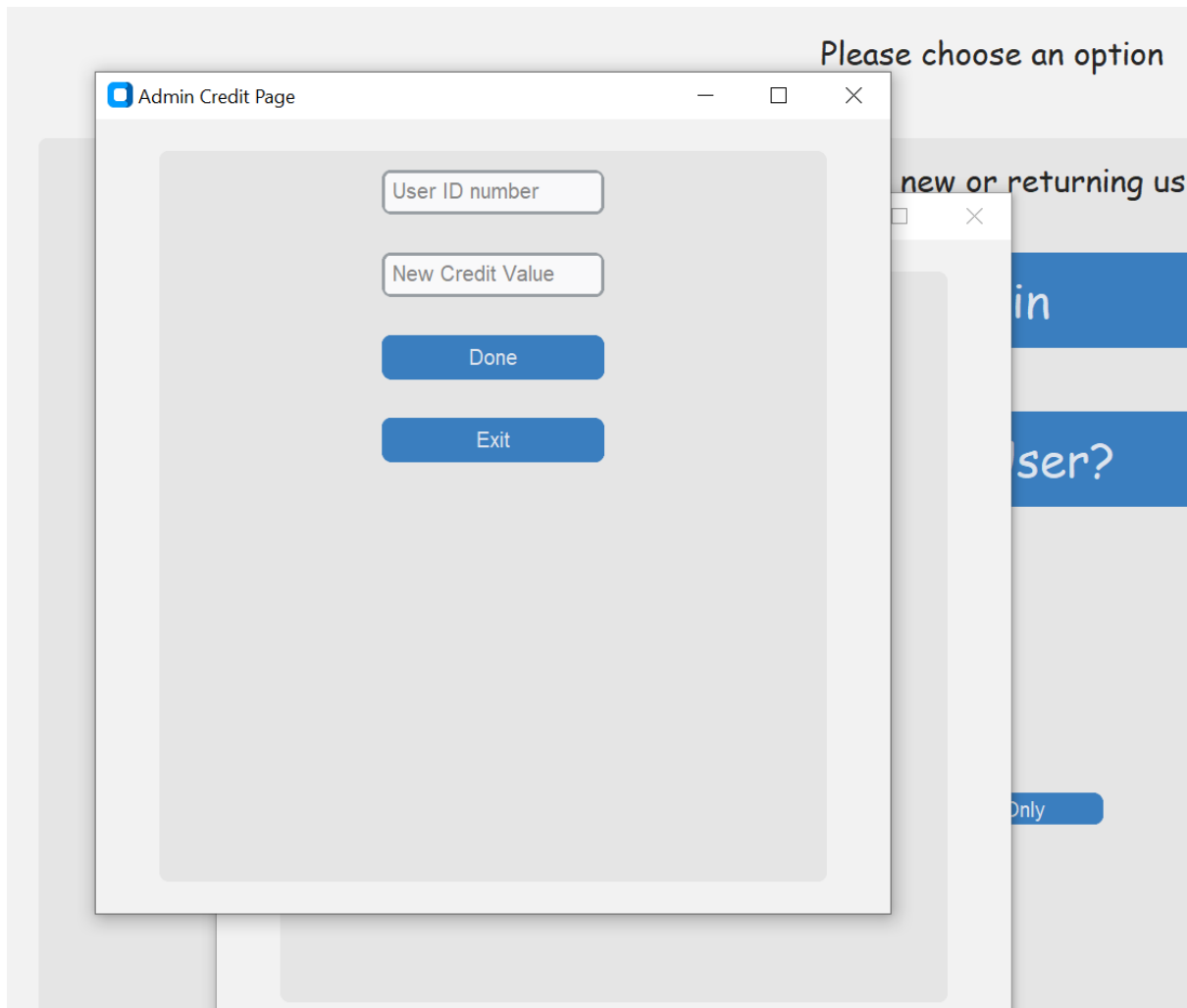
	A	B	C	D	E	F	G
1	3	testemail	Anete	2020	2	5	520
2	2	email	name	year	month	date	credit
3	3	test2@ma	test2	2021	2	2	20
4							

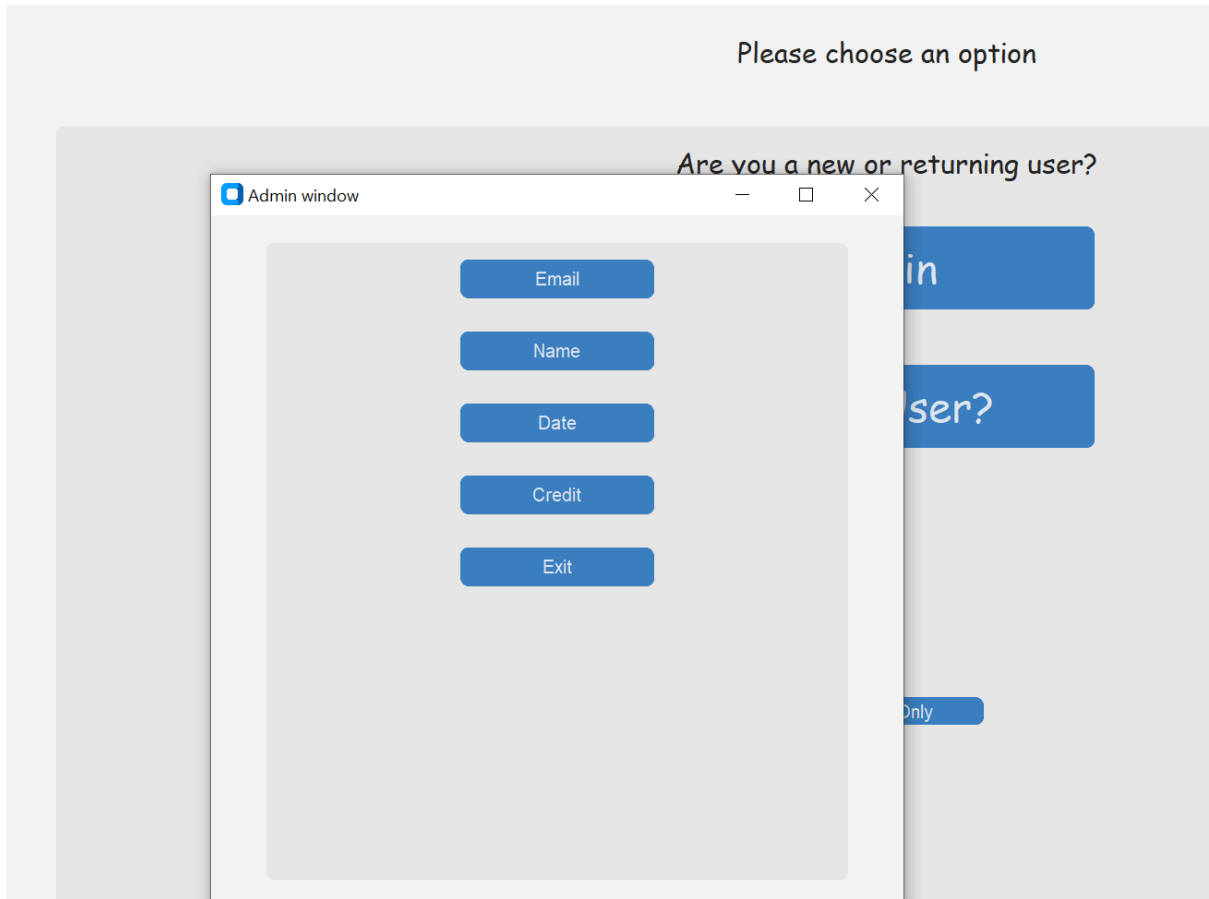




	A	B	C	D	E	F	G	H
1		3 testemail	Anete	2020	2	5	250	
2		2 email	name	year	month	date	credit	
3		3 test2@ma	test2	2021	2	2	20	
4								

Fig 3.0





Stage 5 Review

What's been developed:

In this stage I created the admin menu for the kiosk program. This admin menu allows for user details to be changed by an administrator who knows the password and username to get into the admin system. This is necessary incase a user accidentally sets any of their details incorrectly or gets charged the wrong amount of credit per ticket.

How it was tested:

I tested this section by using screenshot evidence of every feature that I had created for the section after explaining what the feature should do. I compared the result to the expected result of the test and identified if the test was a positive or a negative result for the test.

How it meets my success criteria:

This section meets the criteria of "Values on the database being altered depending on the user's actions" when the administrator uses the menu to change the details of a user. The details that the administrator wants to save are added to the database and saves that. I have evidenced this by showing screenshots of the database before and after the

The criteria of "User interface which is easily navigate-able" is also met in this section, with all the buttons and differing windows being labelled correctly for use. The evidence for this can be seen within screenshots of the evidence for other functions, displaying the buttons in the process.

The criteria of "Functional buttons throughout the program" is also met in this section with every button being assigned a functional command which, when clicked, executes the correct function it is assigned. I have demonstrated this in my evidence during this section, demonstrating how each button is clicked and the output of that button in the form of screenshots.

The criteria of "Working entry boxes for user to enter details" is also met in this section in the name and email address section where the administrator can enter the new name or email address that they want the users name or email to be changed. I have evidenced this throughout my stage, displaying screenshot evidence of the values entered the entry fields being saved into the database when confirmed, displaying the database before and after the changes.

The criteria of "Functional dropdown boxes that allow a user to only select a fixed number of options to choose from" has been met in this section in the birthday change menu where the administrator applies a new birthday to the user. I have evidenced this by displaying the dropdown boxes by screenshots, showing the set values displayed in the dropdown boxes.

Where it doesn't meet my success criteria so far:

- "Store facial encodings of users"
- "Correctly display user history in user history window"

Any changes from the design in this section:

This whole section is a change from my design since I had not planned to add this section at first until I realised there is not much of a room for error and an accessible way for administrators to fix any errors that do naturally happen in the course of the running of the program.

Where the project is so far:

The project half complete now, since the kiosk application is finished after this step. What only remains for the project is to create the other side of the application, which is the barrier application which charges the users for tickets.

Stage 6: Barrier program

Now that all the components of my kiosk application are finished, I will create the program for my barrier application which will access the current users and then identify them based on their faces.

Step 1: Barrier setup and facial detection

```
wb = load_workbook(filename='userbase.xlsx') # load workbook
userBase = wb['Sheet1'] # load the correct workbook
userCount = userBase['A1'] # get the value for the amount of current users
userCount = userCount.value

# Initialize the camera
cap = cv2.VideoCapture(0)

# Load the Haar Cascade classifier for face detection
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
```

First I loaded up the userbase workbook for use later in the code and then I also found the value of the user count. I then initialised the camera under the variable cap and loaded the Haar cascade classifier.

```
app = tk.Tk()
screen_width = app.winfo_screenwidth()
wcentre = (screen_width // 2) - (screen_width // 2)

screen_height = app.winfo_screenheight()
hcentre = (screen_height // 2) - (screen_height // 2)

app.geometry(f"{screen_width}x{screen_height}+{hcentre}+{wcentre}")

# Create a frame to hold the OpenCV window
frame = tk.Frame(app)
frame.pack()

# Create a label to display the OpenCV image

image_label = tk.Label(frame)
image_label.pack()

text_label = tk.Label(app, text="")
text_label.pack()
```

Then I created a tkinter app window and found the screen height and width and found the centres of the screen as well. I then created a frame to put inside of the app window and created two labels to put inside of the frame. The first label is the image label which is where I will store the image of the user which will be displayed, and the second label is a text label which will store the text that will be displayed back to the user.

```
def display_frame():
    global empty, userNumber
    # Read a frame from the camera
    ret, frame = cap.read()

    # Convert the frame to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detect faces in the frame
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=0
    |                                     , minSize=(30, 30))

    # Draw a rectangle around the detected faces
    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

I then created a function called display frame, which first will read the frame from the camera, then I reused the gray variable to convert the frame to grayscale. I then reused the faces variable to detect each face in the frame and I created a green box around the detected face.

```
    # Convert the OpenCV image to a Tkinter-compatible image
    image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    image = Image.fromarray(image)
    photo = ImageTk.PhotoImage(image)

    # Display the image in the Tkinter label

    # Update the existing label with the new image
    image_label.config(image=photo)
    image_label.image = photo
```

After that I converted the read image from the camera into an image that will be compatible to read in tkinter and displayed the image in the tkinter label and updated the existing label with the new image.

```

# Start the display loop
display_frame()

# Run the Tkinter event loop
app.mainloop()

# Release the camera and destroy all windows
cap.release()
cv2.destroyAllWindows()

```

I then started the loop and finished the tkinter event loop and released the open cv camera.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Tkinter window opens	Tkinter window opens after code is ran	Tkinter window opened when code is ran	Pass		1.0
2.0	Image in tkinter	Image displays when tkinter window opens	Image is displayed in the tkinter window	Pass		1.0
3.0	Face detected is	Green box tracks the face in the tkinter window display	Too many green boxes glitch around the face	Fail	Issue 3.0	3.0
3.1	Face detected is	Green box tracks the face in the tkinter window display	Green box is displayed around my face	Pass	Resolution 3.0	3.0

Fig 1.0

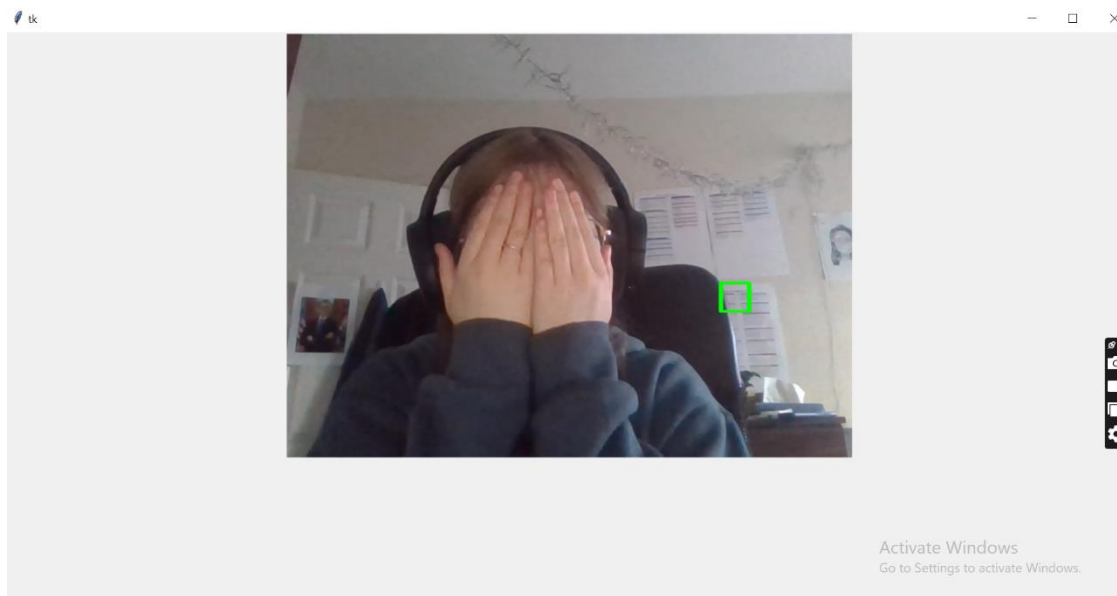


Fig 3.0

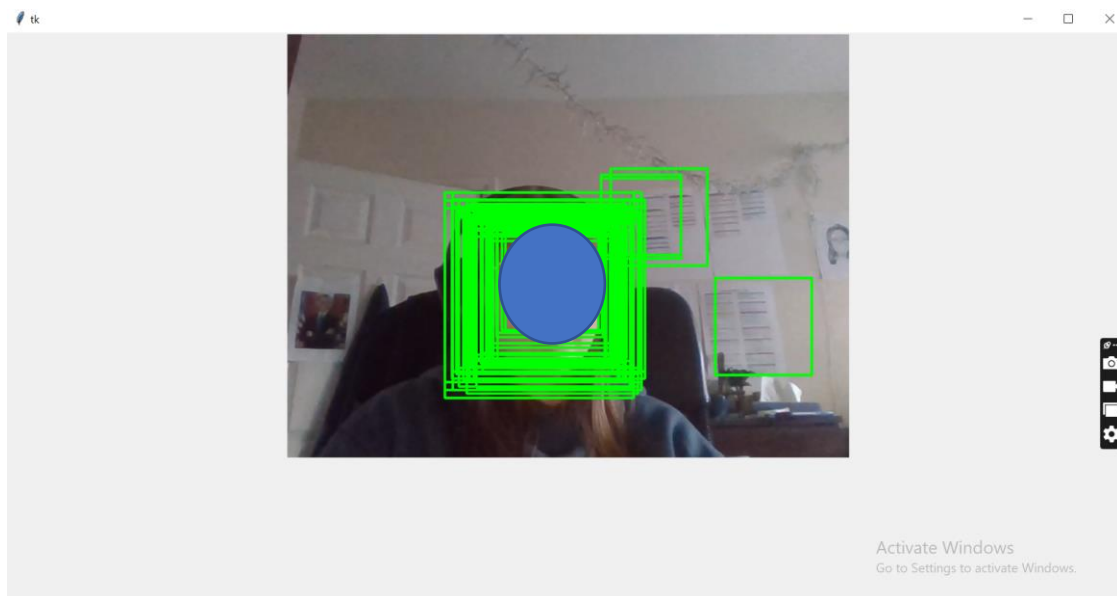
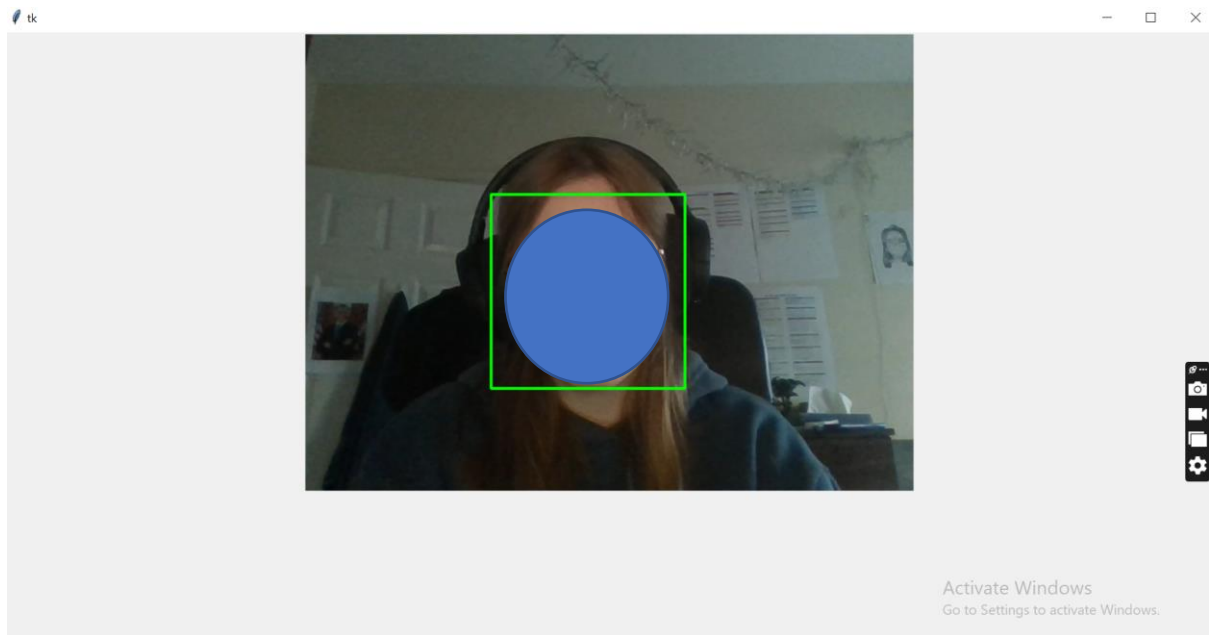


Figure 3.1



Issue and Resolution 3.0

```
# Detect faces in the frame
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=0
                                     , minSize=(30, 30))
```

The issue was caused by me setting the minimum neighbours to 0, causing the face detector to break.

```
# Detect faces in the frame
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5
                                     , minSize=(30, 30))
```

I fixed it by setting the minimum neighbours to 5.

Step 2: Barrier face recognition

Next, I will create the recognition inside of the barrier program which will detect the user inside of the database of users.

```
if len(faces) > 0:
    face_frame = frame[faces[0][1]:faces[0][1] + faces[0][3], faces[0][0]:faces[0][0] + faces[0][2]]
    cv2.imwrite( filename: "temp.jpg", face_frame)
```

First I will check if there is a face being detected in the frame by checking if the frame has more than 1 face. Then I set the image size to be the boundary of the face and then save that boundary under the name temp.jpg.

2 usages

```
def return_empty(image_path):  
    # reading image  
    image = cv2.imread(image_path)  
    # get face encodings from the image  
    face_enc = face_recognition.face_encodings(image,  
    if not face_enc:  
        return True  
    else:  
        return False
```

```
if not empty:  
    # getting face encodings for second image  
    image_2 = face_encode.find_face_encodings("temp.jpg")  
  
    # compare temp file to userbase, same code as kiosk  
    userCount = userBase["A1"].value  
    counter = userCount + 1  
    for i in range(1, counter):  
        print(i)  
        # getting face encodings for first image  
        image_1 = face_encode.find_face_encodings(f"{i}.jpg")  
  
        # checking both images are same  
        is_same = face_recognition.compare_faces(known_face_encodings: [image_1], image_2)[0]  
        print(f"Is Same: {is_same}")
```

First, I used the return empty command from the face encode module to find if the image has a face in it. It will determine if the image has a face if the value for face encodings is not an empty variable so it will return false. If the variable is empty, then it will return true. If the variable empty is false, then it will find the face encodings of the temp image and then the user count of the database currently and then creates a counter for the for loop. The for loop will loop from the 1 to the maximum amount of users in the database. When looping it will print the current user number that it is on and then find the face encodings of that image of the user it is looping at currently and then it will compare the current looping image to the encodings of the temp file and store that as a Boolean value in the variable is same and then it will print the is same value.

```

if is_same:
    userNumber = i
    print(f"The current user is user number {i}")

    break
else:
    print("The images are not same")

```

After the is same variable is found and it is true, it will set the user number to be the current looping number and print that the current user is the looping number and break out of the loop. If the is same variable is not true, then it will display that the images are not the same.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Barrier recognises face	Is same is printed to be true and the correct user number is printed to be my user number	Is same is printed to be true and the correct user number printed to be the user's user number	Pass		1.0

Fig 1.0

```

1
Is Same: True
The current user is user number 1

```

Step 3: Charging the user the correct amount

Now that the program is able to correctly find the user ID of the current user, I will now charge the user the correct amount for a ticket depending on the age of the user.

```
if is_same:|
    is_same = False
    text_label.config(text="Face Detected!", fg="green")

    # find the existing credit of the user and print the name of the user
    print(userBase["C" + str(userNumber)].value)
    coord = "G" + str(userNumber)
    precredit = userBase[coord].value

    # find the users birthday
    y = userBase["D" + str(userNumber)].value
    m = userBase["E" + str(userNumber)].value
    d = userBase["F" + str(userNumber)].value
    birthdate = date(year=int(y), month=int(m), day=int(d))
    today = date.today()

    # calculate the users age
    age_days = today - birthdate
    age_days = age_days.days
    age_years = age_days // 365
    print(age_years)
```

If the face has been detected (calculated by the is same variable being true), I will change the variable to be false so it can loop again and then I will set the text label to display that the face has been found. Then I printed the users name and found the current credit of the user and the birthday of the user. Using the birthday, I calculated the age of the user and then printed the age in years.

```
if age_years < 18: # child fare
    ticket_price = 3
    message = "Child fare charged, please proceed through the barrier"
elif age_years < 25: # student fare
    ticket_price = 4
    message = "Student fare charged, please proceed through the barrier"
elif age_years > 65: # concession fare
    ticket_price = 3
    message = "Concession fare charged, please proceed through the barrier"
else:
    ticket_price = 5 # regular fare
    message = "Adult fare charged, please proceed through the barrier"
```

Next I found the individuals ticket price depending on their age. Users under 18 will be charged a child fare of 3

pound, users under 25 but over 18 will be charged a student fare of 4 pound, users over 65 will be charged a concession fare of 3 pound and all other users will be charged a standard fare of 5 pound.

```
if precredit - ticket_price < 0:
    # not enough credit window
    barrier_window = tk.Tk()
    barrier_window.geometry(f"600x300+350+150")
    barrier_window.attributes('-topmost', True)

    text = tk.Label(barrier_window, font=("Arial", 20),
                    text="Not enough credit, add more at the kiosk")
    text.pack(pady=100, )

    barrier_window.after(ms=8000, lambda: (barrier_window.destroy(), display_frame()))
    barrier_window.mainloop()
else:
```

Next I created validation to ensure that the user has enough credit to be charged than the calculated ticket price. If the user does not have enough credit, then the program will open a new window informing the user that they do not have enough credit and must add more at the kiosk.

```
else:
    postcredit = precredit - ticket_price
    print("Credit charged")
    userBase[coord] = postcredit
    barrier_window = tk.Tk()
    barrier_window.geometry(f"600x300+350+150")
    barrier_window.attributes('-topmost', True)

    text = tk.Label(barrier_window, font=("Arial", 15), text=message)
    text.pack(pady=100, )

    barrier_window.after(ms=8000, lambda: (barrier_window.destroy(), display_frame()))
    barrier_window.mainloop()
```

If the user does have enough credit to be charged, then I will calculate the credit after the user has been deducted the price and print that the credit has been charged and then create a new window to inform the user that their account has been charged the specific ticket type that the user has been charged and then after 8 seconds the window will destroy itself and rerun the function display frame for the next coming user.

```

    else:
        text_label.config(text="no entry", fg="red")

        # Schedule the next frame to be displayed
        app.after(ms= 30, display_frame)

```

If the value for is same is false, then the text label will display no entry and then rerun the display frame function after 30 milliseconds.

Test Number	What I'm testing	Expected Outcome	Actual Outcome	Pass/Fail	Issue/Resolution	Fig
1.0	Text label	Displays no entry when the user is not found and display entry when the user is found and has been charged	No entry displayed when face isn't clear, and entry is displayed when the face is detected and charged	Pass		1.0
2.0	Child fare	When age is under 18, child fare gets charged and is displayed to the user	Child fare is charged, and the user is told that a child fare is charged	Pass		2.0
3.0	Student fare	When age is above 18 but below 25, student fare is charged and displayed to the user	Student fare is charged and displayed to the user	Pass		3.0
4.0	Concession Fare	When age is above 65, concession fare is charged and displayed to the user	Concession fare is charged and displayed to the user	Pass		4.0
5.0	Regular fare	When age is between 25 and 65, user is charged regular fare and displayed to the user	Regular fare was charged and displayed it to the user	Pass		5.0
6.0	Insufficient credit	When the user has less credit than the target ticket price, display credit error.	Credit error window opened telling the user to add more credit	Pass		6.0

Fig 1.0

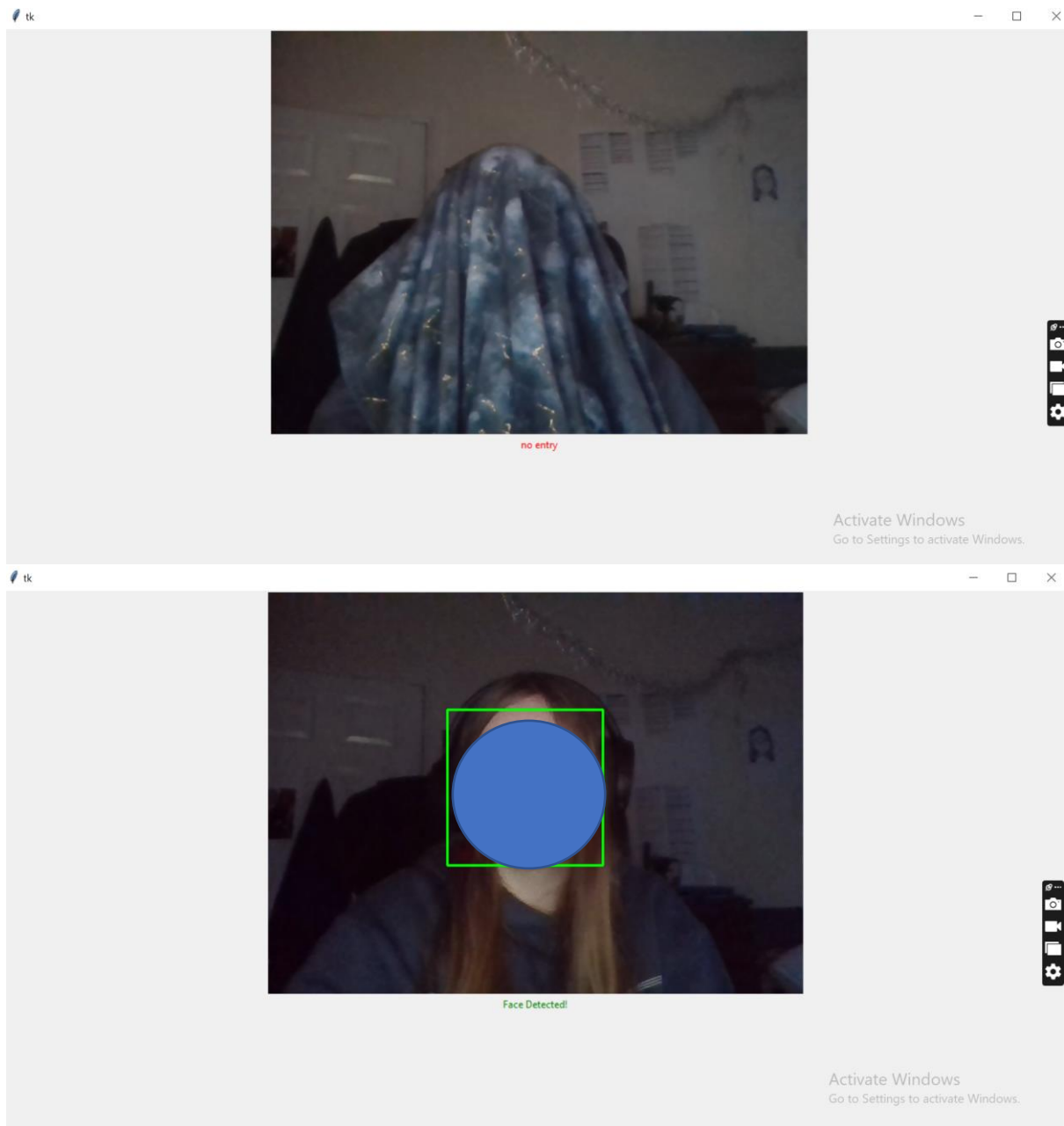


Fig 2.0

	A	B	C	D	E	F	G	
1	3	testemail	Anete	2020	2	5	250	
2	2	email	name	year	month	date	credit	
3	3	test2@ma	test2	2021	2	2	20	
4								

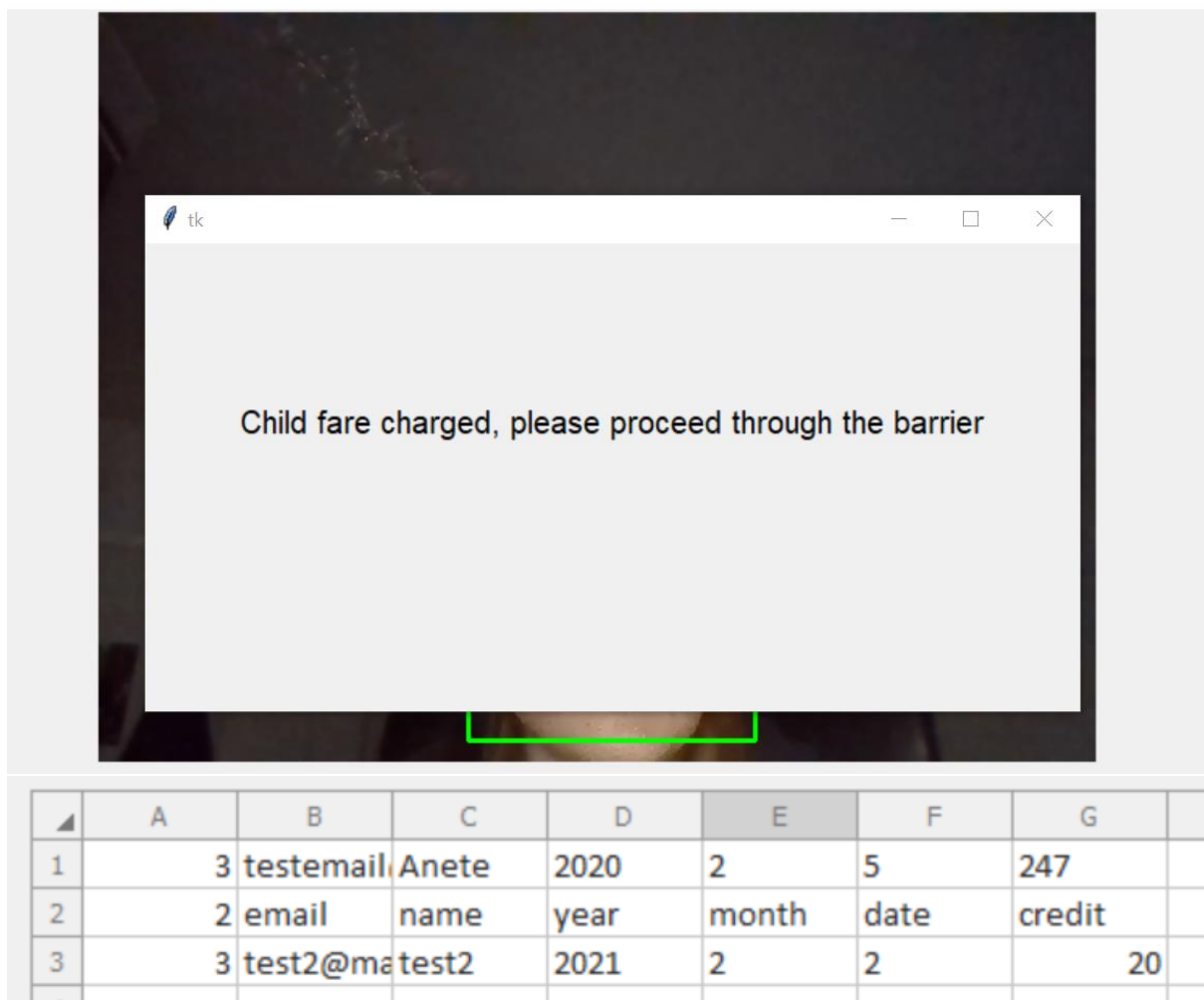


Fig 3.0

	A	B	C	D	E	F	G
1	3	testemail	Anete	2000	2	5	247
2	2	email	name	year	month	date	credit
3	3	test2@ma	test2	2021	2	2	20

tk

tk

Student fare charged, please proceed through the barrier

Face Detected!

tk

Student fare charged, please proceed through the barrier

Face Detected!

	A	B	C	D	E	F	G
1	3	testemail	Anete	2000	2	5	243
2	2	email	name	year	month	date	credit
3	3	test2@ma	test2	2021	2	2	20
4							

Fig 4.0

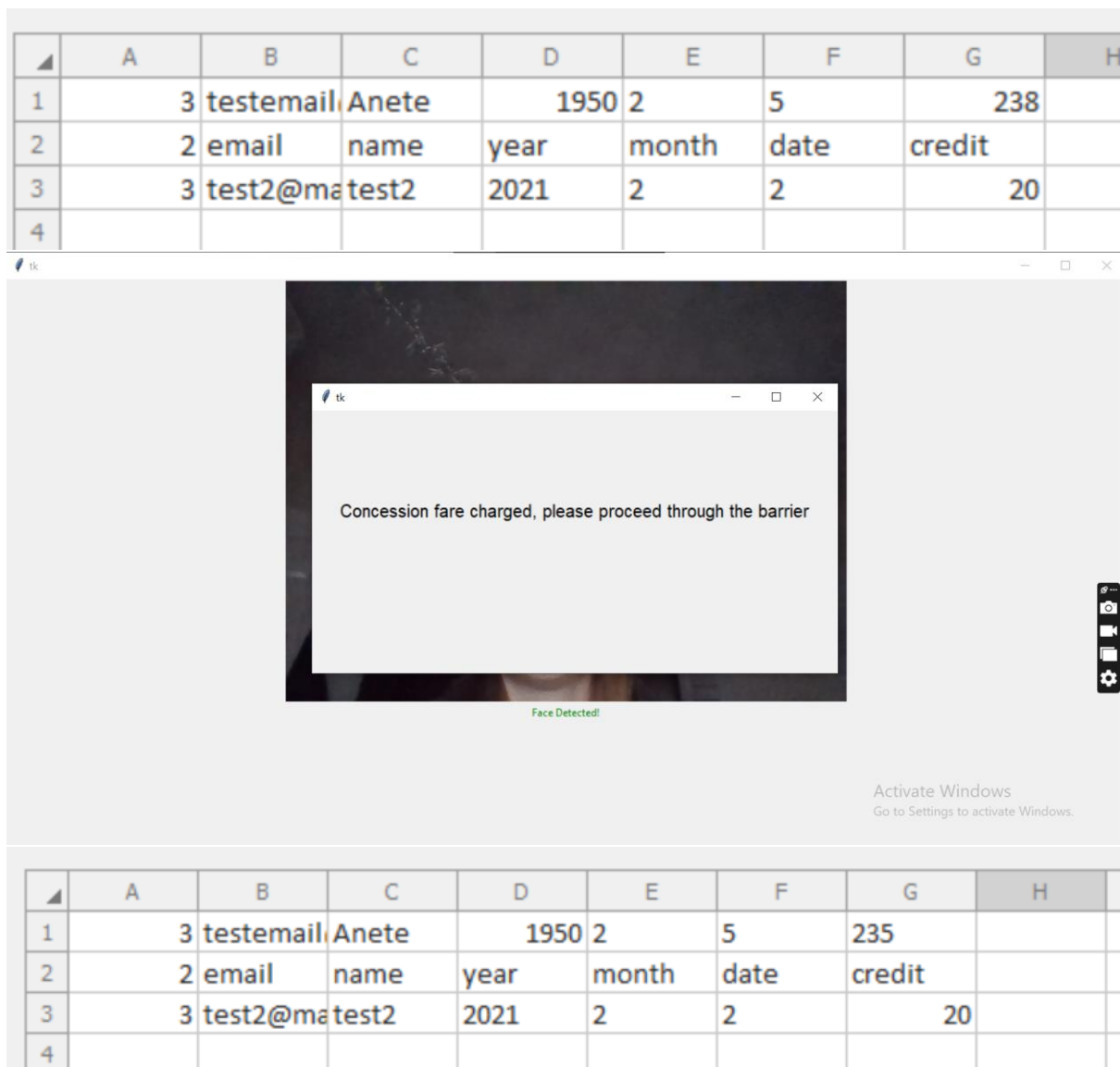


Fig 5.0

Explorer

userbase.xlsx

×

	A	B	C	D	E	F	G
1		3 testemail	Anete	1990	2	5	243
2		2 email	name	year	month	date	credit
3		3 test2@ma	test2	2021	2	2	20
4							

tk

tk

Adult fare charged, please proceed through the barrier

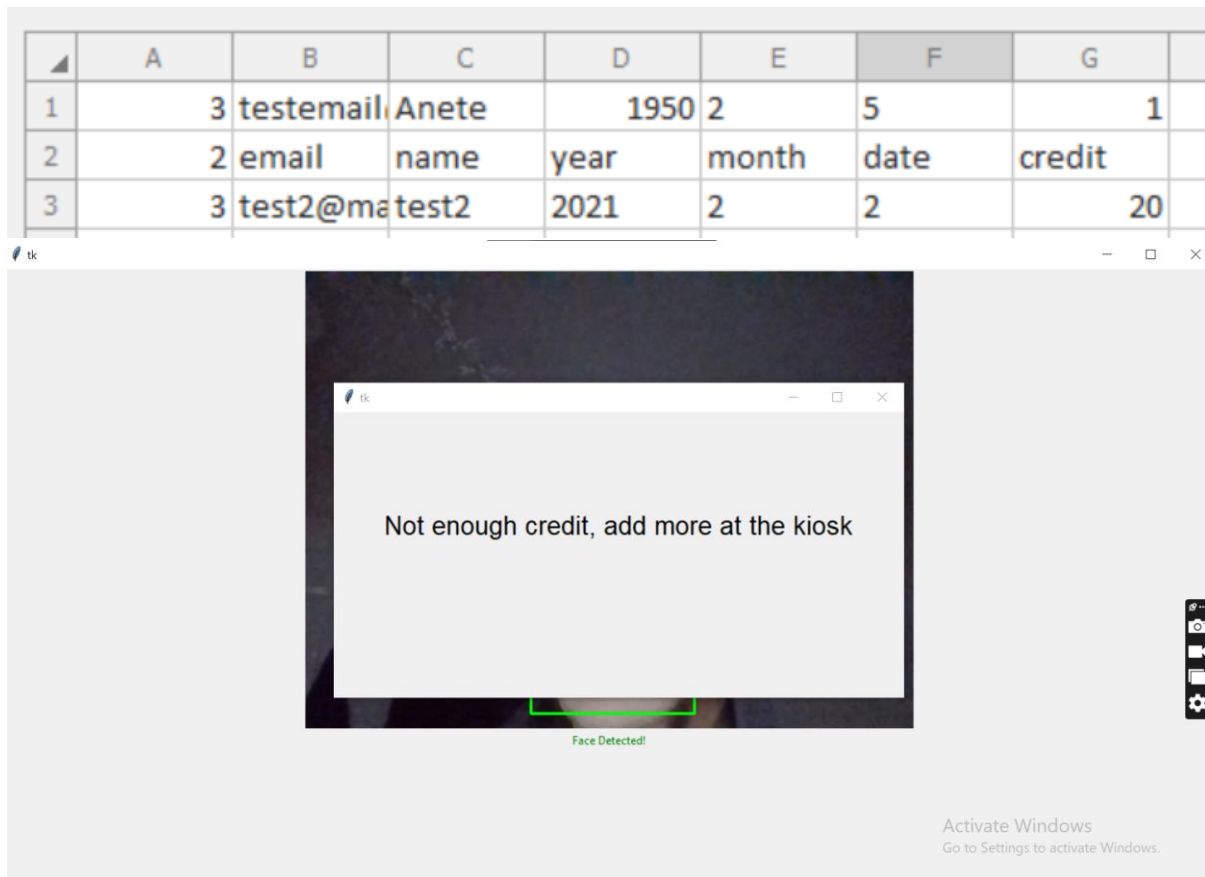
Face Detected!

Activate Windows

Go to Settings to activate Windows.

	A	B	C	D	E	F	G
1		3 testemail	Anete	1990	2	5	238
2		2 email	name	year	month	date	credit
3		3 test2@ma	test2	2021	2	2	20
4							

Fig 6.0



Stage 6 Review

What's been developed:

In this section I developed the other part of my project, the barrier application program. In this program I create a new tkinter window which displays the live camera feed. Onto that camera feed it detects the face of users who approach it and then check their face against the database of users faces that is collected from the kiosk program. Then it calculates the age of the user and deducts the correct amount according to the age from the user's credit.

How it was tested:

I tested this section using screenshots of each step that I designed, making sure that functionality of each section is displayed through its screenshots. I stated the expected outcome and the true outcome and compared them for if the test passed the test.

How it meets my success criteria:

The program meets the success criteria "Be able to retrieve details of a user on a database" when it retrieves the existing credit of the user to deduct credit from them.

The program meets the success criteria "Ability to compare a face to an already stored face" when it compares the face of the user currently in the view of the frame to the images of users already available.

The program meets the success criteria "A clear user display which conveys correct information based on the inputted data" when the barrier displays whether the barrier is open and closed and when it

informs the user if they lack credit or the ticket type that the user has been charged. I have evidenced this in the form of screenshots throughout the stage.

The criteria "Values on the database being altered depending on the user's actions" has been met when the current credit of the user is changed to the new credit after the user is charged credit for their ticket. I have evidenced this in screenshots of the database before and after, including screenshots of the user being told they have been charged.

The criteria "Correctly displayed information when an account is charged for money" has been met when the user is charged money depending on their ticket, with the screen displaying the type of ticket they have been charged. I have evidenced this by displaying the ages of my user entry currently and the display shown when I am charged.

The criteria "Charge the correct ticket price depending on user age and save the amount charged" has been met and evidenced by screenshots of the database before and after the account has been charged, displaying a change in the credit of the user and the current age of the user.

The criteria "Displayed if barrier is open or closed" has been met and evidenced throughout my code through screenshots of the barrier being closed when a face isn't detected and screenshots of the barrier being open when the face is present in the frame.

The criteria "Barrier program refreshes after a few seconds to allow for the next user to be detected" is met in the program by the program refreshing and rerunning the function after 8000 milliseconds.

The criteria "Inform user when lacking credit for a ticket" has been met by the program and evidenced by a screenshot of the program informing the user with a window telling them that their account lacks credit and they need to add more credit onto their account.

Where it doesn't meet my success criteria so far:

- "Store facial encodings of users"
- "Correctly display user history in user history window"

Any changes from the design in this section:

I have not made any changes to my program from the proposed design that I made for it.

Where the project is so far:

The project is now finished. I have created the two main aspects of my program which work together to create the whole project.

Testing

Post-development testing

Kiosk

Test No.	What is being tested	Input data or actions	Expected outcome	Actual Outcome	Pass /Fail	Video name and Time stamp
----------	----------------------	-----------------------	------------------	----------------	------------	---------------------------

1	New user button in initial menu function	Click the button opens up the new user information window	New user window opens when clicked	New user information menu opens when clicked	Pass	Newuser.mp4 00:03
2	Cancel button in new user information menu	Clicking closes the window and takes back to the start	Information window closes	Button closes the information menu	Pass	Newuser.mp4 00:05
3	Confirm button in new user information menu	Clicking opens the new user take picture window	New user information window closes, and capture image menu opens	Information menu closes	Pass	Newuser.mp4 00:08
4	Capture image menu capture image button	Clicking takes a picture of the user and then opens the image in a new window	New window opens displaying the taken image giving an option to the user to retake or confirm	Taken image is displayed	Pass	Newuser.mp4 00:10 – 00:22
5	No option in the image confirmation pop-up	Clicking no retakes the image and redisplay it	Pop-up and window close and a new window opens displaying the new image	Pop up closed, window closed, and new window opened displaying the image	Pass	Newuser.mp4 00:24
6	Yes, option in the image confirmation pop-up	Clicking yes opens the next menu for the user details to be inputted	Image window closes and pop-up closes, and new window opens for the user to enter their details	Image window closed and new window opened	Pass	Newuser.mp4 00:27
7	New user details name and email entry field	Entry fields allow for keyboard input	Keyboard input displays in the selected entry field.	Keyboard input is displayed in the correct field	Pass	Newuser.mp4 00:34
8	New user details dropdown boxes	Dropdown boxes display correct information when clicked and allow for options to be chosen	Clicking an option selects and displays it in the field	Correct option is displayed after it is selected	Pass	Newuser.mp4 00:45

9	Email validation	If the email that is entered is not in a correct format it is not accepted	Error popup opens when confirm button is pressed	Error popup opened	Pass	Newuser.mp4 00:47
10	Birthday validation	If the entered birthday is not a valid birthday it is not accepted	Error popup opens when confirm is pressed	Error popup opened	Pass	Newuser.mp4 00:55
11	New user details confirm button	Clicking the button accepts all the entry fields and saves them to the database	Current window closes and new user credit window opens	New user credit window opened	Pass	Newuser.mp4 01:00
12	Credit window buttons	Clicking the selected button adds the specific amount of credit to the database	The credit window closes, and the program goes back to the initial menu	Credit window closed	Pass	Newuser.mp4 01:00
13	Saving onto the database	The new user details are saved onto the database	When database is refreshed, the new details are displayed on a new record	The correct details are displayed on a new record	Pass	Newuser.mp4 01:01
14	Login button in the initial menu when the user does not have an account	Clicking the login window button scans the detected face and loops 3 times comparing it to every face in the database and determines no account	Opens up a new window informing the user that they do not have an account	New window opened, informing that the user must make an account	Pass	Log_in.mp4 00:05-01:17
15	Login error window exit button	Clicking the exit button closes the current window and reopens the initial menu	Window closes and initial menu opens again	Window closed and went back to the menu	Pass	Log_in.mp4 01:19
16	Login button in the initial menu when the user has an account	Clicking the login button scans the face and compares it to the database until it finds a match and opens the login window menu	Initial menu closes and the login menu opens	Login menu opened	Pass	Log_in.mp4 01:21-1:36
17	Login menu update details button	Clicking opens the update details menu	Update details menu pop-up opens	Update details popup opened	Pass	Log_in.mp4 01:38

18	Name update button	Clicking closes the pop-up and opens an menu to enter the new name	Pop-up closes and new window opens	Name update window opened	Pass	Log_in.mp4 01:44
19	New name menu	Entry field allows for a new name to be enter and the confirm button saves that name and closes the window when clicked	New name entry displays input characters and confirm button closes the window	Input characters are displayed, and the window closes when button is clicked	Pass	Log_in.mp4 01:51
20	New email menu entry	Entry field allows for a new name to be enter	Displays inputted characters inside of the entry	Characters are displayed in the entry	Pass	Log_in.mp4 01:55
21	New email confirmation button	When the confirmation button is clicked, and the email is valid the user email is saved	Closes email entry window	Email window closed	Pass	Log_in.mp4 02:16
22	New email validation	When the confirm button is clicked and the email is invalid, it will display an error	Error popup will open	Invalid entry pop up opened	Pass	Log_in.mp4 02:09
23	Add credit button in login menu	When the button is clicked, a popup will open giving the user an option of credit	Pop-up opens	Credit pop up opened	Pass	Log_in.mp4 02:19
24	£20 button in credit pop-up	Clicking the button adds 20 credits to the users existing credit	Pop-up closes	Credit pop up closed	Pass	Log_in.mp4 02:21
25	£10 button in credit pop-up	Clicking the button adds 10 credits to the users existing credit	Pop-up closes	Credit pop up closed	Pass	Log_in.mp4 02:26
26	Exit button in credit pop-up	Clicking closes the pop-up	Pop-up closes	Credit pop up closed	Pass	Log_in.mp4 02:30
27	Login window exit button	Clicking the button closes the login menu and reopens the initial user menu	Login menu closes and initial user menu opens again	Login window closed	Pass	Log_in.mp4 02:33
28	Log in details saved	After the log in menu is closed, all the details are saved to the database	Correct field on the database shows changed data depending	The correct details were changed	Pass	Log_in.mp4 02:41

			on the users changes once the database is refreshed	inside the database		
29	Admin only button in the initial user menu	Clicking the button opens a small window for an admin login	Admin login window opens	Admin login window opened	Pass	Admin.mp4 00:05
30	Admin entry fields	Admin username and password fields display inputted keyboards	Respective entry fields display the inputted keyboard characters	The correct entry field displayed the inputted characters or * in the place of characters for the password	Pass	Admin.mp4 00:16
31	Confirmation button – incorrect entry	Adding the incorrect details into the login entries and clicking the confirm button will open an error message	Error pop-up opens	Invalid entry pop up opened	Pass	Admin.mp4 00:22
32	Confirmation button – correct entry	When the username and password is correct and the confirmation button is clicked, the admin menu will open	Admin login closes and the admin menu opens	Admin login closed and admin menu opened	Pass	Admin.mp4 00:38
33	Admin window email button	When clicking the email button, the email menu opens	Email window opens	Email window opened	Pass	Admin.mp4 00:46
34	Email window entry fields	When typing into the entry fields they display the inputted characters	Inputted characters are displayed in the respective entry field	Inputted characters were displayed in the correct respective entry fields	Pass	Admin.mp4 00:54
35	Email validation	When the wrong email is input and confirmed an error popup appears	Error pop-up opens	Invalid entry pop up opened	Pass	Admin.mp4 00:56
36	Done button	When clicked the current entry fields are saved into the database and	Information pop-up opens	Information pop up opened	Pass	Admin.mp4 01:03

		information pop-up opens				
37	Exit button	When clicked it closes the admin email menu	Admin email window closes	Admin email window closed	Pass	Admin.mp4 01:07
38	Admin window name button	When clicking the name button, the name menu opens	Name window opens	Admin name window opened	Pass	Admin.mp4 01:12
39	Name window entry fields	When typing into the entry fields they display the inputted characters	Inputted characters are displayed in the respective entry field	The inputted characters were displayed in the respective entry fields	Pass	Admin.mp4 01:21
40	Name menu done button	When clicked the current entry fields are saved into the database and information pop-up opens	Information pop-up opens	Information pop up opened	Pass	Admin.mp4 01:25
41	Name menu exit button	When clicked it closes the admin name menu	Admin name window closes	Admin name window closed	Pass	Admin.mp4 01:28
42	Admin window date button	When clicking the date button, the date menu opens	Date window opens	Date window opened	Pass	Admin.mp4 01:33
43	Date menu ID entry	When typing into the ID entry field it displays the inputted characters	Inputted characters are displayed in the ID entry field	ID entry opened	Pass	Admin.mp4 01:37
44	Date menu dropdown boxes	When clicking the dropdown boxes, they display the correct set of values and when value is clicked it is set to the dropdown box	Clicked options are displayed in the dropdown box	Clicked option was displayed in the correct dropdown box	Pass	Admin.mp4 01:56
45	Date menu date validation	When clicking done and the date is invalid, it displays an error message	Invalid entry pop up opens	Invalid entry pop up opened	Pass	Admin.mp4 02:09
46	Date done button	When clicked the current fields are saved into the database and	Information pop-up opens	Information pop up opened	Pass	Admin.mp4 01:57

		information pop-up opens				
47	Date exit button	When clicked it closes the admin date menu	Admin date window closes	Admin date window closed	Pass	Admin.mp4 02:11
48	Admin menu credit button	Clicking the credit button opens the admin credit menu	Admin credit menu opens	Admin credit menu opened	Pass	Admin.mp4 02:16
49	Admin credit menu ID entry	When typing into the ID entry field it displays the inputted characters	Inputted characters are displayed in the ID entry field	Inputted characters are displayed in the ID entry	Pass	Admin.mp4 02:19
50	Admin credit menu credit entry	Typing into the entry field will display the inputted characters	Inputted characters are displayed in the credit entry field	Inputted characters are displayed in the credit entry field	Pass	Admin.mp4 02:26
51	Credit done button	When clicked the current entry fields are saved into the database and information pop-up opens	Information pop-up opens	Information pop up opened	Pass	Admin.mp4 02:28
52	Credit exit button	When clicked it closes the admin credit menu	Admin credit window closes	Admin credit window closed	Pass	Admin.mp4 02:31
53	Admin menu exit button	When clicked it closes the admin menu and re opens the admin login window	Admin menu closes and the admin login window opens	Admin menu closed and login window opened	Pass	Admin.mp4 02:33
54	Admin login exit button	When clicked it closes the admin login window	Admin login closes	Admin login closed	Pass	Admin.mp4 02:36
55	Admin changes onto database	Changes made in the admin menu get saved to the database	When the database is refreshed it displays the new data	Database displayed the correctly changed data	Pass	Admin.mp4 02:49

Barrier

Test No.	What is being tested	Input data or actions	Expected outcome	Actual Outcome	Pass/Fail	Fig
1	Camera feed facial detection	Camera feed is detected, and the user is detected in the feed	Camera feed is displayed to the user and green box is displayed around face	Green box is around the face in the displayed camera feed	Pass	1
2	Face detected against database – Not enough credit	When a face is recognised by the database it displays the appropriate message and doesn't charge the account	Window opens displaying not enough credit	Window opened displaying not enough credit	Pass	2
3	Face detected against database – child ticket	When a face is recognised by the database it displays the appropriate message and charges the account 3 credit	Window opens displaying child fare has been charged	Window opened displaying child fare has been charged	Pass	3
4	Face detected against database – concession ticket	When a face is recognised by the database it displays the appropriate message and charges the account 3 credit	Window opens displaying concession fare has been charged	Window opened displaying concession fare	Pass	4
5	Face detected against database - Adult ticket	When a face is recognised by the database it displays the appropriate message and charges the account 5 credit	Window opens displaying adult fare has been charged	Window opened displaying adult fare has been charged	Pass	5
6	Face detected against database - Student ticket	When a face is recognised by the database it displays the appropriate message and charges the account 4 credit	Window opens displaying student fare has been charged	Window opened displaying student fare has been charged	Pass	6

7	Detection label	If the face is detected then display in green that face is detected, if not then display in red that face isn't detected.	Label displayed with the correct colour and text depending on if the face has been found	Label displayed correctly when face isn't detected or charged as not open and displayed as open when face is detected and charged	Pass	7
---	-----------------	---	--	---	------	---

Fig 1

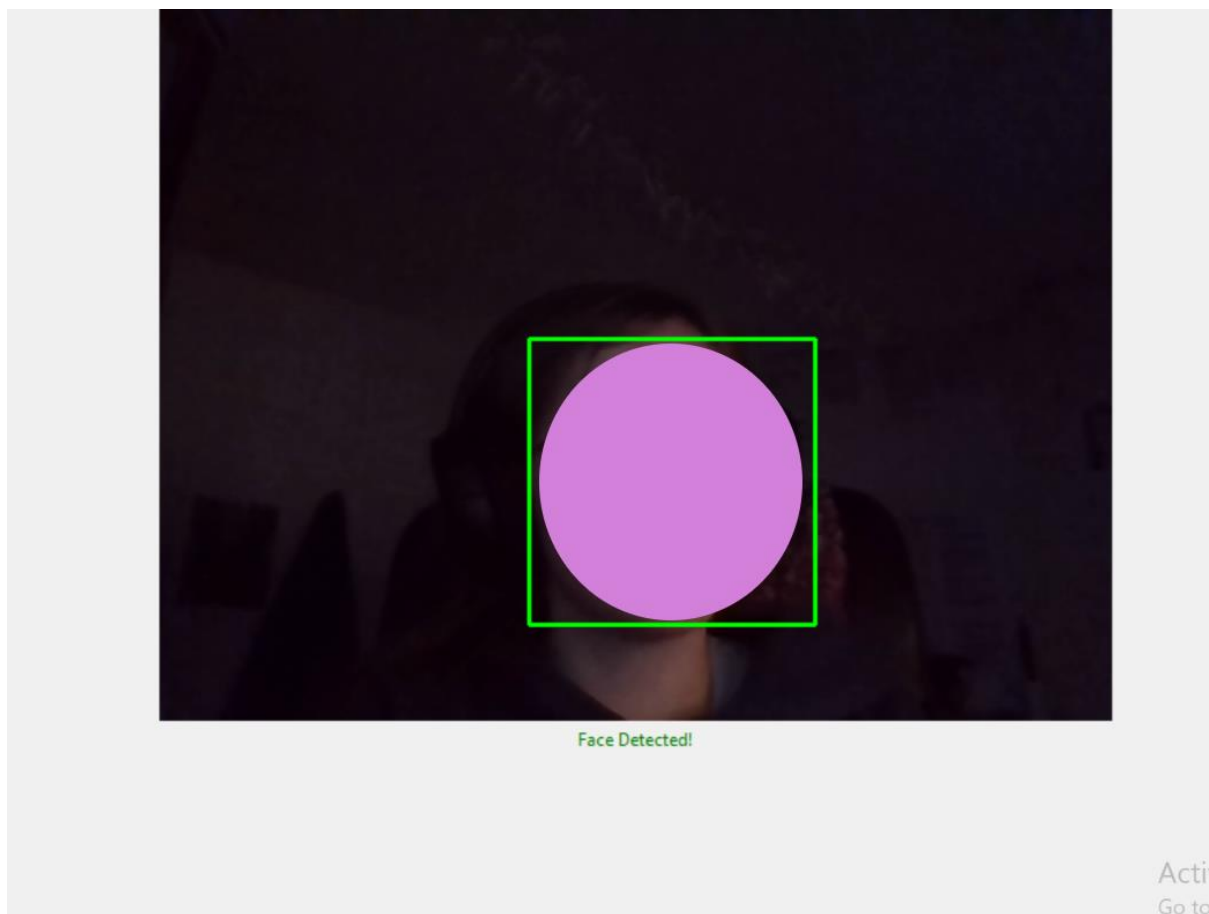


Fig 2

	A	B	C	D	E	F	G
1	5	newemail	New Nam	2007	2	5	0
2	2	email	name	year	month	date	credit

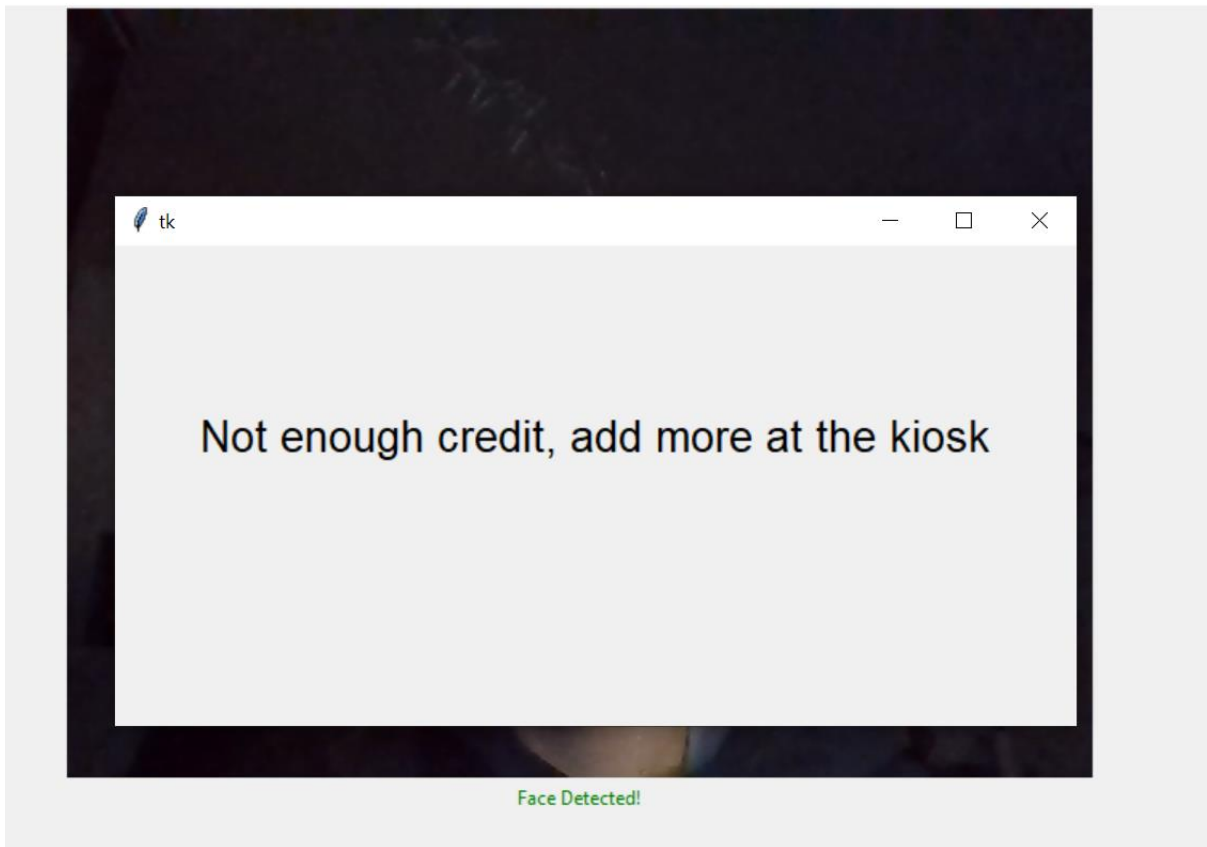


Fig 3

	A	B	C	D	E	F	G
1	5	newemail	New Nam	2007	2	5	40
2	2	email	name	year	month	date	credit

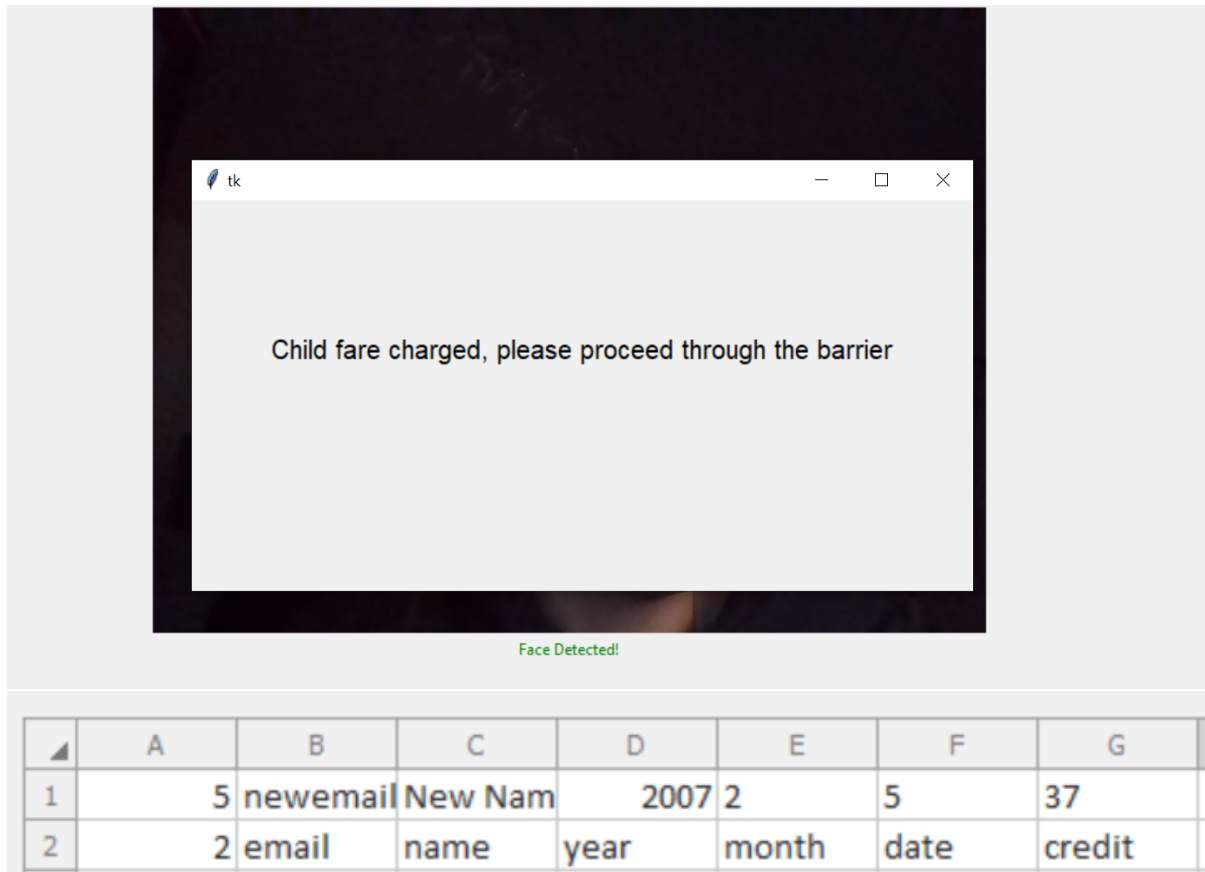
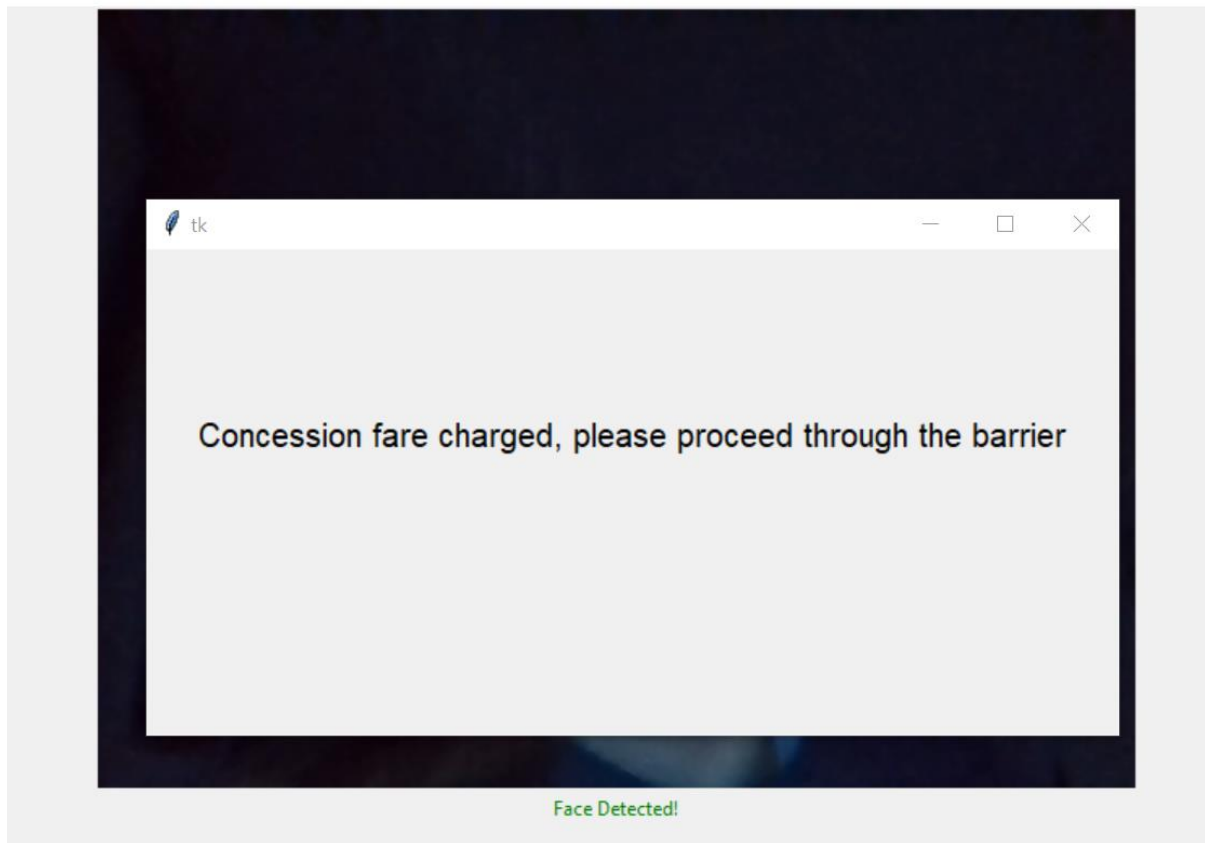


Fig 4

	A	B	C	D	E	F	G
1	5	newemail	New Nam	1950	2	5	40
2	2	email	name	year	month	date	credit
3	3	iwantit@l	Karen	1983	4	22	60
4	4	yake@gm	jake	2006	3	24	16
5	5	newmail@	testing na	1985	10	10	111
6							



	A	B	C	D	E	F	G
1	5	newemail	New Nam	1950	2	5	37
2	2	email	name	year	month	date	credit
3	3	iwantit@l	Karen	1983	4	22	60
4	4	yake@gm	jake	2006	3	24	16
5	5	newmail@	testing na	1985	10	10	111
6							

Fig 5

	A	B	C	D	E	F	G
1	5	newemail	New Nam	1990	2	5	40
2	2	email	name	year	month	date	credit
3	3	iwantit@l	Karen	1983	4	22	60
4	4	yake@gm	jake	2006	3	24	16
5	5	newmail@	testing na	1985	10	10	111

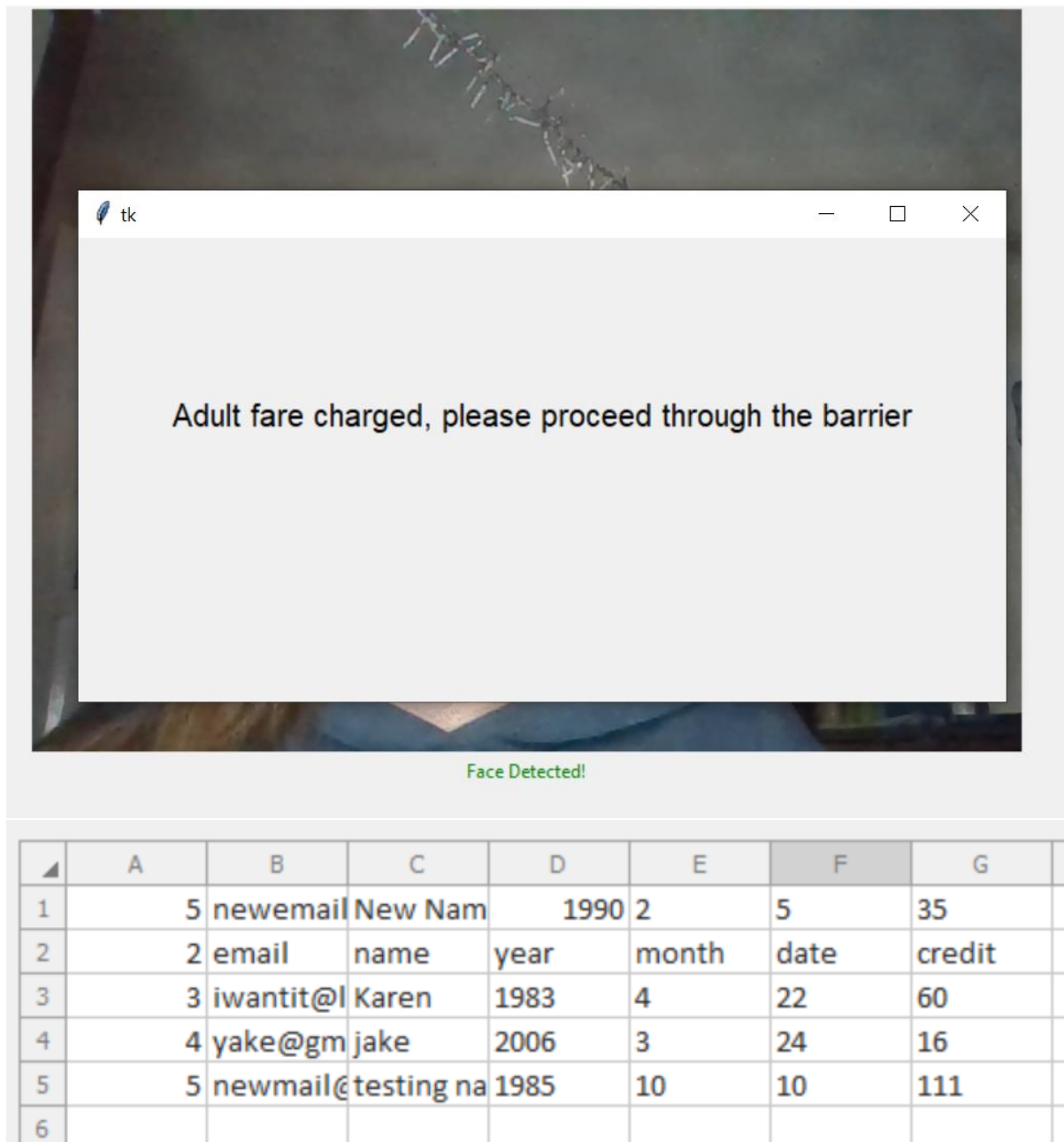
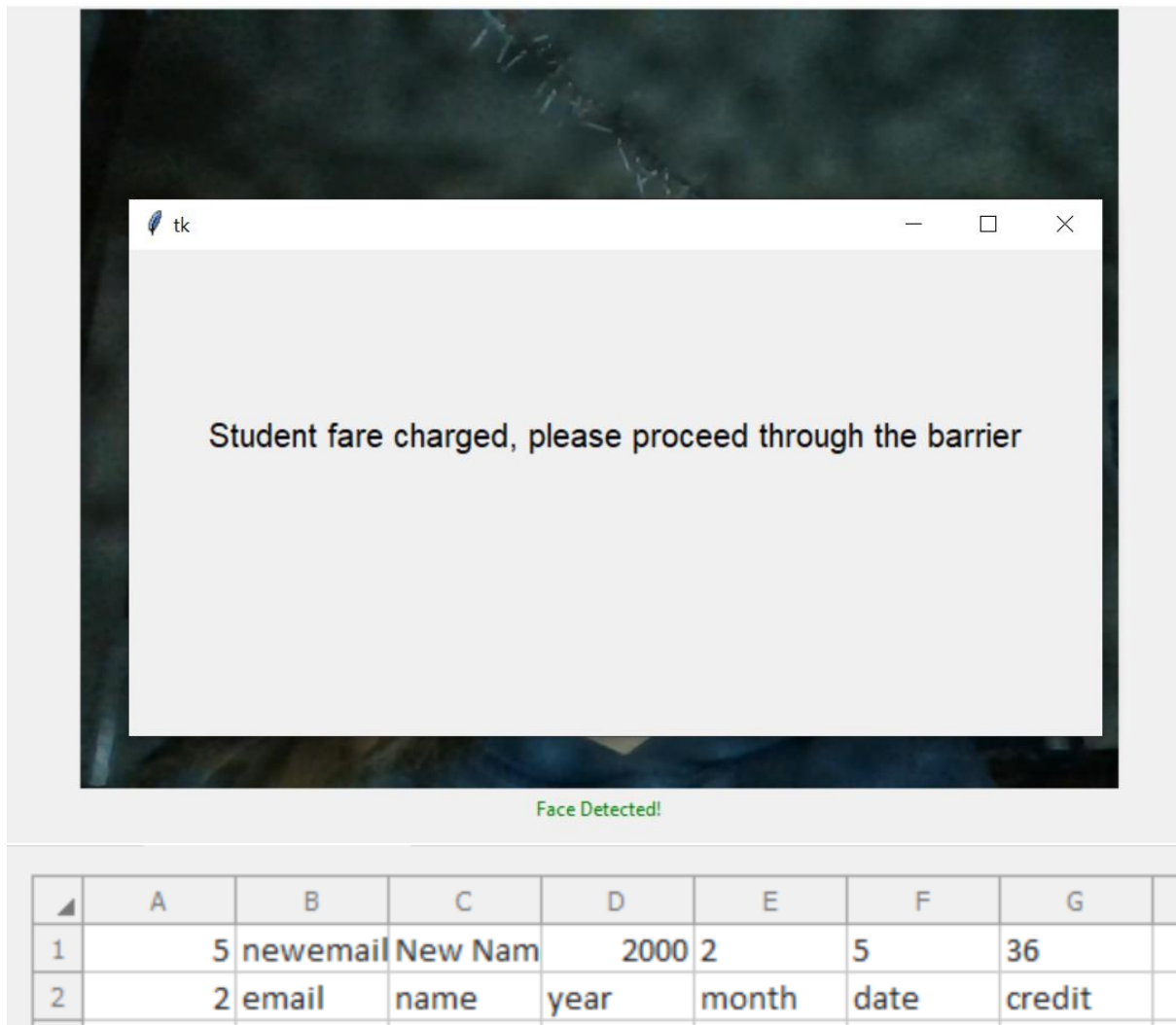


Fig 6

	A	B	C	D	E	F	G
1	5	newemail	New Nam	2000	2	5	40
2	2	email	name	year	month	date	credit
3	3	iwantit@l	Karen	1983	4	22	60
4	4	yake@gm	jake	2006	3	24	16
5	5	newmail@	testing na	1985	10	10	111
6							



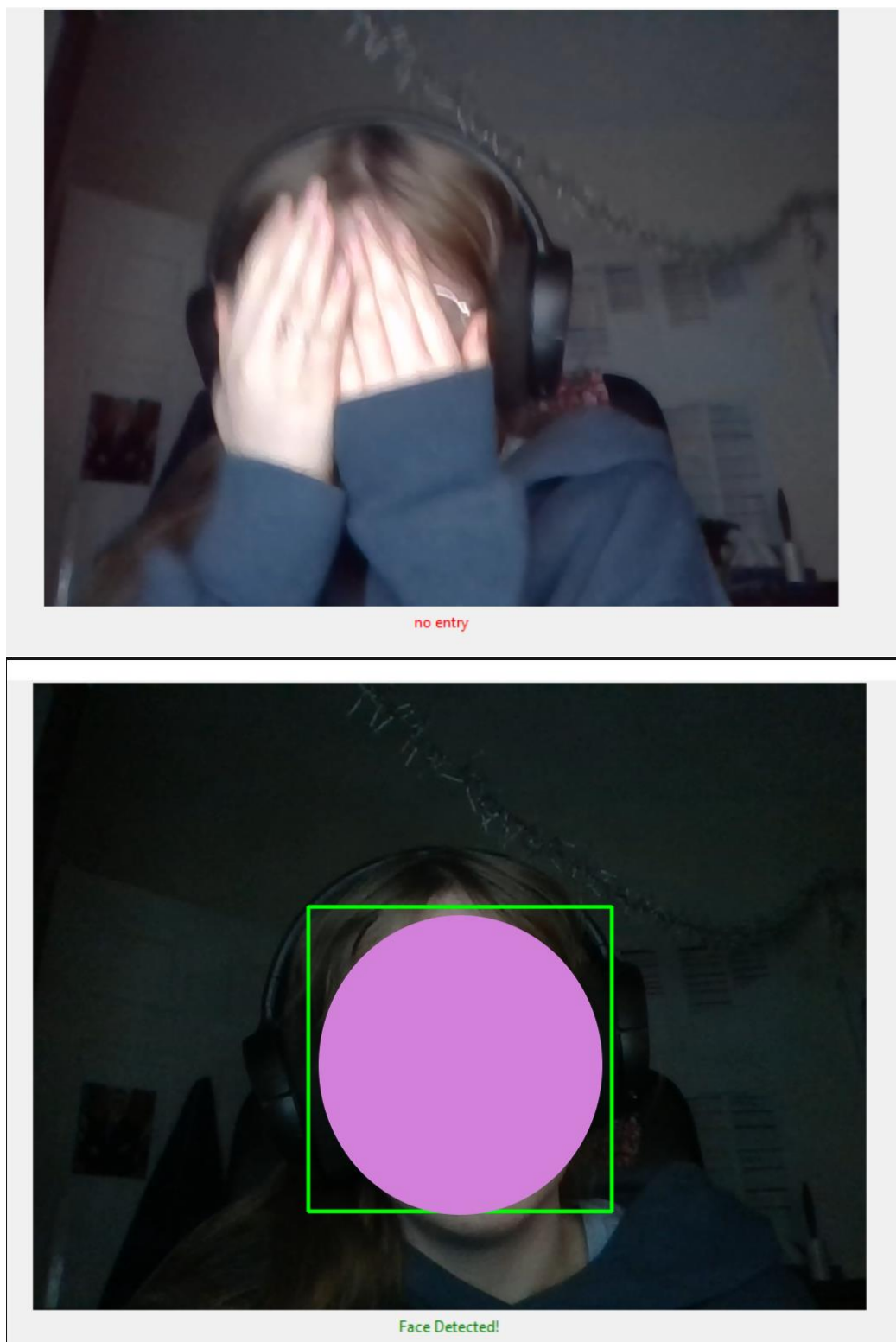
tk

Student fare charged, please proceed through the barrier

Face Detected!

	A	B	C	D	E	F	G
1	5	newemail	New Nam	2000	2	5	36
2	2	email	name	year	month	date	credit

Fig 7



Stakeholder testing

For my stakeholder testing, I will ask three out of four of my stakeholders for their input on the design of the program. I will ask both of my frequent rail users for their input since they are the users of the program and they have more of a stake in this part of the program creation process than the rail ticket officers. I will only ask one of the rail ticket officers since they personally do not provide much of a varying perspective than the frequent rail users. I will ask the users questions mainly about the usability features of the program rather than the functionality since the blackbox testing and the whitebox testing during the development will cover the functionality aspect of the code.

These are the questions I will be asking:

How was your overall experience using the software?

Was the user interface visually appealing?

Was it easy to navigate the application?

Were the instructions clear?

Were you charged the correct ticket price for your age?

Did you encounter any issues?

The following are the responses that I received from the three stakeholders:

Q1) How was your overall experience using the software?

John:

"My experience was good, the program functioned properly, and the program ran quickly."

Mark:

"The software was quick and easy to use, the key functionality of the program functioned as intended."

Sandra:

"My overall experience was positive, and I didn't encounter any bugs, but the user interface is quite ugly, and some sections didn't have instructions present, but it was simple to use and was fairly quick."

Q2) Was the user interface visually appealing?

John:

"Yes, the colour scheme was pleasant to look at and the instructions were big enough to read."

Mark:

"It was too bright for me, and the font used was hard to understand but the design of the UI was good just the colour and font choice, maybe a dark mode button to change how it looks?"

Sandra:

"The blue colour wasn't very appealing to me, combined with the white it looked quite dated, and the font choice of comic sans was not very suitable for the specific application of the program."

Q3) Was it easy to navigate the application?

John:

"Yes, except for the new user menu where I had to enter my details, instead of having Y, M and D for the dropdown boxes, it would be clearer if they were labelled as year, month and day."

Mark:

"It was quite easy to navigate, the buttons were labelled so that it was easy to understand what they did."

Sandra:

"It was easy to navigate to the correct sections and understand where I am supposed to click since all the buttons were clearly labelled and large enough to read."

Q4) Were the instructions clear?

John:

"The instructions were prompt and effective, I didn't have any issues understanding what to do."

Mark:

"There were not many instructions but the ones that I saw were clear, there was not description to what your program does when you first open the program."

Sandra:

"After I clicked the button to take a picture as a new user, it was quite confusing if I was supposed to click the button again as I didn't know if it was processing or not, when it was, so maybe add some text to tell the user that its loading? I also had the same issue when it's trying to recognise my face when logging in, the window closed, and I wasn't sure if I had broken it, or it was loading when it was loading."

Q5) Were you charged the correct ticket price for your age?

John:

"Yes, I was charged an adult ticket, and I am an adult."

Mark:

"I was charged the correct amount for my age the student fare."

Sandra:

"Yes, I am 29 and I was charged the adult ticket."

Q6) Did you encounter any issues?

John:

"Only one, just using the scroll down for the age it wasn't really easy to see that you had to click down on the side of the box and not scroll."

Mark:

"No issues with the program itself just the minor issues that I had with the UI."

Sandra:

"I did not encounter any issues within the program"

Stakeholder testing review

It was clear that none of the users had any issues with the functionality of the program and that it worked according to how it should work, as everybody answered no to question 6 (aside from John, which was more so an issue with communication rather than the functionality of the program). This displays that the functionality of the program was good and everything functioned to the intended usage.

Two out of the three users I surveyed had issues with the user interface, both not liking the font used and Mark had issues with it being in light mode whilst Sandra had issues with the colour scheme in general. This would be fixed, as suggested by Mark, by having options available to the users to choose the user interface colours themselves, choosing between light mode and dark mode. This is a small issue with my usability, slightly hindering how usable it is to a user, making them less enticed by the program, which is why I would fix it soon in the near future.

All users generally enjoyed using the application and commented on its simplicity and speed of use. The users all said it was easy to navigate since the buttons were clearly labelled but some aspects were not clearly enough explained. This shows that the usability of the program was good and users were able to use the program with ease and the usability features that I included were successful.

In the last question, all users stated that they did not have any issues with the functionality of the program whilst running it, not encountering any errors during the whole use of the program. This implies that the program as successfully robust as a program and will not break during usage.

Evaluation

Success criteria comparison

Key: Pass / Fail / Partial pass

Criteria	Pass or fail	Evidence
Display taken image on new user window		
Recognise a face from camera feed		
Be able to store details of a user on a database.		

Be able to retrieve details of a user on a database		
Store facial encodings of users		
Ability to compare a face to an already stored face		
A clear user display which conveys correct information based on the inputted data		User Feedback
Values on the database being altered depending on the user's actions		
Be able to create a new entry on the database including all relevant details that would be required to identifying a new user		
User interface which is easily navigate-able		User Feedback
Clearly displayed instructions for the user-on-user creation window.		User Feedback
Choice of credit for user after creating a window		
Correctly display user history in user history window		n/a
Charge the correct ticket price depending on user age and save the amount charged		User Feedback and test
Displayed if barrier is open or closed		
Barrier program refreshes after a few seconds to allow for the next user to be detected		
Inform user when lacking credit for a ticket		
Functional buttons throughout the program		User Feedback
Working entry boxes for user to enter details		
Functional dropdown boxes that allow a user to only select a fixed number of options to choose from		
Validation of email addresses entered in the new user entry stage		

Why some criteria weren't met

I didn't meet the criteria of "Correctly display user history in user history window" since I did not create the user history window as a part of my log in window as I had planned. This was due to it not being a necessary part of my program to run and it would've been too complex to create for its unnecessary and limited functionality. The lack of this section does not limit the usage of my program at this stage, but it would if my program was actually implemented into the real world as it would help administrators track payments that users name incase there are any wrong payments. Since my program does not use actual credit, it does not matter too much to the user if they are incorrectly

charged but this would be an issue once real credit would be implemented since users would not want to be incorrectly charged money.

Why some criteria were partially met

The criteria of “Store facial encodings of users” was partially met since the facial encodings of the user themselves is not stored but the faces are, which the facial encodings are taken from. This is the reason for it partially being met, since the face encodings can be found from the stored images, but it is not directly storing the encodings. The reasons for not storing the facial encodings directly were since I wasn’t sure of how I would store the encodings at the start of the program, but I knew how to store the faces using open CV, so I chose to store the faces. This also benefited me during testing period since I was able to see exactly what each stored file contained and who's face it was when I had errors sorting the wrong faces for users, but if I used facial encodings I would not have been able to check if the stored face had the correct encodings without using software to compare encodings. Storing facial encodings is more beneficial to my code since it uses less space within the program and is faster to process. I have abstracted the faces slightly by removing the background from the saved images, but the images still contain a lot more data than just storing a facial encoding. The facial encodings are also a lot more safe to store since it masks the identity of the users who are on my database, since if somebody was able to access my database of users and find the images, they would have access to images of users.

The criteria of “User interface which is easily navigate-able” was only partially met as shown in my user feedback in the stakeholder testing as not all users agreed that it was fully easily navigable. Some users had issues knowing whether they should press buttons at certain sections and wished that there would be more instructions. One user stated that they wished the user interface had more instructions during the start, describing what the program does and what the user needs to do and the functionality of the buttons and different programs which can be very easily implemented into the code. Other users didn't not have any other issues with the instructions, and everybody said that other sections of the code were very easy to follow the code since all buttons were clearly labelled and all screens had instructions on there, describing what the user must do.

The criteria of “Clearly displayed instructions for the user-on-user creation window.” was also only partially met since not all stakeholders agreed on it being clearly displayed. Two out of three users had an issue with the instructions for the user creation window, where one of the issues was that the user wasn’t sure that once the image taking process is running, that it was actually running, and not knowing if they need to click the button again since the program did not indicate that it was currently processing the camera feed. This can be easily fixed by adding a window that pops up once the take picture button has been pressed, informing the user to wait whilst the program is processing the taking of the picture, making it more clear to the user. Another user indicated that the dropdown boxes weren’t very clear, being labelled Y, M, and D instead of year, month and date. I can easily also fix this issue by changing the titles of each of the dropdown boxes in the menus to the non-abbreviated titles to make it easier for users to understand.

Changes during Development

During the development, one of the largest changes I made to the program was getting rid of the user history section in the log in feature of the program. I decided to make this change since it would be

quite hard to store every user's history and would require a larger amount of storage space. I would either have had to create individual lists that get stored somehow for each user or create a large 3D array, containing the history of all users. This would be quite hard to store, since with the credit I would have to contain the date and time, the transaction type and the amount of credit added or gained. This would also be hard to display to the user since a user could have more history than what can be displayed on the screen so then it could possibly cut off weirdly. It would also take long to process every individual history entry and then display it back to the user. For these reasons I decided currently not to implement the history feature since it is not crucial to the running of my program, it is just a convenient addition to it.

An addition that I made to my program was the addition of the admin menu to the user menu. I decided to add the admin menu to make my program more complex and create a way to fix any errors that a user may encounter whilst using the application. For example, if a user accidentally sets their email to the wrong email, they could alert a staff member who can use the admin section to manually change the email to the correct email, without having to alter the database directly by opening it and accessing every possible user's data.

Improvements to the program

An improvement to the program would be changing the font of the program to a more serious font such as Calibri or Arial. Users during the stakeholder testing said that the font was not an appropriate font for use of an application such as this and that it was not visually appealing at all.

Another improvement that I would like to add to my program would be storing the faces of my users as face encodings instead of full images. There are many benefits to storing them as facial encodings, such as increased security incase someone enters my system and accesses the data on my system, if they saw user's faces, they would easily be able to identify users but facial encodings cannot be used to turn into faces so it would keep the identity of users anonymous. Another advantage would be that it would save a lot of space, since facial encodings are just 2D arrays that include numbers which take up a lot less space than an image file which must store a lot more data per user. It also makes processing the faces a lot quicker during the log in stage of the program and the barrier section of the program since it must find the encodings of each face anyway.

Future additional features

A feature that I did not implement into my program was validation for leap year birthdays. Whilst I validated birthdays to be correct, I did not check if a person input 29th February on a leap year, meaning that if a user tried to enter a leap year birthday, they would be met with an error message even though it is a real birthday.

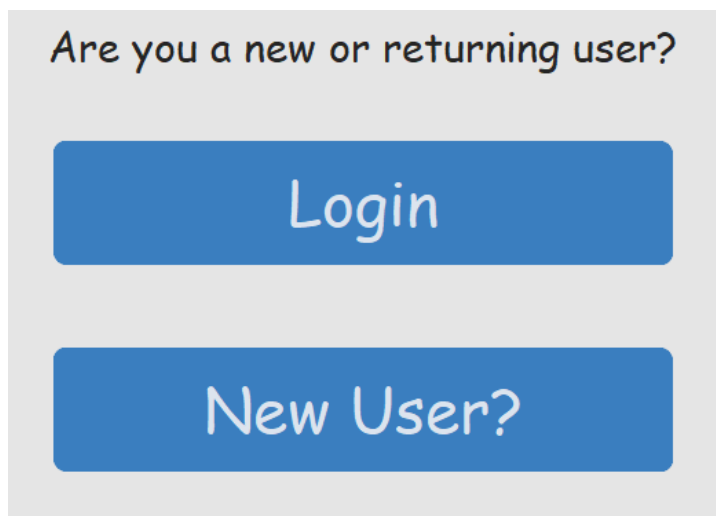
Another feature that I would need to add if the userbase got larger would be implementing a better searching algorithm for the users when comparing faces. Currently I have a linear search of the users faces images, finding the encodings every time but this makes the program quite slow so to improve my program I would store the images as face encodings instead and use a different sorting algorithm.

If the program is fully implemented into the usage of it in the real world, then I would probably need to implement SQL instead of using an excel document as my userbase. SQL is good for large databases since it is a lot faster at processing query's than excel, since that's what it is designed for. When the program would be in real life use, the database would get large so using openpyxl and an excel spreadsheet could slow down the program majorly. SQL also has integrated security built into it, making the database a lot more secure.

Another feature I would like to add in the future would be a background colour changer, from light mode to dark mode or vice versa as per request of my stakeholders. One of my stakeholders said that the light background of the screen was too bright for them to use, so to avoid this and make it easier for everyone to use, I would add a feature that allows for users to choose if they would like to choose light mode or dark mode.

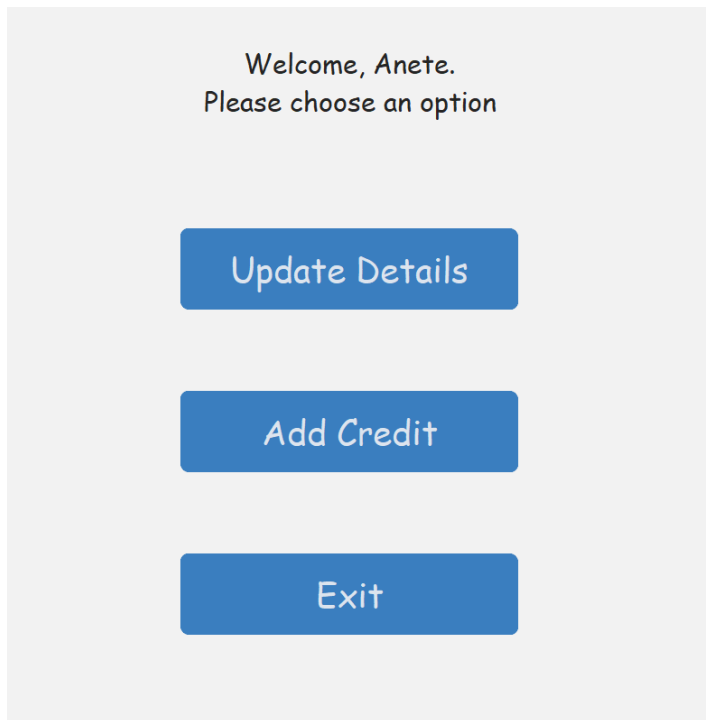
Usability features

Throughout my program, I have well labelled buttons where the function corresponds well to the label and inside of windows, I have text describing what the user must do within this window. This makes navigating the program easy, meaning that users will not get lost inside of the program. The colour of the buttons contrasts the colour of the rest of the screen which makes the buttons more emphasised, so users are immediately drawn to them.

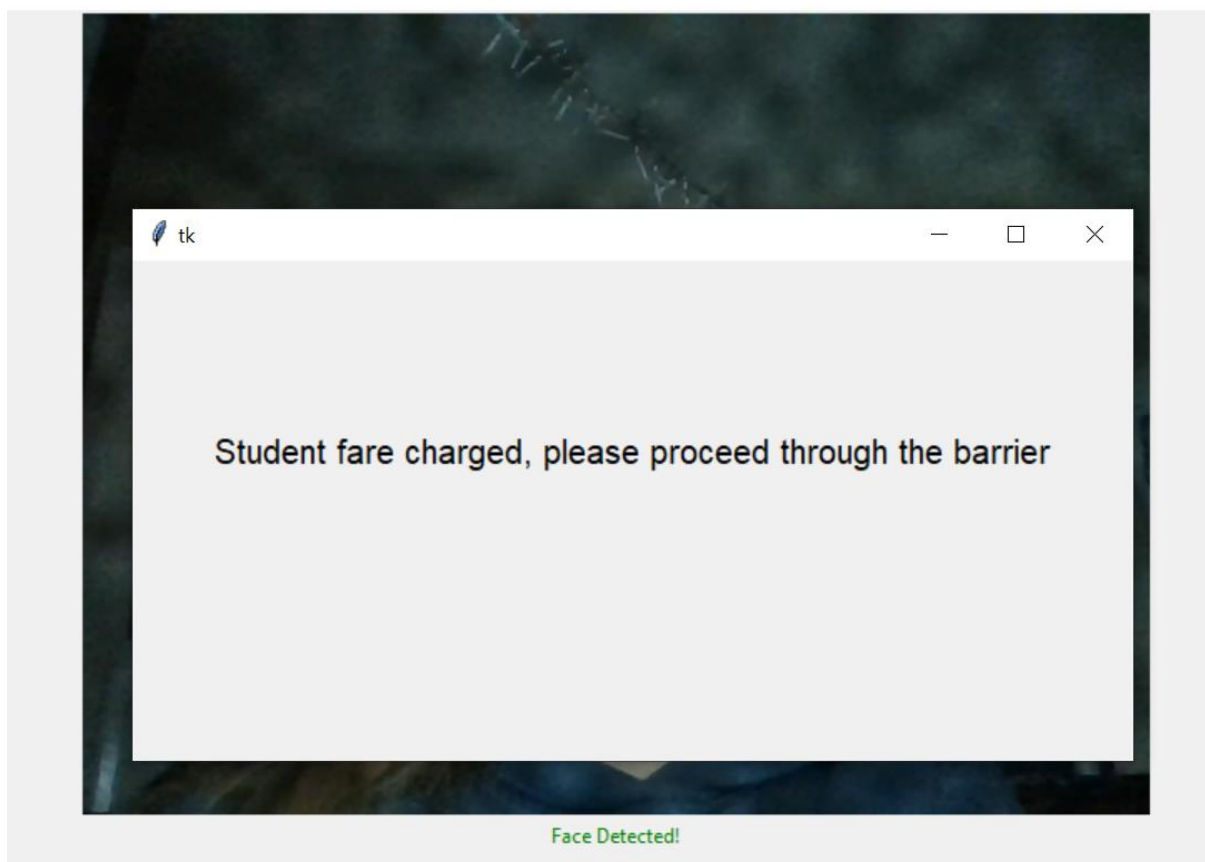


The font on the buttons is larger than the font used in the instructions, so they are easier to read and more clear.

I have also implemented a consistent colour scheme throughout the program, which creates familiarity for the user and does not confuse them or overwhelm them. The colour scheme I have currently used is a white main colour with blue accents for buttons.



The barrier program has very good usability, since the users do not actually need to do anything within the program itself except walk up to it and read the display once it has processed the face for instructions on what to do.



The instructions cover the face of the user once the user has been detected to hide the identity of the user from the following users as the next user comes to use the program. The instructions are in a

large font so that they are clearly visible and easy to read by the users and are placed in the centre of the screen to also draw the attention of the user to the instructions.

Future maintenance

As time passes, python libraries get frequent updates which may cause certain parts of my program to not work exactly as before. To maintain the program, I would have to be aware of these updates and be ready to alter the code to match the new library code. This would be easy to do since each individual action inside of my program is contained in functions and large parts are divided into modules, allowing rewriting code to be a lot easier. By using functions and modules, it means that the code inside the module/function is isolate from the code outside of it, so it only affects the code that is inside of the same area, making it a lot easier to change the program and reduce errors.

Limitations

One of my biggest limitations is that I do not have a touch screen computer to test the kiosk program on. For real implementation I would have wanted the kiosk program to be touchscreen for the user to use since it is easier for them to use instead of using a trackpad, but I do not own a touchscreen device that I can use python on to test if the functionality of it works. Touchscreen would be easier to use for users since it saves time and is simpler to control for the users.

Another big limitation of my program is the fact that I do not have a physical barrier that I can test my barrier application on. When a user is charged for a ticket, I just have to state that they have been charged and that the barrier is open when no physical barrier has been adjusted at all. This limits how I can code my program since I cannot test adding barrier functionality to my code, I just have to print that the barrier is opened or closed.

Another limitation of my program is that it cannot detect when users are identical twins, which would have to make them use the same account and charge them from the same tickets. This is due to their faces being identical so that when the program detects them and recognises them it cannot understand which the difference between them, since they would have the same facial encodings received from their images.

Another limitation of my program is lighting, where if the user is in bad lighting, it will not detect their face or be able to find the facial encodings of the face since it is not clear enough. This may cause errors sometimes since the program thinks there is a face since the program would have found it using the face detecting but it wouldn't be able to find the encodings of the face, which would cause an error with the face encodings function.

Another limitation of the development and running of the program is the computer that I am testing and running the program on. Since it is not particularly fast as a computer and does not have a graphics card (has integrated graphics) it is slower at completing things such as finding face encodings of users which are more graphics based. This slows down the processing speed of fetching and decoding of the program, which makes it less easy to use for users who are testing it.

Another limitation of my program is that my program does not utilise real money. This is since I am unsure of how to connect a real banking account to a python program and since I am not actually using the program in a real scenario, I do not need to use real money. Instead of money I use arbitrary "credit" which is easily added to an account by just clicking the buttons in either the user creation menu or the log in menu. I also cannot actually add real money if I wanted to since I do not have a

bank account that would receive the money added for the application to test it even if I knew how to code it.

Preventing the limitations:

- Use a computer that has a touch screen.
- Getting a physical barrier to implement into the program
- Find a more accurate facial recognition that can differentiate between twins
- Ensure that lighting near the user is always good
- Ensure that the pc that the program is being ran on has a GPU and CPU which is of adequate processing power
- Including an actual way to add real money onto the program

Finished program
"Stage1.py"

```

1  import cv2
2  import face_recognition
3
4
5  # function to find the face encodings of the inputted user
6  def find_face_encodings(image_path):
7      # reading image parameter
8      image = cv2.imread(image_path)
9      # get face encodings from the read image
10     face_enc = face_recognition.face_encodings(image)
11     # return the first value of the face encodings array
12     return face_enc[0]
13
14     # define a video capture object
15     vid = cv2.VideoCapture(0)
16
17     # load haar cascade
18     face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
19
20
21     # always loop
22     while (True):
23
24         # Capture the video frame by frame
25         ret, frame = vid.read()
26
27
28         # Convert the frame to grayscale
29         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
30
31         # Detect faces in the frame
32         faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5
33                                             , minSize=(30, 30))
34
35
36         # Draw a rectangle around the detected faces
37         for (x, y, w, h) in faces:
38             cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

```

```

39
40
41     # Display the resulting frame
42     cv2.imshow('frame', frame)
43
44     #if a face is present, save the face as temporary frame
45     if len(faces) > 0:
46         face_frame = frame[faces[0][1]:faces[0][1] + faces[0][3], faces[0][0]:faces[0][0] + faces[0][2]]
47         cv2.imwrite("temp.jpg", face_frame)
48
49         # getting face encodings for an exisiting image
50         image_1 = find_face_encodings("1.jpg")
51         # getting face encodings for temporary frame
52         image_2 = find_face_encodings("temp.jpg")
53
54         # checking both images are same
55         is_same = face_recognition.compare_faces([image_1], image_2)[0]
56         print(f"Is Same: {is_same}")
57
58     # the 'q' button is set as the
59     # quitting button you may use any
60     # desired button of your choice
61     if cv2.waitKey(1) & 0xFF == ord('q'):
62         break
63
64
65     # After the loop release the cap object
66     vid.release()
67     # Destroy all the windows
68     cv2.destroyAllWindows()

```

“face_encode.py”

```
1 import cv2
2 import face_recognition
3
4 # create the return empty function, taking image path as its parameter
5 def return_empty(image_path):
6     # reading image parameter
7     image = cv2.imread(image_path)
8     # get face encodings from the image
9     face_enc = face_recognition.face_encodings(image, model="small")
10
11     # if face encodings don't exist, return True
12     if not face_enc:
13         return True
14
15     # if face encodings exist, return False
16     else:
17         return False
18
19 # create the find face encodings function, taking image path as its parameter
20 def find_face_encodings(image_path):
21     # reading image
22     image = cv2.imread(image_path)
23     # get face encodings from the image
24     face_enc = face_recognition.face_encodings(image, model="small")
25
26     #if face encodings cannot be found, print error
27     if not face_enc:
28         print("error")
29
30     # when face encodings are present, return the first set of encodings from the face_encodings array
31     else:
32         return face_enc[0]
```

"Thekiosk.py"

```
1 import customtkinter as ctk
2 from openpyxl.reader.excel import load_workbook
3
4 import new_user
5 import log_in
6 import admin
7 # selecting gui color and theme
8 ctk.set_appearance_mode("light")
9 ctk.set_default_color_theme("dark-blue")
10
11 wb = load_workbook(filename='userbase.xlsx') # load workbook
12 userBase = wb['Sheet1'] # load the correct workbook
13 userCount = userBase['A1'] # get the value for the amount of current users
14 userCount = userCount.value
15
16 font20 = ("Comic Sans MS", 20)
17 font25 = ("Comic Sans MS", 25)
18 font30 = ("Comic Sans MS", 30)
19 program = True
20
21
22 app = ctk.CTk() # initialise the app
23
24 # screen size geometry
25 screen_width = app.winfo_screenwidth()
26 wcentre = (screen_width // 2) - (screen_width // 2)
27
28 screen_height = app.winfo_screenheight()
29 hcentre = (screen_height // 2) - (screen_height // 2)
30
31 geometry = f"{screen_width}x{screen_height}+{hcentre}+{wcentre}"
32
33 app.geometry(geometry) # sets the size of the screen
34 app.title("Kiosk application") # application title bar
35
```

```

36 def login():
37     app.destroy() # close the previous window
38     same = log_in.login_check()
39     if same == True:
40         log_in.user_editmenu(geometry)
41     else:
42         log_in.loginFail(geometry)
43
44 def newuser():
45     def cancel():
46         newuser_confirmation.destroy() # destroy the confirmation window
47         print("Cancel")
48
49     def confirm():
50         global userCount
51         userCount += 1 # increment user count by one
52         userBase['A1'] = userCount # update user count
53
54         strUserCount = str(userCount)
55         filename = strUserCount + '.jpg' # set the filename to the usercount.jpg
56
57         app.destroy() # destroy the app
58         newuser_confirmation.destroy() # destroy new user confirmation
59
60     def call_take_picture():
61         new_user.take_picture(filename, newuser_window)
62
63     # new user window initialises
64     newuser_window = ctk.CTk()
65     newuser_window.title("New User Window")
66     newuser_window.geometry(geometry)
67
68     # create the text label informing user to stand still
69     label = ctk.CTkLabel(newuser_window, text="Please stand still and look at the camera.",
70                         font=font20)
71     label.pack(pady=200)
72

```

```

109 label.pack(pady=12, padx=10)
110
111 # login button calls the login command when clicked, stored in frame
112 loginButton = ctk.CTkButton(master=frame, text='Login', command=login, font=font30, height=60, width=300)
113 loginButton.pack(pady=20, padx=5)
114
115 # new user button calls the newuser command when clicked, stored in frame
116 newUserButton = ctk.CTkButton(master=frame, text='New User?', command=newuser, font=font30, height=60,
117                             width=300)
118 newUserButton.pack(pady=20, padx=15)
119
120 # admin button calls the admin command when clicked, stored in frame
121 adminButton = ctk.CTkButton(master=frame, text="Admin Only", command=admin.admin)
122 adminButton.pack(pady=160)
123
124 app.mainloop()
125

```

“new_user.py”

```

1 import face_recognition
2 from openpyxl.reader.excel import load_workbook
3 import customtkinter as ctk
4 import cv2
5 from CTkMessagebox import CTkMessagebox
6 import time
7 from PIL import Image, ImageTk
8 import re
9
10 from CTkScrollableDropdown import CTkScrollableDropdown
11
12 # declare v1 as false
13 v1 = False
14 # declare fonts
15 font15 = ("Comic Sans MS", 15)
16 font20 = ("Comic Sans MS", 20)
17 font25 = ("Comic Sans MS", 25)
18 font30 = ("Comic Sans MS", 30)
19
20 wb = load_workbook(filename='userbase.xlsx') # load workbook
21 userBase = wb['Sheet1'] # load the correct workbook
22 userCount = userBase['A1'] # get the value for the amount of current users
23 userCount = userCount.value
24 userCount += 1 # increment usercount by 1
25 userBase['A1'] = userCount # save usercount
26
27 def setData(letter, userNumber, variable):
28     coord = letter + str(userNumber) # find the coordinate it needs to be set to
29     userBase[coord] = variable # set the variable to the set coordinate on the userbase
30
31
32 def credit(geometry, userCount):
33     # initialise credit window
34     credit_window = ctk.CTk()
35     credit_window.title("Credit Window")
36     credit_window.geometry(geometry)
37
38     # credit label, telling user to choose credit option
39     creditTxt = ctk.CTkLabel(credit_window, text="Choose how much credit you would like to add",
40                             font=font30)
41     creditTxt.pack(pady=30)

```



```

41     creditTxt.pack(pady=30)
42     print(userCount)
43
44     def nocredit():
45         setData("G", userCount, 0) # use setdata function to set the users credit to 0
46         credit_window.destroy()
47         wb.save('userbase.xlsx')
48
49     def fivecredit():
50         setData("G", userCount, 5) # use setdata function to set the users credit to 5
51         credit_window.destroy()
52         wb.save('userbase.xlsx')
53
54     def tencredit():
55         setData("G", userCount, 10) # use setdata function to set the users credit to 10
56         credit_window.destroy()
57         wb.save('userbase.xlsx')
58
59     def twentycredit():
60         setData("G", userCount, 20) # use setdata function to set the users credit to 20
61         credit_window.destroy()
62         wb.save('userbase.xlsx')
63
64     # no credit button calls no credit command when clicked
65     noButton = ctk.CTkButton(credit_window, text="No Credit", command=nocredit, height=60, width=250,
66                             font=("Comic Sans MS", 25))
67     noButton.pack(pady=30)
68
69     # five credit button calls five credit command when clicked
70     fiveButton = ctk.CTkButton(credit_window, text="£5", command=fivecredit, height=60, width=250,
71                               font=("Comic Sans MS", 25))
72     fiveButton.pack(pady=30)
73
74     # ten credit button calls ten credit command when clicked
75     tenButton = ctk.CTkButton(credit_window, text="£10", command=tencredit, height=60, width=250,
76                              font=("Comic Sans MS", 25))
77     tenButton.pack(pady=30)
78
79     # twenty credit button calls twenty credit command when clicked
80     twentyButton = ctk.CTkButton(credit_window, text="£20", command=twentycredit, height=60, width=250,

```

```

79     # twenty credit button calls twenty credit command when clicked
80     twentyButton = ctk.CTkButton(credit_window, text="£20", command=twentycredit, height=60, width=250,
81                                  font=("Comic Sans MS", 25))
82     twentyButton.pack(pady=30)
83
84     credit_window.mainloop()
85     wb.save("userbase.xlsx")
86
87
88 def take_picture(filename, newuser_window):
89     def check_result():
90
91         image = cv2.imread(filename) # load the image taken
92
93         face_enc = face_recognition.face_encodings(image) # find face encodings of the image taken
94
95         if face_enc:
96             # create a message box asking if the image is fine, options are yes and no
97             msg = CtkMessageBox(title="", message="Is this image fine?", font=font20,
98                                icon="question", option_1="Yes", option_2="No")
99             response = msg.get()
100
101             if response == "Yes":
102                 global v1
103                 v1 = True
104
105         # create the cap variable that stores the video capture
106         cap = cv2.VideoCapture(0)
107         # Load the Haar Cascade classifier for face detection
108         face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
109
110         start_time = time.time()
111         while v1 == False:
112             current_time = time.time()
113             total_time = start_time - current_time
114             if total_time > 10:
115                 # create a message box telling the user there was a detection error
116                 msg = CtkMessageBox(title="Detection Error!", message="Unable to detect face, adjust your environment!",
117                                    icon="warning", option_1="Exit", option_2="Retry")
118

```

```

118
119     if msg.get() == "Exit":
120         newuser_window.destroy()
121         break
122
123     ret, frame = cap.read()
124     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) # set the video color to gray
125     faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30)) # find the faces inside of the gray capture
126
127     # Draw a rectangle around the detected faces
128     for (x, y, w, h) in faces:
129         cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
130
131     # Check if a face is detected
132     if len(faces) > 0:
133         # Save the frame as an image file without the green box
134         face_frame = frame[faces[0][1]:faces[0][1] + faces[0][3], faces[0][0]:faces[0][0] + faces[0][2]]
135         cv2.imwrite(filename, face_frame)
136
137         # display the image to the user and ask if it's okay
138         img = Image.open(filename)
139         img = img.resize((int(img.width * 3), int(img.height * 3))) # Increase the size of the image by 200%
140         tk_img = ImageTk.PhotoImage(img)
141
142         img_window = ctk.CTkToplevel()
143
144         # create the geometry of the window
145         screen_width = img_window.winfo_screenwidth()
146         wcentre = (screen_width // 2) - (screen_width // 2)
147
148         screen_height = img_window.winfo_screenheight()
149         hcentre = (screen_height // 2) - (screen_height // 2)
150
151         geometry = f"{screen_width}x{screen_height}+{hcentre}+{wcentre}"
152
153         # initialise the image window window
154         img_window.title("New User Image")
155         img_window.geometry(geometry)
156         img_window.transient(newuser_window) # make img_window a transient window of newuser_window
157

```

```

157
158         img_label = ctk.CTkLabel(img_window, image=tk_img)
159         img_label.pack()
160         check_result()
161
162     img_window.destroy()
163
164     newuser_window.destroy()
165
166     # initialise the new details window
167     newdetails_window = ctk.CTk()
168     newdetails_window.title("New User Window")
169     newdetails_window.geometry(geometry)
170
171     # display instructions of the window to the user
172     label = ctk.CTkLabel(newdetails_window,
173         text="Please enter your name, email address and date of birth (Year, Month, Date)",
174         font=font20)
175     label.pack(pady=20)
176
177     # create name entry field
178     name_var = ctk.StringVar()
179     nameEntry = ctk.CTkEntry(master=newdetails_window, placeholder_text="Enter your Name", width=250, height=30,
180         border_width=2, corner_radius=10, font=font15)
181     nameEntry.pack(pady=30)
182
183     # create email entry field
184     emailEntry = ctk.CTkEntry(master=newdetails_window, placeholder_text="Enter your Email Address", width=250,
185         height=30, border_width=2, corner_radius=10, font=("Comic Sans MS", 15))
186     emailEntry.pack(pady=30)
187
188     # create the list of possible values for dates, months and years
189     dates = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
190         15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31]
191
192     months = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
193
194     years = [2024, 2023, 2022, 2021, 2020, 2019, 2018, 2017, 2016, 2015, 2014, 2013, 2012, 2011, 2010, 2009,
195         2008, 2007, 2006, 2005, 2004, 2003, 2002, 2001, 2000, 1999, 1998, 1997, 1996, 1995, 1994, 1993,
196         1992, 1991, 1990, 1989, 1988, 1987, 1986, 1985, 1984, 1983, 1982, 1981, 1980, 1979, 1978, 1977,
197         1976, 1975, 1974, 1973, 1972, 1971, 1970, 1969, 1968, 1967, 1966, 1965, 1964, 1963, 1962, 1961,

```

```

194     years = [2024, 2023, 2022, 2021, 2020, 2019, 2018, 2017, 2016, 2015, 2014, 2013, 2012, 2011, 2010, 2009,
195             2008, 2007, 2006, 2005, 2004, 2003, 2002, 2001, 2000, 1999, 1998, 1997, 1996, 1995, 1994, 1993,
196             1992, 1991, 1990, 1989, 1988, 1987, 1986, 1985, 1984, 1983, 1982, 1981, 1980, 1979, 1978, 1977,
197             1976, 1975, 1974, 1973, 1972, 1971, 1970, 1969, 1968, 1967, 1966, 1965, 1964, 1963, 1962, 1961,
198             1960, 1959, 1958, 1957, 1956, 1955, 1954, 1953, 1952, 1951, 1950, 1949, 1948, 1947, 1946, 1945,
199             1944, 1943, 1942, 1941, 1940, 1939, 1938, 1937, 1936, 1935, 1934, 1933, 1932, 1931, 1930]
200
201     # create a year dropdown box
202     yearbox = ctk.CTkComboBox(newdetails_window, width=250, values="Year")
203     yearbox.pack(pady=30)
204     CTkScrollableDropdown(yearbox, values=years, justify="left", button_color="transparent")
205
206     # month dropdown box
207     monthbox = ctk.CTkComboBox(newdetails_window, width=250, values="Month")
208     monthbox.pack(pady=30)
209     CTkScrollableDropdown(monthbox, values=months, justify="left", button_color="transparent")
210
211     # date dropdown box
212     datebox = ctk.CTkComboBox(newdetails_window, width=250, values="Day")
213     datebox.pack(pady=30)
214     CTkScrollableDropdown(datebox, values=dates, justify="left", button_color="transparent")
215
216     def confirm():
217         email = emailEntry.get() # retrieve the email entered
218         global userCount
219         if yearbox.get() != "Y" and monthbox.get() != "M" and datebox.get() != "D": # validate the buttons to have values
220             if monthbox.get() == "2" and (datebox.get() == "29" or datebox.get() == "30" or datebox.get() == "31"): # validate the february month
221                 CTkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
222             elif (
223                 monthbox.get() == "4" or monthbox.get() == "6" or monthbox.get() == "9" or monthbox.get() == "11" and datebox.get() == "31": # validate other months
224                     CTkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
225             elif not re.match(r"^[a-zA-Z0-9]{4,}@[a-zA-Z0-9]{4,}\\.([a-zA-Z]{2,})$", email): # validate the email
226                 CTkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
227             else:
228                 # set the inputted data to the correct fields
229                 setData("F", userCount, datebox.get())
230                 setData("E", userCount, monthbox.get())
231                 setData("D", userCount, yearbox.get())
232
233                 setData("C", userCount, nameEntry.get())
234                 setData("B", userCount, emailEntry.get())

```

```

227         else:
228             # set the inputted data to the correct fields
229             setData("F", userCount, datebox.get())
230             setData("E", userCount, monthbox.get())
231             setData("D", userCount, yearbox.get())
232
233             setData("C", userCount, nameEntry.get())
234             setData("B", userCount, emailEntry.get())
235             setData("A", userCount, userCount)
236             wb.save('userbase.xlsx')
237
238             newdetails_window.destroy()
239
240             # run the credit function
241             credit(geometry, userCount)
242             wb.save('userbase.xlsx')
243
244             # confirm button calls the confirm command
245             confirmbutton = ctk.CTkButton(newdetails_window, text="Confirm", command=confirm,
246                                         height=60, width=250, font=font25)
247
248             confirmbutton.pack(pady=20)
249             newdetails_window.mainloop()
250             wb.save("userbase.xlsx")
251

```

“log_in.py”

```
1 import cv2
2     import face_recognition
3     from CtkMessageBox import CtkMessageBox
4     from openpyxl.reader.excel import load_workbook
5     import customtkinter as ctk
6
7     import face_encode
8     import update_details
9
10    # declare the fonts
11    font15 = ("Comic Sans MS", 15)
12    font20 = ("Comic Sans MS", 20)
13    font25 = ("Comic Sans MS", 25)
14    font30 = ("Comic Sans MS", 30)
15
16    wb = load_workbook(filename='userbase.xlsx') # load workbook
17    userBase = wb['Sheet1'] # load the correct workbook
18    userCount = userBase['A1'] # get the value for the amount of current users
19    userCount = userCount.value
20
21
22    def login_check():
23        global userNumber
24        global same
25
26        # store the video feed in the cap variable
27        cap = cv2.VideoCapture(0)
28        success, frame = cap.read() # when successful store it as a temporary file
29        cv2.imwrite("temp.jpg", frame)
30        is_same = False # declare is_same
31        breaking = False # declare breaking
32        count = 0 # declare count
33
34        while not is_same and not breaking:
35            bound = userCount + 1 # create the upper boundary of the for loop
36            for i in range(1, bound):
37                count += 1 # increment the count
38                print(i) # print the current variable looping through
39                # getting face encodings for first image
40                image_1 = face_encode.find_face_encodings(f"{i}.jpg")
41                # getting face encodings for second image
```

```

34     while not is_same and not breaking:
35         bound = userCount + 1 # create the upper boundary of the for loop
36         for i in range(1, bound):
37             count += 1 # increment the count
38             print(i) # print the current variable looping through
39             # getting face encodings for first image
40             image_1 = face_encode.find_face_encodings(f"{i}.jpg")
41             # getting face encodings for second image
42             image_2 = face_encode.find_face_encodings("temp.jpg")
43
44             # checking both images are same
45             is_same = face_recognition.compare_faces([image_1], image_2)[0]
46             same = is_same
47             print(f"Is Same: {is_same}")
48             if is_same:
49                 userNumber = i
50                 is_same = True
51                 break
52             else:
53                 print("The images are not same")
54
55             if count > userCount * 3: # break the program when looped too many times
56                 breaking = True
57                 break
58     return is_same
59
60
61 def updateDetails():
62     # create a message box for the update details option for the user, with name, email and exit as an option
63     msg = CTKMessageBox(title="Update details", message="What detail would you like to update?", icon="question",
64                         option_1="Email", option_2="Name", option_3="Exit")
65     response = msg.get()
66
67     if response == "Email":
68         # when email is pressed, call update email function
69         update_details.updateEmail(userNumber)
70
71     elif response == "Name":
72         # when name is pressed, call update name function
73         update_details.updateName(userNumber)
74

```

```

76 # this function adds credit to a user's account
77 def addCredit():
78     # determine the coordinate in the userBase dictionary where the user's credit is stored
79     coord = "G" + str(userNumber)
80
81     # get the current credit value for the user
82     credit = userBase[coord].value
83
84     # create a message box asking the user how much credit they would like to add
85     msg = CtkMessageBox(title=f"You have £{credit}.", message="How much credit would you like to add?", icon="question",
86                         option_1="Exit", option_2="£10", option_3="£20")
87
88     # get the user's response to the message box
89     response = msg.get()
90
91     # if the user selected £10, add £10 to their credit and save the updated userBase to a file
92     if response == "£10":
93         userBase[coord] = userBase[coord].value + 10
94         wb.save('userbase.xlsx')
95
96     # if the user selected £20, add £20 to their credit and save the updated userBase to a file
97     elif response == "£20":
98         userBase[coord] = userBase[coord].value + 20
99         wb.save('userbase.xlsx')
100
101     # save the updated userBase to a file
102     wb.save('userbase.xlsx')
103
104
105 # this function creates a login window with options to update details, add credit, or exit
106 def user_editmenu(geometry):
107     def exit():
108         login_window.destroy()
109
110     # create a new login window
111     login_window = ctk.CTk()
112     login_window.title("Login Window")
113     login_window.geometry(geometry)
114
115     # get the user's name from the userBase dictionary
116     Name = userBase["C" + str(userNumber)].value

```



```

115     # get the user's name from the userBase dictionary
116     Name = userBase["C" + str(userNumber)].value
117
118     # display a welcome message with the user's name
119     label = ctk.CTkLabel(login_window, text=f"Welcome, {Name}.\nPlease choose an option",
120                          font=font20)
121     label.pack(pady=50)
122
123     # create a button to update the user's details
124     updateDetailsButton = ctk.CTkButton(login_window, text="Update Details", command=updateDetails,
125                                         height=60, width=250, font=("Comic Sans MS", 25))
126     updateDetailsButton.pack(pady=30)
127
128     # create a button to add credit to the user's account
129     addCreditButton = ctk.CTkButton(login_window, text="Add Credit", command=addCredit, height=60, width=250,
130                                     font=font25)
131     addCreditButton.pack(pady=30)
132
133     # create a button to exit the login window
134     exitButton = ctk.CTkButton(login_window, text="Exit", command=exit,
135                               height=60, width=250, font=font25)
136     exitButton.pack(pady=30)
137
138     # start the login window's main loop
139     login_window.mainloop()
140
141
142     # this function creates a window to display an error message if there is a problem with the user's login
143     def loginFail(geometry):
144         def exit():
145             loginerror_window.destroy()
146
147         # create a new login error window
148         loginerror_window = ctk.CTk()
149         loginerror_window.title("Login Error Window")
150         loginerror_window.geometry(geometry)
151
152         # display an error message
153         label = ctk.CTkLabel(loginerror_window,
154                             text="There was an error with your log in.\nIf you do not have an account make one in the New User menu or contact admin.",
155                             font=font20)
156
157         # display an error message
158         label = ctk.CTkLabel(loginerror_window,
159                             text="There was an error with your log in.\nIf you do not have an account make one in the New User menu or contact admin.",
160                             font=font20)
161         label.pack(pady=50)
162
163         # create a button to exit the login error window
164         exitButton = ctk.CTkButton(loginerror_window, text="Exit", command=exit,
165                                   height=60, width=250, font=font25)
166         exitButton.pack(pady=30)
167
168         # start the login error window's main loop
169         loginerror_window.mainloop()
170
171

```

“update_details.py”

```

1  import re
2
3  from CTkMessageBox import CTkMessageBox
4  from openpyxl.reader.excel import load_workbook
5  import customtkinter as ctk
6
7  # load workbook from file
8  wb = load_workbook(filename='userbase.xlsx')
9
10 # select the first sheet in the workbook
11 userBase = wb['Sheet1']
12
13 # get the value of the first cell in the sheet
14 userCount = userBase['A1']
15
16 # define some fonts
17 font15 = ("Comic Sans MS", 15)
18 font20 = ("Comic Sans MS", 20)
19 font25 = ("Comic Sans MS", 25)
20 font30 = ("Comic Sans MS", 30)
21
22 def update_email(user_number):
23     def new_email():
24         # declare new_email_var as a global variable
25         global new_email_var
26         # check if the email format is valid
27         if not re.match(r"^[^@]+@^[^@]+\.[^@]+", email_entry.get()):
28             # display an error message
29             CTkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
30         else:
31             # get the coordinate for the user's email in the userBase dictionary
32             coord = "B" + str(user_number)
33
34             # get the new email from the input field
35             new_email_var = email_entry.get()
36
37             # update the user's email in the userBase dictionary
38             userBase[coord] = new_email_var
39
40             # save the updated userBase to a file
41             wb.save("userbase.xlsx")

```

```

40         # save the updated userBase to a file
41         wb.save("userbase.xlsx")
42
43         # close the email window
44         email_window.destroy()
45
46         # create a new window for the user to enter their new email
47         email_window = ctk.CTk()
48         email_window.title("New Email")
49         email_window.geometry("400x150")
50
51         # get the current email for the user
52         coord = "B" + str(user_number)
53         current = userBase[coord].value
54
55         # display the current email for the user
56         current_txt = ctk.CTkLabel(email_window, text=f"Your current email is \n{current}", font=font20)
57         current_txt.pack()
58
59         # create an input field for the user to enter their new email
60         email_entry = ctk.CTkEntry(master=email_window, placeholder_text="Enter your New Email Address",
61                                   width=200, height=25, border_width=2, corner_radius=10)
62         email_entry.pack()
63
64         # create a confirm button for the user to submit their new email
65         confirm_button = ctk.CTkButton(email_window, text="Confirm", command=new_email,
66                                       height=60, width=250, font=font25)
67         confirm_button.pack()
68
69         # start the mainloop for the email window
70         email_window.mainloop()
71
72
73     def update_name(user_number):
74         def new_name():
75             # declare new_name_var as a global variable
76             global new_name_var
77             # get the coordinate for the user's name in the userBase dictionary
78             coord = "C" + str(user_number)
79
80             # get the new name from the input field and capitalize the first letter of each word

```

```

80     # get the new name from the input field and capitalize the first letter of each word
81     new_name_var = name_entry.get()
82     new_name_var = new_name_var.title()
83
84     # update the user's name in the userBase dictionary
85     userBase[coord] = new_name_var
86
87     # save the updated userBase to a file
88     wb.save("userBase.xlsx")
89
90     # close the name window
91     name_window.destroy()
92
93     # create a new window for the user to enter their new name
94     name_window = ctk.CTk()
95     name_window.title("New Name")
96     name_window.geometry("300x100")
97
98     # get the current name for the user
99     coord = "C" + str(user_number)
100     current = userBase[coord].value
101
102     # display the current name for the user
103     current_txt = ctk.CTkLabel(name_window, text=f"Your current name is {current}", font=font20)
104     current_txt.pack()
105
106     # create an input field for the user to enter their new name
107     name_entry = ctk.CTkEntry(master=name_window, placeholder_text="Enter your New Name",
108                             width=200, height=25, border_width=2, corner_radius=10)
109     name_entry.pack()
110
111     # create a confirm button for the user to submit their new name
112     confirm_button = ctk.CTkButton(name_window, text="Confirm", command=new_name,
113                                   height=60, width=250, font=font25)
114     confirm_button.pack()
115
116     # start the mainloop for the name window
117     name_window.mainloop()
118

```

“admin.py”

```
1 import re
2
3 import customtkinter as ctk
4 from CtkMessageBox import CtkMessageBox
5 from openpyxl.reader.excel import load_workbook
6
7 from CtkScrollableDropdown import CtkScrollableDropdown
8
9 wb = load_workbook(filename='userbase.xlsx') # load workbook
10 userBase = wb['Sheet1'] # load the correct workbook
11
12
13 def admin():
14     # create a new window for the admin login
15     adminlog_window = ctk.CTk()
16     adminlog_window.geometry("250x300")
17     adminlog_window.title("Login Page")
18
19     # create a frame to hold the login widgets
20     frame = ctk.CTkFrame(master=adminlog_window)
21     frame.pack(pady=20, padx=40, fill='both', expand=True)
22
23     # create a label for the login page
24     label = ctk.CTkLabel(master=frame, text='Login Page')
25     label.pack(pady=12, padx=10)
26
27     # create an input field for the admin to enter their username
28     user_entry = ctk.CTkEntry(master=frame, placeholder_text="Username")
29     user_entry.pack(pady=12, padx=10)
30
31     # create an input field for the admin to enter their password
32     user_pass = ctk.CTkEntry(master=frame, placeholder_text="Password", show="*")
33     user_pass.pack(pady=12, padx=10)
34
35     # define a function to handle the admin login
36     def admin_login():
37         # get the username and password from the input fields
38         username = user_entry.get()
39         password = user_pass.get()
40
```

```

41 # check if the username and password are correct
42 if username == "admin" and password == "Password":
43     # display a success message
44     CTKMessageBox(title="Login", message="Login Successful!",
45                   icon="info", option_1="Okay")
46
47     # close the login window
48     adminlog_window.destroy()
49
50     # create a new window for the admin page
51     admin_page = ctk.CTk()
52     admin_page.geometry("500x500")
53     admin_page.title("Admin window")
54
55     # create a frame to hold the admin page widgets
56     frame = ctk.CTkFrame(master=admin_page)
57     frame.pack(pady=20, padx=40, fill='both', expand=True)
58
59     def admin_email():
60         # create a new window for the admin email page
61         admin_email_page = ctk.CTk()
62         admin_email_page.geometry("500x500")
63         admin_email_page.title("Admin Email Page")
64
65         # create a frame to hold the email widgets
66         frame = ctk.CTkFrame(master=admin_email_page)
67         frame.pack(pady=20, padx=40, fill='both', expand=True)
68
69         # create an input field for the admin to enter the user's ID number
70         user_count_entry = ctk.CTkEntry(master=frame, placeholder_text="User ID number")
71         user_count_entry.pack(pady=12, padx=10)
72
73         # create an input field for the admin to enter the new email address
74         email_entry = ctk.CTkEntry(master=frame, placeholder_text="New Email Address")
75         email_entry.pack(pady=12, padx=10)
76
77         def done():
78             # check if the email address is valid
79             if not re.match(r"^[^@]+@[^@]+\.[^@]+$", email_entry.get()):
80                 # display an error message if the email address is invalid
81                 CTKMessageBox(title="Error", message="Invalid Entry!", icon="cancel")

```

```

80         # display an error message if the email address is invalid
81         CtkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
82     else:
83         # get the user's ID number from the input field
84         user_id = user_count_entry.get()
85
86         # create a coordinate for the user's email address in the user base
87         coord = "B" + str(user_id)
88
89         # update the user's email address in the user base
90         user_base[coord] = email_entry.get()
91
92         # display a success message
93         CtkMessageBox(title="Email", message=f"User {user_id} email successfully changed",
94                       icon="info", option_1="Okay")
95
96         # save the updated user base to a file
97         wb.save('userbase.xlsx')
98
99     def email_exit():
100         # destroy the admin email page
101         admin_email_page.destroy()
102
103     done_button = ctk.CtkButton(master=frame, text="Done", command=done)
104     done_button.pack(pady=12, padx=10)
105
106     exit_button = ctk.CtkButton(master=frame, text="Exit", command=email_exit)
107     exit_button.pack(pady=12, padx=10)
108
109     # start the main event loop for the admin email page
110     admin_email_page.mainloop()
111
112     def admin_name():
113         # create a new window for the admin name page
114         admin_name_page = ctk.Ctk()
115         admin_name_page.geometry("500x500")
116         admin_name_page.title("Admin Name Page")
117
118         # create a frame to hold the input fields and buttons
119         frame = ctk.CtkFrame(master=admin_name_page)
120         frame.pack(pady=20, padx=40, fill='both', expand=True)

```

```

119 frame = ctk.CTkFrame(master=admin_name_page)
120 frame.pack(pady=20, padx=40, fill='both', expand=True)
121
122 # create an input field for the user's ID number
123 user_count_entry = ctk.CTkEntry(master=frame, placeholder_text="User ID number")
124 user_count_entry.pack(pady=12, padx=10)
125
126 # create an input field for the new name
127 name_entry = ctk.CTkEntry(master=frame, placeholder_text="New Name")
128 name_entry.pack(pady=12, padx=10)
129
130 def done():
131     # get the user's ID number and new name from the input fields
132     user_id = userCount_entry.get()
133     coord = f"C{user_id}"
134
135     # update the user base with the new name
136     user_base[coord] = name_entry.get()
137
138     # show a message box to confirm the name change
139     CtkMessageBox(title="Name", message=f"User {user_id} name successfully changed",
140                   icon="info", option_1="Okay")
141
142     # save the updated user base to a file
143     wb.save("userbase.xlsx")
144
145 def email_exit():
146     # destroy the admin name page
147     admin_name_page.destroy()
148
149 # create a "Done" button to save the new name
150 done_button = ctk.CTkButton(master=frame, text="Done", command=done)
151 done_button.pack(pady=12, padx=10)
152
153 # create an "Exit" button to close the admin name page
154 exit_button = ctk.CTkButton(master=frame, text="Exit", command=email_exit)
155 exit_button.pack(pady=12, padx=10)
156
157 # start the main event loop for the admin name page
158 admin_name_page.mainloop()
159

```



```

158     admin_name_page.mainloop()
159
160     def admin_date():
161         # create a new admin date page
162         admin_date_page = ctk.CTk()
163         admin_date_page.geometry("500x500")
164         admin_date_page.title("Admin Date Page")
165
166         # create a frame to hold the widgets
167         frame = ctk.CTkFrame(master=admin_date_page)
168         frame.pack(pady=20, padx=40, fill='both', expand=True)
169
170         # create a user ID input field
171         user_count_entry = ctk.CTkEntry(master=frame, placeholder_text="User ID number")
172         user_count_entry.pack(pady=12, padx=10)
173
174         # list of dates in a month
175         dates = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27,
176                 28, 29, 30, 31]
177
178         # list of months in a year
179         months = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
180
181         # list of years to choose from
182         years = [2024, 2023, 2022, 2021, 2020, 2019, 2018, 2017, 2016, 2015, 2014, 2013, 2012, 2011, 2010, 2009,
183                 2008, 2007, 2006, 2005, 2004, 2003, 2002, 2001, 2000, 1999, 1998, 1997, 1996, 1995, 1994, 1993,
184                 1992, 1991, 1990, 1989, 1988, 1987, 1986, 1985, 1984, 1983, 1982, 1981, 1980, 1979, 1978, 1977,
185                 1976, 1975, 1974, 1973, 1972, 1971, 1970, 1969, 1968, 1967, 1966, 1965, 1964, 1963, 1962, 1961,
186                 1960, 1959, 1958, 1957, 1956, 1955, 1954, 1953, 1952, 1951, 1950, 1949, 1948, 1947, 1946, 1945,
187                 1944, 1943, 1942, 1941, 1940, 1939, 1938, 1937, 1936, 1935, 1934, 1933, 1932, 1931, 1930]
188
189         # create a year dropdown menu
190         year_box = ctk.CTkComboBox(frame, values="Year")
191         year_box.pack(pady=30)
192         # add scrollable functionality to the year dropdown
193         CtkScrollableDropdown(year_box, values=years, justify="left", button_color="transparent")
194
195         # create a month dropdown menu
196         month_box = ctk.CTkComboBox(frame, values="Month")
197         month_box.pack(pady=30)
198
199         # add scrollable functionality to the month dropdown
200         CtkScrollableDropdown(month_box, values=months, justify="left", button_color="transparent")
201
202         # create a date dropdown menu
203         date_box = ctk.CTkComboBox(frame, values="Day")
204         date_box.pack(pady=30)
205         # add scrollable functionality to the date dropdown
206         CtkScrollableDropdown(date_box, values=dates, justify="left", button_color="transparent")
207
208     def confirm():
209         # get user ID from input field
210         userID = userCount_entry.get()
211
212         # check if all dropdowns have been changed from default values
213         if yearbox.get() != "Y" and monthbox.get() != "M" and datebox.get() != "D":
214             # check if February and day is 29, 30, or 31
215             if monthbox.get() == "2" and (
216                 datebox.get() == "29" or datebox.get() == "30" or datebox.get() == "31"):
217                 # show error message
218                 CtkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
219             # check if month is April, June, September, or November and day is 31
220             elif (
221                 monthbox.get() == "4" or monthbox.get() == "6" or monthbox.get() == "9" or monthbox.get() == "11") and datebox.get() == "31":
222                 # show error message
223                 CtkMessageBox(title="Error", message="Invalid Entry!", icon="cancel")
224             else:
225                 # update userBase dictionary with new birthday values
226                 userBase['F' + str(userID)] = datebox.get()
227                 userBase['E' + str(userID)] = monthbox.get()
228                 userBase['D' + str(userID)] = yearbox.get()
229
230                 # show success message
231                 CtkMessageBox(title="Date", message=f"User {userID} birthday successfully changed",
232                               icon="info", option_1="Okay")
233                 # save changes to userbase.xlsx
234                 wb.save("userbase.xlsx")
235
236         # create a done button with the text "Done"
237         doneButton = ctk.CTkButton(master=frame, text="Done", command=confirm)
238         doneButton.pack(pady=20)

```

```

238
239     # define a function to destroy the adminDate_page when called
240     def dateExit():
241         adminDate_page.destroy()
242
243     # create an exit button with the text "Exit" and assign it the dateExit function
244     exitButton = ctk.CTkButton(master=frame, text="Exit", command=dateExit)
245     exitButton.pack()
246
247     # start the mainloop for the adminDate_page
248     adminDate_page.mainloop()
249
250 def adminCredit():
251     # create a new admin credit page
252     adminCredit_page = ctk.CTk()
253     adminCredit_page.geometry("500x500")
254     adminCredit_page.title("Admin Credit Page")
255
256     # create a frame for the page
257     frame = ctk.CTkFrame(master=adminCredit_page)
258     frame.pack(pady=20, padx=40, fill='both', expand=True)
259
260     # create an entry field for user id number
261     userCount_entry = ctk.CTkEntry(master=frame, placeholder_text="User ID number")
262     userCount_entry.pack(pady=12, padx=10)
263
264     # create an entry field for new credit value
265     credit_entry = ctk.CTkEntry(master=frame, placeholder_text="New Credit Value")
266     credit_entry.pack(pady=12, padx=10)
267
268     def done():
269         # get the user id number from the userCount_entry field
270         userID = userCount_entry.get()
271
272         # create a coordinate string for the user id number
273         coord = "G" + str(userID)
274
275         # set the credit value for the user id number
276         userBase[coord] = credit_entry.get()
277

```

```

277
278         # show a message box indicating that the credit value was successfully changed
279         CTKMessageBox(title="Email", message=f"User {userID} credit successfully changed",
280                       icon="info", option_1="Okay")
281
282         # save the userbase to an excel file
283         wb.save("userbase.xlsx")
284
285
286     def creditExit():
287         # destroy the adminCredit_page window
288         adminCredit_page.destroy()
289
290         # create a "Done" button with the command to execute the 'done' function
291         doneButton = ctk.CTkButton(master=frame, text="Done", command=done)
292         doneButton.pack(pady=12, padx=10)
293
294         # create an "Exit" button with the command to execute the 'creditExit' function
295         exitButton = ctk.CTkButton(master=frame, text="Exit", command=creditExit)
296         exitButton.pack(pady=12, padx=10)
297
298         # start the main event loop for the adminCredit_page window
299         adminCredit_page.mainloop()
300
301     def adminPageExit():
302         admin_page.destroy()
303         admin()
304
305         # create an "Email" button with the command to execute the 'adminEmail' function
306         adminEmailButton = ctk.CTkButton(master=frame, text='Email', command=adminEmail)
307         adminEmailButton.pack(pady=12, padx=10)
308
309         # create a "Name" button with the command to execute the 'adminName' function
310         adminNameButton = ctk.CTkButton(master=frame, text='Name', command=adminName)
311         adminNameButton.pack(pady=12, padx=10)
312
313         # create a "Date" button with the command to execute the 'adminDate' function
314         adminDateButton = ctk.CTkButton(master=frame, text='Date', command=adminDate)
315         adminDateButton.pack(pady=12, padx=10)
316

```

```

316
317     # create a "Credit" button with the command to execute the 'adminCredit' function
318     adminCreditButton = ctk.CTkButton(master=frame, text='Credit', command=adminCredit)
319     adminCreditButton.pack(pady=12, padx=10)
320
321     # create an "Exit" button with the command to execute the 'adminPageExit' function
322     adminExitButton = ctk.CTkButton(master=frame, text='Exit', command=adminPageExit)
323     adminExitButton.pack(pady=12, padx=10)
324
325     # start the main event loop for the admin_page window
326     admin_page.mainloop()
327
328
329     else:
330         # display a message box with the title "Detection Error!" and the message "Incorrect Password or Username"
331
332         msg = CTkMessagebox(title="Detection Error!", message="Incorrect Password or Username",
333                             icon="cancel", option_1="Exit", option_2="Retry")
334
335         # check if the user clicked the "Exit" button in the message box
336
337         if msg.get() == "Exit":
338             # destroy the adminlog_window window
339
340             adminlog_window.destroy()
341
342
343     # create a "Login" button with the command to execute the 'adminlogin' function
344     button = ctk.CTkButton(master=frame, text='Login', command=adminlogin)
345     button.pack(pady=12, padx=10)
346
347     # define a function 'adminexit' that destroys the 'adminlog_window' window
348     def adminexit():
349         adminlog_window.destroy()
350
351     # create an "Exit" button with the command to execute the 'adminexit' function
352     exit_button = ctk.CTkButton(master=frame, text="Exit", command=adminexit)
353     exit_button.pack()
354
355     # start the main event loop for the 'adminlog_window' window
356     adminlog_window.mainloop()
357
358 admin() -> admin_login() -> if username == "admin" and pass...
348     def adminexit():
349         adminlog_window.destroy()
350
351     # create an "Exit" button with the command to execute the 'adminexit' function
352     exit_button = ctk.CTkButton(master=frame, text="Exit", command=adminexit)
353     exit_button.pack()
354
355     # start the main event loop for the 'adminlog_window' window
356     adminlog_window.mainloop()
357

```

“Barrier.py”

```
1 from datetime import date
2 from tkinter import *
3
4 import cv2
5 import tkinter as tk
6
7 import face_recognition
8 from PIL import Image, ImageTk
9 from openpyxl.reader.excel import load_workbook
10
11 import face_encode
12
13
14 wb = load_workbook(filename='userbase.xlsx') # load workbook
15 userBase = wb['Sheet1'] # load the correct workbook
16 userCount = userBase['A1'] # get the value for the amount of current users
17 userCount = userCount.value
18
19 # Initialize the camera
20 cap = cv2.VideoCapture(0)
21
22 # Load the Haar Cascade classifier for face detection
23 face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
24
25 # Create a Tkinter window
26 app = tk.Tk()
27 screen_width = app.winfo_screenwidth()
28 wcentre = (screen_width // 2) - (screen_width // 2)
29
30 screen_height = app.winfo_screenheight()
31 hcentre = (screen_height // 2) - (screen_height // 2)
32
33 app.geometry(f"{screen_width}x{screen_height}+{hcentre}+{wcentre}")
34
35 # Create a frame to hold the OpenCV window
36 frame = tk.Frame(app)
37 frame.pack()
38
39 # Create a label to display the OpenCV image
40
41 image_label = tk.Label(frame)
```

```

41 image_label = tk.Label(frame)
42 image_label.pack()
43
44 text_label = tk.Label(app, text="")
45 text_label.pack()
46
47 # Create a function to display the OpenCV window in the Tkinter frame, define is_same as false
48 is_same = False
49
50
51 def display_frame():
52     global empty, userNumber
53     # Read a frame from the camera
54     ret, frame = cap.read()
55
56     # Convert the frame to grayscale
57     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
58
59     # Detect faces in the frame
60     faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5
61                                           , minSize=(30, 30))
62
63     # Draw a rectangle around the detected faces
64     for (x, y, w, h) in faces:
65         cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
66
67     # if there is a face present in the frame, save it as a temporary image
68     if len(faces) > 0:
69         face_frame = frame[faces[0][1]:faces[0][1] + faces[0][3], faces[0][0]:faces[0][0] + faces[0][2]]
70         cv2.imwrite("temp.jpg", face_frame)
71
72         # find if the face is clear enough to find encodings of it
73         empty = face_encode.return_empty("temp.jpg")
74
75         # if the face is clear enough
76         if not empty:
77
78             # compare temp file to userbase, same code as kiosk
79             userCount = userBase["A1"].value
80             counter = userCount + 1
81

```

```

81
82     # getting face encodings for second image
83     image_2 = face_encode.find_face_encodings("temp.jpg")
84
85     # loop between the values of 1 and the current user count(+1)
86     for i in range(1, counter):
87         global is_same
88         # print the current user image comparing the temporary image to
89         print(i)
90         # getting face encodings for current user count image
91         image_1 = face_encode.find_face_encodings(f"{i}.jpg")
92
93         # checking both images are same and printing whether they are
94         is_same = face_recognition.compare_faces([image_1], image_2)[0]
95         print(f"Is Same: {is_same}")
96
97         if is_same:
98             userNumber = i
99             # finding the distance level between images
100             distance = face_recognition.face_distance([image_1], image_2)
101             distance = round(distance[0] * 100)
102
103             # calculating accuracy level between images and printing it
104             accuracy = 100 - round(distance)
105             print("The images are same")
106             print(f"Accuracy Level: {accuracy}%")
107             is_same = True
108
109             break
110         else:
111             # if the images are not a match, print that they are the same
112             print("The images are not same")
113
114     # Convert the OpenCV image to a Tkinter-compatible image
115     image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
116     image = Image.fromarray(image)
117     photo = ImageTk.PhotoImage(image)
118
119     # Update the existing label with the new image
120     image_label.config(image=photo)
121     image_label.image = photo

```

```

120 image_label.config(image=photo)
121 image_label.image = photo
122
123 # if is_same is true, update the label to say face detected and is_same to be false so it loops through again
124 if is_same:
125     is_same = False
126     text_label.config(text="Face Detected!", fg="green")
127
128     # find the existing credit of the user and print the name of the user
129     print(userBase["C" + str(userNumber)].value)
130     coord = "G" + str(userNumber)
131     precredit = int(userBase[coord].value)
132
133     # find the users birthday
134     y = userBase["D" + str(userNumber)].value
135     m = userBase["E" + str(userNumber)].value
136     d = userBase["F" + str(userNumber)].value
137     birthdate = date(year=int(y), month=int(m), day=int(d))
138     today = date.today()
139
140     # calculate the users age
141     age_days = today - birthdate
142     age_days = age_days.days
143     age_years = age_days // 365
144
145     if age_years < 18: # when child, set price to 3 and message to be child fare
146         ticket_price = 3
147         message = "Child fare charged, please proceed through the barrier"
148     elif age_years < 25: # when student, set price to 4 and message to be student fare
149         ticket_price = 4
150         message = "Student fare charged, please proceed through the barrier"
151     elif age_years > 65: # when concession age, set price to 3 and message to be concession fare
152         ticket_price = 3
153         message = "Concession fare charged, please proceed through the barrier"
154     else:
155         ticket_price = 5 # when adult, set price to 5 and message to be adult fare
156         message = "Adult fare charged, please proceed through the barrier"
157
158     # if not enough credit to afford the specific age ticket
159     if precredit - ticket_price < 0:
160         # initialise not enough credit window

```



```

159     if precredit - ticket_price < 0:
160         # initialise not enough credit window
161         barrier_window = tk.Tk()
162         barrier_window.geometry(f"600x300+350+150")
163         barrier_window.attributes('-topmost', True)
164
165         # window text, not enough credit
166         text = tk.Label(barrier_window, font=("Arial", 20),
167                         text="Not enough credit, add more at the kiosk")
168         text.pack(pady=100, )
169
170         #destroy barrier window after 8 seconds and re display the frame, restarting the program
171         barrier_window.after(8000, lambda: (barrier_window.destroy(), display_frame()))
172         barrier_window.mainloop()
173
174     # if the user does have enough credit
175     else:
176         # deduct fare from current credit and save it to the users credit
177         postcredit = precredit - ticket_price
178         print("Credit charged")
179         userBase[coord] = str(postcredit)
180         wb.save("userbase.xlsx")
181
182         # create a barrier window displaying that they have been charged
183         barrier_window = tk.Tk()
184         barrier_window.geometry(f"600x300+350+150")
185         barrier_window.attributes('-topmost', True)
186
187         # display the set message of credit charged
188         text = tk.Label(barrier_window, font=("Arial", 15), text=message)
189         text.pack(pady=100, )
190
191         # after 8 seconds, destroy the pop up and restart the program
192         barrier_window.after(8000, lambda: (barrier_window.destroy(), display_frame()))
193         barrier_window.mainloop()
194     else:
195         # if they are not the same, display no entry in text label
196         text_label.config(text="no entry", fg="red")
197
198     # Schedule the next frame to be displayed after 30 milliseconds
199     app.after(30, display_frame)

```

```

198     # Schedule the next frame to be displayed after 30 milliseconds
199     app.after(30, display_frame)
200
201
202     # Start the display loop
203     display_frame()
204
205     # Run the Tkinter event loop
206     app.mainloop()
207
208     # Release the camera and destroy all windows
209     cap.release()
210     cv2.destroyAllWindows()
211

```